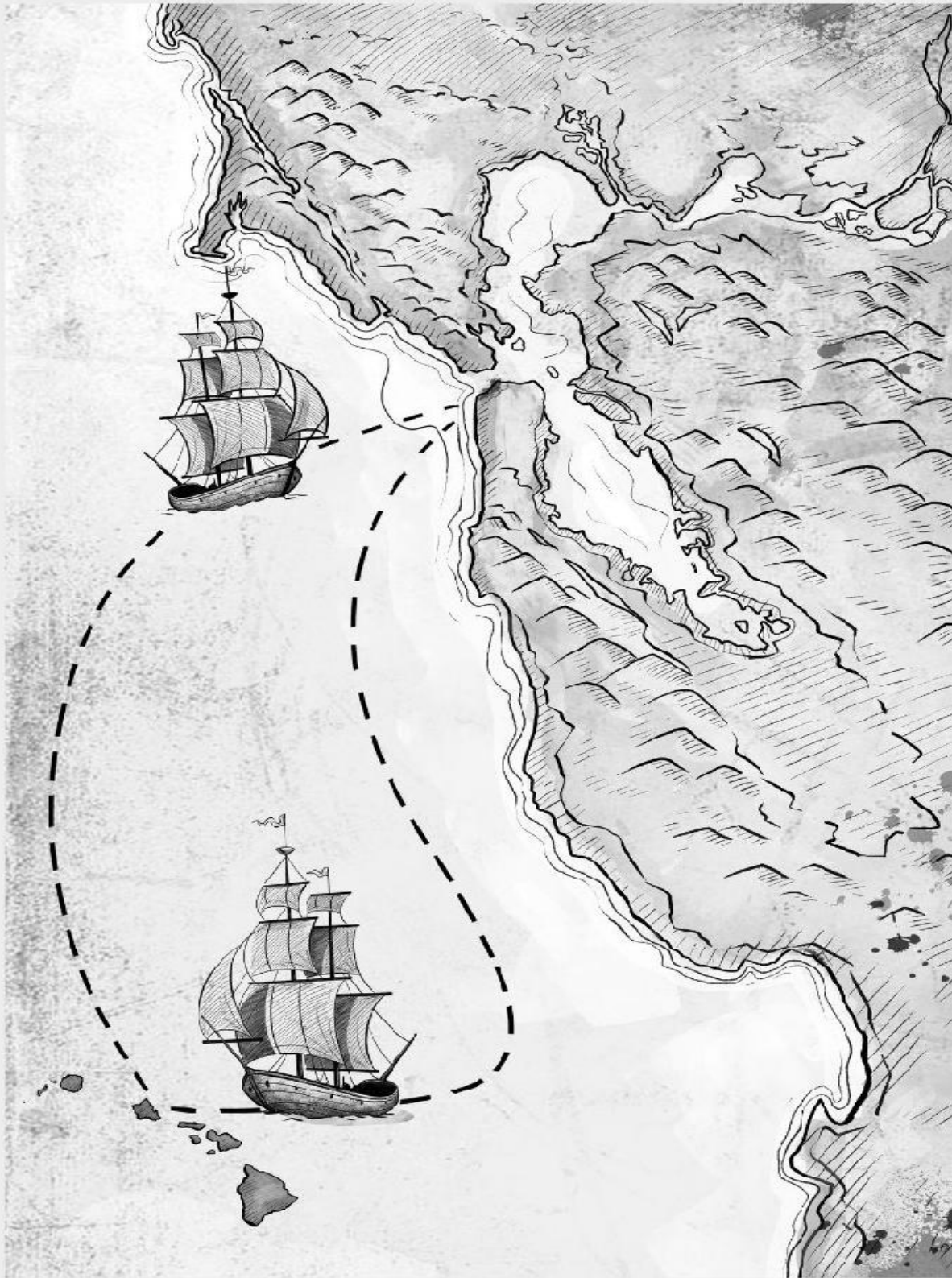




Staff Engineer

Leadership beyond the management track

Will Larson

资深工程师

管理层之外的领导力

威尔·拉森

© 2020 威尔·拉森

资深工程师

[资深工程师](#)

[致谢](#)

[前言](#)

[序言](#)

[概述](#)

[资深工程师原型](#)

[资深工程师实际上做什么？](#)

[标题重要吗？](#)

[以Staff级别运作](#)

[做重要的事情](#)

[编写工程策略](#)

[管理技术质量](#)

[与权威保持一致](#)

[要领导，先跟随](#)

[学会永远不出错](#)

[为他人创造空间](#)

[建立同行网络](#)

[向高管展示](#)

[在你所在的地方获得头衔](#)

[晋升材料](#)

[找到你的赞助人](#)

[员工项目](#)

[进入房间，并留在那里](#)

[保持可见性](#)

[决定更换公司](#)

[找到合适的公司](#)

[面试 Staff-plus 职位](#)

[协商你的报价](#)

[故事](#)

[Michelle Bu - Stripe支付产品技术主管](#)
[Ras Kasa Williams - Mailchimp的资深工程师](#)
[Keavy McMinn - Fastly的资深首席工程师](#)
[Bert Fan - Slack的资深工程师](#)
[Katie Saylor-Miller - Etsy的前端架构师](#)
[Ritu Vincent - Dropbox的资深工程师](#)
[Rick Boone - Uber基础设施副总裁的战略顾问](#)
[Nelson Elhage - 前Stripe的资深工程师](#)
[Diana Pojar - Slack的资深数据工程师](#)
[Dan Na - Squarespace 的 Staff Engineer 和团队负责人](#)
[乔伊·埃伯茨 - Split 高级职员软件工程师](#)
[达米安·申克尔曼 - Auth0 首席工程师](#)
[德米特里·彼得拉什科 - Stripe 基础设施主管技术顾问](#)
[斯蒂芬·万 - Samsara 职员工程师](#)

[结束语](#)

[资源](#)

[关于 Staff-plus 工程师的补充资源](#)
[Staff-plus 工程师在组织中扮演什么角色?](#)
[管理 Staff-plus 工程师](#)
[设计 Staff-plus 面试流程](#)
[员工晋升阶梯](#)

资深工程师

致谢

写这本书需要一个村庄的力量，尤其是在 2020 年这个混乱的一年里，大部分工作都是在那段时间完成的。要感谢的人太多了，很难知道从哪里开始，但实际上只有一个地方：那就是分享他们故事的人。

谢谢你，[米歇尔](#)，[卡萨](#)，[基维](#)，[伯特](#)，[凯蒂](#)，[里图](#)，[瑞克](#)，[纳尔逊](#)，[戴安娜](#)，[和](#)，[喜悦](#)，[达米安](#)，[德米特里](#)，[和](#)[斯蒂芬](#)。我同样感谢那些为我们贡献故事的人们[staffeng.com](#)，他们的故事没有收录在这本书中。每一个故事都是独特的声音，非常值得一读。

[Tanya Reilly](#)的序言非常精彩，我正热切地等待 Tanya 出版关于 Staff 工程的权威书籍，以取代这本。封面插图由[Luciana Guerra](#)绘制，让人想起古老的海图，水手们在其中航行，面对着与目前笼罩着 Staff 工程师角色的不确定性相同的境况。TechWriters Discord 上的 Greglas 是本书拥有合理字体和格式的唯一原因。在[TechWriters Discord](#)中的社区提供了无尽的建议和支持，特别感谢 Gergely、Shawn 和 Uma。[Laurel](#)对整本书进行了文字编辑；如果你在读完两章不一致的 capitalization 后没有把它扔进壁炉，那要感谢她。

许多人审阅了各个章节，特别感谢[西德](#)，[加里](#)，[帕特](#)，[格尔盖伊](#)，[皮特](#)，[和](#)[汤米](#)。我还要感谢所有写推荐语的人以及二十多位[贡献了拉取请求的人](#)。

前言

我的副本一个优雅的谜题威尔·拉森的第一本书，带着一丝焦虑。我对这本书没有冒犯之意——它充满了见解，我推荐它。但这是一本为管理者写的书，而我是在办公室的经理读书会上读的。我不是经理；我是一名首席工程师。我不确定我是否应该在那里。

对于选择了“技术领导”职业道路的工程师来说，培养技能通常意味着感觉自己身处错误的房间。随着我们行业的发展和应对越来越大的问题，越来越多的公司认识到，需要那些“见过世面”的工程师来推动技术战略，领导跨团队和组织的项目，并通过建模良好的工程实践来提高每个人的水平。但是，在任何学科中蓬勃发展都意味着找到可以学习的资源和社区——这可能需要在技术领导岗位上发挥一些创造力。

哦，我们有很多关于技术的书籍、聚会和会议，但是从“高级”级别开始（甚至更早），技术技能是不够的。成功通常意味着解读业务需求、传达明确的方向、化解迫在眉睫的危机、说服团队就权衡达成一致，或者仅仅是产生良好的影响。工程书架上没有太多关于这些主题的内容。相反，工程师们阅读商业或管理书籍，挑选出可以应用于技术决策、架构等主题，以及可以在没有直接权力的情况下发挥作用的技巧。目前，经理读书会通常是我们拥有的最好的学习社区。（而且，需要明确的是，我们很感激被邀请！请继续邀请我们，经理朋友们。）

高级工程师资源的缺乏是一个更大的问题的一部分：很容易忘记这份工作到底是什么。晋升到“高级软件工程师”以上的工程师可能会发现当他们探索一个未定义的新角色时，他们会感到孤独，他们会努力理解“影响力”这个神秘的概念，以了解他们是否在做正确的事情，并努力适应以季度或年来衡量的反馈周期，而不是以冲刺来衡量。

这不是故意的疏忽：管理者只是并不总是知道如何支持他们最资深的工程师。当你的下属应该就最重要的问题向你提供建议，而不是反过来时，你如何确定他们正在做正确的事情？你应该期望那些将成为组

织其他成员榜样的工程师具备哪些技能和行为？而且，不可避免的问题是，他们应该编写多少代码？

没有公司通用的职业阶梯，或者职位头衔根本不一致，这无济于事。这就是为什么当威尔推出 staffeng.com 并将各种角色封装到集体“staff-plus”中时，会让人感到如此欣慰。好吧，我们仍然有截然不同的头衔，但至少我们现在有一个词来描述我们所谈论的角色类型。我已经听到“staff-plus”在使用，而以前我们会绞尽脑汁地描述那些比高级工程师更资深的工程师。

staffeng.com 立即在科技界引起轰动。我从未见过一个全新的网站如此迅速地成为某个主题的权威资源。威尔定义并描述了 staff engineer 的角色，提供了关于如何成为一名 staff engineer 的明确建议，并阐述了他所说的“组织层面的脊椎按摩疗法”，这是胜任这项工作所必需的。他擅长于处理一个熟悉但模棱两可的主题，围绕它画一个清晰的圆圈，并解释我们其他人一直在看但一直没有看到的东西。伴随建议而来的是我们一直需要的社区的开始。许多访谈（与 staff engineer、首席工程师、高级首席工程师、架构师、技术主管、技术顾问等等——我告诉过你头衔是令人困惑的）展示了通往员工角色及其他职位的各种途径；大多数读者都能够找到一个路径感觉熟悉的人，我希望，这是可以实现的。

这正是我们现在需要在职工程方面进行的写作，我很高兴威尔以书本形式提供了它。就像一个*优雅的谜题*，你即将阅读的这本书提供了基于真实生活经验的清晰、务实和实用的领导力建议。但这一次，高级工程师是听众。好吧，是听众之一。无论你是一名正在思考自己应该做什么的高级工程师，一名正在职业道路之间选择的中级工程师，还是一名想要让你的最高级工程师获得成功的经理，你都会在这里找到智慧。

软件工程的风险每年都在增加，而且这种情况不会很快停止。正如威尔所说，“就像洛拉克斯在他广受欢迎的儿童读物中为树木代言一样，高级工程师为他们公司的技术代言。”我们对榜样工程师的技能和行为

的要求将直接影响我们编写的代码、我们部署的算法、我们做出的决策以及我们认为可以接受的模式。

我很高兴技术领域的领导者终于有了这种指导。我希望这只是为我们这些在经理阅读小组中感到有点格格不入的人提供的众多书籍和资源中的第一本。

Tanya Reilly
Squarespace首席工程师。

序言

当人们问起我写第一本书时，一个*优雅的谜题*，我说我花了十年时间写了一半，另外花了六个月写了另一半。它的创作有时是一种挑战，而且有很多我想在最终产品中改变的东西，但创作它是我个人的亮点。作为一名作者，你应该警告未来的作者不要写书，但我没有这样的警告，即使是对我自己：我想再写一本书。

问题是，写什么书？我可能在某个时候会对工程管理有更多的话要说，但我现在肯定没有更多的话要说了。我担任经理的时间比担任开发人员的时间要多，而且还有其他人更适合撰写关于有效开发的文章。我希望有一天能写一本关于基础设施工程的书，但我正在努力花更少在未来几年里，减少对基础设施的思考时间。

最终，我回到了两个核心问题。有什么领域是今天挑战我的？有什么话题是我认为一本书可以朝着积极的方向推动科技行业的？符合这两个标准的一个问题是Staff工程师的角色。在大多数职业中，人们随着资历的增长越来越确定自己的角色，但我的经验是，许多工程师在达到他们的第一个Staff角色后会失去方向感。眼睁睁地看着人们追求Staff工程师的角色十年或更长时间，然后发现他们讨厌这项工作或觉得自己没有能力成功，这真是太可怕了。

除了亲自深入研究寻找和胜任这些Staff-plus角色的主题之外，我还意识到不同的人达到这些角色时有非常不同的经历。我合作过的一些最有才华的人都在努力通过高级工程师级别。每次他们尝试时，他们都会遇到系统性的障碍，将他们的晋升推迟一个绩效周期。

当我开始写这本书时，我的第一步是列出章节和主题。盯着那个大纲，我很快意识到我不能独自写这本书。这导致了一系列的访谈，在访谈中，我得以从人们达到他们的第一个Staff工程师角色的非凡故事以及他们在晋升后如何在这些角色中运作的故事中学习。这些故事，加上我作为一名经理支持、晋升和招聘Staff-plus工程师的经验，慢慢地变成了这本书。

我希望资深工程师帮助你完善你对技术领导力的愿景，以及你如何朝着这个愿景发展。

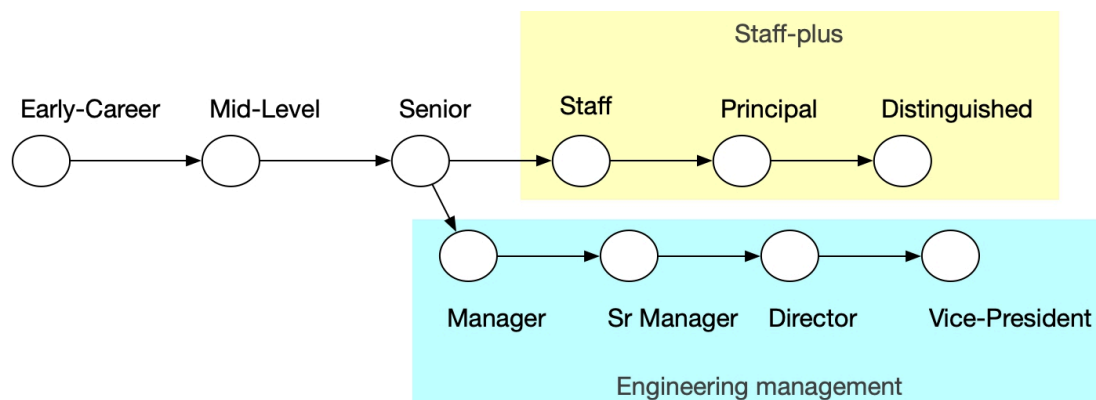
概述



在大多数科技公司，你将达到高级软件工程师，即[职业级别](#)对于软件工程师来说，在五到八年内。在职业级别上，你公司的职业阶梯不会要求你努力争取下一次晋升；进一步晋升是一种例外，而不是预期。这也是许多工程师第一次有机会进入工程管理的时候。

在过去的几年里，我们看到涌现出大量关于解锁工程管理职业道路的书籍，比如卡米尔·富尼耶的《[管理者之路](#)》，朱莉·卓的《[成为经理](#)》，拉拉·霍根的《[弹性管理](#)》，甚至还有我自己的。[一个优雅的题目](#)工程管理职业生涯并非易事，但有地图可以帮助你导航。

如果你想在成为工程经理的情况下提升你的职业生涯呢？许多公司会兴奋地告诉你，他们有两条软件工程职业道路。工程管理是第一条道路，第二条是技术领导。技术领导道路上的人的头衔通常是资深工程师和首席工程师。这条道路的存在本身就是一种进步的标志，但要使其既易于进入又具有影响力，还有很多工作要做。



典型的双轨工程职业阶梯

本书对最常见的职称序列进行了标准化：从高级到专家，然后是首席，最后是杰出。它使用“专家+”一词作为专家、首席和杰出职称的总称。许多公司只拥有这些职称的一个子集，随着团队的成长而慢慢增加，但只有一种技术领导职称的公司几乎总是使用专家。少数公司使用另一种序列，但它们是少数。

围绕技术领导职业道路存在普遍的模糊性，使得回答关于专家+角色的看似简单的问题变得困难。如果你是一名高级工程师，并且想成为专家工程师，你应该发展哪些技能？仅仅技术能力就足够了吗？大多数人是如何进入这些角色的？你的经理在帮助你前进的过程中扮演什么角色？你会喜欢成为一名专家工程师吗，还是会辛苦多年才能达到一个不适合你的角色？本书将帮助你回答所有这些问题。

鉴于围绕这些角色的广泛困惑，我知道我不能过多地依赖我自己的经验来写这本书。行业内十几位以上的专家+工程师非常友善地分享了他们达到和在专家+角色中工作的经验。他们的智慧创造了一些比我个人所能创造的更丰富、更广泛和更有远见的东西独自写出来的。

如果你已经担任专家+角色，我希望这些文字能激励你在管理轨道之外的领导者之旅。如果你的目标是这样的角色，我希望这本书能为你的追求提供务实的帮助。

你可以从头到尾阅读这本书，但如果你愿意，请随意跳跃。没有正确或错误的阅读方式。

它的章节包括：

- **概述**- 对专家工程师角色的调查，它在不同公司之间的差异，以及为什么职称很重要
- **[以Staff级别运作](#)**- 如何在职称的另一边完成工作
- **[在你所在的地方获得头衔](#)**- 如何在你的公司获得专家+角色
- **[跳槽以获得头衔](#)**- 何时以及如何更换公司可以支持追求 Staff-plus 头衔
- **故事**- 从 Staff-plus 工程师那里收集的关于他们做什么以及如何达到他们角色的故事
- **[资源](#)**- 如果您正在寻找更多，这是一个模板和进一步阅读的集合

每家公司都对 Staff-plus 角色有自己的看法，因此某些部分可能与您的经验不符。如果是这种情况，请接受引起共鸣的部分，并丢弃其余部分！

资深工程师原型

大多数**职业阶梯**为公司内部的 Staff 工程师定义了一套单一、统一的期望。每个人都受益于明确的角色期望，但职业阶梯是一种更适用于人群而非个人的工具。对于 Staff-plus 工程师来说尤其如此，他们的职业阶梯通常掩盖了隐藏在单一称号背后的几个不同的角色。

我与越多的人谈论他们在公司中 Staff-plus 工程师的角色，他们的经验就越能归类为四种不同的模式。大多数公司强调一种或两种模式，并且只有在拥有数百或数千名工程师的公司中才存在一种模式。少数公司没有任何技术领导模式，并将所有经验丰富的工程师推向工程管理。在文学中，反复出现的角色模式被称为原型，例如“英雄”或“骗子”，原型术语对于标记这些 Staff-plus 工程师的常见变体很有用。

我遇到的 Staff-plus 角色的四种常见原型是：

- 这个**技术主管**指导特定团队的方法和执行。他们与一位经理密切合作，但有时他们与一个专注领域的两到三位经理合作。有些公司也有**技术主管经理**角色，这与**技术主管**原型相似，但存在于工程经理阶梯上，包括人员管理职责。
- 这个**架构师**负责关键领域内的方向、质量和方法。他们结合了对技术约束、用户需求和组织层面领导力的深入了解。
- 这个**问题解决者**深入研究任意复杂的问题并找到合适的解决方案。有些人长期专注于特定领域。另一些人在组织领导的指导下，从一个热点跳到另一个热点。
- 这个**右手**扩展了高管的注意力，借用他们的范围和权力来运作特别复杂的组织。他们为大型组织的领导者提供额外的领导带宽。

这个分类法更侧重于**有用**而不是**完整**，但到目前为止，我能够将我交谈过的每一位 Staff-plus 工程师归入这些类别之一。诚然，有些人比其他人更容易分类。

技术主管

Tech Lead's Weekly Calendar					
	Monday	Tuesday	Wednesday	Thursday	Friday
	Primary On-Call This Week!				
8 AM		Prep proposal for Arch Review			
9 AM					
	Standup		Standup	Sprint Planning	Standup
10 AM	1:1		Interview		
	1:1				
11 AM	1:1		1:1	Incident	
	1:1		1:1	Retrospective	
12 PM	Lunch				
1 PM	Focus Block / Coding		Arch Review	Focus Block / Coding	Interview
2 PM		Interview			
3 PM					
					Sprint Demos
4 PM					
5 PM					
	Child Care				

技术主管原型示例日历

以技术主管原型为特色的故事: [黛安娜·波亚尔](#), [丹·娜](#), [里图·文森特](#)

技术主管是最常见的Staff原型，并在其方法和执行中领导一个团队或一组团队。他们能够轻松地确定复杂任务的范围，协调团队解决这些任务，并在过程中解除他们的障碍。技术主管通常掌握团队的背景信息，并维护团队成功所必需的许多重要的跨团队和跨职能关系。他们是团队产品经理的紧密合作伙伴，也是在路线图需要调整时第一个被联系的人。

在他们职业生涯的早期，他们将实施团队最复杂的技术项目，但此时，他们默认将此类项目委派给整个团队。他们这样做既是为了培养他们的队友，也是为了承认团队的影响力随着技术主管的编码块缩小而增长。虽然他们的编码量减少了，但他们仍然是定义团队技术愿景的人，并在团队内部就复杂问题达成一致。

这个技术主管角色对于许多人来说是他们作为Staff工程师的第一次体验。一些力量促成了这一结果。首先，技术主管角色往往在对团队有强烈概念的公司中早期发展，这在使用敏捷方法的公司中很常见，而

且大多数公司在某个时候都会尝试敏捷方法。另一个因素是技术主管的日常工作与您作为高级工程师已经做的工作最相似，使其成为一个相当直观的过渡。最重要的是，一个组织大约需要一个技术主管每八位工程师中就有一位是这种类型，这使得它比其他原型更为常见。

有点令人困惑的是，一些公司使用技术主管作为头衔，其他人则将其用作角色。在这个原型列表中，技术主管是作为一名高级工程师进行操作的一种方法，但执行以下操作很常见技术主管角色，但没有达到职员级工程师预期的影响力。事实上，您会发现编外工程师的行为符合每个人的行为原型。成为一名高级工程师不仅仅是一个角色。它是角色、你的行为、你的影响以及组织对所有这些事情的认可的交集。

架构师

Architect's Weekly Calendar						
	Monday	Tuesday	Wednesday	Thursday	Friday	
8 AM		Prep proposal for Arch Review			Technical discussion on potential acquisition	
9 AM	Org Status Meeting			Team Meeting		
10 AM	1:1		Interview			
	1:1			1:1		
11 AM	1:1			1:1	Incident Retrospective	
	1:1		1:1			
12 PM	Lunch					
1 PM			Arch Review		Interview	
2 PM	Review and give feedback on proposals for this week's Arch Review	Interview	Share notes from Arch Review	Work with pilot team on integration plan for new storage backend	Interview	
3 PM		Urgent sync on scalability for upcoming enterprise launch				
4 PM						
5PM						
		Child Care				

架构师原型的日历示例

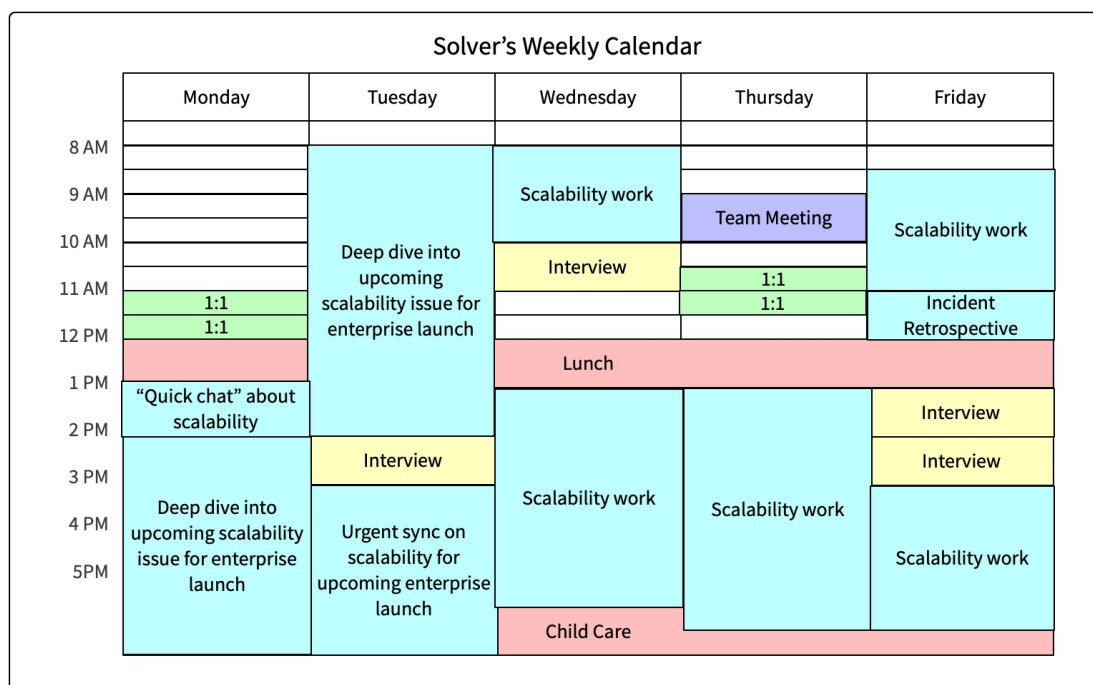
以建筑师原型为特色的故事: [乔伊·艾伯特](#), [凯蒂·赛勒-米勒](#), [基维·麦克明](#)

这个架构师在许多公司中，职称已经过时，但架构师对于在Staff-plus级别工作的人来说，角色仍然存在并且运作良好。建筑师负责公司内特定技术领域的成功，例如，公司的API设计、前端堆栈、存储策略或云基础设施。对于一个值得“架构师”，它必须既复杂又对公司的成功至关重要。

存在一种有害的先入之见，认为建筑师设计系统在隔离，然后将他们的设计传递给其他人来实现。这种情况确实在某些情况下发生，但背诵这种刻板印象会诽谤我采访过的建筑师。有影响力的架构师会投入精力来保持对业务需求、用户目标和相关技术约束的深入理解。他们利用这些洞察力来识别和倡导在其关注领域内的有效方法，并以通过始终如一的良好判断力而获得的组织权威来做到这一点。

这个架构师角色往往会在规模相对较大的公司、代码库异常复杂或耦合的公司，以及正在努力偿还他们在最初冲刺以适应产品市场的过程中所产生的技术债务的公司中演变。有些公司会推动致建筑师仍然深藏在代码库中，另一些则明确期望建筑师一定不能编写代码：这两种模式都适用于某些公司。

问题解决者



Solver 原型示例日历

以 *Solver* 原型为特色的故事: [伯特·范, 纳尔逊·埃尔哈格](#)

这个**问题解决者**是组织中值得信赖的代理人，他们会深入研究棘手的问题，并持续解决这些问题。这个角色的人会被调去解决组织领导层认为至关重要的问题，这些问题要么缺乏明确的方法，要么执行风险很高。

大多数 Staff 级别的角色都需要大量的组织协调，而**问题解决者**通常处理已经被确定为组织优先事项的问题，因此只需要做相对较少的组织层面的调整。另一方面，他们通常在问题得到控制后就停止处理，这可能会产生短暂的感觉，需要巧妙地处理，以避免激怒那些留下维护“已解决”问题的团队。

这个**问题解决者**在那些认为[个人，而不是团队，作为计划和所有权的基本单位](#)。在这样的公司里，常见的是“**问题解决者**”变得比“**技术主管**”更普遍。在传统管理的以冲刺为中心的公司中，你不太可能遇到这个角色，除非这些公司变得足够大或足够长寿，以至于积累了自己的技术债务。

右手

Right Hand's Weekly Calendar						
	Monday	Tuesday	Wednesday	Thursday	Friday	
8 AM					Technical discussion on potential acquisition	
	Prep for Status	Deep dive into upcoming scalability issue for enterprise launch				
9 AM	Org Status Meeting		Onboarding Working Group	Team Meeting		
10 AM	1:1		Interview			
	1:1			1:1		
11 AM	1:1			1:1	Incident Retrospective	
	1:1		1:1			
12 PM	Lunch					
1 PM	“Quick chat” about scalability		Arch Review	Quarterly Planning Offsite	Interview	
2 PM		Interview			Interview	
3 PM		Urgent sync on scalability for upcoming enterprise launch	Eng Budget Review		Scalability Status	
4 PM						
						Send Out Scalability Status Update
5PM	Scalability Status		Scalability Status			
		Child Care				

右手原型示例日历

以右手原型为特色的故事: [米歇尔·布](#), [里克·布恩](#)

这个右手是最不常见的原型，只有当一个组织达到数百名工程师时才会出现，类似于在没有直接管理责任的情况下担任高级组织领导者。[里克·布恩](#)将他的角色比作“[国王之手](#)”在《权力的游戏》中和[里奥·麦加里](#)来自《白宫风云》，以高级领导借来的权力运作。然而，借用权力伴随着[保持深度一致](#)与那位领导的方法、信念和价值观保持一致的义务。

担任此角色的人员会参加其领导的员工会议，并通过消除领导的重要问题来扩大其影响力。在此级别解决的问题绝不是纯粹的技术问题，而是涉及业务、技术、人员、文化和流程的交叉点。左右手经常冲入火场，修改方法，将执行权委托给最合适的团队，然后跳到组织中其他地方的下一个火场。这些角色的乐趣在于你只处理重要问题。悲剧在于，当这些问题得到解决时，你总是在处理下一个问题。

哪个适合你？

当您考虑这些原型中哪一个适合您时，首先要反思哪些类型的工作能让您充满活力，然后考虑您的公司有哪些职位空缺。

所有公司都需要能够胜任的工程师技术主管角色，这使得它成为获得您的第一个 Staff 工程角色最容易实现的原型。强调个人所有权而非团队所有权的公司通常会尽早发展问题解决者。另一方面，严格按照冲刺或敏捷方法运营的公司往往会较晚发展该角色，甚至永远不会发展。在最近涌现的快速增长的科技公司中，架构师和右手角色通常在组织分别达到一百名和一千名工程师时出现，并且在此之前根本不存在。具有其他文化基因的公司通常会更早地发展它们，或者有时永远不会发展。

在这些角色中取得成功需要保持投入；了解哪些类型的工作能让你充满活力至关重要。技术主管和架构师往往多年来与同一批人一起解决相同的问题，从而培养出紧密的团队意识和共同目标。有些月份，他们的重点将是公司的首要任务，有时他们会进展顺利，以至于高管们忘记了他们的团队的存在。

这个问题解决者和右手从一个火灾现场跳到另一个火灾现场，通常与他们每周合作的人进行更多的事务性互动。他们与高管的优先事项紧密结合，并可能因解决领导层最紧迫的问题而获得认可。另一方面，虽然他们名义上会与其他同事组成一个团队，但他们的团队关注领域通常几乎没有重叠，而且他们的社区意识往往有限。

对于每种原型，你都会发现有人喜欢它并觉得它非常有价值，同时也有人觉得这项工作令人绝望。虽然朝着适合你的原型努力很重要，但同样值得记住的是，在你三十岁或[四十年职业生涯](#)，您将有足够的时间来体验每一种原型。

资深工程师实际上做什么？

Staff-plus 工程师的角色很大程度上取决于团队的需求以及特定工程师的优势。根据我的经验，Staff-plus 工程师的职责会随着时间的推移而变化。不过，通常情况下，他们的主要重点是从事对公司具有战略价值的项目/工作，同时推动技术设计并提升团队水平。 -[黛安娜·波亚尔](#)

任何在聚会上被亲戚逼问解释软件工程师实际上做什么的人都知道，解释这项工作可能具有挑战性。随着时间的推移，您可能已经为您的亲戚创建了一个引人注目的答案，但是当他们的同事凑过来问：“Staff 工程师是做什么的？”时，许多人的大脑一片空白。

最直接的答案是，Staff 工程师会继续做很多让他们作为高级工程师获得成功的事情：建立关系、编写软件、协调项目。但是，这是一个具有误导性的答案。Staff 工程师确实会做这些相同的任务，但是以前这些任务是他们工作的核心，而现在它们是辅助任务。他们的日常安排会因[按原型](#)而略有不同，但是所有原型都有一个共同的基础：设定和编辑技术方向、提供赞助和指导、将工程背景注入到组织决策中、探索以及[Tanya Reilly](#)称之为[作为粘合剂](#)。

设定技术方向

当我可以促进一个领域的技术愿景的设定，并让人们朝着这个愿景前进时，我感觉最有影响力。我想我们都会同意，我们希望我们的代码比现在更好，或者在某些方面有所改进架构方面。然而，我发现人们常常对想要变得更好有一种模糊的感觉，但并不清楚他们想要的东西是什么。我喜欢帮助团队就他们试图达到的目标达成共识（即使我们永远无法达到目标也没关系），并提出一个实现目标的大致计划。 -[乔伊·艾伯特](#)

就像[《老雷斯的故事》](#)在他的热门儿童读物中为树木代言一样，高级工程师为他们公司的技术代言。技术不能为自己说话，需要有效的倡

导者。成功推进技术的人是务实的、深思熟虑的，并且更关注进步的长期趋势，而不是将每个单独的决定视为成败攸关的危机。可以将其视为技术方面的兼职产品经理。

一些高级工程师被明确聘用来领导一个特定领域，例如[API设计](#)，在其他情况下，他们发现自己[在广泛领域内编辑和调整方法](#)。在所有角色中，一个不变的真理是，设定技术方向的现实更多的是理解和解决你周围组织的实际需求，而不是优先考虑你个人热衷于学习的技术和方法。在早期的角色中，你可能试图影响决策，使其倾向于你所感兴趣的技术选择；但在高级职位上，你首先要对企业和组织负责，其次才是你自己。

指导和赞助

在我目前的职位上，当我看到我赞助的人发布他们已经完成工作的公告，或者当我看到我帮助塑造或改变了一个工程团队的模型时，我感到精力充沛关于一个重要的主题。是这些团队，而不是我，在日复一日地努力构建和支持他们的技术。我根据他们的进展来衡量我的影响，更重要的是，根据进展的方向以及他们的工作与公司目标的对齐程度。-[米歇尔·布](#)

有一种流行的英雄式领导的愿景，它以极富成效的个人为中心，他们的决定改变了公司的未来。这些叙述大多是由公关团队故意设计的，目的是创造一个好故事。通过培养你周围的工程师，你更有可能改变公司的长期发展轨迹，而不是通过个人英雄主义。培养你周围的人的最好方法是建立积极的指导和赞助实践。

有时人们在他们的职业阶梯中看到了对指导的要求，并试图机械地勾选那个框，这很可惜，因为指导是Staff-plus角色中最有价值的活动之一。分享你的经验和建议，并建立持续的关系以了解接受者的背景，是一项具有高度影响力的工作。最有效的Staff工程师会将适度的指导与更多的赞助结合起来：直接在天平上施加影响，以帮助提升和支持

你周围的人。如果你还没有读过，Lara Hogan已经写了一篇关于赞助和指导之间区别的经典文章，[赞助是什么样的？](#)

提供工程视角

我在高于单个项目和团队的更高级别的工程讨论中占有一席之地。我们有定期的staff工程会议，我们在会上讨论跨团队的问题，这些问题本质上既有技术性的，也有非技术性的。-[丹·娜](#)

高效的组织简化日常决策。一个很好的例子是审查潜在企业客户合同的过程。早期，会有一些合同签署后，产品和工程团队不支持。在发生几次之后，该过程将在审查步骤中包括更多的利益相关者，并且随着时间的推移，合适的人将在合适的时间出现在合适的地点。

即使是擅长做出日常决策的公司，在出现意外决策时也常常会遇到困难。这种决策既有时效性又很重要，甚至在做出决策之前，将合适的人员聚集在一起都具有挑战性。这种情况经常发生[组织结构重组](#)发生时，没有有价值的意见可以改变结果。同样，对于不常出现的职位（例如，在早期公司中，您每年可能会雇用一名高管或Staff-plus工程师），面试环节通常不会在重要的维度上评估候选人。对于某些公司而言，甚至像路线图计划之类的事情也属于这一类。

Staff-plus工程师通常是那些意外地被拉到[房间里](#)这种决策正在发生的地方。这使他们有机会在仍然可以改变结果的情况下，将工程背景和观点注入到决策中。这些对关键决策的短暂投入具有不应有的影响力，并且可以让您注入工程视角，否则这些视角将被忽略。请记住，您代表的是整个工程部门的利益，而不仅仅是你自己的利益。

探索

在我目前在孵化器中的角色中，我整天都在进行原型设计，但在我之前的技术主管角色中，我做了很多不同的事情。-[里图·文森特](#)

[爬山](#)是一个简单的优化算法。想象一下，你站在某座山上，想要到达山顶。你转一圈，找到附近最高的点，然后走到那里。到达那里后，你再次转一圈，从新的位置找到附近最高的点，然后去那里。如果你一直这样做，你就会到达你所在山的山顶。然而，想象一下你在雾天尝试这样做。因为你看不了很远，你可能会到达附近最高的点，然后才意识到有一个更高的点就在视线之外。

爬山法并不能解决所有问题，但它非常有效，以至于许多公司都在努力采取其他方法。这可能是一家以消费者为导向的公司，难以支持企业交易，或者是一家成熟的公司，难以与规模较小的竞争对手的发布节奏竞争。甚至可能出现这样的情况：你目前的业务非常有价值，[很难优先考虑新业务](#)，即使有价值的业务的增长率正在下降。

从长远来看，公司要么学会探索，要么逐渐消失；这不是一个可以忽视的挑战。简单地分配一个已经掌握了爬山法的团队[去做探索性的工作](#)远非万无一失，所以许多公司采取了不同的方法。他们找到几个具有广泛技能的值得信赖的人，分配一些资源，并在几个月后回来看看他们发现了什么。其中一位工程师通常是 Staff 工程师。

这也不一定总是业务问题；它可能是公司系统难以解决的任何模棱两可的重要问题。它可能是将你的基础设施成本降低一个数量级。它可能是确定一个需要六个月而不是三年的多区域战略。它可能是解决突然意识到你的主数据库只剩下三个月的磁盘空间，而且你无法升级到更大的容量（根据我的经验，在快速增长的初创公司中，这是一个非常常见的问题）。

这是公司所做的最有价值和风险最高的工作之一。要被信任从事这项工作，需要大量的组织信任，包括获得业务部门足够的尊重，这样如果你失败了，那反映的是问题，而不是你。

作为粘合剂

[Tanya Reilly](#)写了一篇精彩的文章，[作为粘合剂](#)，它抓住了成功的高级工程师的另一个核心要素：做那些需要的，但往往是看不见的工作，

以保持团队前进并交付其工作。这并不光鲜，但高影响力的组织通常会有一名或多名高级工程师在幕后工作，加快[最重要的工作并确保它完成](#)。

但你还会写软件吗？

在结束对高级工程师角色的任何讨论时，如果不谈谈高级工程师们聚集在一起时问的第一个问题：“你还有时间写软件吗？”那是不礼貌的。答案当然是，这取决于情况！

[拉斯·卡萨·威廉姆斯](#)说：“我仍然定期贡献代码——当然比我团队中的其他工程师少；但重要的是，我坚持“手到键盘”的工作，以确保我的技术战略（和其他宏观层面的决策）是由我团队其他成员的实际经验所告知的。”

[凯蒂·赛勒-米勒](#)说：“我是一名前端架构师，但到目前为止，我最近一直在写的主要内容是SQL，因为我正在做大量的数据分析。我一直在查看我们的性能指标，以找出需要改进的领域，以及修复哪些问题对提高性能和业务指标最有影响。我会在这里和那里写一些JS或PHP，但主要是为了帮助解除团队的阻塞或运行与性能相关的小型实验。”

[乔伊·艾伯特](#)说：“你的职位越高，你的工作就越与代码无关。当然，与人事经理不同，你仍然有非常技术性的倾向，甚至通过负责人，你可能至少会做一些编码。然而，你的职位越高，你的工作就越是关于指导和培养你周围的人（以及更广泛的人），通过建立你公司的公共技术品牌来建立你的团队，注意到可以改进或纠正的更大的技术趋势，帮助为你的团队或公司设定技术愿景，并倡导为技术债务项目提供资源。”

大多数人写一些，有些人不写，但没有人写得像他们职业生涯早期那样多。偶尔会有一周完全是编码，但这些不会是常态，如果它们发生得太频繁，通常是工作的一个迹象[专注于舒适而非重要的事情](#)即使你不写很多东西，你也会阅读给你审查你同事的代码，并进行相当数量的代码审查。

缓慢但有回报

Staff-plus 工作的一个共同主题是时间框架更长。在你的职业生涯早期，很容易迷恋上软件开发快速的反馈周期——编写、测试、发布、重复——而你在这个级别所做的大部分工作都将这种反馈循环替换为需要数周、数月甚至数年的循环。当你第一次承担 Staff-plus 角色时，这些较长的时间框架会让人感到非常沮丧。这是正常的以 Staff-plus 工程师的身份结束某些日子，感觉自己一事无成——坚持下去！

影响和个人成长存在于更长的时间框架内，虽然我交谈过的每个人都希望他们偶尔能有更多的时间来编写代码，并且承认有些日子担心他们没有完成太多工作，但他们都不后悔过渡到目前的角色。

标题重要吗？

如果你安全地依偎在高级工程师舒适的怀抱中[职业级别](#)，你可能会想，你是否应该追求 Staff 这个头衔。这需要投入大量的时间和精力，还需要相当多的运气。这项投资值得你付出时间吗？

答案当然是，有可能！通常情况下，Staff-plus 职称带来的三个一致的优势是：

1. 允许你绕过非正式的资历衡量标准，
2. 方便进入“房间”，
3. 增加当前和职业薪酬。

第四个潜在的优势是，有些人发现头衔赋予了他们更多的自主权来选择他们从事的项目，但另一些人发现，自主权的增加被对业务相应的责任增加所抵消。

非正式的资历衡量标准

当我和[纳尔逊·埃尔哈格](#)谈到达到Staff级别是否允许他承担新的工作时，他回答说：

“允许”这个问题很有趣，可能不是完全正确的问题，因为关于谁获得什么类型的角色，几乎没有任何官方政策。大多数事情都依赖于更非正式的资历衡量标准。

许多科技公司将自己描述为追求精英制度，定义为创造条件让有才华的员工自然而然地升到顶端。鉴于没有任何被广泛接受的个人价值衡量标准，这些公司开始依赖Nelson恰当地称之为“非正式资历衡量标准”的东西。虽然这些衡量标准被认为可以客观地评估想法，但它们纯粹的非正式性成为一种广泛的偏见载体，并且经常将信心与能力混为一谈。

从不断重新确立自己能力的状态中解脱出来，是Staff职称的一个关键优势，这一点经常被提及。虽然并非我交谈过的每一位Staff及以上级别的工程师都提到这些非正式的衡量标准，但那些不符合公司对资深技术专家刻板印象的人却经常提到。

[基维·麦克明](#)分享了，

当你拥有一个头衔时，你就不必花费那么多精力来展示你的资历。它有助于为他人设定背景。从一开始你就更受尊重，这一点非常明显。

Staff-plus 的头衔允许你将之前花费在证明自己上的精力重新投入到你所评估的核心工作中。如果你发现自己没有投入太多精力来证明自己，那就太好了！也许你在现在的公司工作了足够长的时间，并且已经多次证明了自己，以至于这不再是一个问题。如果你做发现你的时间被转移到证明和重复证明自己上，那么这个头衔将为你节省大量时间，供你重新投入。

在房间里

Staff-plus 头衔的另一个常见优势是“在房间里”。[丹·娜](#)描述为，

我在高于个人项目和团队的更高级别的工程讨论中占有一席之地。我们有定期的员工工程会议，在会上我们讨论跨团队的问题，这些问题既有技术性的，也有非技术性的。作为一个假设的例子，在这种类型的会议中，我会很乐意提出我认为工程入职流程中的缺点。

对于任何重要的决定，都有做出核心决定之前的准备时间，以及之后的一切。在更高级的职位上，你通常可以在相对容易整合的时候提供意见，否则你的反馈可能不会被采纳——尽管它非常有价值——因为相关的推广或实施已经进展得太远了。

薪酬

小公司的薪酬往往相当随意，加薪来自与你的经理直接谈判。在这样一家公司晋升到 Staff-plus 职位甚至可能不会带来相应的薪酬增长。然而，大多数公司在达到一到两百人时，会为每个职位引入薪酬范围。这些薪酬范围通常会确保你的薪酬随着职位的晋升而增加。

任何公司薪酬最高的职位往往是执行和高级管理职位。随着公司的发展，他们通常会在管理和工程职位之间建立薪酬映射，这样达到 Staff-plus 职位（有时是 Sr Staff 或 Distinguished 职位，而不是最初的 Staff 职位）将显著提高你的薪酬。

即使你目前的公司对 Staff-plus 工程师职位的薪酬与高级工程师职位的薪酬没有太大区别，[有些公司会这样做](#)。在你的整个职业生涯中，你可以选择朝着这样的公司发展，并且拥有 Staff-plus 的头衔将有意义地增加你一生的收入。

接触有趣的工作

许多人担任 Staff-plus 职位，认为这将使他们能够接触到最引人注目或最令人兴奋的工作。在某种程度上这是真的，但这取决于[Staff 原型](#)在贵公司最普遍的情况。例如，[解决者](#)通常可以接触到最有趣的工作。相反，一个[技术主管](#)如果他们那样做，可能会破坏他们的团队。

在我交谈过的人中，获得有趣工作最有效的方法是一开始就被雇佣去做，例如[里图·文森特](#)他被聘用启动Dropbox的产品孵化器，以及[基维·麦克明](#)他被聘用设计Fastly的API策略。

这并不总是奏效。有时，有趣的工作会清晰可见，但仍然无法获得。你将过于受业务需求的约束，无法出于个人兴趣追求某个项目。在早期的职位中，你或许可以将这类项目偷偷塞进你的待办事项中，但现在你有责任树立良好的行为榜样。即使在项目对公司来说是最好的情况下，你通常也会决定将这个机会传递给另一位工程师，因为他们会比你从中受益更多。

不同，而不是更好

即使头衔很重要，但你不一定非要追求这个角色。即使你喜欢Staff-plus头衔的特权和福利，重要的是要认识到它们是建立在一个非常不同的工作之上的。[米歇尔·布](#)在给追求Staff头衔的人的建议中捕捉到了这一点，

如果你更专注于获得 Staff 职位，而不是专注于做能让你充满活力的事情，很容易最终陷入一个你不想要的职位。成为一名 Staff-plus 工程师，尤其是一名广义的 Staff-plus 工程师，与成为一名高级工程师是非常不同的工作。重要的是退一步，思考一下这是否真的是你想要的工作。

高级职称的优势是真实存在的，对于某些人来说，这些优势将他们的职业生涯从以生存为特征转变为具备成功必要先决条件的状态。然而，许多人发现他们的 Staff 职位提高的期望消除了过去让他们兴奋的工作。在你的职业生涯中，很少有选择是没有后果的，而这并不是其中之一。

物质的，但不是魔法

你偶尔会遇到一位工程师，他认为获得某个职称是他们与一项重要的成就或机会之间唯一的障碍。这些人可能会表达沮丧，例如，“如果我拥有 Staff 职称，我就可以决定我们团队的技术栈。”

提高的组织权威确实为解决问题提供了新的工具，但在管理良好的组织中成功地保持组织权威需要大量的细微差别和克制。如果你有一个问题，并且认为你的职称是唯一阻碍你的因素，我想向你保证，专注于发展你的方法和技能将比职称更有影响力。一旦你接近边缘，职称会让你越过边缘，但它永远不会像你期望的那样发挥作用。

这条规则唯一不变的例外是，女性和少数族裔通常确实发现，一旦他们获得 Staff-plus 职称，他们花费的时间和精力会大大减少，以证明

自己。职称并没有为他们解锁新的能力，但它确实减轻了他们整个职业生涯中一直背负的一些负担。

以Staff级别运作



有人给我的最好的建议之一，也是我确保传递给其他 staff 工程师的建议是，存在一种误解，认为你成为一名 Staff 工程师，然后你就可以控制你所做的工作，每个人都会听你的，并按照你想要的方式去做。这与实际发生的情况完全相反！ - [凯蒂·赛勒-米勒](#)

许多工程师专注于 Staff-plus 职业道路，因为工程经理的道路有太多的会议，或者需要与其他同事进行太多的协作，而且，如果你以这种心态开始 Staff-plus，你会感到惊讶的。虽然 Staff 工程师的角色通常被定位为高级工程师之后的连续步骤，但它确实是一个不同的角色，你将越来越多地花时间做你以前很少做或根本不做的的工作。

在 Staff-plus 角色中存在一个显著的学习曲线，最初会让大多数人感到困惑。部分挑战在于，你所做的许多工作都有一个慢得多的反馈周期。当您取代发自内心的编码时，延迟的反馈最初会让人感到非常沮丧。[REPL](#)在指导、关系建立和策略方面进展不平衡。

本章讲述的是如何克服学习曲线，学习如何以 Staff 工程师的身份运作，以及找到这个角色中个人感到满足和组织变革的部分。

主题

在本书的访谈中，以及我领导和指导 Staff-plus 工程师的经验中，有一些主题不断涌现，成为个人发展的基石。它们不是一切你将在角色中做什么，但它们是你能最有可能产生巨大影响或意外做出限制职业发展的举动的地方。

1. **做重要的事情**为了充分利用你的工作时间，特别是随着你职业生涯的深入和生活责任的扩大。
2. **撰写工程策略**指导您的组织如何通过其架构、技术选择和组织结构来支持公司的业务目标。
3. **维护技术质量**随着公司的发展和时间的推移，保持公司架构和软件的质量。
4. **与权威保持一致**随着时间的推移，保持有效的领导。技术领导角色依赖于来自另一位（通常是管理层）领导的代理授权，而持续获得该授权取决于保持一致、值得信赖和可预测。
5. **要领导，先跟随**对事物应该如何运作有一个生动的认识是一种强大的领导工具，但学习将你的愿景与你的同伴和领导的愿景融合起来也至关重要。
6. **学会永远不出错**从追求正确转向理解和沟通。停止花费你的社会资本来修复因冲突而破裂的关系，并学会与那些具有不同优先事项和观点的人合作。这还带来了一个额外的好处，那就是向你的经理抱怨你的人会减少。
7. **为他人创造空间**以便你的团队比你的贡献更强大。
8. **建立同行网络**审查困难的决定，并在你的角色的权威开始缓和反馈时给你诚实的反馈。

一位精明的读者会注意到其中讨论的两个关键主题**资深工程师实际上做什么？**此主题列表中缺少以下内容：第一个是“指导和赞助”，第二个是“成为粘合剂”。这两个概念对于 Staff-plus 工程师的成功至关重要，但最终，我认为关于这些主题的规范文章已经存在，阅读它们比阅读我淡化的重述更有帮助。对于指导和赞助，花一些时间阅读 Lara Hogan 的**赞助是什么样的？**，并作为粘合剂，花时间阅读Tanya Reilly 的文章，该文章使用了短语“**作为粘合剂**”。

当你在这些领域有意识地进行实践时，你将从一个新晋的Staff工程师慢慢进步为值得信赖的组织领导者。也就是说，这些并不能涵盖你所做的一切。有时你会发现你的角色与工程总监的角色惊人地相似，有时又会奇怪地熟悉你职业生涯中以前的工作。

这种广泛的职权范围是描述这些角色具有挑战性的部分原因。如果你特别关注某个缺失的主题，请查看[学习的其他资源](#)附录。

做重要的事情

我开始使用“充满活力”而不是“有影响力”。“有影响力”感觉是以公司为中心的，虽然这很重要，但“充满活力”更注重内在。找到充满活力的工作是我在Stripe工作这么久的原因，追求有影响力的工作。-[米歇尔·布](#)

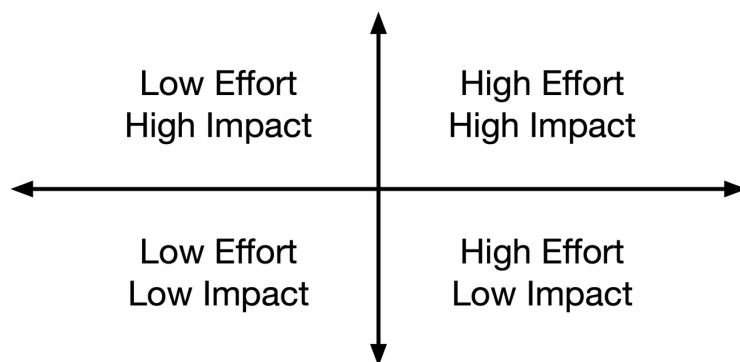
我们所有人的生命时间都是有限的，在生命的倒计时中，我们把一部分时间投入到工作中。即使对于那些最专注于事业的人来说，你的生活也会充满工作之外的许多事情：支持你的家庭、孩子、锻炼、成为导师和学员、爱好等等。这是一个丰富生活的标志，但一个副作用是，随着你职业生涯的深入，你工作的时间会变得越来越少。

如果你在职业生涯中不断进步，那么即使你可用于工作的时间减少，对你影响力的期望也会不断增长。你可以尝试少睡觉或剥夺自己感受完整所需的非工作活动，但你最终会发现，你的工作对你的牺牲保持着冷漠的态度，而不是奖励它。只有通过[根据你的生活调整你的职业节奏](#)你能长期维持自己吗？

的确，调整自己的节奏成为了一个持续、成功的职业生涯的核心挑战：越来越高级的职位要求你完成越来越多的事情，并且用越来越少的时间完成。这两个约束条件之间的边缘会随着你的晋升而变得越来越窄，但如果你采取一种深思熟虑的方法，它仍然是可以行走的。

首先，讨论一下几种常见的绊脚石：*吃零食*，*梳妆打扮*，和*追逐幽灵*。然后我们将进入正题：如何做你致力于真正重要的事情？

避免吃零食



高影响和低影响象限，按高努力和低努力划分

Hunter Walk 建议大家[避免在安排工作优先级时“吃零食”](#)如果你在一个运转良好的组织中，在某个时候，你会发现既有高影响力又容易的事情已经做完了。这让你需要在直接转向困难且高影响力的事情，或者转向容易且低影响力的事情之间做出选择。后者——容易且低影响力——就是 Walk 所说的吃零食。

当你很忙的时候，这些零食会给你一种成就感，使它们在心理上得到回报。不过，你不太可能从做这些事情中学到很多东西，其他人可能也同样有能力完成它们（并且对于其中的一些事情来说，这可能是一个很好的发展机会），而且与做一些影响更大的事情相比，存在着巨大的机会成本。

花一些时间在零食上，让自己在更大的成就之间保持动力是可以的，但你必须对自己花在高影响力工作和低影响力工作上的时间保持诚实。在高级职位上，你更有可能自己决定你的工作，如果你没有刻意跟踪你的工作，很容易发现自己几乎没有做高影响力的事情。

停止梳理羽毛

如果说“吃零食”是做容易且低影响力工作的广泛类别，那么有一种特别诱人的吃零食子集，我称之为“梳理羽毛”。梳理羽毛是做低影响

力、高可见性的工作。许多公司将高可见性和高影响力混为一谈，以至于他们无法区分梳理羽毛和影响力，这就是为什么在一些公司的高级工程师花费大部分时间做价值可疑但经常在公司会议上得到认可的工作并不罕见。

如果你对[职业发展](#)持短期看法，那么在评估影响力时，针对你当前组织的病态进行优化是最佳途径：前进并光荣地梳洗打扮。但是，如果你正在考虑发展自己，以便在你的[当前角色在复杂性方面不断增长](#)或跨多个组织取得成功，那么在有价值的工作和自我成长之间取得平衡更为重要。

这也是选择一家公司工作时要考虑的重要因素！深入了解一家公司的价值观，并确保它与你预期的个人成长相一致。如果一家公司的领导层完全由那些将精力集中在表演性的紧迫性或效忠行为上的人组成，那么当你在公司中的成功取决于这些活动时，不要感到惊讶。

更糟糕的是，要成为一个成功的梳洗者，需要对你实际影响的批评几乎刀枪不入，而且如果你的精力被转移到梳洗上，你的真正工作将会受到影响。通常，这意味着你需要成为高级领导的虚荣心雇佣，或者以公司认为领导者看起来和行动的方式来展示自己。如果那不是你，那么你试图用你的良好判断力换取公司成功的尝试最终会失败：你将因缺乏真正的影响力而受到追究，而其他符合公司对领导者外貌期望的人将以某种方式向上滑动。

停止追逐幽灵

很多人会认为，公司作为理性的优化者，会避免在低影响、高投入的项目上花费大量时间。不幸的是，情况并非总是如此。一位新的高级领导加入公司并立即推动某事，这种情况非常普遍[一种从根本上误解了当前挑战的战略转变](#)他们之前处境的阴影如此牢固地掌握着他们对新公司的理解，以至于他们错误地将熟悉的事物判断为本质。

作为一名高级领导者，你必须控制住自己的自负，以避免大规模地投入到毫无意义的工作中。当你在招聘过程中，不断地被告知你被聘用

来解决一些严重的问题时，这可能会出人意料地具有挑战性——你是新聘请的救世主。当然，你的直觉是对的！在改变现状之前花时间去了解它，总是会以结果回报你的努力。

我最近与某人进行了一次讨论，他认为新的高级领导者故意地即使他们怀疑这些努力会失败，也要推动重大变革。这样的改变使组织越来越依赖新的领导者，并确保任何事情做如果一切顺利，功劳会直接归于新领导，而不是他们的团队。如果你是这种领导方式，请明白你很糟糕，花时间提升自己，直到整个公司的福祉和成功对你来说比被认为至关重要更重要。

存在问题

既然你已经吃完零食、打扮完毕、追逐完幻影，现在是时候从另一个方向思考了：你应该做什么？寻找有意义的工作的第一个地方是探索你的公司是否正在经历生存风险。公司在永恒的[迭代淘汰赛](#)，在未来的成功和生存到未来成为现在之间取得平衡。如果你即将输掉其中一轮，那就始终关注那里。

钱不够用了，[就像我在Digg的经历一样](#)，可能是最明显的问题，但并非所有存在问题都是财务问题，例如 [Twitter的失败鲸鱼稳定性挑战](#)或适应由 Covid-19 大流行引起的转变。

如果你的公司正在发生可怕的事情，那么那里就是你应该参与的地方。如果问题得不到解决，其他一切都无关紧要。

在有空间的地方工作并且注意

存在问题通常不是投入精力最有效的地方，但当四面楚歌时，效率并不是首要考虑因素。你应该涌向存在问题，但如果一个问题不是存在问题，那么你应该怀疑是否要将你的精力投入到每个人都已经关注的地方。人们经常追逐领导的首要任务，但由于有这么多人希望在那里发挥他们的影响力，因此通常很难产生有意义的影响。

相反，最有效的工作场所是那些对你的公司很重要，但仍有足够的空间来实际工作的地方。哪些是未来将变得至关重要的优先事项，你可以在提前做好出色的工作？哪些领域做得还行但可以在你的支持下做得很好？

有时候你会发现一些工作值得关注，但一个组织却无法关注，通常因为其领导层不重视这项工作。在一些公司，这是开发者工具方面的工作。在另一些公司，这是包容性工作。在大多数公司，这是[粘合工作](#)。

几乎总是有大量的空间来做这种无人关注的工作，所以你可以在上面取得快速的初步进展，这感觉像是一个值得投资的好机会。但在某个时候，你会发现这项工作需要支持，而为一个公司天生就会忽视或贬低的工作争取支持是相当具有挑战性的。你早期的胜利会被冷漠和不协调慢慢侵蚀，你最初的影响会被时间的长河所吞噬。

这是否意味着你不应该做包容性工作？不，这不是我希望你从中得出的结论。有时，一个组织不关注的领域非常重要，以至于你会想要倡导它开始关注。教会一家公司重视它不在乎的东西是你所能做的最困难的工作，而且它经常失败，所以你应该尽可能少做，但不能更少。作为一名高级领导者，你有一项超越最大化公司认知影响的道德义务，但重要的是要认识到你所面临的困难，并相应地安排你的努力。

培养成长

一个经常投资不足（例如，有很多工作空间）同时又具有高度杠杆作用的领域是发展你周围的团队。*招聘*通常有很多人参与其中，尤其是在优化[招聘流程](#)方面，但入职、指导和辅导在许多公司完全被忽视，尽管它们至少 [与招聘对公司工程速度的影响一样大](#)。

如果你开始每周花几个小时来培养你周围的团队，很可能这会在你编写的技术规范 and 拉取请求被遗忘很久之后，成为你的遗产。

编辑

令人惊讶的是，许多项目只需一个小小的改变就能成功，只需一个快速的修改就能开启新的机会，或者只需一次对话就能达成共识。我认为进行这些小的改变、快速的修改和简短的对话是[编辑](#)你团队的方法。

凭借你的组织特权、你在公司建立的关系以及你从经验中获得的远见卓识，你通常可以通过投入最小的努力来改变项目的结果，这是你能做的最有价值的工作。

它特别有价值，因为它快速、简单，对你和被你帮助的人都具有高度的激励作用，并且在做得好的时候影响巨大。（此外，如果做得不好，它会非常令人沮丧，所以你的方法很重要！）

完成事情

一种特殊的编辑是帮助完成一个无法自行结束的项目。通常，你会遇到一位职业生涯早期的才华横溢的工程师，他们已经在做这项工作，但无法获得支持或弄清楚如何将他们的项目重新调整为可完成的工作。令人惊讶的是，指导队友如何将项目调整为可完成的项目，然后借用你的特权来推动正确的摩擦点，会将六个月的苦干变成两周的冲刺，并且产生几乎相同的影响。

我们[只有完成项目才能获得价值](#)，让一个项目越过终点线是它从风险转化为杠杆的神奇时刻。花时间完成工作总是值得的花费。

只有你能

最后一类重要的工作是你独一无二能够完成的那种。当然，有些工作你比其他人做得更快或更好，但更重要的是，如果你不做，就根本不会发生的那种工作。

这项工作是你特别擅长的和你真正关心的事物的交集。它可能是[撰写你公司的技术战略](#)人们实际上会遵循，它可能是说服一位优秀的候选

人加入，它可能是改变你的首席执行官关于如何偿还技术债务的想法，它可能是[打造一个有洞察力的API](#)。

无论它是什么，如果你不做就不会发生的事情是你从事重要工作的最大机会，而且随着你职业生涯的深入，这个类别将变得越来越窄、越来越深。

为什么它重要

假设你在职业生涯的第二十年面试一个新职位。面试你的人会理解你对你之前的任何项目或公司的真正影响吗？不，我保证他们不会。相反，你会发现自己被一系列令人惊讶的主观衡量标准所评判：你积累的声望，你拥有的头衔和你工作过的公司，你的幕后声誉，以及你在面试过程中的表现。

你无法逃避主观的面试实践，但你可以通过做有价值的工作来有意识地积累专业知识。事实上，这是你职业生涯中唯一可行的长期赌注：专注于重要的工作，做能发展你的项目，并且朝着重视真正经验的公司前进。

编写工程策略

我有点觉得写关于工程策略的文章很难，因为好的策略相当枯燥，而且写起来也很无聊。而且我认为当人们听到“策略”时，他们会想到“创新”——[卡米尔·富尼埃 \(Camille Fournier\)](#)

很少有公司了解他们的工程策略和愿景。这种不确定性的一个后果是行业普遍认为这些文件很难编写。在某些对话中，你可能会觉得你在谈论一些神秘的东西，但这些只是平凡的文件。现实情况是，好的工程策略是枯燥的，而且更容易写出一个有效的策略，而不是一个糟糕的策略。

要编写工程策略，请编写五个设计文档，并提取出相似之处。这就是你的工程策略。要编写工程愿景，请编写五个工程策略，并预测它们在两年后的影响。这就是你的工程愿景。

如果你无法抗拒在过程中加入你最精彩的想法的冲动，那么你可以将它们包含在你的预备工作中。把你所有最好的想法写在一份巨大的文档中，删除它，并且永远不要再提及它们。现在这些想法已经从你的脑海中消失了，你的头脑已经为接下来的工作做好了准备。

持久有用的工程策略和愿景是迭代的、自下而上的组织学习的成果。因此，所有的学习都有助于你组织的策略和愿景，但你的贡献不必如此抽象。即使你没有直接负责这项工作，也有一些实际的步骤可以采取你可以采取措施来推进你组织的战略和愿景，从现在开始。

何时以及为什么

在深入探讨创建有效战略和愿景的方法之前，一个好的起始问题是：“我应该在何时以及为什么实际创建它们？”战略是主动调整的工具，使团队能够快速且自信地行动。战略使每个人——不仅仅是少数被授权的人——能够做出快速、自信的决策，否则可能需要花费他们一周的讨论时间。战略也是缩小你众多可能未来的砖块，使其有可能写出

一个现实的愿景。如果你意识到你已经重复讨论了三四次相同的问题，那么是时候写一个战略了。当未来过于模糊，无法确定值得进行的投资时，是时候写另一个愿景了。如果这些听起来都不像是熟悉的问题——现在就去其他工作，稍后再回来。

编写五个设计文档

设计文档描述了你在特定项目中做出的决策和权衡。你的公司可能会称它们为 RFC 或技术规范。也会出现更奇怪的名字；Uber 令人困惑地称它们为 DUCKS，直到后来他们[标准化为 RFC](#)。一个好的设计文档描述了一个具体的问题，调查了可能的解决方案，并解释了所选方法的细节。有很多格式可供选择；一些可以开始思考的地方是[设计文档](#)、[Markdown](#) 和 [Git](#)，[Google 的设计文档](#)，和[远程文化中的技术决策和协调](#)。

一个给定的项目是否需要设计文档取决于个人判断，但我发现了一些有用的规则。对于任何其功能将被众多未来项目使用的项目，您都应该编写设计文档。对于任何对您的用户产生有意义影响的项目，您也应该编写设计文档。对于任何需要超过一个月工程时间的工作，您都应该编写设计文档。

一批五个设计文档是编写有效策略的理想成分，因为设计文档具有糟糕策略所缺乏的东西：基于现实的详细细节。同一团队中两个善意的工程师很容易以不同的方式解释一个抽象的策略，但是当您实施一个特定的解决方案时，很难保持一致。

在您写作时的一些建议：

- **从问题开始。**问题陈述越清晰，解决方案就越明显。如果解决方案不明显，请花更多时间澄清问题。如果您在阐明问题时遇到困难，请向五个人展示您所拥有的东西，并询问他们缺少什么：新鲜的眼睛总是能看到真相。
- **保持模板简单。**大多数公司都有一个设计文档模板，这是一个很好的遵循模式。但是，这些模板通常会扩展以服务于太多的目

标。过度加载的模板首先会阻止人们编写设计文档。首选最小的设计文档模板，该模板允许作者选择最有用的部分，并且仅对风险最高的项目坚持详尽的细节。

- **一起收集和审查，独自写作。**您个人不太可能拥有所有相关的背景信息，以编写关于给定主题的最佳设计文档。在深入研究该过程之前，请从具有相关观点的人那里收集意见，特别是那些将依赖于您的设计文档输出的人。但是，要怀疑将这种协作过程带入编写设计文档本身。大多数人都是比编辑更好的作家。这意味着通常更难将一个集体文档编辑成清晰的文字，而不是确定一个作者来编写一个清晰的文档。广泛收集观点，但独自写作。只是要小心不要爱上你所写的东西，直到之后你已经和别人一起审查过了。
- **宁要好，不要完美。**写一份好的文档并让别人看到，比为了稍微好一点而推迟要好。在对别人的设计提供反馈时，这一点尤其重要；很容易陷入期望他们的设计和你最好的设计一样好的陷阱。特别是当你变得更资深时，强迫每个设计都达到你自己的最佳作品的标准是有害的。专注于推动设计变得更好，而不是固执地认为你自己的最佳作品是相关的质量标准。

写出优秀的设计文档需要大量的练习。如果你想改进你的设计文档，我最好的建议是重新阅读你的设计之后你已经完成了它们的实现，并研究你的实现偏离计划的地方——是什么导致了这些偏差？哦，当然，继续写更多的设计文档。

将这五个设计文档综合成一个策略

在你的组织编写了五个设计文档后，坐下来一起阅读它们。寻找在多个设计中出现的有争议的决定，特别是那些难以达成一致的決定。我最近的一个例子是陷入关于 Redis 是否适合作为持久存储还是仅作为缓存的争论。与其在每个设计文档评审中从零开始，如果我们回顾一下我们最近关于使用 Redis 的决定，反思我们是如何做出这些决定的，并将它们写成一个策略，岂不是更容易？

好的策略指导权衡，并解释该指导背后的理由。糟糕的策略在没有解释的情况下陈述一项政策，这使它们与制定政策的背景脱钩。没有背景，你的策略很快变得难以理解——他们为什么决定这样做？——并且随着底层背景的变化而难以适应。在思考如何编写自己的策略时，可以阅读以下几个有趣的策略[负责任的创新框架](#)和[Slack 中大型技术变革是如何发生的](#)。

如果你是[好战略](#)，[坏战略](#)转换——这本书完全改变了我对战略的看法——那么你会注意到，这个战略的定义是“诊断”和“指导政策”部分，将“连贯行动”推迟到设计文档中。

关于撰写战略文件，我最好的建议是：

- **从你所在的地方开始。**在制定战略时，我们很容易被工作中固有的巨大模糊性所麻痹，但你只需要投入进去并开始写作。等待缺失的信息是行不通的：每个缺失的文档都有其充分的理由。你写的任何东西都需要改变，如果你写了一些特别糟糕的东西，你会很快意识到需要改变它。你现在所处的位置永远是最好的起点。
- **写出具体内容。**一直写到你开始概括，然后停止写作。如果你不能具体说明，那就等到你写了更多的设计文档。具体的陈述会产生一致性；一般的陈述会产生一致性的错觉。
- **要有主见。**好的战略是有主见的。如果他们没有主见，那么他们就不会为决策提供任何清晰度。然而，仅仅有主见是不够的。你还需要展示你的工作。
- **展示你的作品。**在成长过程中的数学课上，你必须展示你的解题过程才能获得满分。在这里也是一样，你必须展示你观点背后的理由。展示你的解题过程可以增强对文档初稿的信心，但更重要的是，通过展示你的解题过程，你可以让其他人在你工作的基础之上进行修改和扩展，以适应不断变化的背景。

你写的一些最好的策略，在当时可能觉得太明显而不值得写。“我们应该什么时候编写设计文档？”是一个值得编写的策略。“我们对哪些用例使用哪些数据库？”是一个值得编写的策略。“我们应该如何将我们的单体应用迁移到服务？”也值得编写。当我们抛弃将策略视为展示才

华的想法时，我们可以开始编写更多的策略，并且可以更随意地编写它们。如果最终没有被使用，你总是可以在以后弃用它。

将五个策略推断成一个愿景

随着你收集到越来越多的策略，要推断各种策略如何相互作用将变得越来越具有挑战性。也许你的一个策略是[减少运行软件](#)并更多地依赖云解决方案，但你的另一个策略是尽可能地倾向于将复杂性转移到数据库。如果你发现一个数据库可以让你转移大量的复杂性，但你的云供应商没有提供该数据库，你如何调和这些策略？

选择你最近的五个策略，推断它们在未来两到三年内的权衡将如何发挥作用。当你编辑矛盾并将线索编织在一起时，你就写了一个工程愿景。最终版本将给你[Tanya Reilly](#)称之为[对未来的坚定信念](#)，这让你更容易理解你现有的策略是如何相互关联的，并简化了编写能够经受时间考验的新策略。

对于一个有用的愿景，需要关注以下几点：

- **写两到三年后的计划。**公司、组织和技术都变化得足够快，以至于对未来进行过于长远的思考是令人担忧的。如果你写一个六个月后失效的愿景，那也是行不通的——你实际上会制定多少个策略在那六个月的窗口期内？尽量关注两到三年后的情况；如果你是一家相当成熟的公司，你可以稍微扩大这个范围。
- **立足于你的业务和你的用户。**有效的愿景立足于服务你的用户和你的业务。这种紧密的联系使愿景与你的领导团队的核心价值观——用户和业务保持一致。糟糕的愿景将技术复杂性视为一种自我辩护的理由——这种观点永远不会被你公司的领导层所认同。
- **保持乐观而不是大胆。**愿景应该是雄心勃勃的，但不应该是大胆的。它们应该是可能的，但如果可能的话，应该是最好的版本。写下如果每个项目都能按时完成且没有重大挫折，你能完成什么。不要写你认为拥有无限资源才可能实现的事情。

- **保持具体和明确。**愿景越具体就越有用。笼统的陈述很容易达成一致，但无助于调和冲突的策略。要比你觉得舒服的程度更详细一些。愿景中的细节通常是说明性的，而不是声明性的，它能让人们品尝到未来的味道，而不是提供一种具有约束力的承诺。
- **保持在一到两页的长度。**现实情况是，大多数人不会阅读长篇文档。如果你写的东西有五六页长，读者就会开始放弃阅读，无法完成（或者会非常快速地浏览，而不会关注细节）。强迫自己写一些简洁的东西，并通过链接到其他文档来引用额外的上下文，供那些想要完整细节的人使用。

在你完成愿景的撰写之后，人们通常采取的第一步是在整个工程组织中广泛分享它。愿景背后有大量的工作——每个策略有五个设计文档，一个愿景有五个策略——很难不是当你完成时感到兴奋。如此兴奋以至于很容易感到沮丧，那么，当对你的策略的反应几乎总是平淡的时候。反应平淡有几个原因。首先，你的愿景的核心受众是编写策略的人，这是一个相对较小的群体。其次，一个伟大的愿景通常是如此显而易见以至于它带来的厌烦感多于兴奋感。

不要用最初产生的兴奋感来衡量愿景。相反，通过阅读两年前的设计文档和上周的设计文档来衡量它；如果有了显著的改进，那么你的愿景就是好的。

管理技术质量

当我可以帮助改进一个善意的、解决了实际需求的提案时，我感到特别有影响力，但是起草该提案的团队缺乏经验或背景来编写一个好的计划来抓住这个机会。在这种情况下，拥有一个结构良好的计划可以帮助大大缩小范围，同时获得大部分价值，因此更快地展示影响力。 - [德米特里·彼得拉什科](#)

如果说工程师、工程经理和技术主管们有什么可能达成共识的话，那就是技术质量存在危机。一个容易识别的诊断和治疗方法是：我们的工程师没有优先考虑质量，我们需要雇佣更好的工程师或重新培训我们现有的工程师。当然，如果你觉得更舒服，你可以随意用“产品经理”或“高管”来代替“工程师”。这是一个引人注目的叙述，有一个明确的恶棍，它方便地将责任从工程领导身上转移开。然而，就像大多数将责任转移到权力最少的人身上的叙述一样，它既没有帮助，也是错误的。

当你接受低技术质量源于糟糕决策的前提时，你就会开始寻找错误的判断，并且公司里必须有人是罪魁祸首。是前任首席技术官吗？是那位带着紧张微笑看着你的资深工程师吗？还是所有人？如果不是这些人，而且更奇怪的是，甚至也不是你的错呢？

在大多数情况下，低技术质量并不是危机；它是预期中的正常状态。工程师通常在做出决策时会做出合理的质量决策，而成功的公司会随着规模扩大、转型或向上转向企业用户而逐渐提高其质量标准。在一个运转良好且成功的公司里，你之前的大部分技术决策都无法达到你当前的质量阈值。弥合当前技术质量和目标技术质量之间的差距，不是失败，而是有效工程领导的常规且重要的组成部分。

问题

作为一个工程领导团队，你的目标是在保持适当的技术质量水平的同时，尽可能多地将精力投入到核心业务中。你必须平衡多个时间范围

内的质量，而这些时间范围通常具有冲突的需求。例如，为了赶上下周的截止日期而推出关键的合作伙伴关系，与构建一个支持下个季度启动速度提高十倍的平台，你所做的工作将截然不同。

正如贵公司的技术质量标准会随着时间的推移而变化一样，您管理技术质量的方法也会随之发展：

1. 修复**热点**这些热点正在引起直接的问题
2. 采用**最佳实践**已知可以提高质量
3. 优先考虑**杠杆点**在软件更改时保持质量
4. 对齐**技术向量**在您的组织如何更改软件方面
5. **衡量技术质量**以指导更深入的投资
6. 启动一个**技术质量团队**创建质量系统和工具
7. 运行一个**质量计划**用于衡量、跟踪和创建责任制

当我们深入研究这一系列方法时，请记住选择最便宜、最直接且可能有效的工具。技术质量是一场长期的游戏。没有所谓的胜利，只有学习和赢得继续玩下去的机会。

登上楼梯

深入研究手头的挑战，直到找到一个值得解决的普遍性问题，这其中有一种特殊的乐趣。然而，同样重要的本能是快速解决当前的情况，然后继续处理下一个紧迫的问题。

当您考虑为您的团队和组织做出正确的质量改进时，通常最有效的方法是从最轻量级的解决方案开始，只有当早期的努力在规模的压力下崩溃时，才转向大规模的解决方案。如果您无法让团队采用正确的代码检查，那么您推出全面质量计划的尝试注定要失败。虽然后者在规模上可能更有效，但它们执行起来要困难得多。

所以，先做快速的事情！

即使它不奏效，你也会从推出简单的事情失败中学到更多、更快，而不是从推出困难的事情失败中学到更多。然后你就能更快地进入改进后的第二次迭代。随着时间的推移，你会朝着全面的方法发展，但没有必要操之过急。不要为了企业级协调的风险而放弃早期组织的轻松、快乐和纯真，而没有适当的需求。

将这些阶段呈现为要攀登的线性阶梯是很方便的，但实际组织很少这样使用它们。你更有可能修复一个质量热点，推广一个最佳实践，开始运行一个架构审查，废除那个架构审查，然后回去进行一段时间热点分析。过早的流程增加的摩擦比价值更多，并且很快就会暴露其无效性。如果某些东西不起作用，尝试让它工作一段时间，然后庆祝它的消亡。

热点

当面对质量问题时，第一反应通常是识别出一个流程失败，这必然需要一个流程解决方案。如果部署导致中断，那是因为作者没有正确遵循代码测试流程，所以现在我们将要求每次提交都进行测试——这会给那些懒惰的开发者一个教训！

有一个关于萨班斯-奥克斯利法案的老笑话：它并没有降低风险；它只是清楚地表明了事情出错时该责怪谁。不幸的是，这个笑话在许多组织推出流程时，并没有幽默感。问责制有其作用，但更重要的是理解手头的问题，并尝试直接解决它，而不是创建流程驱动的问责制。

流程推出需要人们改变他们的工作方式，你不应该轻易地承担这项工作。与其寻求流程改进，不如从戴上性能工程师的思维模式开始。衡量手头的问题，确定问题发生的主要区域，并精确地关注该区域。

之前未经测试的部署示例可能受益于直接向部署工程师提供关于改变其测试习惯的反馈。或者，也许你更好地承认你的软件设计容易出错，并采用“定义错误不存在”的方法，如[软件设计哲学](#)。

如果你有一个开发速度问题，它可能是优化测试运行时间，将你的 Docker 编译步骤移动到[RAM 磁盘](#)，或者使用中描述的技术[软件设计X射线](#)找到需要改进的特定文件。

[系统思考](#)是我职业生涯中遇到的最具变革性的思考技术。不过，有时它可能像海妖一样，引诱你修复当前可能最好丢弃的系统。当然，你可以推出一个新的培训计划教你的团队如何编写更好的测试，但或者，你可以直接删除一个测试文件，其中发生了98%的测试失败。这就是优先处理热点的非凡效率，以及为什么它应该是你用来提高技术质量的首要技术。

在某个时候，你可能会发现你的组织产生质量问题的速度比你修复热点的速度还要快，那时就该转向采用最佳实践了。

最佳实践

我曾经在一家没有团队计划流程的公司工作。随着时间的推移，工程主管越来越沮丧，因为无法预测目标日期，并强制我们使用[Scrum](#)。在强制执行后，一位经理在维基上写了Scrum流程。我们正在使用Scrum的消息发布了。经理们告诉他们的团队使用Scrum。任务完成！

当然，没有人开始使用 Scrum。每个人都继续做他们以前做的事情。承认错误是很尴尬的，所以工程主管宣布采用 Scrum 是一个重大胜利，没有人忍心说不一样的话。

这个悲伤的故事反映了许多公司试图推广最佳实践的方式，这也是为什么最佳实践声誉不佳的原因之一。从理论上讲，组织应该在解决质量热点之前采用最佳实践，但我建议在热点分析之后再采用实践。采用最佳实践需要一定程度的组织和领导成熟度，这需要一段时间才能培养。

当您推出一项新实践时，请记住[好的流程是演变而来的](#)而不是强制执行的。研究其他公司如何采用类似的做法，记录您打算采取的方法，

与一些参与的团队一起试验该实践，消除粗糙的边缘，根据挑战改进文档，然后才进一步推广。仓促的过程是失败的过程。

同样重要的是限制并发流程推广的想法。如果您试图让团队同时采用多个新实践，那么您就是在与自己争夺他们的注意力。如果您正在考虑恢复或修改其中一项新实践，这也使得以后更难确定影响。这有点严厉，但我开始相信，您应该在任何给定时间将自己限制为一次推广一项最佳实践。将你所有的精力集中在一个实践的成功上，而不是将资源分散在少数几个实践上。

一次采用一项新的实践也迫使你仔细考虑优先考虑哪一项。选择你的下一个流程听起来很容易，但通常不清楚哪些最佳实践是真正的最佳实践，哪些只是熟悉的或著名的。真正的最佳实践必须有研究支持，而关于这个主题的最佳研究来源是[Accelerate](#)。

虽然Accelerate的所有建议都是数据驱动的，而且非常好，但我发现早期采用最有帮助的少数建议是版本控制、基于主干的开发、CI/CD和生产可观察性（包括开发人员负责他们编写的系统），以及以小的、原子性的更改工作。我有很多其他的实践想要提倡（谁没有花一个职业生涯来提倡[更好的内部文档](#)），但我不再像以前那样信任我的直觉了。

当你被太多需要解决的热点问题压得喘不过气时，就会从解决热点问题过渡到采用最佳实践。下一个过渡，从最佳实践到杠杆点，发生在你发现自己想在进行中的最佳实践生效之前采用新的最佳实践时。与其[增加你正在进行中的最佳实践采用限制](#)，不如转向下一个工具。

杠杆点

在“热点分析”部分，我们讨论了如何利用性能工程师的思维方式来识别需要解决的正确问题。优化对于你已经存在的问题非常有效，但它有意不适用于未来：性能工程最严重的罪过是将精力投入到未经证实的问题上。

然而，当你观察软件随时间的变化时，会发现少数几个地方，额外的投入可以长期保持质量，既能防止严重的质量问题，又能降低未来质量投入的成本。

我称这些为质量杠杆点，其中三个最具影响力的点是接口、有状态系统和数据模型。

接口是系统之间的契约。有效的接口将客户端与封装的实现解耦。持久的接口暴露所有底层本质复杂性，而不暴露任何底层偶然复杂性。令人愉悦的接口是[热切地辨别，辨别地热切](#)。

州是任何系统中改变起来最困难的部分，而对改变的抵制使得有状态系统另一个关键的杠杆点。状态比其他系统更快地变得复杂，并且具有一种惯性，使得以后改进它的成本相对较高。当您将围绕安全性、隐私和合规性的业务义务纳入其中时，更改您的有状态系统将变得更具挑战性。

数据模型是接口和状态的交集，将你的有状态系统的能力限制在你应用程序认为合法的范围内。一个好的数据模型是严格的：它只暴露它真正支持的东西，并防止无效的各州表达。一个好的数据模型能够容忍随时间的演变。有效的数据模型甚至一点也不聪明。

当你在工作中识别出这些杠杆点时，花额外的时间有意识地处理它们。如果是接口，则针对模拟实现集成六个客户端。如果是数据模型，则表示六个真实场景。如果有状态的，则执行故障模式，检查一致性行为，并建立类似于生产场景的性能基准。

把你学到的一切都拿出来，整理成一份技术规范文档，并在你的团队中推广。收集同行专家的行业反馈。即使在你开始实施之后，也要倾听现实的声音，并对变化保持开放的态度。

投资于杠杆点的隐藏力量之一是，你不需要完全的组织协调来实现它。要编写技术愿景或推广最佳实践，你需要那种支持，这就是为什

么我建议从杠杆点开始。但是，如果你已经耗尽了杠杆点带来的可及影响，那么可能是时候转向推动更广泛的组织协调了。

技术向量

高效的组织将其大部分努力集中于共同的愿景。如果将每个技术决策绘制为网格上的一个向量，那么这些向量指向同一方向的越多，随着时间的推移，您将完成的越多。相反，我合作过的一些最令人印象深刻的工程师创造了具有非凡幅度但方向不一致的向量。最终，这些工程师在试图领导组织时损害了他们的组织。

一种确保技术方向一致的万无一失的解决方案是将所有相关决策传递给同一个人，并且他们的职称中包含[架构师](#)。这种方法效果很好，但扩展起来具有挑战性，并且架构师的决策质量会随着他们远离在实际流程中对实际代码进行实际工作而降低。在另一个极端，您可以允许每个团队做出独立的决定。但是，允许任何工具的组织是一个拥有统一不受支持工具的组织。

用于对齐技术向量的基本工具包括：

- **提供直接反馈。**当人们遇到不一致时，第一个答案通常是流程变更，但相反，首先要直接向您认为不一致的个人提供直接反馈。就像他们错过了您的背景一样，您也错过了他们的背景，一次快速的对话通常可以避免多年的不必要流程。
- **完善您的工程策略**从[技术规范到战略再到愿景](#)。
- **将你的方法封装在你的工作流程和工具中。**清晰愿景的文档很有帮助，但有些人根本不会研究你的文档。精心设计的工具创建的工作流程，比培训和文档更能培养习惯。例如，配置一项新服务可能需要访问一个网站，该网站要求你添加该服务的技术规范链接。另一种方法可能是阻止部署到生产环境，如果该服务没有建立随叫随到的设置，并且当前有人随叫随到，并且该人还必须启用他们的推送通知。

- **在入职期间培训新团队成员。**在人们养成习惯后改变他们的习惯是相当具有挑战性的，如果你试图让人们采用新的做法，这会令人沮丧。但是，如果你在人们加入时将他们指向正确的方向，那么这种习惯动力将有助于保持一致。
- **使用康威定律。**康威定律认为，组织构建的软件反映了他们的结构。如果你的组织结构不良，这将导致紧密耦合或混乱的软件。但是，如果你的组织设计是有效的，这也是提高质量的力量。
- **管理技术变革**使用[架构审查](#)，[投资策略](#)，和[采用新工具的结构化流程](#)大多数不一致源于缺少上下文，而这些是组织利用点，可以将上下文注入到决策中。许多组织从这里开始，但这是我建议最后打开的工具箱。如果没有明确的愿景，你如何提供一致的架构审查？为什么在人们设计完东西之后，而不是在他们的入职过程中告诉他们你的策略？

无论你使用哪种方法来对齐你的技术向量，这项工作往往需要花费数月甚至数年的时间。你写完愿景文件，组织立即支持它的辉煌，这是不可能的。更有可能的是，它会堆积灰尘，直到你投入精力来建立支持。

大多数公司可以将上述从热点修复到向量对齐的技术结合起来，形成一种成功的技术质量管理方法，希望这对你来说也是如此。然而，许多人发现这些方法还不够，你需要转向更重量级的方法。在这种情况下，第一步始终是测量。

测量技术质量

在软件工程中，测量的愿望通常超过了我们的测量水平。[Accelerate](#)确定了衡量速度的指标，这些指标对于定位流程和工具问题非常有效，但这些指标从以下方面开始之后代码已经合并。你如何衡量你的代码库的质量，以便你可以识别差距，提出行动计划，并评估你为改进所做的努力的影响？

有一些过程测量与有效的变更相关。例如，你可以测量每个拉取请求中更改的文件数量，因为较小的拉取请求通常质量更高。你还可以测量代码库中每个文件的代码行数，假设非常大的文件通常难以扩展。这些都可能很有帮助，我甚至建议测量它们，但我认为它们充其量只是代码质量的代理测量。

我的经验是是有可能有效地衡量代码质量，这归结于制定一个极其精确的质量定义。你对质量的定义越详细，衡量代码库就越有用，对于那些希望提高他们工作领域质量的人来说，就越有指导意义。这种方法在以下内容中有详细描述：[构建进化型架构](#)和[回收不合理的软件](#)。

在您的质量定义中可以考虑包含的一些代表性组件：

- 代码中有多少百分比是静态类型的？
- 有多少文件有相关的测试？
- 你的代码库中的测试覆盖率是多少？
- 模块之间的公共接口有多窄？
- 有多少百分比的文件使用了首选的 HTTP 库？
- 端点是否在冷启动后 500 毫秒内响应请求？
- 有多少函数具有危险的先读后写行为？或者对主数据库实例执行不必要的读取？
- 有多少端点在单个事务中执行所有状态变更？
- 有多少函数获取低粒度锁？
- 有多少热点文件在超过一半的拉取请求中被更改？

您可以不同意这些属性应该存在于您的代码库的质量定义中：您的定义应该特定于您的代码库和您的需求。重要的是开发一个精确的、可衡量的定义。在定义的发展过程中会有分歧，而且您必然会随着时间的推移而改变定义。

在您制定了定义之后，这是一个工具化可能真正具有挑战性的领域，而工具化是获得有用指标的必要条件。工具化的复杂性是在实践中采用这些技术的最大摩擦点，但如果您能克服它，您将解锁一些非常棒的东西：一个真实的、动态的质量分数，您可以随着时间的推移进行

跟踪，并用它来在您的方法中创建一种清晰的对齐，而概念上的对齐无法做到。

在定义和工具化了质量之后，您的下一步是在投资于*质量团队*或者一个*质量计划*一个专门的团队易于协调，其带宽可预测，并且通常是更容易的起点。

技术质量团队

一个*技术质量团队*是一个软件工程团队，致力于在你的代码库中创造质量。你可能会称这个团队为开发者生产力、开发者工具或产品基础设施。无论如何，该团队的目标是在你公司的软件中创造和保持质量。

这不是有时被称为质量保证团队的团队。虽然两个团队都对测试进行投资，但技术质量团队的职责范围更广，从工作流程到构建到测试到界面设计。

当你启动这样一个团队时，从三到六人的固定团队规模开始。拥有一个小团队会迫使你毫不留情地优先考虑他们的路线图的影响，并确保你将保持对可实现目标的关注。随着时间的推移，这个团队将积累需要扩大投资才能维护的系统，Jenkins集群就是一个常见的例子，你会想要[确定团队规模](#)作为更广泛的工程组织的一个职能。经验法则在这里很棘手，但也许每十五个产品工程师就有一名工程师从事开发者工具工作，此外还有你的基础设施工程投资。

这些团队很少有产品经理，通常由一名或多名 Staff-plus 工程师以及工程经理合作伙伴来担任这个角色。有时他们会聘用技术项目经理，但通常是在他们进入运营一个“*质量计划*”，如下一节所述。

在启动和运营这些团队时，一些成功的要素是：

1. **信任指标胜过直觉。**你应该有办法衡量每个项目。质量是一个复杂的系统，在这种地方，你的直觉很容易欺骗你。同样，随着你

在公司变得越来越资深，你的经验将不再反映大多数其他人的经验。你已经知道粗糙的地方，如果你发现一个新的，你将是第一个排队获得帮助的人，但大多数其他人不会。指标让你保持诚实。

2. **保持你的直觉新鲜。**代码和流程会随着时间的推移而改变，而你离开构建产品功能的每一周，你的直觉都会变得陈旧。大多数人发现团队嵌入和团队轮换是最好的方式保持你的直觉相关。其他人会监控聊天中的问题，以及与产品开发人员进行健康的 1:1 讨论。最优秀的人会同时做这两件事，并随时掌握他们的指标仪表板。
3. **倾听并向你的用户学习。**有一种流行的“品味水平”的说法，暗示着有些人天生就知道什么是好看的。在设计有效的优质投资方面，人们的差异很大，但这不是一种天生的技能。最优秀的人专注于深入了解他们的用户想要完成什么并且优先考虑用户需求，而不是实施限制。

工具的采纳和可用性比原始能力更重要。一个难以使用的强大工具只会吸引少数高级用户，但大多数人会忽略它。放慢速度，把这些细节做好。隐藏所有[偶然复杂性](#)观看一位工程师第一次尝试使用你的工具，不要提供任何帮助。改进不足之处。再做十次！如果你没有对你的工具进行用户研究，那么你就是注定的作为一个高质量的投资团队。

4. **少做事情，但把它们做得更好。**当你为整个工程组织构建时，你做得好的任何事情都会加速整个组织的发展。你做得不好的任何事情，包括几乎很棒但有太多粗糙之处的事情，都会拖累所有人。虽然几乎总是这样，做少数最重要的事情比做许多平庸的项目贡献更大，但在你试图向整个组织推广工具和工作流程的情况下，这一点尤其如此（组织过程中的在制品限制仍然适用！）。
5. **不要囤积影响力。**集中式质量团队和他们所服务的团队之间存在着根本的紧张关系支持。通常情况下，集中式团队倾向于采用全

局最优方法，但这会让在非典型领域或工作负载上工作的部分团队感到非常不满。一个典型的例子是一家公司用 JavaScript 编写其后端服务器，并且不允许其机器学习工程师使用 Python 生态系统，因为他们不想支持两个生态系统。另一个例子是一家公司标准化地使用 REST/HTTP2/JSON 用于所有 API，而一个特定的团队想要使用 gRPC。这里没有完美的答案，但重要的是建立一种深思熟虑的方法，[在探索的好处和标准化的好处之间取得平衡](#)。

一个使用上述方法的成功的技术质量团队将毫无疑问地比相同数量的工程师直接从事产品工程工作更有效率。实际上，折现的开发者生产力（本着[折现现金流](#)的精神）是理论上衡量这种团队影响的正确方法。仅仅是理论上，因为这样的计算主要是在评估你的自信心。

即使你非常成功，你总会有一堆高影响力的工作积压，你想承担但没有足够的带宽来完成。组织不会做出纯粹理性的团队资源决策，你可能会发现你缺乏完成重要项目的带宽，同样也无法获得批准来招聘更多人加入你的团队。

当你的团队有比你能承担的更多的高影响力工作时，这是一个好兆头：如果你不选择要承担的项目，那么你思考得不够广泛。这意味着如果你有积压的工作，你不应该一定尝试扩大你的技术质量团队。但是，如果你发现有你无法完成的关键质量工作，那么可能是时候开始一个[质量计划](#)。

质量计划

一个[质量计划](#)根本不是计算机代码，而是一个由专门团队领导的倡议，旨在维护整个组织的技术质量。质量计划承担着实现组织目标软件质量水平的广泛职责。这些相对来说并不常见，但你可能遇到过的类似情况是，一个事故计划负责公司的事故回顾和补救。

运行高质量程序的技术组成部分是上面讨论的那些，所以在这里我们将重点关注如何有效地管理程序。你的第一步是找到一位技术项目经理，他可以共同领导该项目并操作其机制。虽然你可以在没有技术项

目经理的情况下，在组织计划的信息方面取得相当大的进展；然而，这是一个陷阱。你会被在大型组织中单独驱动程序的协调开销压垮。

运营组织计划是[一个被广泛撰写的主题](#)，但核心方法是：

1. **确定一个计划发起人。**没有一个有权力的发起人，你无法改变一个组织的行为。组织的行为方式是因为这是他们当前约束条件下的最佳解决方案，如果没有有影响力的人的支持，你无法改变这些约束条件。
2. **生成可持续的、可重复的指标。**运行一个计划的人每周花费四个多小时手动维护他们的数据集是很常见的。这行不通。你的数据会有漏洞，你将无法在后面的步骤中将你的数据与自动化集成，而且你将耗尽精力来完成真正的工作改变；刷新指标仪表板没有内在价值。
3. **为每个受影响的团队确定计划目标，并为他们实现这些目标提供明确的途径。**你的计划必须为每个受影响的团队确定具体目标。例如，减少他们测试中的测试不稳定或更快地完成事件修复。然而，至关重要的是，你要提供成功的路线图！很多计划要求其他团队参与，但没有提供关于他们如何完成自己部分的明确指示。计划负责人是主题专家，不要把你的策略卸载给每个团队，让他们独立地重新发明。
4. **构建工具和文档，以支持团队实现其目标。**一旦你确定了团队实现你的项目目标的明确路径，弄清楚你如何能帮助他们做出这些改变！这可能是提供“黄金示例”，说明事情应该是什么样子，或者是一个示例拉取请求，将一段具有挑战性的代码重构为新的模式。这可能是提供一个测试脚本来验证迁移是否正确。这可能是自动生成转换提交，以便在没有工程师自己编写的情况下进行测试、验证和合并。尽可能多地避免每个团队都必须深入理解你试图取得进展的问题领域。

5. **创建一个目标仪表盘并广泛分享。**一旦你将你的项目目标传达给每个团队，提供仪表盘，帮助他们了解他们当前的状态、他们的目标状态，并提供关于他们（希望的）进展的强化反馈。最好的仪表盘既是每个团队工作的记分卡，也为每个团队提供关于下一步工作重点的线索。

你的仪表盘应该支持三个不同的缩放级别。完全缩小的级别可以帮助你评估你的项目的影响。完全放大的级别可以帮助单个团队了解他们剩余的工作。介于两者之间的第三个级别可以帮助组织领导者追究其团队的责任（并支持你的项目发起人提出具体的、明确的要求来追究这些领导者的责任）。

6. **为那些落后于目标的人发送程序化的提醒。**人们都很忙。他们不会总是优先考虑你的项目目标。或者，他们可能在做出你要求的改进方面做得非常出色，但后来又使用过时的做法倒退了。使用提醒来引导团队关注他们应该为你的项目目标采取的下一步工作。记住，注意力是一种稀缺资源！如果你用提醒邮件或ping浪费了人们的时间，他们就不会关注下一个。
7. **定期与你的发起人审查项目状态。**项目试图在组织优先事项上取得进展，而这些优先事项与团队的目标并不自然地一致。许多团队难以摆脱其本地优先事项，以完成全局优先事项。因此，必须与你的发起人一起审查你的总体进展，并将他们指向优先考虑项目工作的团队。有效地利用你的发起人来弥合不一致的优先事项将对你的成功至关重要。

在很多方面，一个项目只是一场无休止的迁移，而且[适用于迁移的技术也适用于程序](#)。

如果你把所有这些步骤都做对了，你就在运行一个真正伟大的程序。这可能会让你觉得工作量很大，而且哇，确实如此：很多程序都出错了。失败的三个主要原因程序是：

1. 纯粹从流程的角度运行它，并脱离你试图完成的事情的现实，

2. 纯粹从技术的角度运行它，并认为你可以跳过倡导你的目标和倾听你试图激励的人们的必要步骤，
3. 试图以一个人的身份涵盖这两种观点——不要单打独斗！

一个糟糕的程序很像一个效率低下的非营利组织：目标是正确的，但很少有资金到达预期的目标。无论你决定如何衡量技术质量，在运行你的质量程序时，最重要的是始终记住程序不是目标。目标是创造技术质量。组织程序规模庞大，并产生巨大的动力，即使在它们停止工作后，惯性也会推动它们前进。保持你的程序足够精简以取消，并保持足够的自我批评以取消，如果它停止驱动质量创造。

从小处着手，慢慢增加

当你意识到你的实际技术质量已经大大落后于你的目标技术质量时，自然的第一个反应是恐慌并开始推出大量的技术和解决方案。不可避免地，把所有的配料都倒进锅里效果不好，更糟糕的是，你甚至不知道该保留哪些部分。

如果你发现自己在技术质量方面苦苦挣扎——我们都会经常这样做——那么从一些小的事情开始，并对其进行迭代，直到它起作用。然后添加另一种技术，并对其进行迭代。慢慢地朝着真正有效的东西构建，即使这意味着要经受住行动不够快的指责。当涉及到复杂的系统和相互依赖性时，快速行动只是表面现象。有条不紊的行动才能完成任务。

与权威保持一致

在我的工作中，我们经常会连续几周不在同一个房间里，但我仍然必须像他的直接代理人一样运作。所以我走进一个房间，心想：“如果马修在这里会怎么做？他会想问什么问题？他对这个问题有什么指导？”因为我不能总是跑回去向他澄清，所以至关重要的是要对他世界观有深刻的理解。这对于我保持作为他的代表所需的非常深的信任，并有效地执行他的战略和愿景至关重要。人们需要确信，如果马修在那里，我总是会给出与他相同的答案。 -[里克·布恩](#)

一种常见的误解是，权力会让你变得强大。许多渴望担任更高级职位的人认为，他们最终可以按照自己的方式做事。他们认为头衔本身就创造了灵活性和自主性。他们相信，阻碍他们的摩擦会变成一群蝴蝶，消散在风中。

现实情况要微妙一些。

头衔带有被称为组织权威的那种权力，而这种权威是由更大的组织权威借给你的。被授予的东西也可以被收回，而保持组织权威取决于与授予赞助者（通常是你的直接经理）保持深度一致。为了在Staff-plus职位上保持有效，你必须学会与组织权威保持一致的艺术。

超越安全网

放弃你剩下的期望，认为公司旨在帮助你走向成功。现在你你是负责帮助公司、你的团队和你的经理走向成功的人之一。

大多数成熟的科技公司都成功地创建了一个可预测的晋升管道，从职业生涯早期加入的人一直到获得高级工程师职称。获得Staff职称的过程通常比之前的职称更复杂，但通常在你的工程经理的支持下完成。在整个过程中，你可能会对你的经理指导你的发展并为你的持续成功提供安全保障感到满意。在达到Staff职位后，你的安全网将不复存

在，或者充其量，安全网会很短，你完全有能力跳过它，进入等待你的深渊。随着你进一步进入高级Staff和杰出工程师职位，情况将越来越真实。

Staff-plus职位是领导职位，而在领导职位上，让你走到今天的支持系统将会消失。通常是突然之间，你现在需要调整你周围的部分，以实现你自己的成功。

随时为总统效力

当[里克·布恩](#)他将自己描述为优步基础设施副总裁的战略顾问，并将自己的角色比作[国王之手](#)在权力的游戏，和[里奥·麦加里](#)从[白宫风云](#)他经常说：“我为总统服务。”在这两个例子中，权力都来自于与更大权力的紧密联系，这是一种在 Staff-plus 角色中运作的绝佳思维模式。对于那些你的权力主要通过你的个人行为 and 随着时间的推移产生的影响而积累起来的之前的角色来说，这可能是一个困难的转变。

如果你和你的经理已经合作多年，那么在这段时间里，你们已经进行了一种微妙的、潜移默化的调整。在其他情况下，一位新的高管会加入，他们熟悉如何支持这些角色，并会带来一份他们希望如何合作的明确地图。然而，这两种情况在很大程度上都超出了你的控制范围，因此，制定你自己的方法来与你的经理进行向上对齐是有价值的。

为了与你的经理保持一致，需要关注的领域包括：

- **永远不要让你的经理感到意外。**没有什么比让你的经理感到意外更能迅速摧毁信任了。领导一个大型组织通常需要同时处理多个项目和问题，而意外会威胁到领导者的节奏。重大或频繁的意外也会让人质疑领导者是否真正为他们的组织承担责任。总的来说，每次让你经理感到意外时，都应将其视为一次需要从中吸取教训的事件，并努力防止重蹈覆辙。
- **不要让你的赞助人给你带来意外。**大多数人对他们的经理抱有极高的期望，例如，他们总是会记得传递与你当前工作相关的信息。经理们会努力做到这一点，他们中的一些人在这方面做得非

常出色，而另一些人则不太擅长。如果你的经理在这方面做得不好，你当然应该给他们反馈，但你也应该采取积极的行动来促进信息的流动。这可能是[每周电子邮件更新](#)或者在你团队的频道中分享你本周重点的 Slack 话题。在 1:1 的会议中，挖掘反馈！询问是否还有其他你应该关注的领域，以及你当前的优先事项与你的经理的优先事项是否一致。如果你们继续互相感到惊讶，那么[确定控制措施](#)你将用来合作。

- **提供你经理的背景信息。**如果第一步是避免你的行为让你的经理感到惊讶，下一步是帮助你的经理不要被更广泛的组织所惊讶。如果团队对一项新政策感到沮丧，或者你的内部工具无法满足需求，主动将这些信息反馈给你的经理。明确表示你正在不是给他们带来一个需要解决的问题，而不是传递你认为有用的信息。意见是有帮助的，但如果你能找到数据，数据会更有帮助。

有时你会听到有人贬低一位同事，说他们非常擅长“向上管理”。当然，向上管理也有一些破坏性的方式，比如有人控制信息来掩盖问题或歪曲事实，但从根本上说，向上管理是为了增加你和你的经理之间的带宽，减少摩擦。与你的经理建立一种深思熟虑的伙伴关系，远比在他们没有达到你的期望时感到失望更有用。

在不产生过多摩擦的情况下施加影响

作为领导者，成长的一部分是在于发展你对世界应该如何运作的看法，如果没有这种看法，你无法达到 Staff-plus 级别。对事物应该如何运作有清晰的认识，可以提高你的判断力，并使你能够积极主动地采取行动。当你达到领导力的下一个阶段时，你越来越需要将你的愿景与更高级别的组织领导者所持有的愿景融合。

你解决这个问题的第一个方法可能是用另一个领导者的愿景取代你的愿景，这种方法对某些人有效，但对许多人来说，这意味着放弃那种促使他们作为具有强烈判断力的积极主动的领导者取得成功的视角。相反，我建议提高对你所持有的价值观与组织运作方式之间的区别的认识，并找到一种在不被[踢出房间](#)。

的情况下倡导这些价值观的方法。人们只能如此迅速地改变，而组织是由人组成的。如果你在方法上是深思熟虑的，那么随着时间的推移，你将能够极大地影响你组织的领导者，但如果你学会始终在每一步都保持紧密一致，你才能获得这段时间。

要领导，先跟随

这是关于采取全局思维并在本地应用它。这意味着将你团队的（技术）计划/路线图与整个工程部门的技术战略对齐；并有意地决定何时偏离该路径，以满足你团队直接利益相关者的需求。这意味着与你团队的经理合作，采纳其他团队在招聘、入职和生产运营方面的成功实践；并分享你团队中对其他人有益的实践。这意味着从公司范围内的业务/产品战略中获取背景信息，并将其转化为它如何影响你团队的直接项目 - [拉斯·卡萨·威廉姆斯](#)

几年前，我所在的公司聘请了一位新的工程总监，首席技术官 (CTO) 正在谈论为什么这位新总监是一位了不起的员工。这位新总监的决定性成就？对领导力和管理之间区别的最佳解释。事实证明，这不是一种特别有效的评估招聘方式，但这是一个有趣的话题。

定义领导力和管理是一个被过度讨论的领域，很难再添加更多内容，但大致来说，管理是一种特定的职业，而领导力是一个人可以在任何职业中展示的方法。

我对领导力的看法在过去几年里发生了一些变化，但主要集中在两个具体属性上。首先，领导者对事物应该如何运作有足够完善的看法这样他们就可以依靠他们对事物之间差异的区分是以及他们应该 是来识别主动的、一致的行动，以缩小差距。其次，他们足够关心这个差距，从而真正尝试那些缩小差距的行动。

如果你只看到差距而不采取行动，你可能是一个有远见的人，但你是惰性的。如果你在没有明确目标的情况下采取行动，许多人会认为你是一个领导者，但你的影响将是随机的、武断的和低效的。将两者结合起来，再加上一些运气，很可能在你的职业生涯中走得很远，这些都是我合作过的那些成功过渡到Staff-plus工程师或高级管理职位的人的共同特征。

但这种领导力只能带你走这么远，就我个人而言，我花了年的摸索才明白为什么我的领导方式在刚加入公司时为我带来了如此多的早期成功，但随着时间的推移，我的贡献是如何逐渐被接受的。我慢慢学到的教训是，在你学会如何追随之前，你不可能成为一个有效的长期领导者。

我认为这是我在过去几年里学到的最重要的一课：最有效的领导者花更多的时间跟随，而不是领导。这个想法也出现在“[第一个追随者造就一个领导者](#)”，“但有效的领导者不会将世界划分为领导者和追随者的二分法，而是与周围的人一起进出领导和追随者的角色。

有很多方法可以将这种方法付诸实践。

1. 清楚地了解你真正的优先事项是什么，不要在所有出现的事情上分散精力。如果有些事情你不同意，但只是在很小的程度上，让其他人带头解决它。这里一个有用的问题是，“我们在这里做的事情在六个月后对我有意义吗？”如果不会，那就抓住机会跟随。
2. 迅速支持其他正在努力做出改进的领导者。即使你不同意他们最初的方法，一个值得信赖的人领导一个项目几乎总会得到一个好的结果。如果一个值得信赖的人正在领导一个项目，而你仍然不舒服让他们继续前进，考虑一下为什么你缺乏影响他们的能力，以及你是否不擅长给予反馈。
3. 明确地让你的反馈是非阻塞的。这可以将代码审查评论归类为“可选的挑剔”，但也可以编写详细的反馈，但将其传递给某人，并提到你想分享你的观点，而不是必须改变他们的方法。

如果你一直在为此挣扎，我对此表示同情。我也为此挣扎过。当你拥有足够强大的世界观来领导时，你将开始在你周围聚集其他人，他们依靠你来维持那个世界的物理学，并且容忍任何偏离你愿景的行为会让你觉得你让他们失望了。但这是让你达到一个成功水平但阻碍下一个成功的缩影：持续的增长需要学会将你的世界观融入你周围人的世界观中，加速你周围的整体进步，即使这意味着容忍偏离你的愿景。

你独自能完成的事情远不及你通过创造领导者所能完成的事情。要成为一个伟大的领导者，花时间学习如何跟随。

学会永远不出错

我提出我认为对我们来说最好的情况，人们可以不同意。而且，你知道，他们经常这样做。我更多的是在引导和影响，而不是说，“我有权告诉你该怎么做。”我从未见过这种风格运作良好。 - [基维·麦克明](#)

大多数人都和那些认为自己永远不会犯错的人一起工作过。在每次讨论中，他们都会凑上前，挺起肩膀，以一种强硬的方式扮演决策者的角色。他们会一直争论下去，直到他们的观点获胜或时间耗尽。他们通常是对的，但这种正确的方式会让人感到窒息。随着他们在公司任职时间的增加，他们可能会认为自己变得非常有说服力，但通常这是一种以同事的无奈为特征的说服方式。

我合作过的一些技术领导者找到了一种永远不会犯错，同时又不主导会议的方式。在正确的同时，为他人创造空间。对我来说，一直体现这种方法的人是[富兰克林·胡](#)，我经常看到他通过致力于为每个人找到最佳结果、愿意放弃自己的初始立场以及默认总是存在额外的背景信息可以将看似冲突的观点统一起来，从而化解激烈的讨论。

要成为一名高级技术领导者，你必须对技术和架构建立深刻的见解。要作为这样一位领导者运作，你必须培养同样深刻的实用主义和对技术宗教的不可知论，以保持对自己的怀疑。这可能感觉像一个悖论，但这是你每天都需要走的路。

倾听、澄清并观察现场气氛

很多时候，你会看到工程师们充满信心地进入讨论，认为自己的观点是正确的，并且希望让房间里的其他人同意他们的方法。这种心态使每次会议都变成一场零和博弈。即使在他们的方案被采纳的“最佳情况”下，他们也没有从房间里的其他人那里学到任何东西，而且房间里的其他人也不太可能充满活力地离开。

最有效的工程师在每次会议中都以达成以下目标为目的：就手头的问题达成一致，了解房间内的需求和观点，并确定需要采取哪些措施来统一方法。他们将每次会议都视为项目更广泛背景以及他们与房间里的人的关系中的一轮。如果房间准备好同意并推进解决方案，他们就会让团队采用该方法。如果房间没有准备好，他们不会强迫它发生。

要擅长这一点，你需要掌握三种方法：通过问题倾听、明确目的以及了解如何观察现场气氛。

通过问题倾听是一种积极倾听的形式，其目标是理解房间里其他人的观点。以良好的意图提出好的问题，可以开启对话，为他人创造空间 and 安全感，让他们提出自己的问题。好的问题是带着学习的愿望提出的，而且是具体的。它们使对话更加清晰。它们使回答者摆脱了捍卫自己立场的义务。在一次可能引起争议的会议中，在你分享你的观点之前，先问三个好问题，你会看到你周围的房间发生变化。

好的会议始于明确的目的和议程，但许多会议并不符合这个定义，特别是临时讨论。如果你发现自己参与了一个目标不明确的对话，那么**明确目的**。花点时间问一下你对小组希望完成的目标的理解是否正确。最好的方式是将其包装成一个澄清性的问题，例如，“确认一下，我们的目标是决定是否将项目发布推迟两周？”

请注意，如果过于频繁地使用定义目的，可能会产生干扰。在这种情况下，它不会帮助澄清对话，而是会造成对话的混乱。在大多数情况下，如果其他人已经尝试过了，尽量避免使用它。多次重构失败的会议几乎总是以安排另一次会议结束。

最后，在每次会议中，你都必须**观察情况**。通常，人们会对谈话感到沮丧，并试图强行达成一致，这给讨论带来了很大的压力，以至于不太可能得出好的结论。如果房间里的人意见分歧太大，那么就确定一个能够花更多时间一起深入研究的小组，或者确定一个合适的当事人，在房间外进行升级。如果抽屉里的东西太多，就不要试图把它关上。

如何练习

如果这些行为对你来说不是天生的，那也没关系；练习的机会就在身边。对文档的每一条评论都是一个机会。每一次会议都是一个机会。每一个拉取请求都是一个机会。

每周开始时，选择一个你希望在主持的会议中明确使用的技能。如果你有一个特别困难的会议要召开，花一些时间在脑海中或与同事一起练习如何使用这些方法来促进进展，尽管存在挑战。

混蛋

上述方法在大多数情况下都有效，但并非总是如此，一个值得注意的例外是当你与一个混蛋打交道时。在这种情况下，混蛋是指那些拒绝给予团队同意、不愿意妥协或不听取意见的人。这样的人还没有意识到他们的职业生涯更多地取决于是否容易参与，而不是技术上是否正确。

处理混蛋的两种最有效的方法是：

1. 包括一个他们不能在会议上表现得像个混蛋的人（比如他们的经理或首席技术官）
2. 在会议之前投入大量精力与他们保持一致，这样他们会感到被倾听，并且不太可能破坏讨论

花时间做这两件事可能会让人觉得很荒谬，但它们往往是最有效的，尤其是当你与一个不经常打交道的混蛋打交道时。如果这个人是你负责的领域的人，或者你经常与之共事的人，那么你会有一些不同的义务。在这种情况下，尽可能友善地给他们反馈，同时保持诚实。再给一次。记录这两次，如果你没有看到任何改进，那么通过面对面或视频讨论，将你的担忧告知他们的经理，包括具体的文档。

认识到你的头衔所赋予的权力可以保护你免受许多这类人的影响也是有用的，所以无论你遇到谁，他们对你的态度都比对其他人要好。如

果这种行为对你来说感觉是临界的，那么对其他人来说可能更过分。

它如何提供帮助

这种方法之所以强大，是因为更复杂的项目更多地是因为个人冲突而脱轨，而不是因为技术复杂性，而且这是一种用伙伴关系取代紧张关系的可重复方式。感觉它很慢，因为它可能需要更长的时间才能开始，但最终它很快，因为你更有可能在没有中断的情况下完成工作。

此外，作为高级领导者的长寿与保持人际关系和取得杰出成就同样重要。你会看到很多人一时风光无限，但后来却缺乏支持，无法取得进展。如果你想避免这种命运，就要学会永远不会犯错，并且永不停歇地练习。

为他人创造空间

在这一点上，我花更少的时间倡导特定的技术或项目，而花更多的时间授权他人倡导他们认为重要的技术和项目。我还努力成为一个知识和支持的来源，人们可以联系我以获得反馈，尤其是在跨领域的产品决策以及向组织其他部门展示想法方面。

- [米歇尔·布](#)

衡量你作为Staff-plus工程师长期成功的最佳标准之一是，你周围的组织越来越多地受益于**但不依赖于**你的贡献。因为很多人都是通过成为组织的“首选”人员而达到他们的第一个Staff-plus角色，所以从必不可少到相邻的过渡可能很困难。

这种转变需要学习有意识地为你的团队创造空间，并且归结为积极地让他们参与讨论、决策，并最终用赞助来代替重复那些让你首先获得Staff成功的经验。

讨论

当你专注于最大化你的个人影响力时，一个好的讨论是指快速结束，并获得合理的答案，参与者之间达成一致，并且参与者之间有积极的感受。当你开始考虑创造空间时，一个好的讨论的定义会大大扩展！

这个更广泛的定义取决于让更多的人参与进来，并在没有你个人贡献的情况下达成一系列好的决策。在这个新的世界里，一个好的讨论是指最终你不需要参加的讨论。当你做出一个关键贡献时，对此感到高兴，然后思考一下需要发生什么，才能让其他人下次做出同样的贡献。

随着思维方式的转变，我发现以下几种技巧有助于在讨论中创造更多空间：

- 将你的贡献转向提问。提出正确的问题[有助于避免失误](#)，而且也让更多人更容易参与进来

- 如果你看到会议中有人没有参与，就把他们拉入讨论。最好是拉一次一个人进入讨论。当你广泛地向所有人开放，甚至试图一次拉两三个人时，会让人感到困惑
- 成为做笔记的人。这有助于消除人们认为做笔记是“低地位”的偏见，同时也让原本要做笔记的人可以腾出时间来做出更多贡献。而且，它还能让你专注于其他事情，而不是发言！
- 如果你意识到有人应该参加讨论但却缺席了，那就成为在下次会议中把他们拉进来的人。与会议协调人沟通，让他们知道 **включить** 他们是有价值的

当你越来越忠实地遵循这些原则时，你在会议中的发言会减少，而你对组织的影响力会增加。

决策

在你的职业生涯中，很多时候成功在于做出正确的决定，但你需要一段时间才能意识到，在某个阶段，做决定本身并不是工作。[里图·文森特](#)很好地描述了这种转变，

也是在那个项目中，我的经理帮助我理解了作为技术主管的最初想法是行不通的。最初，我在想，“我会把它分成二十块，分配出十八块，把最难的两块留给自己”，我的经理推动我把困难的部分委派给团队，以扩展和发展他们。

另一方面，很难将你的[判断](#)传递给其他人，尤其是在复杂的决策方面。幸运的是，可以采取渐进的方法，将越来越复杂和重要的决策转移到更广泛的团队中。

- **写下来。**费曼算法中概括了一个广为人知的的天才模型：“1) 写下一个问题。2) 非常努力地思考。3) 写下解决方案。”这种神秘的天才观既难以接近又令人沮丧。这也是不现实的，但如果我们不写下我们的思考过程供他人参考，人们很难知道它是不现实的。通过写下寻找答案的过程，以及答案的理由，我们周围的人可以开始从中学习我们的决定，而不是仅仅被[指导](#)通过他们

- **尽早传阅**，并在你对某个决定形成定论之前这样做。大多数人很难从已形成的观点中退回来，通过尽早收集反馈，可以更容易地整合反馈，并让人们参与到决策过程中，这样他们就可以看到你的思考轨迹，以及最终的输出。
- **将形式与内容分开**，并停止对其他人的决定提供形式上的反馈。如果一条反馈意见不会对项目的成功产生有意义的改变，那么可以考虑不提供。如果它有用但不是关键，可以私下提出建议，而不是把会议拉到你的范围内。
- **不要试图展示价值**。一些资深人士觉得他们需要对所有事情发表意见，以证明他们的资历。另一些人则要求每个决定都必须完全反映他们曾经做出的类似决定。这两种情况都以对影响的不安全感为中心，并阻止其他人成长为领导者
- **改变你的想法**。尊重同事的最大表现之一是倾听他们的意见，然后在事后改变你的想法。如果高层领导不改变他们的想法，那么很快每个人都会将虚张声势与成功联系起来。

让人们参与你所做的决定，并分享你的决策方法，是培养你周围团队的重要组成部分，但做出决定又如何呢？*他们的？*

赞助

通过让人们参与你的讨论和决定，你让他们参与到你的工作中。这是一个成长、参与和向周围人学习的好方法，但在某个时候，你必须采取下一步行动。

不要让他们参与你的工作，而是让工作成为他们的。

这最后一步是**赞助**为那些让你晋升到 Staff-plus 角色的那种工作寻找其他人。当重要的工作交给你时，你的第一个问题应该是，“谁能胜任这项工作，并从中获得成长？”看看你是否能让他们领导这项工作，然后与他们合作，为他们的成功搭建项目。你的方法会是什么？他们可能需要考虑哪些初步的担忧？他们应该尽早与哪些利益相关者讨论这个问题？

当你发现新的重要工作时，比如发现你的工具或流程中的一个缺口，想想还有谁可以生成这项工作，然后和他们坐下来，让他们把你计划写的提案整理出来。然后像支持你自己的提案一样，为他们的提案建立支持。

重要的是，当工作变成他们的时，你必须让它成为他们的。提供建议、给出意见、提供背景信息，但最终赞助包括让他们采取你不会采取的方法。它最终可能会变得很糟糕，他们会从中吸取教训——就像你从职业生涯中的错误中吸取教训一样。它最终可能会变得非常好，然后你反而会学到一些东西。

虽然赞助应该成为你解决问题的默认方法，但不应该不是你唯一的工具。大多数 Staff-plus 工程师发现，直接参与一些项目对于保持他们对软件、工具和组织如何在实践中运作的了解非常重要。如果你需要一个经验法则，可以写一份赞助日记，确保你每个月至少赞助别人几次——如果你发现自己赞助的频率低于这个频率，那就深入挖掘是什么阻止了你。

相反，如果你回顾过去，想不出过去几个月你直接参与过什么工作，那也值得纠正。

如果你不这样做会怎么样？

如果你通过成为关键公司领导的“首选人物”巩固了 Staff-plus 角色的最后一块垫脚石，那么你就已经了解到，为组织领导解决一个紧急问题是获得认可的最可靠途径之一。如果你已经成为技术远见者，你的想法渗透到公司的架构路线图中，那么你就已经了解到掌控公司技术未来的大门是多么强大。

很难放弃这些。

然而，这种模式的最佳情况是一家[暂时蓬勃发展，直到那个人离开](#)。更为常见的最糟糕的情况是一家公司受到你个人局限性的限制，而唯一能容忍被你限制的公司是一家不会成长的公司。

要想长期保持一家真正成功的公司的领导地位，唯一的方法就是不断为他人创造空间，让他们获得认可、奖励和工作，而这些成就了你现在的地位。这可能会让人感到非常不舒服，但别担心：反正总会有新的工作等着你。

建立同行网络

当我与越来越多的Staff-plus工程师交流职业建议时，最一致的建议是建立一个从事类似工作的同行个人网络。不是每个人强调这种方法，但超过一半的人提到了它，对于那些提到它的人来说，这往往是他们的第一个也是最强烈的建议。

[里图·文森特](#)说，

对我来说，最有影响力的是拥有很多我认为是导师的人，通常是朋友、前任经理和与我共事过的人。我与过去与我共事过、了解我并且我信任的人进行相当数量的每月定期午餐、咖啡聊天和晚餐。正是这些关于职业挑战和成长的对话让我达到了我职业生涯的现在的位置。

[基维·麦克明](#)提到她的网络是获得诚实反馈的重要途径，

我首先想到的是找到你的同伴或支持网络。就像管理一样，你爬得越高就越孤独，找到仍然会挑战你，你可以一起集思广益的同伴非常重要。即使他们在你类似的工作领域，甚至在不同的公司，也没关系。

[纳尔逊·埃尔哈格](#)同样分享道，

对我来说，培养一个良好的高级工程师个人网络也非常有价值。我会和他们非正式地聊聊我们正在做的事情和思考的事情。当你有人脉关系时，你可以非常坦诚地了解人们正在看到的问题以及他们正在考虑的解决方案。

虽然知道你应该建立一个网络很有帮助，但有些人很难弄清楚如何做。在各种[建立你的人脉网络的策略](#)，最常见的两种策略是：容易被找到和内部建立人脉。

保持可见

Staff-plus工程师对社区的潜在需求非常大，因此建立人脉最简单的方法就是让自己容易被找到，成为一名Staff-plus工程师。一种有效的方法是参与围绕Staff-plus工程本身的讨论，比如Joy Ebert's的[一名高级Staff软件工程师实际上做什么](#)或者Keavy McMinn的[在技术领导力道路上茁壮成长](#)。虽然有很多人[写了他们对 Staff-plus 角色的看法](#)，但每个人都带来了新的、有价值的视角。这个话题还有你发挥的空间。

如果写作不是你的菜，那么还有你发声的空间，在技术会议上发言是另一种在更广泛的社区中提高知名度的有效方式。[基维·麦克明](#)描述了她参加会议发言的动机，

主要是，我喜欢在会议上遇到的人。后来，演讲者网络为我带来了工作机会。

如果这些都让你觉得风险很高，那么即使是创建一个 Twitter 帐户或加入几个相关的 Slacks（例如，[#staff-principal-engineering](#)在[Rands Leadership Slack](#)）可能是一个好的开始。

内部网络也是如此

与其专注于公开演讲和写作，[凯蒂·赛勒-米勒](#)的建议是在你当前的公司内部建立你的内部网络，

建立人脉，建立人脉，建立人脉，建立人脉.....你必须真正意识到你在和谁说话，并确保你在多个团队和多个小组之间建立联系，以利用这些网络。

虽然很容易认为建立人脉只发生在外部，但通常在你已经所在的公司更容易做到，在那里，它会在你的工作过程中半有机地、半有意识地发生。这种方法还有一个额外的好处，那就是直接改善你的日常工作。从长远来看，这些人最终会离开并遍布整个行业，从而引导你更广泛的网络。这很有效真的当你是一家相当大或有声望的公司时，效果很好，而当你的公司规模变小或声望降低时，效果会稍差一些。

环境网络

在没有提到发展个人网络的人中，大多数人提到创建一个基于与行业书籍保持同步并在社交网络（尤其是 Twitter）上关注行业领导者的环境学习网络。

[黛安娜·波亚尔](#)的评论是

我广泛使用 Twitter，但我主要是一个消费者，关注很多科技领域的人。我通常关注那些我在会议上见过演讲的人，或者我合作过的人，我发现他们的内容与我相关。这里有几个，排名不分先后：[卡米尔·富尼埃 \(Camille Fournier\)](#)，[拉拉·霍根 \(Lara Hogan\)](#)，[乔希·威尔斯 \(Josh Wills\)](#)，[维姬·博伊基斯 \(Vicki Boykis\)](#)，[大卫·加斯卡 \(David Gasca\)](#)，[朱莉娅·格蕾丝 \(Julia Grace\)](#)，[霍尔顿·卡劳 \(Holden Karau\)](#)，[约翰·奥尔斯帕 \(John Allspaw\)](#)，[慈善·梅杰斯 \(Charity Majors\)](#)，[西奥·施洛斯纳格尔 \(Theo Schlossnagle\)](#)，[杰西卡·乔伊·克尔](#)，[莎拉·卡坦扎罗](#)，[橙皮书](#)。

[达米安·申克尔曼](#)提到，

我尽量关注推特上那些我认为在做有趣事情的人，以及我可以向他们学习的人。有很多人在做有趣的事情，也有很多东西要学习！我想到的名字有：[包括[Aphyr](#)，[Tanya Reilly](#)，和[大卫·福勒](#)]

如果以这种方式建立人脉的想法让你感到不舒服，那么建立一个环境人脉可能是一个朝着正确方向迈出的良好开端。也就是说，你会发现个人人脉更有影响力，找到一种真实的方式来建立人脉，是在你的职业生涯中长期保持影响力的重要一步。

质量胜于数量

一位同事曾经告诉我一个故事，有人决心在业务拓展领域扬名立万，他们会带着一份想要见面的人的名单，从旧金山飞到纽约。他们会寻找那些人当晚可能出现的推文和 Foursquare 签到，然后去那里，买一杯饮料，假装偶然遇到他们。如果运气好的话，他们会尝试以这种方式结识六个或更多的新人。

不用说，你不应该这样做——这完全是对界限的侵犯。此外，这样做甚至没有意义：在建立同行网络时，数量并不重要。相反，专注于与你真正信任、尊重并受其启发的人慢慢建立联系。这才是真正强大的网络，可以帮助你解决最困难的问题和最棘手的情况。

最后，如果你已经读到这一段，并且真的想建立一个网络，但又不确定如何开始，我将分享对我这个内向的人来说有效的方法，我一直在努力寻找一种真实的方法。找到你尊重的人，给他们发一封简短的 1-2 段的电子邮件或私信，提出一个具体的建议请求问题。如果他们回复了，感谢他们，并在六到十二个月后发送另一个问题。如果他们随后向你寻求帮助或问题，尽你所能提供帮助。如果他们没有回复，不要担心；直接离开，不要发表评论。这种方法出奇地有效，最糟糕的情况也完全可以接受：他们只是永远不会回复。

向高管展示

你是否曾向公司高管介绍过一项关键的工程计划，兴致勃勃地走进房间，却沮丧地离开？也许你只讲到第二张幻灯片，就被无关的问题打断了讨论。也许你完成了整个演示，却只听到人们说“干得好”，然后就离开了，没有任何有用的讨论。事后，你不太确定发生了什么，但你知道情况不太妙。

在你的职业生涯早期，你可能不会经常与公司高管互动。当然，如果公司足够小，你可能会，但这并不是常态。然而，随着你职业生涯的深入，你的影响力将越来越受到你有效影响高管能力的影响。虽然[与权威保持一致](#)是影响高管的先决条件，但你还需要培养一些新的沟通技巧。

为什么这很难

每个人在他们的职业生涯中都与一个糟糕的高管共事过，但大多数高管并不糟糕。几乎所有的高管都很出色一些事情；只是经常这样一些事情不是你和他们交流的主题。当你把对你领域的不熟悉与手头话题的有限时间结合起来时，沟通就成了一个挑战。

不过，这些都是常见的沟通挑战，而与高管沟通可能会出乎意料地困难，原因不太明显：高管已经习惯于以一种特定的方式来消费经过预处理的现实。

任何一位高管几乎总是非常擅长一种消费信息的方式。他们觉得最舒服以这种特定的方式消费数据，并且围绕他们的沟通系统经过优化，以这种方式与他们沟通。我认为这是一种预处理现实的方式，而对于给定的高管来说，以错误的方式预处理信息，经常会造成双方都无法解释的沟通不畅。

例如，一些高管在模式匹配方面有着非凡的天赋。他们在任何演示中的第一反应是提出一系列详细的、看似随机的问题，直到他们可以根

据自己之前的经验进行模式匹配。如果你试图向这位高管做一个结构化的、学术性的演示，他们会感到无聊，你也会浪费大部分时间来展示他们不会消费的信息。其他高管会无视你说的任何没有与特定数据或数据集相关联的内容。你会充满信心地进行演示，因为你知道你的数据在附录中，但他们会越来越不相信你的提议是未经支持的。

在大多数其他情况下，沟通不畅会产生延迟而不是错误。不过，当你与高管沟通时，你通常不会有第二次机会在做出相关决定之前讨论给定的主题。在讨论之前进行投资，以避免事后的哀叹。

如何有效沟通

与高管有效沟通的基础是清楚地了解你首先为什么要与他们沟通。你可能习惯于与人们沟通以改变他们的想法或告知他们关于你的项目，但这里可能不是这种情况。当你与高管沟通时，几乎总是以下三种情况之一：计划、报告状态或解决不一致。

虽然这些是不同的活动，但你的目标始终是从高管那里尽可能多地获取观点。如果你进入会议是为了改变他们的想法，你可能会显得不够灵活。进入会议是为了了解如何与他们的优先事项保持一致。你会显得很有策略性，并且可能会带着足够的信息离开，以便调整你现有的计划，使其在高管新提出的重点或约束范围内发挥作用。

提取他们观点的最佳方法是编写结构化文档。写作迫使你全面思考你的信念和数据。结构确保你让读者专注于重要内容。芭芭拉·明托（Barbara Minto），她的[《金字塔原理》](#)是关于有效商业沟通的最有影响力的著作，她也是结构的忠实粉丝：

控制你呈现想法的顺序是清晰写作所必需的最重要的行为。最清晰的顺序总是在给出被总结的各个想法之前给出总结性的想法。我再怎么强调这一点也不为过。

有很多结构可以奏效，但我特别建议每个文档的开头段落都遵循SCQA格式：

- **情境 (Situation)**：相关的背景是什么？例子：在产品功能发布方面，我们已经落后竞争对手两年了。去年，我们的工程团队增加了一倍，但发布的功能却比前一年少。
- **难题**：为什么目前的情况存在问题？例子：我们计划今年再次将工程团队扩大一倍，但根据去年的经验，我们认为这会进一步降低速度，同时大幅增加我们的组织预算。
- **问题**：要解决的核心问题是什么？示例：我们是否应该继续推进今年将工程团队扩大一倍的计划？
- **答案**：你对提出的问题最好的答案是什么？例如：我们应该停止在未来六个月内招聘，专注于凝聚我们现有的团队。根据当时的进展，我们应该更新今年剩余时间的招聘计划。

在许多讨论中，一个结构良好的开篇段落足以引发重要的对话。虽然在这些情况下，您可能不会讨论文档的其余部分，但编写文档的过程仍然是提炼您思想的重要一步。

相对较少的人为其整个文档采用正式结构，但至少有一种流行的格式，有些人认为它很有价值：来自上述书中的明托金字塔原理 (Minto's Pyramid Principle)。首先，将您的提案集思广益成一系列支持您答案的论点。一旦你把它们都写下来，就把它们分成相关的论点。将这些组塑造成三个顶级论点，每个顶级论点最多有三个子论点支持。递归地应用这种方法，确保每个论点总结其最多三个子论点。按重要性降序排列每个组中的论点。到那时，你就完成了。

虽然我个人觉得SCQA立即有用，但我承认，当我第一次尝试遵循金字塔原理时，它给我的情感反应与盯着粗野主义建筑一样。通过实践，它在我身上成长，但我仍然建议大多数人首先采用SCQA作为核心实践，并且只有在您收到反馈说您的演示文稿难以理解时才采用整个金字塔原理。

在您编写结构化文档后，从您的同事和利益相关者那里收集有关它的反馈。在演示之前与利益相关者保持一致，有时称为“[nemawashi](#)”，在减少意外方面非常有效。您的一些同事应该有向高管演示的经验，并且会对改进提出有用的反馈。

对于演示本身，设置一个明确的议程，但不要专注于死记硬背。与执行领导的精彩会议的定义是参与讨论，而不是解决议程上的每个主题。有些人会认为这是一个有争议的立场，更喜欢通过其行动项目来衡量每次会议，但这忽略了这些会议中通常更有价值的关系建立和发展方面。

要避免的错误

即使您在为执行演示做准备方面做得很好，这些事情有时也会出错。没有什么可以避免每条坏路，但您可以避免大多数经常导致这些会议失败的反模式。

永远不要反驳反馈对于一位高管来说，拥有重要的反馈意见，但却无法在当下找到合适的框架来传达是很常见的。你希望他们无论如何都要给出反馈，而不是憋在心里，可能之后就忘了。如果你表现出对反馈的抵触，他们就会开始吞下他们的评论，而你从会议中获得的相对较少。专注于收集反馈；不要担心你是否同意，直到你之后有更多的时间。如果有一个需要做出的决定，你不同意，那么你应该注入一两个可能改变他们想法的相关数据，但之后，就放手吧。通过反思反馈并在之后改变他们的想法，比在会议中继续反驳更有效。

不要逃避责任或问题。很多人试图向他们的领导隐瞒问题，而这总是会变得很糟糕。成功的人将告知高管视为一种解脱：一旦问题摆在桌面上，你就可以朝着解决它而不是隐藏它前进。如果一位高管在会议中嗅出了问题，这一点尤其正确。接受反馈，不要逃避它。通过同意他们的观点并在之后提供更多数据，你将建立更多的信誉。通过与他们争论，你将损害你的信誉。

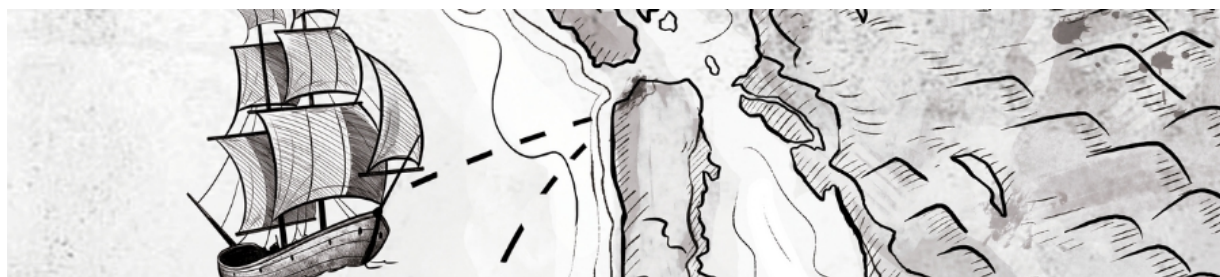
不要提出没有答案的问题。给新领导人的一条常见建议是“永远不要给你的经理带来没有解决方案的问题”。这通常不是一个好建议，但如果你向一位高管提出一个没有提议答案的问题，那么在他们的脑海里，他们会想是否需要聘请一位更高级的领导来补充或取代你。除非你有一个让人们可以团结起来的提议，否则你无法在房间里达成一致。

避免学术风格的演示。你在学校里被教导的关于主题的演示方式或多或少是向高管演示的完全错误的方法。如果你遵循它的教义，*明托金字塔原理 (Minto Pyramid Principle)*将引导你朝着正确的方向前进。

不要执着于你偏好的结果。人们常常过于关注自己想要的结果，以至于把精力都花在抵制那些清晰、不可避免的迹象上，这些迹象表明事情不会那样发生。对于做出“错误”的决定感到沮丧是很常见的，但记住你错过了大量的背景信息是有帮助的。没有什么是一成不变的决定：几乎每一个决定在未来两年内都会被多次重新考虑。

向高管做演示可能会让人感到害怕，而且这可能更多的是建议而不是帮助。如果你想把它归结为一个简洁的建议：将早期草稿发送给参加会议的高管，并询问他们需要更改什么。如果你听取并应用这些反馈，你将在进行过程中弄清楚其他部分。

在你所在的地方获得头衔



我听过的最好的建议是，通常达到Staff级别是运气、时机和工作的结合。 - [伯特·范](#)

大多数科技公司都有一个“职业级别”，旨在成为大多数人达到的最高级别。高级工程师是大多数公司的职业级别。虽然你可能会因为没有足够快地从入门级工程师晋升到中级工程师而被解雇，但大多数公司并不期望你从高级工程师晋升到Staff级别。在中级工程师岗位六年？啊，那是个问题。在高级工程师岗位二十年？当然，没问题。

除了对进步的期望消失之外，公司的晋升制度通常会阻碍你达到职业级别后的进一步发展。有时，已经拥有Staff工程师头衔的人会保护自己的声望不被稀释。在其他情况下，由于团队健康或预算方面的考虑，组织可能不愿在单个团队中拥有多个Staff工程师。但是，我认为最强的摩擦来源是工作性质的变化。Staff工程师不是更好的高级工程师，而是已经开始履行以下职责的人之一[Staff 原型](#)。

即使在你已经发展了成为一名Staff工程师的先决条件技能之后，仍然会有最后一个障碍：让你的公司授予你Staff头衔。对于某些人来说，这个过程相对来说不是什么大事，可能比预期的时间长一两个周期，但最终会成功，而对于另一些人来说，这可能根本不会在他们目前的公司发生。我调查的Staff工程师中，约有三分之二是在他们已经工作的公司晋升获得的头衔，其余三分之一则通过更换公司来获得头衔。

如果追求那种角色是你的目标，那么将晋升到你的职业级别视为一个重置你职业生涯导航方式的机会。从那时起，就没有标准的道路可循。晋升和绩效系统将不再围绕着获得及时的晋升而设计，有时会让人感觉像是在把关。

要更进一步，你将不得不更加有意识地控制你的晋升，本章将分享那些在你的晋升道路上先行一步的人所使用的工具。

找到你的道路

如果你一直依赖你的经理来指导你的职业生涯，那么向自我导向型职业的转变可能会感觉相当突然。有很多关于[管理你的软件职业生涯](#)的书籍，但大多数都关注从你的第一份工作到你成为高级工程师。很少有关关注管理你超越高级职称的职业生涯，这正是本章的重点：

- 你的[晋升材料](#)是你揭开Staff晋升神秘面纱、优先考虑正确的个人发展以确保你实现目标，并激活你的内部支持者和网络以支持你的晋升的基础工具。
- 人们普遍认为，要进入Staff-plus角色，需要成功完成一个[Staff项目](#)。本节讨论了大多数 Staff 工程师所面临的现实不要有一个 Staff 项目，但也描述了如果你在一家需要Staff项目的公司，该如何处理
- 工程师们经常抱怨他们没有“参与”决策，而且他们通常是对的：确实有一个房间，而他们不在里面。但较少被承认的是，你可能没有理由进入那个房间。本节描述了[如何进入房间，以及如何留在那里](#)
- 最后，如果你的公司的领导不知道你是谁，你就不会得到晋升。你如何[在内部变得可见](#)而又不占用所有的资源？

持续应用这些技巧，你就会朝着Staff职称前进，尽管即使是最好的计划也会失败[如果你在一家错误的公司进行这些计划](#)。

机会是不均衡分布的

在追求Staff职位时，你会遇到的一个不方便的事实是，在任何给定的公司里，机会的分配都是不均衡的。如果你的公司领导认为基础设施工程本质上比产品工程“更复杂”或“更具杠杆作用”，那么机会将集中在基础设施团队中。如果你在一个强调发布功能的组织中工作，那么修复你导致的中断比防止未来的中断更容易获得奖励。如果你在公司总部工作，你的工作会比在“分布式办公室”更引人注目。[分布式办公室](#)。

许多公司认为，即使机会明显不均等，他们也有既得利益假装机会是均等分配的。这使得人们很难确信这些动态的存在，但随着你收集更多的数据，趋势会变得清晰。

一旦你认识到这些挑战，你就必须评估它们的可修复程度以及你想优先投入精力的地方。使你的方法与这些不成文的潮流保持一致，而不是重新引导创造它们的河流，要简单得多。如果你选择解决不平等的原因，首先要找到一位支持这项事业的高级赞助人。你只能在内部赞助的情况下改变一个系统。

你应该尝试管理吗？

大多数达到Staff-plus职位的人**不要**花时间在工程管理上，但有些人没有。很容易将此视为一个关键的、改变人生的决定，但这可能有点想多了。如果你想尝试管理，你应该尝试。大多数公司都明白，管理并不适合所有人，并且很乐意让你轮换回工程职位。

那些尝试管理的人会获得更广阔的视角，即使他们回到软件工程职位，也能帮助他们。这是[丹·娜](#)的经验，

我仍然喜欢发布代码和管理团队，我认为在高水平上同时做到这两者的能力对于长期的工程成功至关重要。Charity Majors在这方面有一篇很棒的博客文章，我建议阅读：[“工程师/经理的钟摆”](#)。慈善机构认为，“经理职业道路与工程职业道路”是一种错误的二分法，花时间在两种角色之间交替可以让你在两种角色上都做得更好。这与我自己的经验相符。我之所以成为一个

更好的管理者，是因为我知道在一个计划不周的项目中担任个人贡献者有多么糟糕，而我之所以成为一个更好的个人贡献者，是因为我知道如何在项目进展不顺利时发出警报。

[里图·文森特](#)分享了类似的观点，

我做[钟摆](#)相当多，因为我对职业阶梯两边的很多事情都感兴趣。我对培养人才感兴趣，我真的很喜欢与招聘部门合作，我是那些真正喜欢面试的工程师之一，我喜欢了解团队是如何成长的。但我也真的很喜欢编写代码，在花了一些时间管理之后，我想回到代码中，稍微研究一下。

有些人尝试管理，最终却讨厌它。[乔伊·艾伯特](#)不太喜欢工程管理，

实际上，我在Box工作期间管理了大约一年半的时间，发现我讨厌它（你可以在我的[关于这个话题的博客文章中找到更多信息](#)）。也就是说，我发现在大多数公司里，管理层和员工+的角色之间实际上有很多重叠。

即使乔伊讨厌她的管理经验，她也觉得这可能对她的长期职业生涯有所帮助，

如果我没有在管理层兜圈子，我可能早就成为员工了。也就是说，我不后悔这样做，我学到了很多关于人们如何思考、组织如何运作以及大型项目如何优先排序的知识。所有这些都继续帮助我完成了IC轨道上的工作，并可能帮助我进一步晋升为高级员工。虽然我确实认为当我到达员工级别时，速度明显放慢了，但实际上我对下一个级别不太确定——我认为如果没有它，我很有可能会在员工级别停留更长时间。所有这些都说明，即使我没有采取最直接的路线，我仍然学到了很多对我的长期发展有帮助的东西。

对于考虑这种转变的人，我给出的最后一个警告是，人员管理不仅仅是最大限度地提高你成为一名员工工程师的轨迹。作为一名经理，你

将对你所支持的人产生深刻的影响，如果你带着错误的动机去做这件事，你会后悔这段经历，但远不及你的团队那么后悔。如果你有动力帮助你的团队成长和成功，那就去做吧；如果你只是为了自己而做，那就别做。

半渗透边界

作为最后的警告，Staff-plus头衔是领导职位。如果现有的领导团队不认为你是潜在的成员，那么获得领导职位是一项独特的挑战。这意味着，不幸的是，那些有特权看起来像他们已经是现有领导团队的一员的人，更容易完成过渡。

如果你读完本章后，越来越感到沮丧，因为你已经在这里做所有的事情，那么你可能正在经历这种结构性的劣势。在我采访的女性中，大约有一半不得不更换公司才能获得Staff职称，而在与男性讨论时，晋升摩擦的出现频率较低。

不要忽视这些经历——它们是真实的，很多人都感到受阻——但也要抱有希望，无论你的身份如何，无论你想如何规划你成为Staff工程师的道路，都有很多成功的榜样。

晋升材料

有些人认为他们的晋升材料是达到Staff+角色的顶峰，但我看到很多人通过采取相反的方法取得了成功：早在他们认为自己可能晋升为Staff之前就开始撰写他们的第一份Staff晋升材料，就像他们可能会使用一个[自我吹嘘文件](#)。以这种方式使用，你的材料就成了实现你目标的地图。

你的公司很可能会有自己的晋升材料格式，并且最终你需要将你的材料翻译成那种格式，然后才能将其提交给内部晋升委员会或流程，但没有必要着急。你将花费更多的时间依靠它作为指南，而不是作为正式审查的正式文件，因此要为前者进行优化。

对于走向你的Staff+晋升，一个有用的通用模板格式是：

- 你的员工项目是什么？你做了什么？该项目的影​​响是什么（包括一个[明确定义的目标](#)）？是什么让这个项目变得复杂？请简短说明，然后链接到支持性设计文档
- 你通过哪些高杠杆的方式改进了组织？
- 你的项目有哪些可量化的影响？（你增加了 1000 万美元的收入吗？你将同比客户支持票据减少了 20% 吗？）
- 你指导过谁，通过哪些成就？
- 什么[粘合工作](#)你为这个组织做什么？这种粘合工作的影响是什么？
- 哪些团队和领导熟悉并支持你的工作？他们看重你工作的哪些方面？一句话概括，如果可能，请包含数据（例如调查数据）。
- 你是否存在实际或感知到的技能或行为差距？可能会阻碍你？对于每一个，你将如何解决这个问题？每句一句话

花一些时间自己写出这些答案是有用的，但晋升到领导岗位并不是一项单人活动——这是你只能在团队成员的支持下才能完成的事情。

我推荐的迭代你的材料的方法是：

1. **回答你为什么要做这件事**许多人选择不追求Staff级别；你应该有一个理由说明为什么这对你很重要。如果你没有，你很可能会发现自己身处一个你不喜欢的角色。

[Michelle Bu警告说](#)，“我对工程师的第一个建议是，他们应该避免以导致他们从事不喜欢的工作的方式进行模式匹配。我对我所做的工作充满活力，与团队合作解决抽象建模和设计问题。经过多轮反馈后，一次又一次地尝试需要一定的毅力。说实话，这并不适合所有人。如果你更专注于达到Staff级别，而不是让自己从事能让你充满活力的工作，那么很容易最终陷入一个你不想要的角色。”

2. **控制你的期望。**晋升，尤其是在这个级别，是经过几个季度、半年甚至几年的努力积累而成的。避免期望立即见效
3. **让你的经理参与进来。**带上你的晋升材料在与你的经理的下一次1:1会议中，告诉他们获得Staff晋升是你的目标。与他们一起审查空白包，并询问他们缺少什么，应该强调什么，以及他们是否建议在工作流程中添加步骤。你的目标是确保他们知道你对此感兴趣，并征求他们对你方法的指导。

[Ritu Vincent 建议](#)，“人们经常来问我，‘我下一步应该做什么才能达到Staff？’我告诉他们的一件事是，对你的经理坦诚地表达你对职业生涯的期望。我早期在1对1会议中犯的一个错误是，告诉我的经理我认为他们想听什么，而不是我实际的感受。”

4. **写下晋升材料。**花一个小时坐下来，把你的所有想法放在第一个版本中
5. **编辑晋升材料。**等两天，重读你的晋升材料并编辑内容、清晰度和上下文
6. **编辑晋升材料与同行。**分享你的晋升材料与几位值得信赖的同事交流以获得反馈，最好是已经担任Staff-plus角色的同事。同事

通常比你更擅长识别你的优势和贡献，而且他们比你的经理更了解你的工作

7. **编辑晋升材料与你的经理沟通。** 分享你的晋升材料与你的经理沟通并请求反馈。特别要求列出需要弥补的差距。询问是否可以在接下来的 1:1 会议中花时间讨论哪些项目和机会既能弥补差距，又能使晋升材料更具说服力
8. **定期审查晋升材料与你的经理沟通。** 在你的职业生涯和以绩效为导向的 1:1 会议中，继续与你的经理一起审查晋升材料。你和你的经理都应该利用它来引导你随着时间的推移展示晋升标准。如果你的直接经理发生变化，这一点尤其重要。维护这类文件并在不同的经理之间进行审查，将有助于减轻经理变更后经常发生的晋升进度损失经理变更

如果你有条不紊地遵循这些建议，那么你将在被提名晋升为 Staff 之前很久就准备好你的第一份 Staff 晋升材料。从那时起，你将利用这份材料来集中你的注意力和与经理的合作关系，朝着这个目标前进。它不一定能让你快速到达那里，甚至可能无法让你在你目前的公司实现，但它会将你的精力集中在能够推动你朝着目标前进的发展和工作上。

当最终到了撰写正式材料的时候，这将是一个将你收集到的内容编辑成官方模板的问题，而不是一个整理多年努力的归档过程。希望一切[在晋升过程中不出差错](#)，然后获得 Staff 头衔。

找到你的赞助人

拥有赞助人也绝对重要。我和我的经理关系非常好，我和我的越级经理关系也很好。我认为这在其中也发挥了很大的作用。

- [里图·文森特](#)

正如我和更多试图达到他们第一个 Staff-plus 职位的人交谈时，大多数人遇到了类似的挑战。许多人错误地评估了自己的影响力，并且还没有完成在该级别上运作的工作：一名 Staff 工程师不仅仅是一名更快的 Senior 工程师。然而，有一大批人已经完成了这项工作——他们[在他们的组织中可见](#)并且已经汇集了一份[强有力的晋升材料](#)——但仍然在努力使这项工作得到认可。

这些人常常对他们的影响力和他们所获得的认可之间的差距感到沮丧，并向他们的经理和同事寻求关于弥合这一差距的反馈。他们被告知[完成一个 staff 项目](#)或者[为他人创造空间](#)。对于那些尚未完成工作的人来说，这是一个很好的建议，但对于那些已经完成这些复选框的人来说，这是一种干扰：他们真正缺少的是一个愿意推动对其现有工作认可的发起人。

人们通常会通过我们一生中评估过我们的其他系统（如学校）的视角来看待晋升系统，但这错误地将绩效评估定义为一项单独的活动。无论你的公司是否进行临时晋升或[使用校准过程](#)，晋升是一项团队活动，并且[朱莉娅·格蕾丝 \(Julia Grace\)](#)，然后是 Slack 的，曾经在一次求职过程中建议我，“不要独自玩团队游戏，你会输的。”

找到你的赞助者

指导你晋升的团队中最重要的成员是你自己。第二重要的人是你的组织赞助者。Lara Hogan 已经[详细撰写了关于赞助的文章](#)，但大致来说，这个人会在有影响力的论坛上为你的工作说话，并在倡导有限的资源（如加薪预算）时为你辩护。

虽然你可能会有各种各样的赞助者，但在晋升的背景下——尤其是晋升到 Staff-plus 职位——这几乎总是需要你的直接经理。他们会将你起草的[晋升材料](#)并将其转换为公司的格式。他们将是校准会议期间倡导你晋升的人，因为其他人会深入研究你的资格。他们也将是必须与你进行诚实对话的人，告诉你你在成为强大的晋升候选人之前仍然存在的差距。

虽然你总是需要你的直接经理作为你的赞助者，但你可能需要额外的赞助。如果你的经理以前从未将任何人晋升到 Staff-plus 职位，他们很可能会感到惊讶或在此过程中犯错。投入精力与你的管理链上更高级建立关系。你不需要花太多时间与你的越级经理相处，但如果他们对你的工作影响不够熟悉，以至于两个月后的会议上都记不起来，那么你不太可能晋升到 Staff 职位。

激活你的赞助者

激活你的赞助者的第一步是明确分享你的目标。“我希望被认可为 Staff 工程师”是一个好的开始。[里图·文森特](#)提到了这是她为寻求 Staff-plus 职位的人提供的最佳建议，

人们经常来问我，“我下一步该怎么做才能达到 Staff 级别？”我告诉他们的一件事是，对你的经理坦诚地表达你对你的职业生涯的期望。我早期在与经理一对一交流时犯的一个错误是，我告诉我的经理我认为他们想听什么，而不是我实际的感受。

一旦他们确定了他们的赞助者，许多人认为他们的工作已经完成：这取决于赞助者来完成繁重的工作。这通常会失败！赞助者是拥有比部署这些资本的带宽更多的组织资本的人，当你为他们协调好各个部分时，他们会最有效地帮助你。问问你的赞助者，你如何支持他们的赞助。拥有你的职业生涯不仅仅是要求东西。它是关于这一点，但更多的是促进这些事情的发生。

与你的赞助者协作审查你的晋升材料是促进这种对话的好方法。专注于询问差距是什么，不要促使你的赞助者编造答案。大多数人忘记了

他们可以用“我不知道”来回答问题，如果你强迫他们回答他们不确定的问题，他们反而会编造无益的答案。如果你一直得到像“从事更大、高影响力的技术项目”这样的答案，那么你提出的方式不对、问题不对或人不对。

一个起始提示是，“如果我这次没有得到晋升，可能的原因是什么？”另一个值得问的问题是，“我能做的最有效的事情是什么， чтобы 使自己成为一个更强的候选人？”也就是说，最好的问题是非常具体的，并且为回答者做了很多工作。想想回答这些问题有多难，相比于像“这个季度我完成了 API 重构，我认为这会证明 Staff 级别的工作，但时间表推迟了很多，最终让我们的产品经理感到沮丧，因为他们的工作被放弃了。我本可以更有效地处理这个项目吗？”这样的问题。后一个问题更容易给出有用的答案，即使回答者不太熟悉项目的细节。

最后，请记住，激活你的赞助者不是在晋升前做一次性的交易。随着时间的推移建立关系，并在他们需要你的支持时努力帮助他们。[与他们的倡议保持一致](#)假设他们需要人们加入一个工作组，自愿付出努力。这些人有很多事情需要别人帮忙，而且他们很清楚哪些人会在晋升前夕才出现。我曾经有一个同事很少来办公室，但是总是在晋升决定前一周来办公室。人们注意到了。

如果这不起作用怎么办？

如果你发现自己和你经理相处不融洽，这和互相喜欢彼此不太一样，那么你不会被提升到领导岗位。你的经理对你的影响以及你所认为的影响有太直接的影响，以至于这种情况不会发生。同样，你可能和你的经理关系很好，然后他离开了公司。你几乎没有注定的，但你的晋升时钟可能会在你与新经理建立关系时重置。（有时，情况正好相反，你的新经理会努力通过代表你来向你证明自己。）

如果你在与经理发生摩擦后立即尝试更换团队或公司，你会欺骗自己。公司通常不允许调动，除非你的经理批准，所以你可能会烧毁一座你站在上面的无名桥。更重要的是，你将失去培养与你没有立即产

生共鸣的人一起工作的技能的机会：这不是一项有趣的技能，但领导力总是包括影响和建立与那些有冲突的目标和风格的人的关系。

如果你已经花了六个月积极地试图让这段关系发挥作用，那么现在可能是时候考虑换团队，也许可以[考虑换公司](#)。在许多情况下，与你的越级经理建立关系非常有帮助，即使你和你的经理没有有效地合作，他也可以帮助你找到一个新的团队。

员工项目

没有明确的期望，也没有任何地方将其列为正式要求，但人们普遍认为，你将完成一个Staff项目才能获得晋升。我想不出任何没有包含一个非常强大的项目的Staff晋升，通常是一个多人项目，工程师是技术负责人。 - [里图·文森特](#)

关于达到Staff-plus角色的一个流行的反复出现的想法是，首先，你需要成功完成一个“Staff项目”。这是一个被认为足够复杂和重要的项目，完成它的人已经证明了自己是一名Staff工程师。然而，无论这个想法多么流行，如果你正在追求一个Staff-plus角色，重要的是要刺穿这些项目中的神话，并专注于那些在你之前走过这条路的人的经验。

关于Staff项目的简短回答是，大多数工程师并没有完成一个作为达到Staff角色的一部分，尽管有很大一部分人确实完成了一个，特别是那些通过在他们成长起来的公司晋升而获得该角色的人。对于那些没有完成一个项目的人来说，通常，要么是因为他们在较长一段时间内积累了成功的记录，而没有一个单一的顶点，要么是因为他们换了公司来获得这个头衔。

我们将深入研究关于staff项目的几个不同角度：

1. 没有完成Staff项目的人
2. 完成了Staff项目的人，包括他们最终没有按计划工作的情况
3. 识别和处理你的员工项目

我们深入探讨一下这些混乱的细节。

不需要 Staff 项目

当我问人们是否有一个 Staff 项目时，一些回答非常简洁：

- [乔伊·艾伯特](#), “我实际上并没有一个 Staff 项目。”

- [黛安娜·波亚尔](#)，“不，我没有被分配‘Staff 项目’，而且这也不是 Slack 晋升过程的一部分。”

有些人甚至对 Staff 项目的概念持怀疑态度。[纳尔逊·埃尔哈格](#)说，

我本能地对这种 Staff 项目的想法有点警惕，部分原因是，我所见过的 Staff 工程师的典型形象是，他们不一定自己运行大型项目或做大事。但他们只是非常有效的专家和路由器，使整个工程组织运行得更好。

还有一些人像[丹·娜](#)或者[达米安·申克尔曼](#)他们绕道工程管理才达到这个职位。Damian 描述了绕过 Staff 项目的情况，

我没有。因为我在 Auth0 的成长方式，我有点“跳过了那部分”。作为一家初创公司的总监，我有机会在技术上领导许多大型、关键的举措，但没有具体/明确的“staff/principal 项目”。

从这些故事中可以清楚地看出，任何告诉你“你”的人必须完成一个 Staff 项目以达到 Staff-plus 职称是错误的：有很多途径可以达到 Staff-plus 职称，而无需进行 Staff 项目，[在工程管理方面的工作](#)其中最突出。

员工项目要求

然而，许多公司确实需要或非正式地强制执行员工项目以进行内部晋升，因此，许多人确实将员工项目作为其角色转变的一部分。

[里图·文森特](#)描述了她在 Dropbox 的经历，

我肯定有一个员工项目。早些时候，Dropbox 最初是一款消费者产品，人们下载并安装在自己的机器上。当我们推出 Dropbox for Business 时，有人要求您的个人和工作 Dropbox 帐户同时工作，包括能够在它们之间切换而无需注销并重新登录。最初的实现是在巨大的时间压力下编写的，它运行着多个

Dropbox 进程。一个用于您的个人帐户，另一个用于商业帐户。我的 Staff 项目是让单个 Dropbox 进程可以在多个用户登录的情况下运行。困难之处在于该项目从内核一直延伸到用户界面。我必须了解 Dropbox 系统的每一层。最初，我们认为这需要六个月的时间，但最终花了十八个月。这在很长一段时间内占用了桌面客户端团队的大部分资源。

[拉斯·卡萨·威廉姆斯](#)加入了一个飞行项目，后来他成为了该项目的负责人，该项目成为了他的 Staff 项目：

我以高级工程师的身份加入了 Mailchimp。我立即被加入到一个项目团队（包括一位工程总监和两位首席工程师），该团队旨在构建 Mailchimp 的第一个内部自助式分析平台。这个项目的关键方面是高效并在高水平上执行。不管好坏，拥有另外两位首席工程师意味着对我的期望可能不会那么高。但我能够立即加入并开始为项目的核心方面做出贡献，而他们几乎没有提供任何帮助；到最后，我成为了团队的主要贡献者之一。我最终会被正式任命为技术负责人，以帮助继续指导该项目的工作，因为它被吸收到我当前的工程组，数据服务中。

很少有公司会写下他们的 Staff 项目要求。它们更常见的是一种“软门槛”，在晋升会议期间被提出，有时会让经理和准 Staff 工程师都感到惊讶。发现这些要求最可靠的技术是你的“必胜”晋升未获批准，但这并不是什么乐趣。几乎同样可靠且不那么令人沮丧的是依靠以下策略：[维护并获得对您的晋升材料的反馈](#)在您尝试晋升之前很久。

为什么你应该做一个 Staff 项目

有时很难确定守门和评估之间的精确界限，而 Staff 项目的前提就存在于那个模糊的领域。承担一个范围巨大的项目，驾驭这种模糊性，并成功交付它，是区分已经达到 Staff-plus 影响的人的有效方法，但也很明显，许多人在没有完成这样的项目的情况下获得了 Staff-plus 角色。

我的建议是，尽管你可以在没有完成 Staff 项目的情况下获得 Staff-plus 角色，但它们是发展自己成为一名工程师的特别宝贵的机会。你将会就我个人而言通过这种项目得到拓展和成长这种方式是你通过其他类型的Staff级别工作无法获得的。

[基维·麦克明](#)描述了她的Staff项目如何帮助她，

我从未听说过它被命名，但我理解这个想法。我确实领导并架构过那种类型的项目——解决棘手的工程问题，对公司产生重大影响——几次，但不幸的是，它们并没有让我得到晋升。但它们确实促进了我的职业发展。这些项目给了我经验、知识和信心，让我能够以不同的方式定位自己。甚至可以公开会议演讲，或者知道“我做过X，可以再次做X。”

尽管每个项目都不同，但有一些典型的特征可以说明为什么它们能如此有效地拓展你作为一名工程师的能力：

- **复杂且模棱两可**- 在你职业生涯的早期，你会得到定义明确的问题，但随着你深入其中，你会越来越多地遇到定义不清或未定义的问题，而Staff项目通常会从范围界定不清但复杂且重要的问题开始。你的项目可能仅仅从断言你公司老化的单体架构正在阻碍产品开发开始。从这个宽泛、不明确（且可能错误）的陈述中，你必须确定一种可行的具体方法。
- **众多且分散的利益相关者**- 最简单的项目始于围绕问题和解决方案的组织协调，但您的 Staff 项目可能两者都没有。这可能是一个管理层认为具有生存风险的领域，但许多工程师认为它已经足够好了。这可能是一个每个人都认为存在问题，但在方法上存在强烈派系分歧的领域，例如，分歧在追求服务战略或重新投资于您现有的单体应用之间。
- **失败很重要的指定赌注**- 这将是一个足够重要的项目，高级领导会在全体组织或全体公司会议上谈论它。这意味着人们会密切关注您的工作，任何失败都会非常明显。成功也将非常引人注目！

如果您满足这些条件，这可能是一个 Staff 项目。这些项目可能会让人非常紧张，这也是它们在培养您方面如此有效的原因。

获得 Staff 项目的访问权限

虽然决定承担 Staff 项目是第一步，但您仍然需要获得访问权限这些项目，这取决于您的管理链是否足够信任您，愿意押注您的成功。

这归结为三个因素。

1. 首先是学习与你的领导团队保持一致，其中一些策略在以下文章中描述：[进入房间](#)和[与权威保持一致](#)。
2. 其次，你需要被认为具备解决当前问题的技术能力，这需要[保持可见性](#)。
3. 第三，这在你的控制范围之外，即你的公司迫切需要解决一个 Staff 级别的问题，这可能需要一些耐心。

你应该追求Staff项目吗？

总而言之，如果你希望在当前公司获得晋升，并且之前没有担任过 Staff 或管理职位，那么你可能需要追求一个 Staff 项目，以在该级别上确立自己的地位。在其他情况下，你可能不需要。

无论如何，值得记住的是，无论是否需要这些项目，它们也是你能找到的最具挑战性的工作之一，并且是能让你成长为一名更优秀的工程师的工作。在短期内追求头衔，避免这些项目可能是最佳选择，但在长期追求自我成长方面，它们是不可替代的。

进入房间，并留在那里

我从工程师那里听到的最常见的挫折之一是，他们没有参与重要决策的制定。他们不理解公司的决策，并且掌握着重要的背景信息，但这些信息似乎被遗漏或忽略了。高级工程师经常将进入“房间”视为他们级别的主要好处，而头衔确实会增加你参与影响你的决策的可能性。

然而，重要的是要记住，没有一个可以进入的“房间”。进入正确的房间不是一次性需要面对的挑战。进入房间将是一个持续的、迭代的职业挑战。这意味着值得擅长！

在你的职业生涯早期，它可能是一个与你的技术主管和产品经理进行的冲刺预计划会议。之后，它可能是一个季度计划会议，一个[架构审查](#)，[绩效校准](#)，工程领导团队，或执行团队。总会有另一个房间要进入。要达到高级水平，你不仅要擅长进入，还要擅长留在这些权力房间里。

进入房间

要进入房间，你需要：

- **为房间带来有用的东西.....**这可以是关于关键项目的详细信息、来自关键团队的背景信息、与房间目的相关的专业知识、在以前的公司运行类似项目或团队的经验、与关键相关客户的关系，或者完全不同的其他东西。
- **.....房间里还没有的东西。**仅仅拥有可以带到房间里的有用的东西是不够的。它还需要是一种房间里还没有的视角。小组比大组运作得更好，所以运营论坛通常为了效率而牺牲冗余和代表性。要被包括在这些房间里，你需要带来一些与当前成员不同的东西。
- **房间里的赞助人。**这些房间的空位有限，并且必须作为一个整体良好地运作。要进入房间，你需要有人赞助你的会员资格。你的赞助人正在将他们的社会资本分配给你的加入，他们的同伴会根

据你在房间里的行为来评判他们。这些房间通常混合了不同的资历级别，所以通常情况下，你的赞助人的经理会在房间里评估他们基于他们赞助的决定你。

- **你的赞助人需要知道你想在那里。**你的赞助人可能在许多不同的房间里，并且可能梦想着离开大多数会议。他们不一定会认为你想参加任何特定的会议，事实上可能会认为你根本不想在那里。确保他们知道你是否想被包括在内。

你如何为房间带来有用的东西将取决于你和你想进入的 room 的具体情况：没有任何单一的模式可以遵循。与你类似情况的人是否已经在房间里也取决于你的具体情况，在某些时候，唯一的选择是等待或寻找另一个可以进入的 room。

另一方面，有时增加你对房间价值的最简单方法是降低包括你的成本。以下是一些效果良好的方法：

- **与你的经理保持一致。**人们会根据领导者的团队与他们宣布的方法的一致程度来评估领导者。如果他们已经宣布转向持续部署，但他们的团队却在呼吁发布列车，那么人们就会对谁在领导产生怀疑谁。如果你与你的赞助人高度一致，你更有可能被赞助进入房间。如果你特别一致，他们更有可能把自己的座位让给你，停止参加。
- **为团队优化。**Stripe 过去的一项运营原则是“为 Stripe 优化”，这种为他人广泛优化的心态可以建立信任和对你判断的信心。
- **说话清晰简洁。**学会简洁地说话：随着你培养出精简的语言，你就能在更少的时间内贡献更多的想法。学会清晰地说话：如果人们不理解你的提议，那么它有多好都没关系。请记住，你有义务让别人理解，而不是其他所有人有义务理解你。
- **保持低摩擦。**很容易陷入这样的陷阱：把每次讨论都看作是阻止即将发生的灾难的最后机会。在这种心态下，每次讨论都接近紧急状态，情绪高涨。这类讨论通常会把时间花在排解挫折感上，而不是取得进展。如果你以能够有效地驾驭困难对话而闻名，你更有可能参与其中。

- **做好准备。**有些公司把他们的工程师当成小孩子，认为即使是非常资深的工程师也不会阅读议程、进行预读或为讨论做准备。容忍和奖励之间存在相当大的差距，如果你花时间在每次会议前整理你的想法，你就会脱颖而出。同样重要的是跟进你承诺的事情。
- **专注并保持临场感。**一旦你进入房间，一定要出现并参与。保持专注和投入。无论你还想做什么，它都会等待。
- **主动承担低地位的任务。**如果有人需要做笔记，请举手。如果有人需要跟进行动项目，请随时提供帮助。优先考虑发挥作用，尤其是在工作不是最令人兴奋的时候。

要进入房间，你必须同时处理分子和分母：不断发展独特而有用的视角，同时在会议的限制范围内更有效地传递这种视角。

留在房间里

进入房间是你的第一个障碍，但第二个障碍是留在房间里。最重要的是继续做那些让你进入房间的事情：将重要的背景信息带入房间，展示一个完美的自己，简洁，灵活。

有一些模式会一直让你被踢出房间：

- **误解房间的目的。**每个房间都有[它自己的目的](#)，如果你试图利用一个房间来对抗现有群体的意图，你就会产生摩擦。对于一个给定房间的功能的外部认知（“他们在领导团队会议上做出所有决定”）与房间如何看待其角色（“我们不做决定，只是提出问题进行讨论”）相差甚远，这是非常常见的。花时间了解房间的运作方式，并尊重这种意图融入其中。
- **固执己见。**随着房间变得更加高级，他们必须讨论非常敏感的话题（薪酬、裁员、晋升、收购等），并且他们每周都有固定的时间一起工作。如果你固执己见，你将产生摩擦，减慢讨论速度，并阻碍小组取得进展的能力。

- **拒绝同意。**高效的团队是由愿意[不同意并承诺 \(disagree and commit\)](#)的人组成的。你通常可以通过拒绝同意来迫使团队接受你的观点，直到想法朝着你的方向发展，但团队的步伐会慢下来，你可能会被排除在外。
- **吸走房间里的氧气。**有些头脑风暴讨论欢迎每一个想法，有些时候你已经转入操作模式来解除项目执行的障碍，你必须根据情况判断正在发生的是哪一种。通常，这来自于[一种展示价值的冲动](#)，但请记住，你之所以能进入这个房间，是因为你过去所做的事情，而不是希望进入这个房间能神奇地把你变成一个全新的人。
- **让你的赞助人难堪。**记住，你之所以能进入某个房间，是因为房间里有人主张接纳你。
- **做事不靠谱或不定期出现。**名额有限，会议负责人会优先考虑那些经常出现的人。

话虽如此，我认为很容易陷入过度担心留在房间里的状态。有时候，你最好考虑一下这个房间是否值得你投入时间。

退出房间

重要的是要记住，虽然有无限个房间可以进入，但没有一个房间是真正能完成工作的地方。如果你能有选择性地决定留在哪些房间里，你就能发挥最大的影响力。虽然我见过很多人因为不被允许进入他们执着于的某个房间而感到不满，但我从未见过有人后悔过早离开一个房间。如果任何一个房间感觉没有用，就退出房间。在退出时，推荐其他人来接替你留下的机会。

保持可见性

当人们，特别是女性和非二元性别者，来找我寻求建议时，他们可能期望我谈论如何成长为一名技术领导者，但当我说“你可能已经具备了技术能力，你需要做的是提升你在公司的声誉”时，他们会感到惊讶。无论好坏，没有良好的声誉，你无法晋升到Staff级别。 - [凯蒂·赛勒-米勒](#)

[伯特·范](#)对于那些试图达到Staff+级别的人来说，最好的建议是：“通常达到Staff级别是运气、时机和努力的结合。”时机是一种特殊的运气，所以你可以进一步简化为运气和努力。

如果你很幸运，那么你就不必刻意追求Staff+的职位。你已经在从事公司最重要的工作，有一位关心并支持你职业发展的、地位优越的经理，并且在公司总部工作。如果你一开始什么都没有，那么晋升将是一个相当大的挑战，但不要妄自菲薄：很容易低估自己在获得好运中的作用。

提高运气最有效的方法之一是在你的组织内提高知名度。当然，也有一些非常快速、非常负面的方法来提高你的知名度，所以我将稍微修改一下这个说法。你的目标是以做好事而闻名，同时尽量减少你为此消耗的组织带宽。

为什么知名度很重要

[凯蒂·赛勒-米勒](#)描述了可见性是晋升为Staff的关键因素

我还没有充分谈论的是沟通和透明度。晋升为Staff的一个重要部分是确保你的工作是可见的，人们知道你的名字，并且你拥有良好的声誉。

Staff-plus角色是领导力角色，通过授予你这样的职位，公司正在把你纳入其领导团队。该团队的现有成员希望确信他们正在扩大队伍，吸纳他们信任的人，如果他们不了解你，他们就无法信任你。

如果你在公司内部的可见性不高，这很可能会被认为是小圈子或把关行为。相反，如果你在内部广为人知，这可能会让人觉得是为了保持对承担领导角色的人的一致期望和标准所必需的代价——如果你不熟悉他们的工作，你如何保持一致性？

简要反思一下包容性组织如何减轻验证人们是否适合加入其领导团队的负面把关方面，这很有趣。答案是，他们设计机制来确保全部潜在领导者都能接触到那些将评估他们是否适合担任领导角色的人。相反，不太包容的组织无意中将访问权限集中在那些最积极地自我宣传的人身上。

内部可见性

创建内部可见性的最佳方式是[做对公司和公司领导重要的事情](#)。这条道路也最符合管理良好的公司评估你的贡献的方式。

但有时这还不够，还有其他一些策略：

- 编写和分发更多长期存在的文档，如架构文档或技术规范
- 领导（并在较小程度上参与）公司论坛，如架构评审、公司全体员工会议和学习圈
- 在 Slack 上为你的团队和同事的工作欢呼
- 你也可以通过电子邮件而不是 Slack 来欢呼
- [分享每周笔记](#)以一种其他人如果感兴趣就可以访问你的笔记的方式，将你的工作分享给你的团队和利益相关者
- 为贵公司的博客投稿
- 参加或甚至主持你团队或组织的办公时间

找到合适的活动组合，这些活动能够发挥你的优势，不会因为工作量过大而负担过重，并且对你来说是真实的。如果你从未宣传过你的工作，自我推销可能会让你感到尴尬。你永远都不想完全失去那种尴尬感——克制会有帮助——但你必须适应其中的一些。

高管可见性

要晋升到领导岗位，最重要的内部可见性是高管可见性。使用[晋升材料](#)，你会在你的经理那里建立知名度，但更进一步是有帮助的。尤其重要的是找到机会与你经理的经理建立关系，但所有积极的、在那一层的知名度都会对你有所帮助。

这些人通常都在房间里，负责批准晋升到 Staff-plus 级别的职位，而且他们很少支持那些他们不了解其工作的人。

外部可见性

将内部可见性工作与外部可见性工作相结合是有帮助的。有很多成功的 Staff-plus 工程师没有外部形象，但是[许多人发现外部可见性有助于他们的职业生涯](#)。

与完全专注于内部相比，建立外部形象的一个优势是，有更多的空间来创建一个利基市场并为自己命名。内部努力往往最终会与你的同行竞争，而外部努力则不会。

就如何为自己和你的工作创造这种可见性而言，它可以是像这样发表会议演讲[基维·麦克明](#)或者[丹·娜](#)例如，参加一个播客节目[米歇尔·布](#)例如，像\t一样，将一个问题变成一个网站和一本书[凯蒂·赛勒-米勒](#)的[ohshitgit](#)或者创建一个像 Stephen Whitworth 的\t一样的邮件列表[高增长工程](#)。

应该你专注于可见性吗？

在你的组织内，你总是可以拥有更高的知名度，但在某种程度上，提高你的知名度可能会减少其他人为自己创造知名度的机会。内部可见性并非严格意义上的零和博弈，但它受到你想看到你工作的人的注意力的限制。

我的建议是使用[晋升包练习](#)识别缺乏可见性是否可能阻碍您的晋升过程。如果是这样，努力清除该阈值，但不要走得太远。可见性是一种短暂的货币。学习和发展自己是一种永久的货币；一旦你完成了清除前者悬崖的最低限度，就专注于后者。

决定更换公司



我被 Fastly 聘为首席工程师。所以，老实说，对我来说，最大的因素是更换公司。我所做的工作类型并没有发生巨大的变化，但更换公司最终使我获得了这个头衔。 - [基维·麦克明](#)

我的父亲是一位经济学教授。在他二十多岁完成博士学位后，他开始在一所大学教书，在那所大学获得了终身教职，并在四十多年后退休。在科技行业工作，这听起来像一个童话故事。

拥有四十年业绩记录的软件公司很少，甚至更少的人的[四十年职业生涯](#)由一个雇主组成。过去有一个梗，许多工程师在每家公司工作一到四年，以最大化他们的股权授予，然后跳到下一家公司。如果这种情况曾经发生过，那么对于那些渴望或达到 Staff-plus 职位的人来说，这肯定不是常见的行为。

相反，一般来说，只要情况支持他们的成功，这些人就会留下来并因留在给定的公司而获得奖励。如果情况发生变化，他们往往会此后不久离开，或者花一段时间耗尽精力，然后在耗尽情感储备后离开。

从高级工程师职位晋升到 Staff-plus 职位需要数年来建立知名度和社交信誉，这使得如果你觉得你只是离升职还差一步。离开，感觉就像是从头开始。

再说一遍，正如.....所描述的那样[基维·麦克明](#)，人们通常通过加入一家新公司来获得他们的第一个 Staff-plus 头衔。即使你拥有所有的内

部信誉，有时离开才是最有效的前进道路。

什么才是对你来说正确的决定？

在进一步探讨之前，我想先承认两种截然不同的跳槽经历：一种是拥有特权的灵活性，另一种是受到严格的限制。你的居留许可可能取决于工作担保的签证。你可能需要赡养一个大家庭。你可能被限制在雇主稀少的地理区域。这些建议主要针对前一种情况，对于那些在技术职业生涯中足够深入以追求Staff职位的人来说，这种情况更为常见。如果这些情况与你的实际情况不符，你应该完全忽略它。

为什么离开有效

最了解你优势的公司是你目前的公司，他们也最有可能给你一个Staff-plus 的职位。然而，实际上授予这个职位取决于太多的环境因素，以至于在实践中并非如此。

如果你的团队资历很深，那么可能很难证明你在 Staff 工程师级别的贡献，因为它已经被归因于你的同伴。你的经理可能预算有限，没有空间再增加一名 Staff 工程师。你可能缺乏内部赞助人。你的公司可能根本不需要额外的 Staff 工程师。这些都可能意味着，虽然你应该得到晋升，但在你目前的公司你不会得到晋升。

相反，当您面试新的职位时，您可以一直面试，直到找到一家能够授予该职位的公司。您也可以故意选择在早期阶段的公司面试，这些公司可能会更看重您的经验。面试过程还会带来一位自动的赞助者——招聘经理——他的激励机制与您的激励机制的匹配程度，永远不会超过面试过程。

技术面试对于成功的预测是不一致且不可靠的，这对行业不利，对公司也不利，但如果你决心获得 Staff-plus 的职位并愿意进行广泛的搜索，那么这对你有利。面试创造了进行“偏见套利”的机会，找到一家

不成比例地重视的公司你。这可能是一家重视在会议上发言的人、您设计 API 的经验或[您的编译器博士论文](#)。

同样，有时你会在一家公司陷入困境，你的声誉阻碍了你的进步。也许你在指出包容性问题后被贴上了“难相处”的标签。也许你在午餐时让一位有影响力的主管感到难堪，他们正在阻止你的晋升。一家新公司让你把这些包袱抛在脑后。

是的，当然，你是否能真的在科技行业留下任何东西始终是一个悬而未决的问题。有时会让人觉得有点小圈子。如果你在科技中心、大型公司工作过，并且工作超过十年，那么你几乎肯定与面试你的人有共同的联系。

如果你在一家公司过得很糟糕，也许你的经理是个恶霸，或者你正在经历自己生活中充满挑战的时期，这会让你觉得有一片乌云笼罩着你的未来。也就是说，就像一般的面试过程一样，推荐信和背景调查非常随机。如果你需要任何进一步的证据，可以看看那些在知名公司一次又一次被雇佣的连续骚扰者。

离开前可以尝试的事情

如果你计划因为缺乏兴趣、兴奋、支持或机会而离开，那么至少先探索一下内部情况是值得的。这让你可以在保留内部网络的同时，仍然获得更换公司的许多优势。根据你公司的规模和增长率，这可能不是你的选择，但有些人每两到三年在同一家母公司内更换角色，并发现这是一种保持参与和学习的有效方式。

另一方面，如果你正在考虑因倦怠或精疲力竭而离开，有时可以协商带薪或无薪休假，在那里你可以花几个月的时间给自己充电，通常与更换内部角色相结合。这在大型公司更为常见。（如果你想知道，不，你的同事休产假不是“休假”或“度假”。）

没有工作就离开

说到倦怠，如果你特别感到倦怠，值得考虑在没有找到下家的情况下辞职。这里有一个相当简单的清单，可以确定这是否适合你的选择：

- 你的签证支持这样做吗？
- 在不工作的情况下，你至少有一年的经济保障吗？
- 你是在远程的高密度就业市场工作，还是对你的下一份工作地点有弹性？
- 你面试表现好吗？
- 你能否问问你为什么在没有找到下家的情况下离职的人，清晰地表达出一个连贯的叙述？
- 是否有人可以对你的工作提供积极的推荐？

如果所有这些都是真的，我不认识任何人后悔休假。但是，请记住，只有那些休了六个月以上长假的人才会经历中感到重生。休假时间较短的人很欣赏他们的假期，但通常只是部分恢复。如果你确实要休假，我强烈建议你把你的经历写下来。即使你不分享你所写的东西，它也会帮助你处理这些经历。

冒险一试

如果你在目前的公司几乎要晋升到Staff级别，那么绝对有另一家公司会给你Staff的头衔。至于你是否会喜欢在那里工作，或者在得到头衔后是否会得到支持，这很难预先确定。如果你的内部声誉受损，或者你一直处于晋升的边缘，但却成了移动标准线的受害者，那么如果头衔对你很重要，你应该认真考虑换工作。在某个时候，你必须听听你目前的公司告诉你什么。

相反，如果你对目前的工作感到满意，只是对头衔不满意，那么考虑一下你是否可以更有意识地追求晋升，而不是离开。很多人在他们的晋升到Staff+的道路上遇到了瓶颈，使用像[晋升材料](#)这样的技巧可以

帮助你摆脱困境。如果你已经尝试了所有的方法，认真对待自己的自我发展，但仍然无法达到目标——也许是时候离开了。

也就是说，很容易过度思考这些事情。很少有人会讲述他们过去十年留在或离开某个工作的故事。

找到合适的公司

获得Staff+职位只有几个神奇的咒语：在换工作时谈判获得头衔，或者找到一个支持性的环境来“原地烘烤”，同时建立你的内部信誉，并获得一位有权势的赞助人的支持，他会为你辩护。这两个咒语中最重要的试剂是选择合适的公司来执行它们。

好消息是，你正在申请一家新公司，虽然你可能会投入/周将精力投入到确定你是否能在那里获得一个Staff职位，你就不需要投入年。另一方面，如果你正在寻找一家可以加入并在其中成长的公司，那么你正在开始一段长达数年的未知组织之旅。这是一个令人望而却步的决定，为选择合适的公司你将对你是否能获得Staff+职位产生相当大的影响。

找到一个不成比例地重视你的地方

如果你的目标是职业速度，那么加入一家由于某种原因不成比例地重视你擅长的事情的公司。例如，Fastly重视[Keavy的API设计经验](#)，而Stripe重视[Dmitry在编译器方面的工作](#)。如果存在一个与你的特定形状相匹配的差距，限制了他们的成功，那么你对公司的影响将是独一无二的。

运作良好的组织会因为你擅长的事情而重视你。不太好的公司会因为你的身份而重视你。例如，一种将攻击性视为领导力的文化确实会提升和重视最具攻击性的人，但这对他们的文化和团队不利。有时你会发现一家公司对你的净价值评估是适当的，因为它重视你的附带价值，而不重视你认为有价值的贡献；这看起来似乎可行，但总的来说，这是令人沮丧的根源。

精英主义者和程序主义者

当您试图确定一家公司进行员工过渡时，在您的决策中需要考虑许多公司价值观。其中一个特别重要的是了解公司的领导层是否从根本上

认同以例外情况为主的“精英主义”世界观，还是以一致性为主的“程序主义”世界观。很少有公司完全存在于这个连续统一体的某一端，但大多数公司都严重倾向于某个方向。

当然，人们不会用这些术语来描述自己。第一种风格几年前会称自己为精英体制。现在这个词已经不受欢迎了，他们会避免使用它，但他们的核心信念仍然完好无损。这种风格在硅谷尤其常见，它以例外情况为导向，并将其努力集中在奉承一小群珍视的个人身上。一般来说，在这些公司里，如果你能与他们认为的高潜力模式相匹配，你就会非常成功。如果你不这样做，你可能会遇到一段艰难的旅程。

您会发现的另一种公司风格认为一致性驱动公平。他们是那种[执行政策而不是例外](#)：他们设计时钟，使其运转，观察它滴答作响，并进行偶尔的维修。这些公司往往更注重结构而非直觉，这可以为更多人创造更广泛的机会，但它们也可能是僵化的官僚机构，沾沾自喜地看着他们的机器压垮一个人。

不可避免地，精英主义者和程序主义者都将他们的世界观视为一种道德立场，并且根据你是谁以及公司的领导是谁，你将拥有截然不同的体验。

在面试过程中探索的一些方法来帮助区分这些思维模式：

- 拥有严格的薪酬范围的公司，以及那些实际上坚持下去往往由程序主义者管理。那些自愿避开乐队的人是精英主义者。
- 为个人创造一次性职位以让他们加入的公司往往由精英主义者管理。那些按照计划职位招聘的公司是程序主义者。
- 采用临时或非结构化面试流程，希望了解你的“感觉”的公司，特别是对于高级职位，往往由精英主义者管理。更多的结构意味着程序主义者。
- 临时进行新的、没有规则的面试的公司往往由精英主义者管理。那些严格评估你，即使他们不让你发光的人，往往是程序主义者。

精英主义或程序主义公司在将你推向 Staff-plus 职位方面并没有天生的优势。相反，这取决于你的身份和公司领导职位人员的身份。根据这些因素的匹配程度，你可以估计你在追求 Staff-plus 职位时会遇到的支持和阻力程度。

原型

大多数公司只雇用一到两个[Staff 原型](#)，即使他们都使用相同的头衔来描述他们正在招聘的职位。如果你想弄清楚特定公司的偏好，最有效的方法是联系他们现有的一些 Staff-plus 工程师，了解他们所做的工作。大多数公司不会刻意考虑他们支持哪种 Staff 工程师，因此直接询问的效果通常不如预期。

足够大的公司最终至少会让一些人在每个原型中运作，但这需要很长时间才能实现，通常之后他们的工程组织已经扩展到数千名工程师。

增长

如果你只在快速增长、成功的初创公司工作过，那么你可能不会想到，在 Staff-plus 工程岗位上可能缺乏额外人员的空间，但令人惊讶的是，增长较慢的公司通常根本没有足够的工作或预算来支持更多担任领导职务的人员。对于尚未达到产品市场契合度的公司来说，这也是一个常见的限制——当公司需要保持高度一致并频繁变更时，领导职位是有限的——对于那些恰好在销售以开发者为中心或技术基础设施产品的公司来说，可能存在例外。

如果你加入一家快速增长的公司，新的 Staff 机会将会自然而然地出现。在增长较慢的公司中，你可能需要等待某人腾出他们目前的职位，另一个职位才会空缺。这并不是说如果你不想加入一家快速增长的公司，你就应该加入——它们通常压力很大，并且运行着永久过时的流程——只是另一个需要考虑的因素。

赞助

在新公司获得Staff-plus的职位需要公司内部有人相信你，并且愿意克服相当多的组织摩擦来为你争取到这个头衔。晋升到Staff-plus需要一位经理和管理链，他们相信你，并且愿意克服更多摩擦来为你争取到这个头衔。如果没有公司内部有权力的领导者愿意将他们的组织资本投资于你，你就无法获得领导职位。

在寻找一家可以追求Staff-plus职位的公司时，一个重要的部分是确定一家你将拥有有效赞助人的公司。在当前公司之外进行面试通常可以有效地找到赞助人：你未来的招聘经理往往有充分的动机来向你提供Staff职位。同样重要的是，你的时间投入很高，但仍然相对地与在一家公司工作两年后才意识到你永远无法实现目标相比，这很低。

最容易找到的赞助商是以前和你一起工作过的人。[飞翔楔形](#)一位高级领导加入一家公司，然后把他以前的同事带过来的模式是一种众所周知的、理应受到鄙视的模式，它依赖于这种内置的推荐人作为赞助人，但如果谨慎使用，它不一定是 `ТОКСИЧНЫМ`。

这也是拥有[外部存在和网络](#)可以极大地帮助你的搜索。那些看过你的演示文稿、阅读过你的博客文章或在你的推文中点头表示同意的人，更有可能在面试过程中以及后来的晋升讨论中成为你积极主动的赞助者。

耐用性

特别是如果你正处于职业生涯的早期，并追求晋升到Staff职位，重要的是要考虑公司的持久性：当你希望你的工作最终能晋升到Staff+时，五年后这家公司还会存在吗？

更微妙的是，你还必须考虑你未来的 Staff 赞助人的寿命。有些优秀的工程领导者正在创造公平获得 Staff-plus 角色的机会，但这些机会可能会很快变成[价值观绿洲](#)一旦发起领导离开公司或改变角色，就无法维持自身。

通过确保你加入的公司的商业模式实际可行，并且领导者与他们组织最高层领导的价值观一致，来降低持久性风险（这样，即使他们离开，你仍然与潜在的发起人保持一致）。

节奏

在整个过程中[四十年职业生涯](#)，有时你会休息好，并寻找一个具有挑战性的、包容性的角色。有时你会感到精疲力竭。接受一个对你的节奏要求超过你目前能够承受的角色，会伤害你自己。在担任技术领导角色时，特别重要的是要确保公司的节奏期望与你能够提供的相符，因为你会被部分地评估为公司节奏的榜样。

以及其他一切

领导角色的职位搜索比典型的软件搜索慢得多，需要几个月而不是几周，并且试图匆忙完成它很少奏效。当你评估一家公司是否是达到Staff-plus角色的有效场所时，你还必须评估你在典型流程中会调查的所有其他方面。

深入挖掘你的耳语网络，寻找有毒的问题和个人。确保使命是你能够多年来保持支持和参与的事情。寻找你能够在工作中学习的人。如果你发现这些Staff标准正在把你拉向一家你原本担心的公司，那么很有可能你会后悔这个决定。

面试 Staff-plus 职位

当你决定面试高级工程师职位时，你大致知道会发生什么。你会[更新你的简历](#)通过[破解编码面试](#)，并对公司进行一些研究，以准备问题。当你进入面试时，你知道这将是五个左右的面试，包括一些编程练习，一些关于技术架构的内容，以及一些文化、行为或职业问题。

如果你能以类似明确的期望开始一个Staff-plus面试流程，那将是非常棒的，但大多数公司都在努力应对他们的Staff-plus面试环节。这可能与担任高级工程师角色时所获得的完全相同的面试。这可能是一个工程经理环节，增加了一个编程问题。也可能完全是其他的东西。

适应在模糊不清的环境中工作是Staff-plus角色的核心部分，因此一个特别乐观的人可能会将Staff-plus面试的状态视为展示你技能的好机会。如果你不那么乐观，你可能会觉得有点沮丧，但一点准备工作可以大大提高这些面试的可预测性。

划清你的界限

你职业生涯的工程领导阶段可能会持续二十年，但如果你从角色的角度来考虑，你如何度过这段时间可能会归结为四到五个关键的决定。每个决定都是一种稀缺资源，你应该有意识地分配它们。在开始面试流程之前，花一些时间完善你愿意参与的流程类型的标准，并反思[适合你的公司](#)。

在面试过程中，你会收到一些信号，表明该公司不太清楚如何面试Staff-plus工程师。因为大多数公司的Staff-plus面试流程都很平庸，所以你不应该自动退出运行不佳的流程，但你应该考虑哪些信号代表了你不愿跨越的界限。

许多担任 Staff-plus 角色的人划出的一条线是，他们不愿意进行面试编程。这通常意味着他们在许多公司用来评估早期职业候选人的那种算法问题上速度较慢或犯更多错误。不练习的人采取这种立场，因为

他们已经认定，一家关心快速编程的公司很可能会滥用其 Staff-plus 工程师。这是一条线吗？你想画画吗？也许，你自己决定。

调试过程

在你画完线之后，接下来，你需要弄清楚你面试的公司实际使用的面试流程。问这些问题可能会让公司重新考虑你的候选资格，但它是总是向招聘团队和招聘经理询问更多关于你的面试过程的细节是合理的。在 Staff-plus 级别，如果你不这样做，几乎是一个值得关注的点不要询问更多细节。公司希望你成功，了解流程是准备工作的重要组成部分。

在开始面试之前，需要理解的最重要的三件事是：

1. 面试形式有哪些，包括他们评估什么？
2. 面试需要做特别的准备吗？
3. 面试官是谁？

一旦你回答了这些问题，剩下的就只是准备工作。记下你希望如何处理不同类型的问题。为任何演示面试准备材料。简而言之研究面试官，以便根据他们的背景调整问题。

这也是一个调试你是否在正确流程中的时刻。如果你的面试小组主要由职业生涯早期和中期的工程师组成，那么很少会产生 Staff-plus 的职位；小组将无法评估你的优势，而且人们通常会抵制比自己更高级的职位。如果没有深入了解你之前的成就，也没有展示机会，那么同样很难展示支持 Staff-plus 职位的专业知识。

如果这是一个错误的循环，但你非常擅长这个循环所衡量的任何东西，那么你仍然有可能获得 Staff-plus 的职位。但是，如果你对这些偶然的衡量标准不太有信心，那么礼貌而建设性地提出你的担忧，也许可以向招聘人员指出[关于设计 Staff-plus 面试循环的资源](#)。不要让势头把你拉入一个不支持你目标的过程。

了解公司内部的晋升时机尤其重要。有些公司会发布带有特定级别头衔的职位，这样你就可以直接申请你认为合适的级别。如果你希望获得高级工程师职位，那么就申请Staff工程师的职位。然而，许多公司将这些职位作为临时头衔，并在稍后确定；其他公司则非常严格。唯一的办法就是询问。

对你来说，更多地控制你的面试过程可能会感觉非常不自然，而且从理论上讲，你可能会因此错过一些机会，但这是一个好的结果：你的目标是找到最好的领导机会，而不是第一个可用的机会。

好好收尾

即使你顺利通过面试过程，也要始终[协商细节](#)，并记住要好好收尾。向你的推荐人简要介绍该职位的详细信息。向面试官发送后续邮件。接受提供的销售聊天，并在其中提出经过深思熟虑的问题。在这种情况下，只要你花时间走完最后一英里，它就是最容易的。

协商你的报价

早在 2012 年，Patrick McKenzie 就写了[薪资谈判](#)，此后它已成为软件工程师薪资谈判的事实指南。这是一篇很棒的文章，也是关于如何协商任何报价（包括 Staff-plus 报价）的入门读物：如果你以前没有过多考虑过这个话题，请从那里开始。

在你的大部分职业生涯中，你获得的报价都是以相对公式化的方式生成的。也许他们有一个薪酬计算器，或者他们根据你之前的薪酬来确定，但这是公司的系统在驱动这些数字。然而，存在一个报价谈判转变的阈值，公司愿意参与定制报价而不是系统驱动的报价，但你会被期望以某种方式直觉到你已经跨越了该阈值——没有人会告诉你。

这种定制报价开始在薪酬方面有更大的灵活性，尤其是在你的报价中的股权部分有更大的灵活性。也就是说，这不仅仅是原始薪酬。还包括公司通常没有灵活性，但愿意为高级领导者做出私人例外的其他方面的报价。

例如：

- 他们的标准合同可能会给予他们在离职后三个月内行使既得权益的权利，但他们可能愿意将一位杰出工程师的行权窗口延长至五年。
- 他们可能在运营上没有准备好支持每个人的提前行权，但可能愿意破例以完成一份高级职员工程师的录用。
- 他们可能愿意提供递延补偿计划，以支持具有税收优惠的支付时间表（这在上市公司中很常见）。
- 如果他们使用的是计量休假，而不是无限制休假，他们可能会接受额外的休假天数。
- 他们可能会支持围绕在家工作、工作时间或在他们目前不支持雇佣员工的州或国家注册成立公司的灵活性。

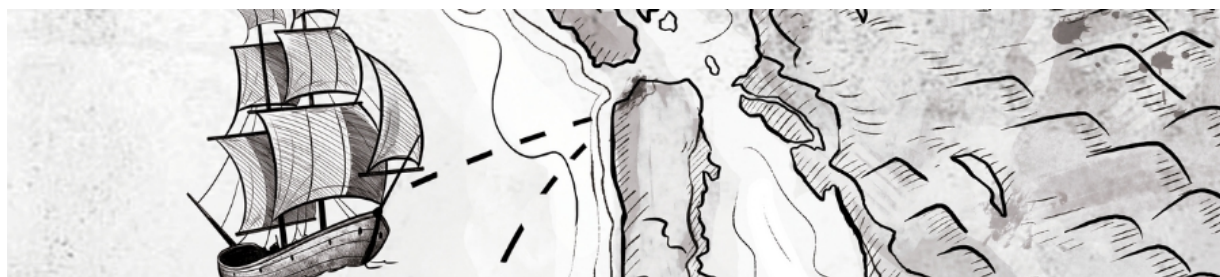
很难知道一家公司的特定职位是否超过了门槛，也很难询问公司里其他的 Staff-plus 员工，因为他们很可能没有就这些方面进行谈判（也

许是因为他们是从内部晋升的，从未获得过机会)。一般来说，如果一家公司有超过 20 名但少于 500 名员工，那么他们不太可能为你做太多的定制工作，除非你担任的是准行政职位——他们根本没有这样做的运营能力。但是，如果一家公司有数千名员工，但只有十几名与你同级别的人，那么他们很可能愿意协商定制条款。

重要的是要对你谈判的内容进行战略性思考。如果你坚持要求所有商务旅行都乘坐头等舱航班，你可能会得到它，但它也可能会传递一个关于你的优先事项的信息，而公司并不喜欢。

无论你试图谈判什么，都要花时间用叙述的方式来构建你的请求，说明为什么这对你很重要。例如，我曾与一位高级候选人合作，他将延长行权窗口的请求置于刚买了房子，并在离职后行使了前一家公司股权的背景下。这比仅仅要求更多的现金补偿要容易接受得多。正确的叙述可以完成谈判，而且不会对你的动机产生负面信号。

故事



在我出版第一本书后不久，我意识到我不是那种喜欢阅读对我作品评论的人。最好的情况下，我对自己感觉良好了一会儿，但通常我只是感到难过。在那段我还在阅读评论的时间里，我读到一条评论说，对于大多数人来说，它太以硅谷为中心了，无法从中受益。

当我开始集思广益，最终写成这本书时，这句话一直萦绕在我的脑海中，我想避免以自己的经历为中心，而排除其他所有人。更重要的是，我很清楚我的职业生涯建立在一套特定的观点、运气和特权之上，我希望这本书能成为那些以不同方式体验这个行业的人的有用指南。

所有这些都是为了说明这本书最好的部分很大程度上依赖于从业者的坦率和深刻的访谈，我很感激能够将这些访谈纳入最后的下一章。即使你到目前为止没有从这本书中获得太多，我希望你能在这些职业故事中找到一些独特的东西。

Michelle Bu - Stripe支付产品技术主管

这个故事录制于 2020 年 4 月。在她的网站上了解更多关于米歇尔的信息[博客](#), [Twitter](#), 和[Linkedin](#).

请简单介绍一下您目前在 Stripe 的职位：您的头衔是什么，您和您的团队通常做什么样的工作？

我是 Stripe 的支付产品技术负责人，直接与我们的首席产品官合作。我支持关键举措，并致力于缓解整个组织中的紧急问题。我通常将 80% 的时间花在一两个大型跨组织设计项目上。我将剩余的 20% 用于审查和支持整个组织的技术和产品设计（特别是 API 设计）。

我一直保持常青的“前三名”文档示例：

Michelle's top 3

Created by Michelle Bu, last modified on Feb 28, 2020

What is this?

This living document describes my current top 3 priorities as Payments Products Tech Lead:

1. Defining the OS spec of the GPTN (75%)
2. Elevate product quality (20%)
3. Build Stripe's product engineering culture (5%)

It serves as an overview of what I'm focused on right now and what you can expect from me in my role. I will not be able to actively prioritize projects and requests that fall outside of these three priorities.

1. Defining the OS spec of the GPTN

Objective

You can read this dev@ email for more history and

context: <https://groups.google.com/a/stripe.com/forum/#!topic/dev/FBaMuzkedBU>

What does this look like in practice?

In Q1 2020, I will spend 75% of my time [working on this project](#) with and

2. Elevate product quality

我管理着两名嵌入到高优先级领域的工程师。这既有助于我扩大我的影响力，也让这些工程师有机会涉足Stripe的许多领域。目前，一位正在研究核心支付API，另一位则专注于改善集成体验。我仍然按照IC级别进行评估——计划是任何时候的报告对象都不超过几个。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

Stripe的Staff-plus角色的大多数工程师都在特定的团队工作。有些Staff-plus工程师也拥有技术主管（Tech Lead）头衔，并在特定产品领域或技术领域承担更广泛的项目。

Stripe有两种Staff-plus工程师：一种是范围深的，另一种是范围广的。

范围广泛的工程师通过从事模糊的、跨组织的项目来产生影响。他们倾向于积累许多不同领域的背景知识，并在整个组织的许多项目中发挥支持作用。这种类型的Staff-plus工程师在我们产品工程团队中最常见。

范围深的工程师往往是特定领域的主题专家。他们领导雄心勃勃的多年项目。这种类型的Staff-plus工程师通常可以在我们的产品基础设施和系统团队中找到。

作为一名Staff-plus工程师，你觉得在哪里最有影响力？

随着我进入目前的支付产品技术主管角色，这种情况随着时间的推移而发生了变化。（作为背景介绍，支付产品由20多个团队组成。我们负责我们大多数面向用户的API和UI库。）

我开始使用“充满活力”这个词来代替“有影响力”。“有影响力”感觉是以公司为中心的，虽然这很重要，但“充满活力”更注重内在。寻找充满活力工作是我在Stripe待这么久的原因，追求有影响力的工作。

当我在团队中直接工作时，我感到最有活力的是能够直接与用户互动，无论是帮助#stripe IRC频道上的用户，还是设计和发布用户可以无缝集成的API。

在我目前的职位上，当我赞助的人发布他们已经完成的工作公告，或者当我看到我帮助塑造或改变了工程团队对重要主题的理解模型时，我感到充满活力。是这些团队，而不是我，在日复一日地努力构建和支持他们的技术。我根据他们的进展来衡量我的影响，更重要的是，根据进展的方向以及他们的工作与公司目标的对齐程度。

最近记忆中的一个具体例子是，另一位高级工程师和我将我们常见的API形状进行了分类：将一些标记为流程，一些标记为引擎，一些标记为配置等。这项工作的目的是为现有API的分类以及讨论和设计新的

API建立一个共享的心理模型和词汇表。人们在看到这些类别后就开始自发地使用它们！正是在这些时刻，我感觉我正在通过传播有用的心理模型和想法来创造杠杆并扩大我自己的影响力。

我花时间在的几个审查论坛上，如API Review，但通常这些论坛更像代码审查。它们发生在设计过程的后期，因此它们在防止不良结果方面比与团队合作以引导良好结果方面做得更好。当我可以为产品团队的工程师提供设计出出色API的工具时，我感觉更有影响力。

你能想到你作为高级工程师所做的任何事情，是你以前无法做或不会做的事情吗？

我在Stripe工作了很长时间（自2013年以来！）。虽然由于我的任期，我一直拥有一定的影响力，但我的支付产品技术主管的角色（以及我直接向CPO报告的事实）确实改变了人们与我互动的方式。我现在在工作中肯定感到更孤独（并且正在积极适应这种新常态）。

有一件事需要适应的是，现在人们期望我对我们正在讨论的任何事情都有自己的看法！当我还是一名直接在团队中工作的高级工程师时，这种情况并不经常发生。我记得在角色改变后不久的一次会议上，我比平时安静了一些，因为我有点累。后来我听说演示者担心我不喜欢他们的提议，因为我什么也没说。这是我第一次意识到人们在我看来拥有自己的观点并支持他们的想法！从那以后，我一直小心翼翼地会议中保持参与，并提供反馈，即使只是明确表示我尚未完全形成任何观点。

有点让人困惑的是，有些人比我没有担任如此显眼的职位时更认真地对待我的意见，并且对我更好。以前，有些人不合作或会驳斥我的意见。我认为经历过这些是件好事。我足够自信（并且受到组织的足够信任），可以就他们的合作向他们提供强烈的反馈，以便我可以确保类似的事情不会发生在像我这样的其他人身上。我现在担心我会失去对这些互动发生地点的可见性。

当您花更少的时间编程时，您如何保持对其他工程师开发经验的同情心？

我担任新职位只有一年，所以我觉得还没有太脱节。也许这会随着时间的推移而改变。我以前是一个较小领域的技术负责人。在那个职位上，我编写了少量软件，并为我团队的一些“运行轮换”做出了贡献，我们在其中对收到的请求进行分类并修复紧急错误。

为了保持我在新职位上的背景，我花了很多时间与直接参与的工程师和产品经理进行一对一的交流执行。仅本周，我就进行了 12 次 30 分钟的 1 对 1 会议。我还关注 Stripe 报告的每一起事件。（我们有一个 Slack 群组，您可以加入该群组，以便自动受邀加入每个事件的 Slack 房间！）事件的嗡嗡声对于调整尤其有用。通过阅读每个事件的详细信息，我能够估计我们的系统现实与我每天思考的理想化架构/产品之间的距离。我想知道工程师遇到的问题的形状，他们陷入的失败深渊，以及开发人员环境如何或没有支持他们摆脱这些深渊。我将自己视为领导层工程师的倡导者，因此对我来说，深入了解我们目前的现实非常重要。

您是否花时间倡导技术、流程或架构变更？

在这一点上，我花更少的时间倡导特定的技术或程序，而更多的时间是授权其他人倡导他们认为重要的技术和程序。我还努力成为人们可以联系以获取反馈的知识和支持来源，尤其是在跨领域的产品决策以及向组织其他部门展示想法方面。

我确实在一些项目中工作，在这些项目中，我会明确地思考理想化的架构和接口。然而，归根结底，迁移到任何理想化状态都将由各个团队完成，所以他们需要感受到一种主人翁意识和赋权。我花很多时间与实际做出日常决策的工程师和产品经理进行直接对话。理想的结果是，我们能够在方向上保持一致，然后他们能够在他们的团队中倡导我们的北极星，并做出良好的本地决策。

对于我现在正在进行的项目来说，这样做要困难得多，因为它涉及到为许多团队定义理想化的架构和许多团队的接口——基本上是所有从事支付工作的团队！我还没有找到一种可扩展的方法来让每个人都参与进来。即使是编写文档（分发信息的最可扩展的方式！）也很困难，因为不同的团队（根据定义）从不同的角度来看待接口，因此对问题和解决方案的非常不同的框架会引起每个团队的共鸣。我们目前的方法是将对我们文档的审查视为用户测试：观察团队中的个人阅读文档，看看他们的光标在哪里，他们对什么做出反应等等。到目前为止，效果还不错！

设计[Payment Intents API（支付意图 API）](#)，这是对我们备受欢迎的Charges API的重新思考，以适应不断变化的支付领域，这是一个我之前参与的类似跨领域的项目。愿景完全被公司里的每个人接受花了两年时间。即使有了这种组织上的支持，我们仍然没有充分发挥其最初理想化设计的潜力。但这并不是一个bug！我们专注于在验证设计的同时为用户提供增量价值。我希望任何足够雄心勃勃的设计项目即使在我不再团队中时也能继续进行。使这项工作发挥作用的一个重要部分是写一切下来。

我们创建了一个规范文档，定义了我们理想化的抽象。即使在今天，在该团队工作的人们仍然将这些抽象概念作为北极星：

Payment abstractions overview

Michelle B

This document is an overview of the core payment abstractions defined by the Payment Flows team. These abstractions purposefully ignore current Stripe abstractions and constraints, but do consider network, regulatory, and payment method realities. Understanding these concepts is critical to understanding our incremental phases & proposed flows, which you can find at the end of this document.

Note that all object, parameter, and enum names in this document are internal names, not the final user-facing API names — we're more than happy to discuss naming possibilities!

如果两个人问了同样的问题，我们会立即将其添加到我们维护的常见问题解答中。我们非常重视每个人的反馈和问题，并将举证责任放在自己身上。最后，我们努力在工作中做到完全透明，甚至创建了一个

决策日志，公司里的任何人都可以用它来跟踪我们的进展。决策日志中的每个条目都简洁地描述了一个产品或技术决策，记录了谁参与了该决策，并链接到详细的支持性技术设计文档，这些文档通常包含完整的问题陈述和替代方案评估。

🔨

Payment Flows decisions

🔗

go/payment-flows-decisions

📄

Instructions

👤

Decisions Template

📅

Historical decision logs

🔍

Looking for a decision? Try these docs.

2019	+gavel: 2019 Payment Flows Decisions
2018	+gavel: 2018 Payment Flows Decisions

📅

2020-03-25

📄

One PaymentIntent never results in more than one successful payment

📄

Elaborated in: [+Decision: One PaymentIntent never results in more than one successful payment](#)

📄

One PaymentIntent never results in more than one successful payment. Additionally, one PaymentIntent only attempts to move funds with one payment instrument at a time.

📄

PaymentIntents are the pay-in mechanism of Stripe to collect a single payment from a single payment instrument.

📄

The linked document elaborates on edge cases like [multicapture](#), split tender, and PaymentIntents that kick off Subscriptions.

👤

@Ellen S @Maddie S @Michelle B @Dan W @Kelvin L

🔗

gavel: Payment Flows decisions

📄

Historical decision logs

📅

2020-03-25

📄

One PaymentIntent never results in more than one successful payment

📄

Elaborated in: [+Decision: One PaymentIntent never results in more than one successful payment](#)

📄

One PaymentIntent never results in more than one successful payment. Additionally, one PaymentIntent only attempts to move funds with one payment instrument at a time.

📄

PaymentIntents are the pay-in mechanism of Stripe to collect a single payment from a single payment instrument.

📄

The linked document elaborates on edge cases like [multicapture](#), split tender, and PaymentIntents that kick off Subscriptions.

👤

@Ellen S @Maddie S @Michelle B @Dan W @Kelvin L

总的来说，我发现对于雄心勃勃的设计项目，极其透明，并且明确说明你是否准备好接受反馈，这对于关心这个话题的人来说效果很好。以下是一些措辞，你可以在我领导的一些项目的（公开）笔记文档的顶部找到：

Feedback and contributions[0] are **welcome and encouraged** on items in “Things we shipped last week.” Items in “Things we’re working on this week” are rough WIPs — feel free to leave comments (especially if you have context that the DRIs are missing), but **please be mindful** that documents and prototypes are still undergoing active iteration and that DRIs are still forming their own ideas and opinions in these areas.

[0] Yes, please make PRs for small things!

Please note that we're not yet ready for feedback on these ideas, as our thoughts are still early! 😊

赞助其他工程师是你角色中的一个重要方面吗？

是的，这是我最喜欢的部分之一！我非常关心与我一起工作的人——他们是我感到精力充沛去工作的主要原因。

对我来说，赞助的一个重要部分是为个人贡献者创造空间，让他们做他们关心的有影响力的工作。我很幸运，在我目前的职位上，我不需要花时间积极证明我的能力，所以我可以花大量时间在支持项目的角色上，并提升他人。我很少觉得我必须“邀功”或让我的名字明确地出现在我帮助过的项目的署名上（尽管当它发生时，总是一种很好的感觉）。对于更开放的项目，有时对我来说，把我的名字借给项目是有用的。例如，我最近启动了一个产品质量指导计划，我在其中扮演更多的协调员角色，选择学员，将他们与导师配对，并偶尔审查他们的工作。我没有做几乎和项目中的导师一样多，但我们能够启动这个全组织范围的项目，因为我赞助了它。

日复一日，我发现我可以作为一个“橡皮鸭”来帮助那些想要就如何驾驭一个复杂的项目或解决一个技术分歧寻求建议的人。我发现这项工作——帮助其他人在不直接参与的情况下取得进展——特别有意义。

最后，我总是记住一份擅长自己工作的人的名单，并为他们辩护，因为他们是可见的与他们的兴趣相符的机会就会出现。但这里有一个平衡。我了解到，人们有时很难对我说“不”。最近，我请一个与我合作的团队的工程师发送一封关于她所做的一些出色工作的电子邮件。她在发送邮件后告诉我，她原本不想这样做，但她也不想对我说“不”。然后她向我展示了她在她的“[拒绝日志](#)”：

feb 21	the not-shipped email	yes - I wrote the email	Well, I dunno, I guess I want to do the right thing for Michelle. It was something that I uniquely would have to do. I would have been OK with it not being done, but it was important to Michelle, and it's important to me to do things that are important to people who are important to me (I lost my words here). Is this even glue? Now that it was sent: I don't regret it because people said such nice things
--------	-----------------------	-------------------------	---

Stripe 是你第一家全职工作的公司，而且你仍然在 Stripe。你是如何达到 Staff 级别的？

我大学毕业后就加入了 Stripe。实际上，我加入 Stripe 后的成长曲线比我同期的其他工程师要慢。与雄心勃勃的大学毕业生相比，我最初四年的发展轨迹很慢。我认为这部分是因为我编码的时间不长（我 2011 年才上了第一门编程课，2013 年加入了 Stripe），部分原因是我在 Stripe 的第一个大型项目是一个 1.5 年的重写项目，但最终被取消了。

当 Stripe 首次引入级别时，我已经在公司工作了两年半，级别为 L2，我们期望应届大学毕业生在 6-18 个月内达到这个级别。老实说，我非常失望，因为我的同龄人已经达到了“高级工程师”级别。那时我已经做了很多有影响力的工作，启动了大部分新加入的产品工程师，并且经常在事件发生时跳出来帮忙。我工作非常努力，做了很多有影响的事情，甚至在我主要项目之外！他们还想让我做什么？我应该不是帮助别人吗？

回想起来，根据级别划分，L2 是 100% 公平的。我努力工作，熬夜加班，因为由于缺乏经验，我在编写好的代码方面自然比其他人慢一些。我还没有为软件开发打下良好的基础，主要是因为我还没有足够的实践经验！我所做的有影响力的工作很有价值，但也不是我唯一能做的事情。当时，我绝对是一个表现出色的 L2。

在早期，我花了很多时间学习产品和支付领域，而不是编写代码。我花了很多时间在 IRC 上帮助用户（开发人员）进行集成。我做了一些较小的任务（错误修复、小功能、创可贴），这些任务在技术上没有挑战性，但对这些用户来说很重要。这类工作并不总是能促进工程师的成长（尽管我确实微调了我的调试技能——我现在是一个非常棒的调试器）。我还通过在 Slack 和工单上过度帮助，以及帮助团队找到我们用户的最佳解决方案，与其他团队和其他工程师建立了关系。在我最初的两年里，我帮助启动了大部分新加入的产品工程师。随着时间的推移，我开始以非常关心我们的用户和成为产品知识的源泉而闻名。（“用户至上”仍然是我最喜欢的 Stripe 运营原则。）

后来我意识到，在那段时间里，虽然我没有有效地朝着成为一名软件工程师的技术方面发展，但我实际上是在学习关键技能，这使我能够非常迅速地从高级工程师晋升到 Staff，再从 Staff 晋升到我目前的职位（总共只用了 3 年！）。事实上，我非常确定正是我在最初几年建立的关系，才使得一个被取消的 1.5 年的技术项目感觉不像是我职业生涯的太大挫折。

当我从事第一个版本的开发工作时，我缓慢而有条不紊地构建了我的技术基础[Stripe Radar](#)和[Stripe Elements](#)。我坚信，只要我认真思考我的技术差距，认真弥补我正在进行的项目中的这些差距，并用让我走出技术舒适区的项目来挑战自己，我就可以有机地培养和实践我的技术技能。更软性的技能，公司内部的联系，用户至上，深入理解产品——这些技能需要更长的时间来学习，并且最终在我建立技术基础后帮助我加速了晋升到 Staff 的道路。

你做过 Staff 项目吗？

我广泛地参与了 Stripe 产品的几乎每个组件。随着时间的推移，我参与的项目通常会衍生出自己的专门团队，特别是当我还是一名高级工程师时参与的两个项目，可能有资格成为 Staff 项目：[Stripe Radar](#)和[Stripe Elements](#)。

通过 Radar，我们从头开始构建了一个全新的产品，对构建什么以及为了尽快向用户推出产品而可以安全地取消范围的内容进行了周全的权衡。当我们在 2016 年 10 月推出时，这是我们有史以来最顺利的产品发布之一。此后，它已成为一款非常成功的产品。

通过 Stripe Elements，我构建了基础设施，从头开始设计了最初的 Card Elements API，并在不到 3 个月的时间内交付生产。这只有通过广泛的内部测试才有可能实现。在构建 Elements 时，我创建了三个具有不同设计框架和设计（质量各异）的小型电子商务商店，以测试其自定义 API 的限制。从那时起，数十名工程师已成功地在代码库中进行开发，它是新的 Stripe Checkout 的所在地，最重要的是，我们对其原始 API 设计几乎没有遗憾。破坏性的 API 更改是随着 API 产品

的扩展以及我们更多地了解开发人员如何在实践中使用它们，总是可以预期的。我们在验证最初的 API 设计方面做得很好，以避免这些重大更改，同时仍能快速交付。

为了确保新工程师能够顺利地加入到涉及大量 IFRAME 恶作剧的相当复杂的产品中，我编写了大量文档。我发现讲故事对于教人们为什么事情需要这样运作效果很好：

Stripe.js mini-map

This walkthrough of Stripe.js is intended to help you formulate a “mini-map” of the world of Stripe.js 🗺.

There are two high-level flows Stripe.js enables today:

1. **Collecting customer data with Elements, Stripe’s pre-built UI components.**
2. **Making API requests of various types (PaymentIntent confirmation, Token creation, etc).** For simplicity, we will focus on Token creation in this document, because other API requests generally work the same way.



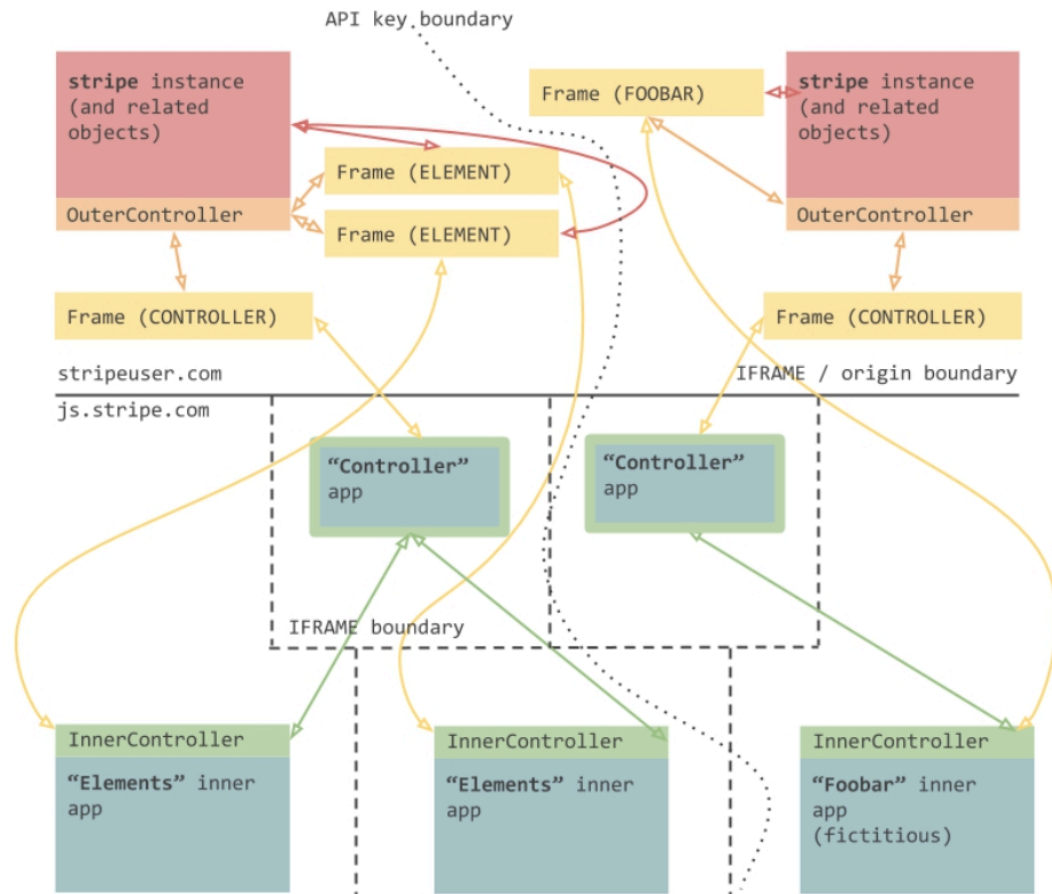
In the following sections, we’ll be digging into these two flows in order to gain a full understanding of how (and why) things work.

🔗 ▼ What the user sees

***NB.** “User” in this context refers to Stripe’s user — the “merchant.” We’ll refer to Stripe’s user’s user as the “customer.”↑*

In this document, we'll define the various components of a Stripe.js feature.

Overview



"Outer" objects

Outer objects **expose user-facing APIs**. For example, Stripe Elements is exposed via `Stripe#elements()` and the `Elements` and `StripeElement` objects.

现在回想起来，自推出以来，这两个项目的产品架构总体上都得到了支持。当时，除了实施这些产品之外，我还需要在产品推出后等待一段时间，才能让产品选择在我们的用户中证明自己，并让技术选择在内部工程师加速发展时证明自己。

我认为这是产品中 Staff-plus 工程师的一个重要标准：不仅要构建能够发布的东西，还要使其顺利推出，并在随着时间的推移继续成功和发展，同时尽可能减少令人后悔的选择。在产品开发过程中，特别是对于新产品，总是会有一些妥协和功能范围的缩小。一位 Staff 产品工程师会深思熟虑地做出这些产品和技术选择，扮演各种不同的用户角色，以做出最佳选择，并为未来的工程师彻底记录粗糙的边缘。

你必须整理一份晋升材料吗？

当我晋升到 Staff 时，我很幸运有一位非常积极地支持我晋升的经理。说实话，当时我真的不明白如何正确地写我的自我评价。我写的是关于我来年想要学习的自我反思发展计划，而不是记录我的工作的影响和范围。我的经理实际上完成了大部分工作，在他的评价中写出了我的影响。

还有其他一些事情帮助了我。首先，我在大部分时间里都和同一个经理一起工作。如果你更换经理，那么你的经理就会失去背景信息，这会将创建连续性的工作推到你身上。其次，我的经理管理着一个相对较小的团队，并且能够花大量时间跟踪和理解我正在从事的工作的细节。如果我向一位支持 10+ 工程师的经理汇报，我可能需要投入更多的工作来准备我自己的晋升材料。

你达到 Staff 级别最重要的两个或三个因素是什么？

回想起来，对我来说一个可能令人惊讶的重要因素是（并且现在仍然是）我的冒名顶替综合症。它使我非常乐于接受反馈；学习和成长，并承担对与我的工作有任何关联的事情负责。它使我主动寻求关于一切事情的反馈，从我对 PR 的评论的有效性到我如何主持一次特定的会议。如果有什么东西坏了（无论是技术上的还是组织上的），我都会感到不安，并且有深刻的、内在的动力去了解它并修复它。Stripe 的产品的任何部分都不是“不是我的问题”。这发展成了两种超能力，作为 Staff-plus 工程师，这两种超能力可能比技术超能力更重要：

1. 真正倾听和同情他人。

2. 深入关心解决所有类型的问题。

当然，冒名顶替综合症是一把双刃剑。它经常让我感到害怕和不自在——早期我总是觉得我会因为不够快或不够有效而被解雇。随着时间的推移，我对自己的优势越来越有信心，但坦率地说，这需要一个很多我的经理和公司领导给予的时间和积极的鼓励。

在产品工程部门工作是否比在基础设施工程部门更难达到 *Staff-plus* 级别？

我认为情况确实如此。我也认为在 Stripe 会稍微容易一些，因为其核心产品是基础设施。这意味着在产品工程部门有很多机会参与需要考虑规模、稳健性、迁移路径和精心设计的接口的项目。

如果你所在的团队主要构建 UI，那么达到 Staff 级别肯定会比较棘手，因为 UI 产品本质上更具临时性，并且允许更多的迭代和实验。要作为 UI 团队的工程师达到 Staff 级别的影响力，你需要能够创造杠杆效应。你可以通过构建精心设计的组件库、实验框架等来实现这一点。

作为产品工程师，构建杠杆效应的另一个方面是创建流程和系统来管理“产品债务”。人们经常谈论“技术债务”，但同样重要的是支持旧版本产品所造成的“产品债务”，并且大规模产品工程的许多困难都与管理产品债务和产品漂移（即需要相互操作的产品随着时间的推移朝着不同的方向发展）有关。我相信，公司产品债务的积累确实会在一定规模上在产品工程部门内部创造 Staff 级别的复杂性角色。

你还记得关于达到 *Staff* 级别的任何对你特别有用的建议吗？

我没有得到太多泛泛而谈的有用的建议。我一路上得到了一些针对具体情况的良好建议，但这些建议总是与手头的情况相关。

对我来说，最有用的普遍学习是适应不确定性。高级职位上的持续成功取决于你适应和发展以适应组织需求变化的能力。

对于想成为Staff-plus工程师的人有什么建议吗？

一些注意事项：

- 我认为我特别幸运，遇到了很好的经理。
- 我的兴趣一直与公司最重要的目标保持一致。（在这一点上，我有点不清楚我的个人兴趣（即，开发者产品、指导）是否一致，或者随着时间的推移，我是否将自己与对公司重要的目标保持一致。我感觉就像是前者，但无论如何，我觉得我一直对我的工作非常感兴趣。）

我可能是公司里最引人注目的产品工程师之一，所以工程师们有时会看到我在做什么，并试图模仿这种模式来成为一名Staff-plus工程师。这感觉很棒，我很幸运能够成为像我这样的人的榜样。

也就是说，我对工程师们的第一条建议是，他们应该避免以导致他们从事不喜欢的工作的方式进行模式匹配。从我所做的工作中获得了极大的能量，与团队合作解决抽象建模和设计问题。经过多轮反馈后，一次又一次地尝试需要一定的毅力，说实话，这并不适合所有人。如果你更专注于达到Staff级别，而不是让自己从事能给你带来能量的工作，那么很容易最终陷入一个你不喜欢的角色。成为一名Staff-plus工程师，尤其是一名范围广泛的Staff-plus工程师，与成为一名高级工程师是非常不同的工作。

相反，追求你觉得能给你带来能量的工作，即使从表面上看，它似乎不会让你达到Staff-plus级别。作为一名成功的Staff-plus工程师，一个重要的组成部分是能够识别和确定新的有影响力的工作范围，并说服其他人相信它的价值和影响。如果你所做的工作能给你带来能量，那么实际上更容易实现这一点，因为你会更喜欢深入思考你的工作！

对于刚开始担任Staff-plus工程师的人有什么建议？

你作为一名高级工程师的工作特定于你的团队和组织，因此务必避免采纳不适用于你情况的建议。例如，当我进入目前的职位时，许多其

他高级工程师专注于撰写个人章程，描述他们希望在未来 1-2 年内完成的目标。这种方法可能对深度范围的工程师非常有效，但对于像我这样需要广泛范围的工程师来说，它并没有那么有用快速响应组织变化和产品战略的转变。

你有没有考虑过工程管理？

我现在确实管理着两名工程师。也就是说，我没有做很多传统经理会做的事情。我没有像招聘经理那样参与招聘，而且我没有经历其他经理会遇到的那种绩效管理情况，因为被选入我团队的工程师已经是高绩效者。

我非常关心Stripe：当我看到任何不合适的地方，我都会感到不安，并想修复它。在一些组织中，我认为这可能会引导我走向工程管理，而不是我目前的角色，我很感激管理不是我唯一的道路。我的优势和兴趣在于产品工程、API设计和执行，我每天都能在我的角色中使用这些优势。

你从哪些资源（书籍、博客、人物等）中学习过？

我喜欢阅读小说，并从伟大的文学作品中了解很多关于世界的信息。我不太喜欢阅读非虚构的商业或技术书籍。当涉及到学习与我的工作更直接相关的主题时，我珍惜我的同伴关系。我的同伴给我有价值的即时反馈，并帮助我理清我一直以来都在思考的答案。

Stripe还有一个名为“实践中的领导力”的项目，所有经理和一些高级工程师都会参加。该项目包括一个关于[适应性领导力](#)的课程，这特别有帮助。从那以后，我将我学到的框架应用于许多情况。

我从来都不是一个向单一导师寻求建议的人。相反，我遵循我过去称之为“弗兰肯斯坦”的自建导师方法，类似于Lara Hogan在她关于的文章中写到的那样[构建一个经理百兽王](#)将我单个导师配对的项目从来没有让我觉得很自然。我倾向于有意识地关注我想成长的特定主题或

领域，并且会倾向于在那个领域表现出色的人，即使他们不是我的“官方”导师。

我大部分时间都在处理困难的、具体的问题，这些问题没有简单的、通用的答案。弄清楚正确的方法需要大量的背景情况，而局外人对此不会有太多的了解。

我最近读过并喜欢的一些非小说作品：

- [Draft No. 4, 约翰·麦克菲](#)：我大部分时间都在工作中写作，并且经常遇到写作障碍。但克服这一点非常重要，因为一篇好的书面交流是传播思想和提升自己的最有效手段。
- [创新公司, 艾德·卡特穆尔](#)：这本书的语气确实让我感到惊讶，但是关于如何大规模地培养创造性的工作环境，这里有很多东西可以学习。随着我们发展我们的产品组织和我们的产品工程职能，我经常思考这个问题。
- [即兴, 基思·约翰斯通](#)：我将我的超能力（尤其是在公司发展壮大时）视为能够快速学习和适应，所以我喜欢阅读关于不同形式的学习和教学的书籍。这本书是关于学习如何表演/即兴表演的，并且推动了关于教育的传统隐喻和叙事。

Ras Kasa Williams - Mailchimp的资深工程师

这个故事录制于2020年7月。了解更多关于Kasa的信息，请访问 [Linkedin](#)。

请简单介绍一下您目前的角色：您在哪里工作，您的职位是什么，以及您和您的团队通常从事什么样的工作。

我是 Mailchimp 的一名 Staff Engineer，在数据服务工程组工作。数据服务可以被视为公司内部数据工程的所在地。我们的小组构建的系统主要支持我们的数据科学和分析师团队（即产品分析师、财务分析师、市场分析师等）。

我一直担任该小组的技术负责人之一，专注于构建可扩展的数据处理管道，为我们的内部分析平台提供支持，并支持关键业务情报计划的推进。我也一直在为团队承担很大一部分工程经理的职责（尽管我不是正式的工程经理）。在这个职位上工作了近 2 年后，我正在积极地将其过渡给另一位工程师。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

在 Mailchimp，一旦你成为一名 Staff Engineer，你就是“工程领导”的成员。由于正式的工程阶梯是几年前才实施的，因此对“成为一名 Staff Engineer 意味着什么？”或者“成为工程领导的成员意味着什么？”的回答可能会有所不同。

在我看来，这是关于全局思考。这意味着与其他成员合作，了解公司范围内的业务/产品战略，并将其提炼为全工程范围内的技术战略，旨在为产品、营销以及我们在其他职能部门的同行提供执行能力。这意味着合作改进招聘、入职、跨团队沟通和生产运营等流程。这意味着合作以提高整个部门的技术和社会技能。

这是关于将全局思考应用于本地。这意味着将您团队的（技术）计划/路线图与全工程范围内的技术战略对齐；并有意识地偏离该路径，以满足您团队直接利益相关者的需求。这意味着与您团队的经理合作，采用其他团队在招聘、入职和生产运营方面的成功实践；并分享您团队的实践，这些实践对其他人有益。这意味着从公司范围内的业务/产品战略中获取背景信息，并将其转化为它如何影响您团队的直接项目。这意味着有意识地为您的团队的个人贡献者创造机会，以提高他们的技能，获得知名度，并访问公司内部的其他人。

当然，我并不总是能做对。但对我来说，这是一种成功的运作模式。

你每天如何度过你的时间？

如前所述，我一直在数据服务部门担任技术主管之一（并承担了很多工程经理的职责）。

作为技术主管，我负责定义团队的技术战略和方法，并对执行情况负责。我努力巩固这项战略对我们内部客户（或业务）的价值，并有效地沟通在需要时在整个公司范围内进行。我每年都会反思几次取得了哪些进展，以及业务内部是否发生了任何需要调整此策略的事情。我仍然定期贡献代码——当然比我团队中的其他工程师少；但重要的是，我坚持“手到键盘”的工作，以确保我的技术战略（和其他宏观层面的决策）能够从我团队其他成员的实际经验中获得信息。对我来说，一个重点是由于我设定的技术方向而发展和培养团队中的其他工程师。更具体地说：我努力帮助我的队友（通过指导或辅导）了解如何进行技术决策，以及如何通过它所解决的客户/业务问题及其提供的价值来支持它；我努力帮助他们了解如何推动工程项目从问题陈述到生产发布/长期运营；我努力帮助他们了解针对不同受众的正确沟通实践（即工程同行、工程管理层、非技术利益相关者等）。总的来说，我努力帮助他们达到一种自给自足的水平，使他们能够在没有我在场或参与对话的情况下，以符合我的技术战略、方向和方法的方式进行操作和沟通。

我们的小组没有产品经理。但是，围绕理解内部客户需求和利益相关者管理等方面的需求仍然存在。最近为我的团队聘请的工程经理对此负责；但我们分担了很多责任。我花了很多时间与内部客户聊天，回答一些一次性的问题，了解对新工作的需求（即新的数据集），推荐前进的道路，设定我们何时能够执行新工作的期望等等。

我们的小组也没有项目经理。但是，同样，这些需求是存在的。我坚信授权每个团队成员负责他们分配的项目的项目管理职责（即向利益相关者提供状态更新等）。我花了很多时间指导项目管理策略，例如主动沟通风险/障碍，以便我可以帮助解除这些障碍，以及如何不断交付增量价值以保持势头。

我花了很多时间构建被动的内部客户和业务背景。我偶尔会阅读我不维护的存储库的合并拉取请求；我会阅读公开共享的技术规范和提案；我偶尔会参加数据科学和分析团队的各种内部演示，他们在会上展示他们完成的项目或正在探索的想法。所有这些都是小策略，但它们有助于我了解情况，并且是我团队季度规划活动的一个关键输入。

作为一名技术主管，我是工程技术主管队列的成员。这是一个由工程部门所有技术主管组成的正式小组，旨在分享背景信息、讨论想法并帮助推进整个工程范围的技术路线图。有时这个小组可能会出现一些临时对话和工作。还有一个为所有员工、高级员工和首席工程师举办的定期电话会议，旨在作为一个提出和讨论问题、在需要时分配负责人和行动项目以及通常与彼此建立社区的空间。我们与 Google 有着密切的合作关系；他们是我们的云提供商。偶尔，我会花时间与我们指定的合作伙伴团队讨论我们遇到的挑战、我们的计划、正确的解决方案方法、有用的正式培训等等。

随着我逐渐退出技术主管的角色，我每天的时间安排肯定会发生变化。但我真的不确定那会是什么样子。

作为一名高级工程师，你觉得在哪里最有影响力？工程师？

如果我能结束一天的工作，并且感觉我帮助我的同事们解除了障碍并保持了前进的动力，这总是一种很有成就感的感受。

这可能意味着帮助我的数据服务团队成员之一思考解决复杂技术问题的各种方案，以及诸如提供即时价值与长期可持续性之间的权衡。或者帮助他们思考如何深入研究一项新技术，以及它是否适用于解决我们的三大问题之一。

这可能意味着帮助另一个团队的同事思考他们在过去一个季度中的可交付成果，这些可交付成果是否提供了真正的商业/客户价值，以及如何围绕这种影响构建一个叙述，从而为晋升提供理由。

这可能意味着发现一项短期的“组织工作”陷入停滞，因为没有达成共识，鼓励一位参与其中的同事承担起做出决定的责任（并从相关方获得适当的反馈），并推动其完成。

这种感觉我一直贯穿我的职业生涯；我享受我的同事们的成功，就像我享受自己的成功一样。但是担任技术主管的角色无疑提升了做这件事的重要性。它从我喜欢做的事情，变成了我喜欢做并且对我要负责的团队的健康也很重要的事情。

你能想到你作为一名高级工程师所做过的，在你达到这个头衔之前你无法做到或不会做的事情吗？

正如之前提到的，一旦你成为 Mailchimp 的高级工程师，你就是 Eng Leadership 的一员。

值得注意的是，高级工程师通常对自己的时间有更多的自主权和所有权。当他们抽出时间做主要职责之外的工作时，他们遇到的阻力较小。这当然是我个人的经历。

我也获得了更多自然的机会来指导、辅导，以及普遍支持团队以外的更多同事。这当然是我在晋升为资深工程师之前就在做的事情；但它明显增加了。这绝对是我工作日中一个有益的部分，所以我很欢迎它。

行业内的同行经常会说头衔并不重要。但我百分之百不同意这种说法；我个人的经验以及在我工作过的公司里的观察都表明情况并非如此。

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

从某种意义上说，我想是的。

我以高级工程师的身份加入了 Mailchimp。我立即被加入到一个项目团队（其中包括一位工程总监和两位首席工程师），旨在构建 Mailchimp 的第一个内部自助式分析平台。

这个项目的关键方面是**高效并以高水平执行**更好或更坏的是，拥有另外两位首席工程师意味着对我的期望可能不会那么高。但我能够立即投入并开始为项目的核心方面做出贡献，几乎不需要他们的任何帮助；到最后，我成为了团队中的主要贡献者之一。我最终会被正式任命为技术负责人，以帮助继续引导该项目的工作，因为它被吸收到我当前的工程组，数据服务中。

这个项目的另一个关键方面是大量的**在整个公司的可见性**该项目团队的工作被归类为公司层面的倡议。这意味着很多高管层面的关注；当然，也伴随着相关的压力。但整个项目团队能够始终保持良好的势头，并最终成功构建了分析平台的初始迭代版本。此外，我的经理和团队的首席工程师有意为我创造机会来展示项目团队的工作；我最终在工程全体会议、公司范围内的全体会议上进行了展示，并在工程招聘活动中共同发表了一次技术演讲。由于该项目的知名度和Mailchimp的整体文化，我能够在加入公司后很短的时间内与公司各个级别的工程师合作，并与其他部门的分析师互动——这是大多数人需要一年左右的时间才能获得的机会。

所以，这既包括做好工作，又包括与更资深的工程师一起高效工作；以及那些资深工程师（和其他人）有意让我获得在整个公司的知名度和访问权限，即使他们是该项目的技术负责人。

重要提示：晋升直到我也担任了一段时间的技术主管并在那里交付了很多价值之后才到来。但这个项目绝对是导火索。

你还记得关于达到 *Staff* 级别的任何对你特别有用的建议吗？

第一个来自我的经理，[马克·赫德伦德](#)。他告诉我用第三人称写我的绩效评估。其想法是，在给别人做评估时，你更有可能赞扬别人，而减少批评。这很简单，但对我来说非常有用。奇怪的是，它在帮助我理解如何围绕我所做的工作以及它为企业提供的价值构建一个连贯的叙述方面发挥了作用。做和它为企业提供的价值。

第二个来自[丹·麦金利](#)，我提到的“Staff 项目”中的一位资深工程师。当我向他们征求意见时，他直接反馈了我的优点和缺点。他指出，我在公司内部建立关系的能力是一项重要的技能，因为工程的人员/社交方面不会消失；事实上，它们是完成任何事情的关键。

第三个来自[科达·黑尔](#)，我提到的“员工项目”中的另一位高级工程师。他谈到了扩大你的影响力。具体来说：

首先，有效地设定技术方向 为了其他工程师；其次，指导他们并发展 他们的才能是你所构建的工作的结果 为了他们。

这个建议是我思考作为技术主管的核心；就像，真正有意识地为团队创造机会，扩展、灵活运用他们的技能，并学到很多东西。

对于刚开始担任 *Staff* 工程师的人，你有什么建议吗？

不要以为你能解决工程部门的所有问题；从我所看到的，这会让你很快感到疲惫和厌倦。慢慢地积累这些事情。希望你因为你已经在 Staff 级别工作而被提升的；所以你不应该做任何非常不同的事情。继续做让你获得这个头衔的出色工作。一旦你觉得你正在适应，就从那里扩展。

沟通和构建叙事是关键。确保写.....很多。在思考问题或想法时，把它写下来（即使你不打算分享）。通常当我如果无法用连贯、简洁

的段落来表达一个问题陈述或想法，那就意味着我需要做更多的研究，或者我很难说服别人这是一项值得的投资。此外，文字可以让你更有效地扩展你的想法和围绕它们的讨论；这比与每一个可能的人安排会议来推销这个想法要容易得多。

尽量让你的经理们的工作更轻松。不要只是给他们带来问题；还要给他们带来（如果可能的话，多个）关于解决该问题的建议/建议，并征求他们的反馈。这样，经理就不必为你做所有解决问题的的工作（他们可能已经有很多事情要处理了），而且他们有机会从他们的经验中汲取经验，帮助你排除/选择你的建议。有趣的是，让别人对你的解决方案为什么不好提供反馈，可能比让他们自己想出一个完整的解决方案要容易得多。

开始有意识地思考如何为与你直接合作的其他工程师创造机会，以提高他们的技能，并让他们有机会接触到公司里他们通常无法接触到的人。

尽早建立关系。这可以在你需要花费一些社会/政治资本来对某件事采取立场的情况下有所帮助。例如，如果你第一次与某人接触是在为招聘问题的对立解决方案而战时，你已经从与那个人的赤字开始了。需要明确的是，你不应该寻求“取悦他人”。但是，如果你事先建立了关系，你与特定人员的工作关系以及在未来工作中进行富有成效的协作的能力可能会容易得多。

坦率地说，尝试应用所有这些东西很多；这只是一个人的观点。把它当作餐厅的菜单；选择与你产生共鸣的东西并尝试应用它。然后，把它传递下去通过与下一个试图晋升为Staff的人分享你的经验。

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

我经常被问到这个问题。

我的确有一个成为首席技术官的目标；但这更多的是一种方向性的东西，以确保我不会对我的职业发展感到自满。在Mailchimp的背景下，晋升为首席工程师对我来说就足以实现这个目标。我能够在目前的水平上交付真正的、切实的商业价值；所以我认为我可以继续这样做，并通过坚持个人贡献者的道路继续感到有回报。

此外，我一直在为我的团队履行Eng Manager的许多职责（但不是正式的Eng Manager）。因此，如果我转到管理岗位，我想开始做的很多事情，我都能够付诸实践并获得经验。所以，我确信这满足了我暂时的任何渴望。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

用各种技术和非技术受众进行关于技术的口头交流是一项技能。这是我每天都在努力培养和磨练的技能。我观察的一个人，从视觉上学习如何正确地做到这一点是[凯尔西·海托华](#)。另一位是我大学里几门软件工程课程的教授。他很少来上课；但当他来的时候，我通常学到的关于软件开发的知识比我在任何其他学术环境中都多。他们都知道如何很好地解释事情，并根据听众进行调整。

我遇到过几个博客，它们帮助我磨练和微调我如何开发技术方法和策略。第一个是[Pete Hodgson 的“实现架构战略”](#)，它提出了一个框架，用于在功能交付和基础架构工作之间实现可持续的平衡。第二个是[James Cowling 的“垫脚石而非里程碑”](#)，它是关于通过大型架构计划交付真正的价值。

Keavy McMinn - Fastly的资深首席工程师

这个故事录制于 2020 年 3 月。在她的网站上了解更多关于 Keavy 的信息[博客](#), [Twitter](#), 和[LinkedIn](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是一名高级首席工程师，在[Fastly](#). Fastly 是一个边缘云平台，提供诸如 CDN 之类的服务。我在 OCTO（首席技术官办公室）工作，该办公室由大约十名直接向首席技术官汇报的首席或杰出工程师组成。OCTO 的每个成员都有自己的重点，而我的重点是担任 API 负责人。

贵公司的 Staff-plus 工程师是做什么的？你如何安排你的时间？

OCTO 中有几种截然不同的首席和杰出工程师。也有直接在工程团队中工作的首席工程师，而不是在 OCTO 中。在 OCTO 团队中，有些人从事互联网标准或学术研究，有些人进行深入的技术研究和原型设计，有些人帮助孵化一个团队来构建全新的东西。作为 API 负责人，我与更广泛的工程组织密切相关。

我们都在做不同的事情，但我们有一个共同的目标，即对事物采取整体、长期和系统性的看法。我们还尝试寻找并帮助解决工程中可能被忽视或遗漏的事情。我们的首席技术官支持我们的工作，但不确定要从事的项目，这取决于我们自己。

我从未想过用百分比来衡量我的时间。有些我的工作分阶段进行，这周做的事情多一些，下周做的事情多一些。我大量的时间都花在写作、研究和与人交谈上。我会定期与构建 API 的团队和经理会面。我会花时间将长期战略分解成小块，进行研究，并撰写一份关于此的提案。然后我必须在公司内部推销该提案。最近写代码的时间少了，但在其他阶段，我会构建演示或工具来支持更广泛的工作。编码工作仍然非常愉快。

作为 Staff-plus 工程师，你觉得最有影响力的地方在哪里？作为 Staff-plus 工程师，你做了哪些在你职业生涯早期不会做的事情？

作为一名普通工程师，很难抽出时间。你必须在定期安排的项目的约束和节奏内工作。作为负责人，你有信任、时间和空间去尝试一些东西。

当你拥有头衔时，你就不必花费那么多精力来展示你的资历。它有助于为他人设定背景。你从一开始就更受尊重，这非常明显。你还可以接触到高管，因此你可以更早地获得信息，并且可能有机会在谈判桌上影响事情。

你是否花时间倡导技术、实践、流程或架构的改变？

我被专门聘用来为 API 制定战略方向。其中一部分是引导技术方向和我们所做的选择。我把它当作一种协作练习。你知道我会做研究的繁琐工作，我分析所有信息，提出权衡方案，并提出建议。我会考虑所有组织和工程团队的背景。

我提出我认为对我们来说最好的方案，人们可以不同意。而且，你知道，他们经常这样做。我更多的是引导和影响，而不是说，“我有权告诉你该怎么做。”我从未见过这种风格能奏效。

对于有争议的决定，我会与来自不同相关团体的代表会面。我会与一群工程师会面，告诉他们我打算推荐什么，并问“你们对此有什么看法？我遗漏了什么？”我还会与管理层和产品方面，以及可能还有法律、文档、安全或不同的人员会面，具体取决于项目。我也做过相反的事情，即先展示东西，然后通过电话获取反馈，而不是仅仅等待人们在文档中留下评论。

你倡导过什么？

在目前的工作中，我一直在努力倡导的一件事是 API 变更的设计文档。因此，在任何人编写任何代码之前，当进行更改的成本很低时，他们会写下用户工作流程，以及可以实现这些工作流程的接口是什么

样的。有时，事实证明，看似简单的事情实际上非常困难，特别是当团队不习惯锻炼这些能力时。

我对倡导的方法是提醒自己，是什么痛点让每个人都感到痛苦，从而促使我们尝试做出改变。我们不是为了理论、代码美观或崇高的担忧而试图做到完美。我把它带回到“这些是每个人都说他们感受到的痛点，而这是一种你知道最终会有助于解决这些问题的方法。”

我帮助其他人关心同样的事情，比如我将开始与更多的工程师一起进行 API 审查。在进行审查时，我尝试帮助教导其他人我在看什么，并在整个过程和对话中给予鼓励。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

在我目前的角色中，这并不是我关注的重点。在 GitHub，我意识到我的资历和任期赋予了我特权，并赞助了一位工程师。我会给他越来越具有挑战性的工作，鼓励他质疑任何不清楚或让他对我的工作感到好奇的事情，并为他争取更多的责任和认可。

你是如何建立组织信任的？

在 Fastly，我从一开始就获得了信任。当我加入时，我被聘请来从事特定的工作。我记得我问，“这有一个时间表吗？”以及他们对战略的看法，并被明确告知他们希望我来弄清楚并告诉他们。所以绝对有很多信任和责任。

当你建立这种信任而不是被聘用时，会有优点和缺点。当你建立信任时，你会同时建立大量的背景信息，这就是我在 GitHub 的工作方式。虽然，我发现在我目前的工作中，没有背景信息进来，成为一双全新的眼睛实际上非常有用。这使得当人们认为“哦，好吧，也许我们一直都是这样做的”时，很容易提出质疑。不被过去束缚可能会很自由。

有一种流行的观点认为，成为一名 Staff 工程师需要完成一个“Staff 项目”。你是否有一个“Staff 项目”，如果有，那是什么？

我从未听说过它被命名，但我理解这个想法。我确实领导并设计了那种类型的项目——解决棘手的工程问题，对公司产生重大影响——几次，但不幸的是，它们并没有让我得到晋升。但它们确实促进了我的职业发展。这些项目给了我经验、知识和信心，让我能够以不同的方式定位自己。甚至可以公开会议演讲，或者知道“我已经做过 X，可以再次做 X”。

公开演讲或公开露面对于达到你目前的水平重要吗？

是的，我认为这对我职业发展来说是一个巨大的因素。我认为这不是必要的，但我认为它肯定会有帮助，它对我很有帮助。我的第一次会议演讲是被邀请去做的——因为组织者认为我有一个有趣的视角可以分享，来自艺术背景。这很可怕，最初我想拒绝。但我母亲说服我答应了。所以，公共演讲一开始是一种比深思熟虑的策略更偶然的方式。

主要是，我喜欢在会议上遇到的人。后来，演讲者网络为我带来了工作机会。

你是如何第一次获得Staff或Principal头衔的？哪些因素对你获得这个头衔贡献最大？

我被Fastly聘为首席工程师。所以，老实说，对我来说，最大的因素是更换公司。我所做的工作类型并没有发生巨大的变化，但更换公司最终使我获得了这个头衔。

有人强烈主张专门聘用我，我相信这有帮助。他们不是我以前直接合作过的人，但他们熟悉我的工作。

远程工作是否影响了你的职业轨迹？

就我所知没有。我一直是一名远程员工，我相信这在能够进行意外的对话方面起到了作用，但我已经做了这么久了。你会刻意努力进行对话并建立关系。此外，我的公司在很大程度上是分布式的。我可以想象，如果远程办公是少数，或者公司没有完全接受分布式办公，这可能会成为一个更大的潜在问题。

你是否得到过关于晋升到Staff职位的建议，这些建议对你特别有帮助？

没有，我得到了一些糟糕的建议。这太老套了，但“这很棒。现在你必须再次证明它。”有些建议会促使人们朝着英雄的方向发展，比如认为你需要发明一些不寻常或神奇的东西才能获得资格。一个人可以朝着很多不同的方向发展。工程阶梯通常会导致这些信念。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

我首先想到的是找到你的同伴或支持网络。就像管理一样，你爬得越高就越孤独，找到仍然会挑战你并且你可以一起集思广益的同伴非常重要。即使他们在你类似的工作领域，甚至在不同的公司，也没关系。

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

我尝试过一次，但没有好的体验。我意识到这根本不是我的热情所在。我对工程管理非常尊重，不会为了正确的理由之外的任何事情去做。正确的理由是支持其他人。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

会议对我来说是一种资源，同时也能与成熟、低调、优秀的工程领导者和工程师一起工作。查德·福勒，以及他的书[《充满激情的程序员》](#)，排在首位。Dave Thomas 是我在刚开始学习 Ruby 时经常参加

其研讨会的另一位人物，他的书[《程序员修炼之道》](#)是另一本伟大的书。

Bert Fan - Slack的资深工程师

这个故事录制于 2020 年 5 月。在他的网站上了解更多关于 Bert 的信息[博客](#), [Twitter](#), 和[Linkedin](#).

请简单介绍一下您目前的角色：您在哪里工作，您的职位是什么，以及您和您的团队通常做什么样的工作。

我是 Slack 平台团队的高级工程师。我很幸运能在我们推出[Slack 应用目录](#)后不久加入 Slack，因此我有机会帮助 Slack 平台发展成今天的样子。

很难概括我所做的工作，但目标始终相同：使开发人员能够在 Slack 的基础上进行构建，从而使我们客户的工作生活更简单、更愉快、更高效。这方面的例子包括构建新的平台功能、提高 API 性能、编写文档以及与合作伙伴、内部集成商和第三方开发人员合作，以确保他们能够构建有影响力的软件。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

在 Slack 担任 Staff-plus 工程师的工作内容非常多样化，具体取决于您所在的部门、团队的组成和规模，以及业务对您的需求。Staff-plus 工程师通常是项目的技术负责人，这意味着他们帮助编写技术规范，从各个利益相关者那里获得反馈，与设计 and 产品部门密切合作以决定要构建什么，并领导项目的技术实施。他们还指导其他工程师，改进我们的面试流程和工程文化，开发工程流程和工具，并为重构和技术债务提供技术指导。Staff-plus 的全部意义在于使其他人能够更好地工作——成为一种力量倍增器。

我的角色包括我提到的所有内容，但我倾向于较少关注特定的技术，而更多地关注技术所实现的功能。因此，我可能会花时间制作几乎肯定会被丢弃的概念原型，或者收集围绕特定用户流程的使用指标，以更好地了解如何改进系统。我经常在 Slack 平台上构建应用程序，以

保持我对开发者体验的真实了解，并积极尝试在其他人的平台上构建，以了解哪些有效，哪些无效。与公司其他工程师相比，我编写的代码进入生产环境的要少得多，我完全接受这一点。

你能想到你作为一名高级工程师所做过的，在你达到这个头衔之前你无法做到或不会做的事情吗？

去年，我对各种产品创意进行了大量实验，其中一些已经演变成我们平台的实际功能。我之所以有机会这样做，部分原因是我通过其他成功交付的项目建立了信任，但拥有 Staff-plus 的头衔似乎让我在选择工作内容方面有了更大的灵活性。

不管好坏，我还参加了很多以前从未参加过的战略和规划会议。如果你曾经想从领导的角度了解香肠是如何制作的，也许可以考虑一下你是否真的想知道香肠是如何制作的。

当您花在实际开发上的时间越来越少时，您如何与事物的实际运作方式保持联系？

我发现最好的方法是与工程师定期进行 1:1 的交流在整个公司范围内，花大量时间倾听。如果你花时间建立关系，让工程师们觉得他们可以对你坦诚相待，你就能了解到很多关于工程的现状。作为一名 Staff-plus 工程师，你对某人的收入或下一次晋升没有实际的影响力，所以如果你让自己容易接近，工程师们可以更坦诚地对待你。

您达到 Staff 级别最重要的两三个因素是什么？您加入的公司、您的地理位置或您的教育背景对您的职业道路有何影响？

我来自一个相当优越的背景——我大学毕业，获得了计算机科学学位，没有债务或学生贷款，这给我带来的一部分好处是，我可以灵活地离开一份工作，而不必担心如何支付房租或是否能找到新工作。我利用这种灵活性，在我想去的地方变得更加挑剔和更有策略。

我承认大多数人没有这个选择，但对我来说，我认为重要的是做那些我觉得有意义的事情——那些我个人使用过的，我认为对世界有积极

影响的事情。而且我相信这些选择已经得到了回报，因为这样的公司会吸引志同道合的人，他们会去其他公司，希望这些公司也能以同样的方式保持一致。这不是精英管理，你的人脉很重要。我得到过工作，因为我和其他人一样在公司的网站上申请过，但也得到过工作，因为我尴尬地给一位我已经多年没说过话的经理发了邮件，但我尊重他，并且知道他尊重我作为一名工程师。我们所处的行业有很多选择如何度过你的时间，如果你足够幸运，能够有灵活性和特权去选择，那么你不定期评估你所做的事情就是在亏待自己。

也许在某个时候，你会成为那种工程师，当你在 Twitter 上宣布你开始一份新工作时，以前和你一起工作过的人会创建一个四年后的日历提醒，那时你已经完全归属，这样他们就有最大的可能性挖走你，但在那之前，你将不得不给那些你想再次合作的人写一些尴尬的电子邮件。

你还记得关于达到 Staff 级别的任何对你特别有用的建议吗？

我听过的最好的建议是，通常晋升到 Staff 级别是运气、时机和努力的结合。以下是我观察和亲身经历过的一系列事件：

1. 与你的经理建立一种关系，在这种关系中，他们完全信任你，你也完全信任他们。坦诚、直接地告诉他们你想要什么。建立这种信任需要你成功地完成他们要求你做的事情。
2. 因为你的经理信任你，当他们听说对公司有重大影响的项目时，他们会支持你领导这些项目。或者，你必须自己找到或创建一个项目，并倡导它的实现。这要困难得多，但仍然是可行的。
3. 成功交付项目。
4. 该项目对公司有重大影响。
5. 因为你成功交付了一个对公司产生重大影响的项目，所以很容易支持你晋升为Staff，你的经理也很乐意这样做。

希望你能明白，运气和时机可能会影响这个简单的计划——如果你和你的经理相处不好怎么办？如果你的经理离开公司或升职了怎么办？

如果你在公司里一个没有意思的领域怎么办？项目？如果项目注定要失败呢？如果项目成功了但没有影响呢？

这些都是有可能发生的，除了有时候你永远不会得到晋升，你应该对自己诚实，并在你处于这种情况时识别出来之外，我无法给出任何通用的建议来克服这些问题。在这种情况下，有时获得晋升的唯一方法就是离开公司去做其他事情。你可能会在职业生涯的后期以比你离开时更高的职位回到公司，但就像一段你重新审视的失败的感情一样，你还想待在那里吗？或者是否有太多的包袱让它永远无法成功？

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

这在工程界算是一个老生常谈的笑话，但很多人进入这个行业是因为他们不喜欢与人交谈，但要胜任你的首席工程师工作，你很可能要花大量时间与人交谈。我认为在你的职业生涯早期，你可以通过专注于编写越来越好的代码来取得进步，但在某个时候，你应该转向专注于更好地与他人合作。信任他人，给予他们做出技术决策的自由（即使是你不同意的！），理解他人的动机，学会给出困难的反馈，知道何时选择你的战斗——这些都是有用的技能。

如果你还没有这样做，试着成为人们想与之共事的工程师。每家公司都有少数工程师，如果你离开你的工作，你会试图绕过一份禁止招揽协议，再次与他们共事。成为其他人的那种工程师，它会为你的职业生涯打开很多扇门。

你有没有考虑过工程管理，以及如果是这样，你如何决定走首席工程师的道路？

在我的职业生涯早期，我曾经告诉我的经理，我担心我迟早要进入管理层，他们说了一些类似这样的话，“别担心！在我们公司，我们有两条不同的轨道，管理轨道和IC（个人贡献者）轨道，并且在IC轨道上有与最高管理职位相当的角色，所以你永远不必为了在公司晋升而转到管理层。”虽然技术上是正确的，但现在对我来说显而易见的遗漏是，在很多公司，管理轨道比工程轨道要清晰得多。

你在工程阶梯上爬得越高，你所能效仿的例子就越少，而且这些例子似乎越来越遥不可及。当你开始深入研究时，你可能会意识到有人在他们的公司被收购时获得了这个头衔，或者他们是某种编程语言或框架的作者，或者他们为公司带来了数千万美元的收入。

我的很多同事因为各种原因进入了管理层，我怀疑其中一个原因可能是我所描述的更明显、更可靠的晋升。但我坚信这不应该是你主要的动机。如果你有开放的日历，看看你经理的日程安排，以及他们每周进行的1:1会议的数量。那是你喜欢的日程安排吗？编写代码的愿望并不是非黑即白的，因为有技术主管经理职位需要编写代码，也有首席工程师以上的职位永远不需要编写一行生产代码，而把大部分时间花在Google Docs或Dropbox Paper上。但在我的职业生涯中，我从未不得不解雇某人或拒绝他们的晋升或撰写绩效评估——我知道我宁愿站在哪一边。

Katie Sylor-Miller - Etsy的前端架构师

这个故事录制于2020年8月。在她的网站上了解更多关于凯蒂的信息 [网站](#), [Linkedin](#), 和 [Twitter](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我在Etsy工作，Etsy是世界领先的手工商品在线市场。我们让卖家把他们的商品拿出来出售，并卖给世界各地的人们。我们真正专注于为人们提供独特的、特殊的或手工制作的物品，这些物品可以替代大型商店的无个性。

我目前在前端系统团队，这是一个产品基础设施团队，负责我们的前端架构，包括我们的PHP视图渲染框架，尽管我现在没有非常积极地参与团队正在做的工作。在过去的几个月里，我一直专注于Web性能——以顾问的身份处理所有与性能相关的事情——改进我们的监控和报告系统，识别需要改进的领域，并为产品团队提供与性能相关问题的帮助。

我认为，许多公司要么忽视网络性能，要么不重视它。当我刚到Etsy时，由于Lara Hogan等人的努力，我们拥有出色的性能文化，但由于几年前的组织变革，我们不再拥有网络性能团队，我认为作为一个组织，我们安于现状，降低了网络性能的优先级。现在，我们正把它重新摆在首位，因为行业内围绕“良好”性能的标准发生了很多变化被定义和衡量，特别是对于搜索引擎优化（SEO）。谷歌真的在推动网络性能成为公司关注的一个标准，作为他们搜索排名的一部分。所以这在很大程度上是一个首要考虑的领域，特别是对于零售商来说。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

对我来说，关于我们如何看待“资深工程师”的角色，有趣的是，我们把一个人可以成为资深工程师的两种不同方式，都放进了一个叫做资

深工程师的桶里。但实际上，有两个不同的桶。

有些人可以通过成为特定主题领域的专家，或者真正承担起技术负责人的角色，从而在他们的职位上变得非常资深，他们推动团队或组织的的技术方法和路线图。还有另一种成为高级工程师的方式，这些人扩大了他们的工作范围和重点，以便他们思考跨领域的问题，他们推动创建跨多个团队运营的系统 and 实践。我认为第二个类别是架构师。这并不是说你不是一个主题专家，而是说你的影响力范围大于作为一个特定团队的技术负责人。

在Etsy，我们几个资历级别：高级工程师I和II，然后是Staff I和II，然后是高级Staff，这被认为相当于总监级别的角色。从技术上讲，我是Staff工程师II，我也是这样看待自己的，但我的具体角色是前端架构师。这意味着我不只是对我团队正在做的事情负责，我还要负责查看Etsy在前端领域所做的一切。未来会是什么样子？我们需要解决哪些问题？我们将如何到达那里？我会考虑所有这些，并倡导能够让我们到达那里的技术方法公司层面。

作为一名架构师，你是否花很多时间进行软件开发？

是的，这很有趣。我是一名前端架构师，但到目前为止，我最近一直在编写的主要内容是 SQL，因为我正在进行大量的数据分析。我一直在查看我们的性能指标，以找出需要改进的领域，以及修复哪些最具影响的问题可以提高性能和业务指标。我会在这里和那里编写一些 JS 或 PHP，但主要目的是帮助团队解决问题，或者运行与性能相关的小型实验，或者如果有什么重要的事情需要完成，但其他人没有时间。

我确实发现我的节奏变慢了，而且随着我的日历上排满了会议，我需要花费更长的时间才能真正找到专门的专注时间来编写代码。所以我想你不会再想让我写太多代码了！我更专注于识别机会领域，然后尝试将其作为我的团队或其他团队应该做的工作来推销。

你每天如何度过你的时间？

我会说 50% 的时间在开会，剩下的时间每天变化很大。有时我用剩下的 50% 的时间写文档，有时我在 SQL 中做大量的数据分析，有时我在 Slack 上与多个团队和角色的人交谈。有时，当项目落到我头上时，我的会议负担会增加一些，我会联系其他团队，了解他们正在做什么，或者试图影响他们做出改变。变化很大。

我发现这些职位的人常常难以量化他们的工作，你有没有发现什么有用的方法来衡量你的影响力？

我很高兴听到你这么说，因为这确实是我非常非常挣扎的事情。我总是有很多不同的项目和讨论同时进行，而且我有一个不好的倾向，就是会被最新的事情吸引或者让我的注意力游移，所以我必须非常认真和有意识地组织我的工作和笔记。我总是看着所有我可能做的事情，然后选择那天对我来说最有影响力或最重要的事情去做，这可能真的很难。

直到我转到架构师的角色，我才意识到我多么依赖冲刺、JIRA看板以及完成一个任务并将其移动到“完成”列的仪式，以此来检查自己，并知道我正在完成我需要完成的事情。现在我没有那种团队环境来帮助我组织我的一天，我不得不更多地依靠我自己的待办事项清单，而且我仍在努力改进我的系统。

有一件事绝对有帮助，那就是确保我记录下我每天完成的所有不同任务——记录会议、电子邮件、Slack讨论等等。然后，当我和我的经理进行正式的季度目标检查时，我回顾我的所有笔记，然后意识到，哇，我帮助工程师修复了六个不同实验的性能问题，或者我影响了这个团队在新功能方面采取了更好的方向，或者我给了那个工程师反馈，帮助他们。这些都是小事，在当时感觉没什么，但加在一起就显示出真正的影响。

作为一名Staff-plus工程师，你觉得在哪里最有影响力？

我最喜欢做的事情是发现一个以前没有解决过的新的或独特的问题，想出一个疯狂的想法来解决这个问题，然后我出色的同事们会采纳这

个想法，并真正地用它来构建一些太棒了。首先要从人们正在做的工作中获取大量信息——看到这个团队在做x方面有问题，而另一个团队在做y方面有问题。然后，将所有这些信息与你的经验以及整个行业正在发生的事情混合在一起，让它在你的大脑中停留一段时间，直到最终一切都点击，你意识到这两个问题更深层的原因是z，所以你想出一个解决这个问题的计划，这真的很难解决。

在我转到架构师角色之前，我的团队拥有我们的设计系统组件。更改或修复我们共享组件的问题非常困难，因为我们没有每个组件的标记和模板的单一来源。公司里的每个人都没有重复使用同一个模板文件，而是将HTML复制并粘贴到许多不同的地方。因此，当我们必须更改组件时，很难找到所有需要更新的地方，因为组件的各个部分分散在不同的地方进行管理——有时在JavaScript中，有时在Mustache中，有时在PHP逻辑中。

所以我有一个疯狂的想法：如果我们扩展我们的自定义PHP框架，以在mustache中启用可重用的模板块来表示我们所有的组件，并且我们能够像在React应用程序中那样轻松地将它们组合在一起，会怎么样？我出去做了一个概念验证，并为该项目写了一份提案，并将其提交给了团队。然后团队真正地接过了球并开始行动，他们构建了支持这个组件系统的基础设施，结果比我自己做的任何事情都要好得多，也更健壮。

我真正喜欢的部分是发现问题，并创造性地思考我们如何解决它，然后四处推销这个提案，并让其他人参与到执行工作中。

从事前端工作的人能否像开发人员生产力或基础设施角色的人一样，为公司创造杠杆作用？

是的，当然。我个人只认识少数几个专门从事前端的职员工程师，而且我认为前端作为一种技能在行业中的价值并没有得到应有的重视。我很幸运能在像 Etsy 这样的公司找到一份工作，这家公司倾向于招聘“全栈”工程师，因为我拥有计算机科学的基础知识——我学的是计算机科学，并且我在整个堆栈中都有工作和理解的经验。但实际上，我

的热情和关注点一直都在前端，因为它就在你的用户面前。我希望看到更多的公司重视前端，因为我相信我们带来了宝贵的技能和独特的思考方式。

就成为一名技术专家而言，我认为一名优秀的技术专家的素质超越了你所使用的技术栈。最终，技术专家需要能够将工程决策视为一系列权衡，而表达这些权衡是一种你可以从技术栈中的任何角度获得的技能。

我还认为，技术专家应该对与其自身专业领域相邻的所有领域有广泛的了解。对我来说，在前端工作，我投入了大量的时间和精力来理解市场营销、业务目标、用户体验、视觉设计、服务器上的视图和业务逻辑层、我们如何将代码发布到浏览器、浏览器如何将所有这些代码转化为网站，以及用户如何与之互动。在所有这些不同领域拥有专业知识使我更容易看到我的技术决策的更广泛影响，并更好地理解这些权衡。

对你的用户有同理心，特别是，对于所有类型的工程师来说，都是一项重要的技能，我认为它可能是在许多基础设施或开发者支持组织中被低估了，他们不明白，是的，他们实际上有用户！我在前端基础设施工作，我们真的努力将自己视为产品工程师——只不过我们构建的产品是供其他工程师使用的系统。所以我们有客户。我们有用户。当我们考虑我们构建的系统的API时，我们正在为用户设计我们的API，我们需要了解我们的用户——也就是产品工程师——才能做好这一点。

所以我个人认为，倾向于前端的人非常适合成为技术专家，因为他们非常习惯于不断思考用户以及用户将如何与他们构建的东西互动。用户同理心是前端人员带来的超能力。

当你自己做的开发工作越来越少时，你如何保持同理心，并了解公司开发的实际情况？

建立人脉，建立人脉，建立人脉，建立人脉。一对一的交流尤其重要，因为我是一名全职远程员工。显然，现在每个人都是远程办公，但作为一个非完全分布式团队的远程员工，你必须真正了解你在和谁说话，并确保你在多个团队和多个小组之间建立联系，以利用这些网络。

在Etsy，我们非常幸运，我们有一些不同的员工资源小组，供员工在公司内部建立联系。我非常积极地参与了技术领域中边缘化性别身份的ERG（MAGIC），这很棒，因为该社区的成员在工程部的每个部门工作。远程员工社区也是如此。我抽出时间指导更年轻的员工，定期进行1:1的交流，并参与Slack讨论，以培养和发展这些联系，因为这对于了解组织中正在发生的事情非常有帮助。我还尽量确保我与整个产品工程部门的工程师交谈，因为产品工程师是我们的客户群。

我正在努力改进的一件事是与管理者建立更多的联系。长期以来，我在个人贡献者轨道上拥有非常好的网络，并且在过去的几个月中，我一直在努力扩大我的网络范围，以包括更多的工程经理。很多时候，我所做的工作需要“无权影响”，我不是自己做决定，而是试图影响其他人正在做的决定，而且很多时候，管理者是最终决定事情的人。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

我很幸运能在 Etsy 与 Lara Hogan 共事几年，她谈论了很多关于赞助的事情，作为一名科技行业的女性，我从赞助中受益匪浅，也看到了它的价值。我肯定投入了很多时间和精力在这上面。

大约一年半前，我的同事 Andy Yaco-Mink，另一位 Staff Engineer，和我注意到产品团队之间并没有一个很好的沟通方法来分享他们正在做的事情，或者与从事产品基础设施的团队联系。为了尝试解决这个问题，我们提出并启动了一个每月一次的会议，我们称之为产品工程会议（Product Engineering Confab）。这是一个开放的论坛，人们可以在这里提出问题，分享他们的工作，庆祝胜利，以及我们基础设施方面的人员分享我们正在做的事情。

我认为我们没有完全预料到的是，这也是一个创造赞助机会的好方法。每个月，Andy 和我都必须弄清楚人们正在做什么，这些事情会很有趣，可以更广泛地分享。什么是已经运行并获得有趣结果的实验？谁在那里做着应该分享的酷事？然后我们会联系这些团队的工程师，说：“你应该来会议上谈谈你一直在做的事情！”这真的很简单。只有五分钟。非常随意，但这是获得公开演讲经验的好方法。

从那以后，我们有几个人来会议上发言，然后继续在公司全体员工会议或当地聚会上发言。至少有一个人最终在大型会议上发表了他们演讲的扩展版本。我们也从人们那里听说，在会议上发表演讲是他们用来证明自己在晋升材料中领导力的证据，这是一种了不起的感觉！

你第一次在你现在的公司获得了 Staff engineer 的头衔。晋升为 Staff 的过程是怎样的？

我被聘为高级工程师，因为当时我们没有直接聘用 Staff 职位，尽管我们后来改变了这一政策。在加入 Etsy 之前，我已经在该行业工作了近十年，但主要是在规模较小、不太知名的公司。在加入 Etsy 之前，我担任前端技术负责人已经超过五年。因此，我已经非常适应作为导师和领导者的角色。我已经花了很多时间与管理层、产品和设计部门密切合作，以及制定路线图和执行计划。总而言之，我觉得我已经完全掌握了技术负责人的角色。

但是，当我来到 Etsy 时，规模比我以前见过的要大得多。工程部门比我以前工作过的任何工程部门都要大几个数量级。我有很多东西要学习，关于如何在一个非常大的规模上运作，以及这与你是一家小公司时有多么不同。我学会了更加注意查看数据：我不得不去自学基础统计学，以理解实验框架。

但从一开始，我就一直在寻找我们可以改进的地方。我进来后说：“哦，嘿，我们没有做这件事。我们应该做这件事。”例如，我注意到人们一直在以任何旧的方式编写设计系统 JavaScript 组件，所以我说“让我们为此制定一个框架和一个标准样板。”这是一件很小的事情，对我来说也很明显，但这是我们实践中的一个重大改进。我认为让一

个人成为 Staff 的很多因素是注意到问题并主动采取行动解决它们，而不是让它们过去。

总而言之，在晋升为 Staff 之前，我在 Etsy 工作了不到两年。我当时的经理是全新的，不了解我的过往记录，所以我们非常密切和协作地一起整理我的材料。我听说过经理主导和 IC（个人贡献者）主导这两种极端情况的经验，我很高兴我最终成为了晋升过程的重要组成部分。特别是作为一名远程员工，我认为除非你积极主动，否则你的很多工作可能会被忽视，因为它发生在 Slack 上、在 pull request 或文档中，而不是在经理们倾向于工作的公开场合。你永远是你自己最好的倡导者，但作为一名远程员工，这一点更为真实。你必须付出很多努力来确保你的成就广为人知。

您达到 Staff 级别最重要的两三个因素是什么？您加入的公司、您的地理位置或您的教育背景对您的职业道路有何影响？

我已经讨论过一些类似的事情：创造力、积极主动、同理心等等。我还没有谈论足够的是沟通和透明度。晋升为 Staff 的一个重要部分是确保你的工作是可见的，人们知道你的名字，并且你有一个良好的声誉。

我很幸运能在一个构建前端基础设施的团队中工作，因为我们自然会给工程部门的每个人写很多关于我们所做工作的电子邮件，所以我们获得了很高的知名度。但基础设施部门更重要的部分是客户服务——帮助那些在 Slack 频道中提出问题或需要解决问题的人。在回到学校完成计算机科学学位之前，我曾在服务行业工作过几年，我总是试图在我与同事的每一次互动中，把我从那段经历中学到的关于客户服务的经验应用起来：随时待命，保持谦逊，专注于真正倾听和理解人们的需求。当你真正关心帮助我们的同事时，这一点就会显现出来。

你认为有些公司特别擅长培养 Staff 工程师吗？

老实说，除了 Etsy 之外，我真的不知道 Staff 工程师在其他公司是如何工作的，所以我完全有偏见！我认为 Etsy 擅长培养 Staff 工程师，

因为我们拥有强大的内部文化，重视技术卓越，同时拥有无责备文化和在更大的世界中做好事的愿望。我认为这导致了真正聪明和善良的人在 Etsy 工作，而智慧和谦逊的结合使人们成为伟大的 Staff 工程师。这种环境会自我滋养，创造出好的榜样，以及想要效仿这些榜样以获得晋升的人。所以我认为总的来说，我们有一群很棒的人在 Etsy 工作或曾经工作过，并为 Staff 工程师树立了良好的榜样。

我认为重要的是要记住，有很多规模较小、不太知名的公司也有很棒的人在 Staff 工程师类型的工作，但他们不被称为 Staff 工程师，而是以其他方式被认为是技术领域的领导者。在许多公司里，那些强大的技术负责人会成为管理者，甚至可能没有 Staff 工程师这样的职位概念。在大公司里，很容易沉迷于 Staff 这个头衔，认为它是最终目标，但请记住，发展你的职业生涯有很多不同的方式。

你还记得关于达到 Staff 级别的任何对你特别有用的建议吗？

有人给我的最好的建议之一，也是我确保传递给其他 Staff 工程师的建议是，人们有一种误解，认为你成为 Staff 工程师后，你就可以控制你所做的工作，每个人都会听你的，并按照你想要的方式去做。这绝对与实际情况相反！你有一个非常明确的目标，那就是长期以来获得晋升，然后你成为了 Staff 工程师，突然之间，一切都变得模糊不清。你从解决一些明确的问题，转变为负责找到正确的问题，然后弄清楚如何说服人们解决这些问题很重要。你将面临与你迄今为止的职业生涯完全不同的挑战。

你对追求 Staff 职位的人有什么建议？

在我获得晋升之前，我曾两次被提名晋升为 Staff 工程师；第三次才成功。我认为最终推动我前进的是，我的主管是我的一个很好的赞助人。所以我的建议是，确保你发展你的关系网，并开始与你的主管或你的副总裁会面，因为这些人是在房间里做出关于你是否获得晋升的决定的人。不是你的同事，也不是你的经理，而是另一群人，所以你应该希望他们知道你的名字和你的工作。在晋升讨论中，你应该希望他们会想，“哦，她发了那封关于这个项目的全公司范围内的工程邮件。”或

者“我经常在 Slack 上看到他回答人们的问题。”或者“他们在那个会议上发言了，不是吗？”

当人们，特别是女性和非二元性别者，来找我寻求建议时，他们可能期望我谈论如何成长为一名技术领导者，但当我说“你可能已经具备了技术能力，你需要做的是提升你在公司的声誉。”时，他们会感到惊讶。无论好坏，没有良好的声誉，你无法晋升到Staff级别。我认为人们希望或期望这是一个精英体制，但事实并非如此。有很多因素会影响你获得Staff级别的职位。

对于应对更高级职位带来的不确定性和模糊性的建议？

重要的是培养大量的自我认知，以了解你追求某件事是因为你想要它，而不是因为它对组织有益。这真的很难做到。你必须准备好放弃你珍视的东西，进行调整，并尝试新的事物。如果你采取的方法不起作用，不要试图强行推进。

我也非常喜欢[Dan Na关于克服摩擦的演讲](#)，因为当你成长为一名技术领导者时，你会不断地体验到这一点。我经常思考“无权影响力”这个概念，因为当你是一名Staff工程师时，你的工作是弄清楚团队或组织需要做什么，使组织围绕这个目标保持一致，并弄清楚如何在你不控制人员配置或最终决策的情况下让人们做到这一点。这需要大量的毅力，并且你必须灵活运用大量所谓的非技术技能来推动事情向前发展。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

我已经说了很多名字，特别是[拉拉·霍根 \(Lara Hogan\)](#)，[丹·娜](#)。我只是喜欢[Julia Evans](#)是的，我真的很幸运能和她合作一个项目。[林恩·丹尼尔斯](#)他过去曾在Etsy工作，经常在博客上发表关于职业发展的文章。[Tanya Reilly](#)作为另一位厉害的职场妈妈，同时也是一位受人尊敬的技术领导者，她给了我很大的启发。在前端领域，[妮可·沙利文](#)，[珍·西蒙斯](#)，和[伊森·马科特](#)仅仅列举几个就给了我很大的启发。我真的

很喜欢读卡米尔·富尼埃的[管理者之路](#)。我从来没有走过管理路线，所以这有点像一个黑匣子，任何能让你深入了解管理世界的知识都是有帮助的，因为作为一名高级工程师，你几乎就像一个没有人员方面的经理。

Ritu Vincent - Dropbox的资深工程师

这个故事录制于2020年3月。在她的网站上了解更多关于里图的信息 [Linkedin](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是Dropbox的一名高级工程师。实际上，我曾经是Dropbox的一名高级工程师，后来离开加入了一家不同的初创公司，然后最近几个月又回到了Dropbox。我回来是因为Dropbox内部出现了一个非常有趣的机会，可以启动一个内部孵化器。我们正在努力促进公司内部的创新。Dropbox在文件同步领域已经成为一个强大的品牌，但现在开始出现很多竞争，所以我们需要做更多的事情，并扩展到新产品。孵化器直接与首席执行官合作，是一个非常小的团队。

我在 Dropbox 工作了足够长的时间，与这里的很多人建立了非常紧密的联系，所以当人们向我推荐这个角色时，听起来真的很有趣。我也当了几年的经理，有点想再次编写代码。把这些放在一起，让我回到了 Dropbox。

孵化器有两个部分。

第一种是更经典的孵化器，公司里的工程师可以提出想法并获得资金加入该计划，并尝试展示产品与市场的契合度或其他形式的进展，以便每隔几个月继续获得资金。目标是让成功的项目发展成为他们自己的业务线，尽管我们还处于早期阶段。

第二部分是永久性的工程师孵化器，并且总是从孵化器内部产生想法，并在很大程度上自主运营。我是那个常设“侦察”团队中的两名工程师之一，我们计划在明年扩大规模。这与我以前做过的任何事情都非常不同，这就是我想要加入的原因。这对我来说是一个巨大的范式转变。老实说，最初的几个月既有趣又令人沮丧，因为当你的主要目标是快速尝试大量新想法时，很难衡量明显的成效，其中许多想法都

不会有任何结果。我不得不学会以更长的时间尺度来思考影响——不是从我今天交付的东西来看，而是从我未来可能影响公司交付的东西来看，

在贵公司，Staff-plus 工程师是做什么的？

我想说，在 Dropbox，Staff Engineer 有两种不同的类型。一种是技术负责人，他们做大量的协调工作，为团队设计工作，并花时间推动项目。另一种更像是一位专家。

当我最初成为 Staff 时，技术负责人绝对是我的个人资料，当时我带领一个由大约八名工程师组成的团队，并推动了一个为期 18 个月的项目。该项目有很多依赖关系，很多棘手的部分。我必须控制围绕该项目的沟通，并弄清楚如何以一种既能帮助他们成长又能完成项目的方式将项目的各个部分分配给团队。

这位专家深入专注于某个特定领域，例如[吉多·范罗苏姆](#)，Python 的创建者。专家们会承担非常复杂的项目并自己执行，通常是其他人无法有效承担的项目。专家比技术主管少。

这些专家主要是在外部招聘的吗？

有一些专家来自行业，比如 Guido 和 ML 团队中许多非常有经验的人，但很多专家最终都是自己培养的。这可能与 Dropbox 相对较晚推出职称有关，这让人们有更多的时间来深入了解我们的技术。

你每天如何度过你的时间？

在我目前在孵化器中的角色中，我整天都在做原型设计，但在我之前的技术主管角色中，我做了很多不同的事情。

我在写代码，但我写得不多，可能只占我时间的 20%。我是桌面客户端领域的技术主管，花了很多时间协调和提供项目指导。我还花了很多时间与招聘部门合作，这是我感兴趣的事情，而不是因为这是必需的。

例如，我参与了专业面试流程的设计、主持汇报和候选人筛选。我还做了很多关于多元化倡议的工作。这就是我在职业生涯中多次尝试工程管理的原因之一，因为我喜欢参与组织发展。

作为一名Staff-plus工程师，你觉得在哪里最有影响力？

我真正为之自豪的一件事是，我对我们的工程级别进行了重大改革。早在2017年，我是少数被选出来参与工程师级别更新的个人贡献者之一，其他大多数人都是主管或经理。我感到自豪，因为新的阶梯影响了Dropbox中从事工程、产品和设计的每个人。

深入思考公司发展如何改变角色和职责也真的很有趣。我们开始从很多不同的背景引进人才，我们希望能够以一种健康的方式奖励每个人。这与我正常的日常职责非常不同，并且在很大程度上将我推到了舒适区之外。

我也为我的Staff Project感到自豪，它在技术上非常复杂。那个项目也给了我一个机会来帮助团队中的很多人成长。多年以后，我收到了一些已经离开公司的工程师发来的邮件，他们说他们因为那个项目而变得更加自信，或者学到了很多東西。

也是在那个项目中，我的经理帮助我理解到，我作为技术主管的最初想法是行不通的。最初我是这样想的：“我会把它分成二十块，分配出十八块，把最难的两块留给自己”，而我的经理推动我把难的部分委派给团队，以扩展和发展他们。

你是否花时间倡导技术、实践、流程或架构的改变？

作为一名技术主管，我花了很多时间来倡导变革。我会参与很多不同的架构和技术讨论，即使是在那些不直接属于我的专业领域，因为人们似乎信任我的直觉。我知道很多工程师都有惊人的技术直觉，但没有Staff Engineer（资深工程师）的头衔，但这个头衔确实使拥有这种直觉变得正式。

我确实更希望将要拥有一个项目的团队来做出关于它的最终决定。如果在我脑海中有有一个非常明确的“正确决定”，我会尝试引导团队朝着那个决定前进，而不是直接进去说“这是正确的决定”。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

我绝对认为自己是一个赞助者。执行是我工作中回报最多的部分之一——我喜欢构建东西——但我一直喜欢帮助人们成长。当我看到我非正式指导或在一个项目中帮助过的人继续做一些伟大的事情时，我感到非常自豪。

作为一名Staff Engineer（资深工程师），尤其是作为一名女性Staff Engineer（资深工程师），我觉得很多人都很仰慕我。在管理职业道路上似乎有更多的榜样，所以我试图把成为榜样作为我的责任的一部分，而不是仅仅埋头编码。我的意思是，我可以只是埋头编码，那也很棒，但我想要帮助其他人，特别是那些患有冒名顶替综合症的人。

我经常遇到有人来找我说，“我不知道如何迈出下一步。”或者“我不知道如何成为一名资深工程师，所以我打算去做一名经理。”我想尽力帮助他们找到自己的道路。作为一名资深工程师，我认为让自己变得可见，并且可以回答人们的这类问题，是这个角色的重要组成部分。

作为一名Staff-plus工程师，你做了哪些你在职业生涯早期无法做到或不会做的事情？

没有头衔我并非不能做任何事情，但这个头衔确实给了我信心。除了头衔之外，另一件给我信心的事是意识到其他人也在与冒名顶替综合症作斗争。后者是我在与一位我认为是我共事过的最有自信的工程师的一次关键对话中学到的，当我与他谈论这件事时，他说：“我质疑我做的每一件事。我回家后会为我那天早些时候说的话而苦恼，以及它是否很愚蠢。”

真正让我相信自己是一名资深工程师的是那次谈话与头衔的结合。它们共同给了我信心去要求更难的项目，或者去要求我的经理给我更多项目去做。

你在 Dropbox 晋升为 Staff engineer 的过程是怎样的？

在我加入 Dropbox 后不久，他们推出了一系列外部职称。在第一个职称评审季，他们只授予了极少数工程师 Staff 职称。当时他们实际上还在校准职称。在第二个评审季，我获得了 Staff 职称。

到了第二个季度，我已经担任技术主管一段时间了，我和我的经理都觉得我显然已经在 Staff 级别上工作了。在评审周期之前，我们确实仔细研究了新的职业级别定义，以找出任何差距，但总的来说，过程非常顺利。

你认为达到 Staff 级别最重要的两三个因素是什么？

对我来说，一个重要因素绝对是知名度。部分原因是我做了很多超出正常工程职责范围的事情。

例如，我曾在一个夏天帮助招聘部门运营实习生项目。在项目期间，我与不同团队的大量实习生导师合作，由于 Dropbox 往往有非常庞大的实习生团队，这意味着我在几乎整个公司都获得了知名度。招聘工作也很有帮助。如果你每个月主持几十次招聘总结会议，并推动招聘和校准对话，那么你将有机会与工程部门的每个人互动。我还帮助进行入职培训，向新员工进行核心工程演示。

拥有赞助人也绝对重要。我和我的经理关系非常好，我和我的越级经理关系也很好。我认为这起到了很大的作用。

这项工作，在一些公司会被称为“粘合工作”，是否直接受到重视？

这项工作在 Dropbox 受到领导层的高度重视。领导层和许多资深工程师都积极参与这些工作，尤其是招聘，这不被认为是粘合工作。话虽

如此，单凭这一点并不能让我晋升到 Staff 级别。关键在于在文化影响力和展示技术实力之间找到良好的平衡。

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

没有明确的期望，也没有任何地方将其列为正式要求，但人们普遍认为，你将完成一个 Staff 项目才能获得晋升。我想不出任何没有包含一个非常强大的项目的 Staff 晋升，通常是一个多人项目，工程师是技术负责人。

我确实有一个 Staff 项目。很久以前，Dropbox 最初是一款消费者产品，人们下载并安装在他们的机器上。当我们推出 Dropbox for Business 时，有人要求你的个人和工作 Dropbox 帐户同时工作，包括能够在它们之间切换而无需注销并重新登录。

最初的实现是在巨大的时间压力下编写的，它运行着多个 Dropbox 进程。一个用于您的个人帐户，另一个用于商业帐户。我的 Staff 项目是让单个 Dropbox 进程可以在多个用户登录的情况下运行。困难之处在于该项目从内核一直延伸到用户界面。我必须了解 Dropbox 系统的每一层。

最初我们认为这需要六个月，结果花了十八个月。这在很长一段时期内占用了桌面客户端团队的大部分资源。

您是否愿意分享一条对您晋升到 Staff 级别特别有帮助的建议？

在我职业生涯的早期，我的本能是要求做那些我觉得自己能够很好地执行的项目，而不是那些更模糊、会促使我成长的项目。我得到的建议是，把自己从舒适区里推出去，要求做团队里最难的项目。要达到 Staff Engineer（资深工程师）级别，你必须知道和做到比你目前知道的更多。重要的是，要不断超越你正在做的事情，不要害怕要求那些你认为对你来说太难的事情。

这与冒名顶替综合症有关，你可能不想尝试任何事情，直到你绝对确定自己会擅长它。但你必须接受这样一个事实，即你可能会彻底失败。没关系，你必须尝试。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

人们经常来问我，“我下一步该做什么才能达到 Staff 级别？”我告诉他们的一件事是，对你的职业生涯目标，要对你的经理超级开放和诚实。我早期在与经理一对一沟通时犯的一个错误是，我告诉我的经理我认为他们想听什么，而不是我实际的感受。

他们会问我对某项工作是否感兴趣，我会想他们为什么要问，他们是否想让我接受这项工作？所以即使我不感兴趣，我也会说我感兴趣。或者他们会问我一个项目进展如何，可能进展得很糟糕，但我会告诉他们进展顺利，以避免让他们失望，而不是说我需要帮助。

在某个时候，我意识到你的经理实际上是在你的团队里的。他们正在寻找一种方法让你成长、提高效率、快乐并成为你能成为的最好的工程师。与你的经理建立有效关系的方式，包括让他们赞助你，就是对他们超级诚实和开放。

当我成为一名经理时，这一点对我来说变得特别明显，因为我想要我希望我团队中的每个人都能成为 Staff Engineer 并获得晋升。我想找到提升他们的理由，并与他们一起努力。

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

我做[钟摆](#)相当多，因为我对职业阶梯两边的很多事情都感兴趣。我对培养人才感兴趣，我真的很喜欢与招聘部门合作，我是那种真正喜欢面试的工程师，我喜欢了解团队是如何成长的。但我也真的很喜欢编写代码，在管理一段时间后，我想回到代码中，稍微研究一下。

一旦我开始指导和管理，我肯定发现自己对职业发展的看法非常不同。钟摆帮助我看到了许多不同的视角。作为一名经理，你对诸如员

工人数和绩效评估等事情负有非常明确的责任。Staff Engineer 的职责非常模糊，并且因公司而异。Staff 角色的这种模糊性导致许多人横向转到管理岗位，而他们本来可以更快乐地留在工程师岗位上。这就是为什么发布更多关于 Staff 角色的信息供人们阅读非常有价值。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

我读了很多书，但我的阅读非常休闲。对我影响最大的是有很多我认为是导师的人，通常是朋友、前任经理和与我共事过的人。我经常与过去与我共事过、了解我并且我信任的人一起吃午餐、喝咖啡和晚餐。正是这些关于职业挑战和成长的对话让我达到了现在的职业高度。

Rick Boone - Uber基础设施副总裁的战略顾问

这个故事录制于 2020 年 4 月。在他的网站上了解更多关于 Rick 的信息[Linkedin](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是 Uber 基础设施副总裁的战略顾问，这意味着我是基础设施领导团队的一员，与工程总监和整个组织的项目经理一起。Uber 的基础设施工程大约有 700 人，分布在六个子组织中，例如处理我们数据中心和服务器的 Metal、Storage、Developer Platform 等。我与副总裁一起处理诸如技术战略、文化战略和特殊项目之类的事情。

战略顾问是一个范围广泛的角色，例如，我可能会参与：

- 评估未来两年的技术需求
- 帮助确定未来六个月路线图中创新的优先级
- 深入研究没有明确负责人的重要领域，并帮助简化正在进行的相关项目
- 了解工程师在大型组织变革之前或之后的感受
- 与需要就某事达成一致但相距甚远且似乎存在沟通问题的两个团队交谈，弄清楚如何帮助他们找到有效的前进道路

这只是一个非常非常广泛的角色，混合了工程、文化、心理学、组织设计和战略。有有这两种方式来描述它，都来自流行文化。第一个就像是[国王之手](#)在权力的游戏，这是我能想到的最好的类比。第二个是[里奥·麦加里](#)从白宫风云，他总是说：“我为总统服务。”在我的角色中，我说我为基础设施副总裁服务。

虽然现在只有我一个人，但之前我们有两个人担任基础设施副总裁战略顾问的角色，我们会根据我们对项目的自然亲和力来分配工作。她经常更关注与经理和领导力相关的项目，而我更关注IC和工程项目——尽管我们仍然设法在两个领域都做事情

战略顾问的角色有点不正统；它是由[Matthew Mengerink](#)在他开始担任基础设施副总裁职位后不久创建的。据我所知，我们的组织和我们的首席技术官办公室是唯一拥有这种类型角色的组织。Matthew创建这个角色是因为拥有来自工程团队内部的完整背景的价值，他希望创建这个反馈循环来为他的决策提供信息。

在 Uber 的基础设施组织中，这是一个特别有价值的角色，因为它是一个非常非常广泛的组织，我帮助提供一个综合的视角来审视它。

这个角色与TPM角色相比如何？

这是一个有趣的问题，因为我前几天还在思考幕僚长和我自己角色之间的区别。在基础设施领导团队中，我们有战略顾问和项目经理，过去，我们也曾有人担任幕僚长的角色。

在我看来，项目经理是一个组织范围内的运营角色。他们在高层次上工作，确保Infra内的主要项目和领域按常规节奏推进和评估，运营工作+倡议等。幕僚长的角色是确保整个领导机器协同工作——所有参与运营和领导Infra的人员、团队、信息传递等都能有效运作。

我的战略顾问角色更多的是利用广泛的领域知识，包括技术和文化，深入了解个人和组织层面的问题，然后融入工程方面的敏锐性。由此，我将综合出一套建议或见解，并将其传递给组织领导或整个领导团队。在日常工作中，我的绝大部分工作是直接与组织主管和项目经理一起完成的——向组织主管提供建议，然后，在他的投入和批准下，与项目经理合作，将这些建议变成现实。

您如何看待与您的赞助人保持一致的重要性？

这很有趣，因为这种一致性是关键——几乎是这个角色的必要条件。马修和我在原则、价值观、世界观、对情商的重视、执行方法和理念上非常一致。在很多方面，我们都是一致的，以至于这几乎是一种共生关系。

与赞助人保持一致对于发挥作用至关重要，但这不仅仅是战略顾问和副总裁之间冷静的联系。这也与里克和马修作为人之间的联系有关，并确保这是一个很好的契合。

在我的角色中，我们经常连续几周不在同一个房间里，但我仍然必须像他的直接代理人一样行事。所以我走进一个房间，心想，“马修会怎么做？他想问什么问题？他对这个问题有什么指导？”因为我不能总是跑回去找他为了澄清，必须深入了解他的世界观。这对于我保持作为他的代表所需的非常深的信任，并有效地执行他的战略和愿景至关重要。人们需要确信，如果马修在那里，我总是会给出与他相同的答案。

这也意味着我必须真正理解他的目标、意图、价值观和原则，以确保我准备好押上我的声誉和信誉来推动它们前进。通常，我的部分职责包括向工程师倡导或翻译他的愿景和/或实施方案，有时是在补充背景信息并不总是已知的情况下。当我这样做时，我必须确保我不仅理解他所做事情逻辑和价值，而且我自己也相信它——否则，倡导就会变得困难，更不用说虚伪了。

这是我刚开始担任这个角色时真正挣扎的事情。Matthew会不断地告诉我，“你是我的代表；你应该随意地用我的名字和角色来推动和执行事情。”这对我来说很困难，因为我以前从未担任过这样的角色。以前我总是用自己的名字和声誉来运作，而现在我是在副总裁的庇护下运作，以及随之而来的一切。随着时间的推移，我学会了如何慎重地使用这把锤子，因为你不想过度使用它。

我也了解到，我有时必须让人们知道我戴的是哪顶帽子。我喜欢指导别人，但有时人们不确定他们得到的是为组织和公司利益工作的战略顾问，还是为那个人及其职业生涯利益工作的导师；我尽量让他们知道我在特定对话中扮演的角色。如果我和我指导的人见面，他们可能想得到关于更换团队，甚至离开组织或公司的建议，他们想知道哪个角度我给出建议。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

我认为最大的区别在于，其他高级工程师主要从事技术工作。他们是领导者，所以他们确实会涉及到情商、沟通、协作、冲突解决、宣传等方面，但仍然有80%的日常工作是由技术问题驱动的。

而对于我来说，可能几周的时间我都在专注于一个关于群体心理学或组织设计的项目。技术问题并不总是驱动我日常工作的纯粹焦点——尽管它们总是存在，即使只是在背景中。

既然你开发的少了，你如何保持对现实的了解？

当我是一名工程师时，我可以被动地做到这一点，因为你在代码中，试图推送提交，处理配置和操作服务的摩擦等等。这种方法不再有效，因为我不太接触代码；所以现在，获得这些数据和意识需要一个积极的过程。

我做的一件事是继续坐在我以前的团队旁边，这样我就可以听到他们工作。也许他们会抱怨服务的稳定性，或者我们工具中的一个漏洞，听到这些很有帮助。

我还经常问人们关于他们的开发者体验的问题。我脑海中保留着一份擅长发现问题并对方法给出反馈的人的名单，我经常联系他们。有时这些联系更结构化，实际上是一份征求意见的调查，还有一些有时这只是一个快速的消息来问候一下。

我还告诉大家，把那些没有严格时间限制的非关键路径工作发给我，我尽量利用这个机会，保持编写实际代码的新鲜感。但我必须小心，不要进入我们实际产品的关键路径，因为我知道我没有足够的精力来维护未来的代码。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

这个特殊职位的一个特别之处在于，它本质上是与副总裁的内置指导。当我刚开始时，他问我：“你五年后想做什么？你的目标是什么？”当时，我真的没有明确的答案。长期以来，我的观点是，在当今时代，能够编写代码，在工作保障和发展轨迹方面，让你处于人类历史上最好的位置之一，这对我来说似乎就足够了。

当我在思考我的目标时，我真正意识到的是，我喜欢成为其他工程师，特别是其他少数族裔工程师的可见参考，并帮助优步或更早职业生涯中的人们。我特别喜欢帮助那些刚进入这个行业，可能仍然有点害怕的人。这是驱动我的一个重要部分，这个角色帮助我意识到并承认了这一点。以前我不认为这是一个有效的目的，但我意识到，如果这是你所爱，如果你对此充满热情，那么你必须去追求它。

指导对我来说很重要的另一个原因是，在我的生活和职业生涯中，我有六个人被认为是我的关键导师。他们每个人在不同的时期都对我的生活产生了巨大的影响和影响——我会如果没有他们过去和持续的指导，我就不会成为现在的我。我非常感谢他们，并且始终意识到他们对我的指导有多大。所以，我总是认识到导师的力量，并希望确保我可以为他人提供这种力量。有时，导师甚至不知道他们的言语或行为如何改变你，他们可能产生的连锁反应，甚至在多年以后。所以，我总是尽量让自己可以作为导师帮助他人，因为你永远不知道什么时候你可以对某人产生那种改变人生的影响，或者如何产生这种影响。可能只是正确的词语，正确的视角，你在正确的时间给他们的正确的推动。

我总是告诉人们，“说真的，如果你需要我，就来寻求帮助。”这是我所做的最令人兴奋的事情之一，我尝试通过几种不同的方式让自己可以提供帮助。

其中一种方式是，我每个月都会为工程部门的大部分新员工开设一门Engucation（将工程和教育融合为一个词）课程。这门课叫做“课程+问题”，实际上只是一个他们可以问我任何关于优步的问题的地方——技术、文化，任何问题——我都会尽可能坦诚。最后，我会告诉他们我的电子邮件，欢迎他们与我联系。很多人在那之后会联系我，

我会给他们关于职业、在优步工作或其他方面的建议。其他时候，我会在办公室里遇到一些人，他们会向我寻求建议。

我想作为一个黑人工程师被大家看到，向其他人展示我们是在这里，这是可行的。一旦我意识到这是我重要的动力，我就知道我必须提高我的公开演讲能力，因为这是作为一个榜样来扩大我的影响力的重要方式。公开演讲曾经让我感到恐惧。我曾经憎恨公开演讲。但因为它是一种接触大量人群的关键方式，我告诉自己我必须学会喜欢它，从那以后，我学会了成为一名有效的公开演讲者，并且实际上爱上了它。它现在是我做的最令人兴奋的事情——就像坐过山车一样；每次我做的时候，我都会感到紧张，但那是一种令人兴奋、有趣的紧张，而且我在做的时候会感到巨大的刺激。

你是否考虑过建立你的外部品牌？

我有几个朋友花时间打造他们的外部品牌，其中一个现在又开始重新投入其中。他意识到他在 Uber 的工作非常紧张，以至于他把外部工作搁置一旁。

我在这方面比较被动。当我有参与一些公开的文章，或者我做公开演讲时，我会在领英上发布链接，但我根本不写自己的内容。我考虑过这样做，我也对这样做感兴趣，但我没有。我倾向于通过说话来思考，所以这样写作需要大量的准备来组织我的想法，而且到目前为止，我还没有花太多时间在外部分这样做。

你最初在你现在的公司获得了战略顾问的头衔。你是作为战略顾问被雇佣的吗？如果不是，晋升到这个职位的过程是怎样的？

我的道路完全是不合常规的。它不是计划好的，而且实际上没有可复制的途径，更像是一系列幸运的事件。之前[罗布·庞库努斯](#)当时担任同样的职务，当他决定离开时，马修让他推荐潜在的继任者。他推荐了我和凯特，我们俩最终都担任了战略顾问的角色。

在此之前，马修和我已经有过几次积极的互动，我们开始意识到我们有相似的观点和价值观。例如，有一次，我们收到了一大堆匿名提交给我们的问答会议的恶意评论，看到我们的文化朝着那个方向发展，我真的很困扰。我站起来在一次问答环节上发言我请大家寻找一种更具建设性的方式来表达他们的担忧，我认为这引起了 Matthew 的共鸣。

当他第一次建议我担任这个角色时，我感到非常自卑。我试图让他撤回这个提议，认为这并不适合我，但最终我还是接受了，并且一直担任这个角色。

在您成为战略顾问的过程中，最重要的两个或三个因素是什么？您加入的公司、您的所在地或您的教育对您的职业道路有何影响？

除了 Rob 的推荐之外，最重要的因素是做与 Matthew 的价值观相符的可见工作。我参与的一个项目是加入工作组，以了解和改进 2017 年的 SRE 文化。工作组在[Susan Fowler's blog post](#)发布之前就已经计划好了，我们的第一次会议恰好在她发布后的三天。我真的认为文化工作组做了一些很棒的工作，我和其他小组成员都为此感到非常自豪，在十八个月的时间里，我们真正地以有意义的方式改变了一个百人组织的文化。

此外，我个人一直对文化、人类心理学和行为领域的事物着迷。在我的职业生涯中，在我工作过的公司里，文化 + 群体心理学通常是将组织从优秀变为卓越的隐藏因素。我已经通过阅读行为经济学、行为科学等方面的书籍和论文来满足我个人的好奇心，因此这种自然的兴趣帮助我走向了现在的方向。

你还记得关于达到 Staff 级别的任何对你特别有用的建议吗？

在我的整个职业生涯中，人们总是告诉我，我的影响力比我意识到的要大得多，潜力也更大。我从来没有听过这些话，对我来说，以及对许多其他工程师，尤其是少数族裔工程师来说，我们花了很多时间怀疑自己。我们很容易只看到不好的一面。我们可能没有意识到，当我

们在会议上充满激情地谈论某件事时，人们真的在倾听我们。有人一直告诉我，我没有意识到我所产生的影响，我的观点不仅有效，而且实际上对组织产生了影响，这真的很有帮助。

另一件有帮助的事情是拥有导师。特别是，我喜欢具有建设性对抗性的导师。我的意思是，他们把我扔进让我非常害怕的事情中，但他们确信我已经准备好了。他们帮助我超越了自己认为可能的范围。这些人通常是我合作过的经理，但我们能够相互学习。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

这可以追溯到如何根据我对组织心理学、文化、指导等方面的广泛兴趣以及技术而走到今天的位置。我从来都不是一个纯粹的工程师，只是整天沉浸在代码中。我从来都不是那样的人，我不得不接受这一点。

对我来说，追随我的热情非常重要。最近，这与指导有关，但也与机器学习等其他事情有关，机器学习一直是我的爱好。我喜欢机器如何产生模仿人们思维方式的见解——这是我对技术+心理学兴趣的完美结合。

所以我一直在激发这些热情，当有机会将这些热情与公司需要的某些东西结合起来时，我就会抓住它们。例如，我之前的在优步的团队正在为容量规划目的生成关于车队利用率的见解，这是一个很好的机会[将我对机器学习和站点可靠性的兴趣结合起来](#)。

小公司让你有机会做很多不同的事情，但达到一定规模的公司也让你有机会专注于你的热情，这让我能够在不成为整天坐在键盘前敲代码的人的情况下，保持影响力和热情。

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

这是我有时会思考的事情，即使现在我还在思考。它在我的可能性清单上，在我的整个职业生涯中，人们都会问：“你有没有考虑过转入管

理层？”

我现在想关注的是成为一名高效的、具有全局观的领导者。最终，我也想培养人员管理技能，也许在不久的将来。吸引我的是，人类行为让我兴奋不已，而人员管理是一个花时间在这方面的绝佳机会。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

你知道，在我职业生涯的前三分之二，我过去非常喜欢阅读尽可能多的技术内容。我会整天在YCombinator或我的RSS feed上阅读关于分布式系统、可靠性等的内容。现在我更喜欢阅读行为经济学、行为科学、人类心理学、组织战略等等。在这些领域，我非常喜欢的一些人是 [丹尼尔·卡尼曼 \(Daniel Kahneman\)](#)，[蒂姆·哈福德 \(Tim Harford\)](#)，[丹·艾瑞里](#)。还有一些很棒的播客 - [《魔鬼经济学》](#)，[《选择学》](#)，[《隐藏的大脑》](#)。

此外，去年我开始整理一份[关于人类大脑和行为的书籍阅读清单](#)我与任何对这些主题感兴趣的人分享。

我仍然关注[r/linux](#)和[r/programming](#)在 Reddit 上，它已经取代了我的RSS 订阅，成为了我发现新读物的方式。

Nelson Elhage - 前Stripe的资深工程师

这个故事录制于 2020 年 4 月。在他的网站上了解更多关于 Nelson 的信息[Twitter](#)和[博客](#)。

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我最近在 Stripe 工作。他们做在线支付处理，这是一个发展非常迅速的初创公司，大约有两千人。工程部门大约有六百人。当我离开时，我实际上没有头衔。如果我再待两个月，我就会成为一名 Staff Engineer（资深工程师），因为经过多年的内部辩论，他们终于推出了头衔。

我最近工作的团队叫做 Payment Architecture（支付架构），这是一个由三到四个相当资深的工程师组成的团队。支付是 Stripe 产品的核心，我们负责支付代码库。我们特别关注代码库的金融基础设施层，并构建我们需要的数据模型和抽象，以支持 Stripe 当前和有抱负的所有产品线。

我们研究了代码结构如何适应组织结构，包括如何在快速增长的组织中构建代码，该组织正在增加团队、产品、国家和支付方式。我们的架构特别重要的是支持在多个办公室和时区之间分散所有权。

我们推动了许多围绕代码质量和代码架构的倡议，并做了一些实施和重写项目。对于每个倡议，我们都制定了指标和目标，让团队承担这些目标，然后给团队工具来帮助他们迁移到新标准。

"Payments Architecture（支付架构）”团队是一个永久团队还是更像一个项目团队？

两者兼而有之。它不是一个具有狭窄项目或范围的超级战术团队。但它也不太可能作为一个团队永远存在下去。我们正在采取一种实验性

的方法来发展我们的架构，目标是在我们前进的过程中修改和更新我们的方法。我们希望这个团队最终会把自己从工作中解放出来。

在贵公司，*Staff-plus* 工程师是做什么的？

很难非常有信心地说，因为 Stripe 只是刚刚引入头衔。谁是 Staff Engineer（资深工程师）并不是公开的，但你可以根据那些从事最重要、最有影响力的事情的人来判断谁是资深工程师。

有一些清晰的 Staff Engineer（资深工程师）原型。一种是从事深入的技术项目，可能是在规划或构建新的基础设施。在支付架构团队之前，我致力于构建[冰糕](#)，也就是我们的静态 Ruby 类型检查器。我花了大约一年的时间和另外两位高级工程师从头开始构建它，这是一个深度、高杠杆的技术工作原型的很好例子。

还有一些 Staff Engineer 花时间处理交叉项目，担任架构师和项目经理的组合，将组织的不同部分整合在一起，以解决一个大问题。通常，这些问题与我们当前的架构或组织结构不太一致，因此需要许多不同团队之间的协作。

还有一些 Staff Engineer 与一个团队或一小群团队合作，他们是团队的守护者愿景。他们会确定团队正在朝着什么方向发展，以及他们希望在一到五年内达到什么目标。他们会在整个组织中构建和分享这一愿景，然后努力实现它。

你每天如何度过你的时间？

这在支付架构角色和 Sorbet 角色之间看起来非常不同。Sorbet 更像是一个“埋头苦干和编码”的项目。在支付架构方面，仍然有一些编码工作，因为我们有一种特定的方法，我们希望尝试并演示我们正在推动的想法。

我也做了相当多的项目管理工作。比如处理任务跟踪器，主持每日站会，弄清楚谁需要帮助或者谁被阻止了。我还花时间和公司和工程组

织中充当沟通纽带，特别是与那些对我们正在构建的工具和模式感兴趣的团队交谈并为他们提供建议。

为了实现这一目标，我花时间参加各种会议，弄清楚技术战略，并且每周花相当多的时间编写设计文档，记录我们发现的问题，并推广我们认为可以解决这些问题的架构形式。最后，我努力向领导层和其他团队解释和推销这些想法，以此来设定议程并倡导他们的投资和优先排序。

作为一名Staff-plus工程师，你觉得在哪里最有影响力？

当然，最容易追踪影响的是[冰糕](#)，在两年内，一个三人团队将 Stripe 从一个动态类型代码库转变为一个基本上是静态类型的代码库。这影响了公司所有六百名工程师在编辑器中的日常体验和开发环境。

也就是说，很难知道那是否真的是最具影响力的项目。有一种更模糊的观点认为，架构战略工作从长远来看会更有影响力。

作为一名 Staff-plus 工程师，你做了哪些你在早期职位上无法做或不允许做的事情？

关于“允许”的问题很有趣，可能不是完全正确的问题，因为关于谁获得什么样的角色，几乎没有官方政策。大多数事情都依赖于更非正式的资历衡量标准。

但即便如此，Sorbet 和支付架构团队都是相对雄心勃勃的项目。例如，Sorbet 需要将三名高级工程师从更具体的项目中抽调出来。启动它们需要高度的组织尊重和信任，才能获得许可和支持，将团队从他们现有的工作中抽调出来，让他们转而从事这些项目一年。

你是否花时间倡导技术、实践、流程或架构的改变？

这在某种程度上是围绕规划过程的季节性现象。优先级排序最终意味着人员配置，而人员配置决策发生在规划期间。

规划季是一个特别紧张的时期，但我或多或少一直在考虑整个工程范围内的优先级排序。这可能是注意到许多工程师遇到的问题，或者看到一些减慢团队速度的事情。这是一个我一直在思考的持续不断的线索，它会周期性地成为一个紧迫的优先事项，我会花时间倡导创建一个团队或解决一个问题。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

我没有花很多时间从这些角度明确地思考，我也想不出明确的例子来描述我当时在做什么。我做过的另一件相关的事情是帮助启动我没有参与的团队。例如，一些团队会成立来接管一个曾经是我能力范围内的系统，我会以密切的顾问身份与他们合作，为他们提供背景和建议。

您在收购 Ksplice 后首次在 Oracle 获得了架构师头衔。您获得架构师头衔的过程是什么？

我不记得是否 [Ksplice](#) 在收购前就已经有了职位。收购后，我在甲骨文工作了一年，职位是架构师，我认为当时这是他们最高的个人贡献者级别。那里肯定存在一些收购职位膨胀的情况。如果我不是通过收购进来的，我不知道我是否能达到那个职位。

Ksplice 被甲骨文收购后，你成为了架构师，你每天所做的工作与收购前相比是否发生了变化？

我大致上做着相同风格的工作。改变的是，我花了更多的时间与甲骨文内部的 Oracle Linux 组织进行交流。我专注于弄清楚我们的产品如何与他们的产品集成，并让他们尽快了解我们的技术，以便他们能够使用它。我之前花时间培训过新员工，但那比在甲骨文发生的速度慢得多，甲骨文的情况是，“我们把你扔进这个 400 人的组织，现在培训他们是你工作的重要组成部分。”

你认为达到 Staff 级别最重要的两三个因素是什么？

我所走的具体道路很大程度上取决于我早期加入 Stripe。我大约第 30 号员工。但我所做的事情，我认为与其他人所做的并不完全相同，是我试图在 Stripe 上建立非常广泛的背景和意识。当只有 15 名工程师时，这相对容易做到；那时没有那么多事情。

但随着公司的发展，我花了很多精力来了解一切工程方面正在发生的事情：团队之间的互动，扩展的痛点。我试图拥有一个异常全局的视角。这帮助我知道哪些问题是重要的，特别是那些被移除一级的重要问题。如果我知道组织的目标是推出一个特定的产品，我就会有视角看到为什么它会很困难，因为之前的这些架构决策，或者这个下游系统目前无法胜任这项任务。

随着组织变得非常庞大，看到那些被移除一级的依赖关系变得越来越困难，试图保持一个广阔的视野和系统层面的视野对此有所帮助。它还帮助我将团队联系在一起，使我成为信息和想法的路由器，以及提案的发起者。

许多团队陷入只关注他们所在的世界，而对他们的内部客户如何与他们集成缺乏了解。这种情况发生是因为他们从未在他们支持的内部客户团队工作过。我帮助团队了解其他团队如何真正使用他们的系统，并将他们与组织中他们应该收集观点的人联系起来，

随着组织的成长，很难保持所有这些背景信息，但对于那些在公司规模较小时没有开始建立这种全局背景信息的人来说，这甚至更难。通过尽早开始你相对于那些稍后开始尝试逆向工程架构和组织依赖关系的人来说，具有巨大的竞争优势。

当我和[基维·麦克明](#)她提出的一个有趣的观点是，有时能够不带完整的历史背景来看待事物是有帮助的。你是否发现你的背景知识有时反而让你难以向前迈进？

当然。我注意到自己有时会参与团队的对话，并且准备好向他们讲述过去七年里每次有人尝试他们正在做的事情以及为什么没有成功的所

有历史。我需要刻意地回顾这段历史，并问自己：“这些信息对他们有帮助或相关吗？”

有时这些信息没有用。另一方面，如果有人尝试做这件事却失败了，可能仍然存在一些非常困难的技术问题。指出这些障碍可能有一些价值，但同样，有勇气再次尝试也很有价值，因为几年过去了，我们已经成为了一个不同的组织。

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

我本能地对这种员工项目的想法持谨慎态度，部分原因是，我见过的Staff工程师的典型形象是那些不一定自己运行大型项目或做大事的人。但他们只是非常有效的专家和引导者，使整个工程组织运行得更好。

也许我最接近Staff项目的事情是，我最终因为一个名为“数据模型Stripe发布计划”的项目而获得了晋升。我领导了这个为期六个月的计划，让一群团队协调处理一些项目，以解决我们数据模型中的弱点，并以具有变革意义的方式推进数据模型。

在某些方面，我认为这不是一个很好的Staff项目例子。首先，我们做了很好的工作，但由于多种原因，它的变革性远低于任何人的期望。其中一些原因在我控制范围内，还有一些原因是问题太难了，组织没有足够的资源在六个月内真正解决它们。

虽然那个项目不一定比我在其他时间段做的工作更好，但这是一个非常引人注目、备受瞩目的角色。它创造了知名度，并在重要方面提高了我在公司的地位。

你能分享一条对你成为Staff工程师有帮助的建议吗？

我学到的一个教训是专注和优先级的重要性。当你拥有我之前谈到的广泛的组织背景时，这一点尤其重要。在任何时候，你都很容易识别出三十件你想做的事情。

偶尔你可以把这三十件事中的每一件都向前推进一点。在一段时间内，这是有成效的，但你需要小心。如果这些事情是不是正在进行，并且你认为应该得到处理，你会有更好的运气一次选择其中一个，并真正集中你的精力，而不是一次在许多不同的项目中投入一点精力。

一个很大的区别是是否已经有团队在处理这三十件事。如果已经有团队在处理它们，但方向不是你认为有效的方式，你可以通过去这三十个团队并帮助他们解除障碍来获得很大的影响力。

最后你不得不说，“我希望我能做所有这些事情，但我不会做所有这些事情。今年我将选择一两个来做，并且我会故意忽略其他的，即使我认为它们是主要问题。”

对于刚开始担任 *Staff* 工程师的人，你有什么建议吗？

一件事是，我非常相信康威定律在指导组织的技术架构方面的重要性。

另一种是建立和投资于你与工程领导层的关系：经理、主管和副总裁。我认为这可能与组织结构有关，但肯定在Stripe，这些人通常有很多隐含的权力，因为他们是遇到问题时可以去找的明显人选。他们对人员配置和优先级排序也有很大的影响。

与他们建立良好的关系很重要，这样你就可以用你的想法影响他们，而且你也可以了解他们正在看到的问题。你需要知道他们的动机是什么，以及他们认为你没有意识到的问题是什么。与领导层更好地保持一致会使很多事情变得更容易。

对我来说，另一件非常有价值的事情是估算。我发现能够查看一个系统并习惯于估计这个东西每秒有多少千兆字节，或者这个数据需要多少存储空间，真的很有价值。你不必做到完美，得到最接近的10的幂通常就足够有用了。

你有没有考虑过工程管理，如果有的话，你是如何决定走上*Staff*工程师这条道路的？

我考虑过，但不是很认真。我对自己有很好的了解，至少现在，我不会真正喜欢这项工作。我想我会发现所有的互动都不是一种可持续的消磨时间的方式。我偶尔希望我对它更感兴趣，因为我确实认为这是一种获得很多权力的方式，但我很幸运地有足够的自我意识来相信，我认为这是正确的，我不会喜欢它，因此也不会擅长它。

你从哪些资源（书籍、博客、人物等）中学习过？

我经常被问到这个问题，因为我拥有异常广泛的常识，但我不知道这些知识是从哪里来的。我对计算、软件和架构非常好奇。我阅读很多不同的东西，而且我花在阅读软件工程 Twitter 上的链接的时间可能比健康的要多。

对我来说，培养一个由其他高级工程师组成的良好个人网络也非常有价值。我会和他们非正式地聊聊我们正在做和思考的事情。当你有人脉时，你可以获得人们所看到的问题和他们正在考虑的解决方案的非常坦率的想法。

我主要是通过我专业上认识的人的朋友的朋友网络，或者一直追溯到我上学的时候，来引导这件事的。这不是我在事后才去寻找的东西。

我偶尔会阅读技术论文，但这不是我积极去做的事情。通常是当有人引用它或在其他情况下出现时。我绝对不会努力系统地跟踪或审查最近的出版物。但我确实认为，对所谓的“基础文献”有一个不错的了解是非常有用的。

Diana Pojar - Slack的资深数据工程师

这个故事录制于 2020 年 4 月。在她的网站上了解更多关于戴安娜的信息[博客](#), [Twitter](#), 和[Linkedin](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是 Slack 的 Staff 数据工程师和数据平台团队的技术主管。我于 2016 年 2 月加入 Slack，是数据工程团队的首批工程师之一。我深度参与了构建许多工具和基础设施，以使数据可用于长期分析。当我加入时，团队刚刚决定使用 Thrift 作为日志记录格式。如果有人想获得见解，他们必须在生产 MySQL 数据库的只读副本之上安排 cronjobs。

Slack 的数据工程团队的目的是使公司中的任何人（数据科学、工程师、产品经理等）都可以访问数据，以便他们可以计算见解、推动业务决策或构建新功能。数据平台团队专注于构建大规模运行的服务和框架，以授权每个需要在数据仓库中处理或使用数据的人。我们团队拥有的一些东西包括：数据发现服务，该服务公开任务、表、列沿袭和通用元数据，事件日志记录结构以及使用事件并在数据仓库中的原始表中公开它们的管道。

Staff-plus 工程师在 Slack 做什么？你每天如何度过你的时间？

Staff-plus 工程师的角色很大程度上取决于团队的需求以及特定工程师的优势。根据我的经验，Staff-plus 工程师的职责会随着时间的推移而变化，但通常他们的主要重点是从事对公司具有战略价值的项目/工作，同时推动技术设计并提升团队水平。

我见过的资深工程师大致可以分为两大类：更注重深度（专家）或更注重广度（通才）。

对于第一类人，那些更注重深度的人通常是特定领域的专家，他们的大部分时间都花在编写代码或处理技术设计文档上，以寻找其专业领域的解决方案。公司面临着独特的挑战，需要主题 matter 专家来推动这些极其困难问题的技术解决方案。例如，在 Slack，随着公司的发展，我们的系统需要扩展和执行，有一位首席工程师，他的主要重点和热情是检测和修复性能问题。

那些注重广度的人通常与领导团队更紧密地合作，影响整个组织或公司的技术愿景，改进流程和文化。由于他们的广度，他们更加灵活，可以根据公司的优先事项和需求在工程组织的各个领域工作。

就我个人而言，目前我更喜欢并专注于广度，以及我如何度过我的时间很大程度上取决于我的团队和组织的需求。我想说，今年到目前为止，我大约 50% 的时间花在[技术领导](#)以及与人们讨论我们应该关注的更大的技术投资，而我 50% 的时间则专注于指导、审查代码、编写代码、处理突发事件和解决关键问题等。这个比例每个季度都在变化。

作为一名 Staff-plus 工程师，你觉得在哪里最有影响力？作为一名 Staff-plus 工程师，你做了哪些你在早期职位上不会做的事情？

就我个人而言，我感觉非常明显的是，那些在我晋升/职称变更之前没有与我合作过的人，对我的信任和尊重有所增加。拥有这个头衔与一个人影响组织/公司路线图和优先事项的能力密切相关——基本上你可以进入“事情发生的地方”。

我可以参与构建对公司直接成功产生影响的事情。倡导这样的项目并参与其中，在早期的职位上是不可能实现的。

我也能够提升其他更年轻的人，让他们发出自己的声音。拥有 Staff+ 的头衔带来了一些其他人没有的特权，我试图利用它来帮助提升我的团队/同事。

你是否花时间倡导技术、实践、流程或架构变革？你倡导过什么？

我实际上花费大量时间倡导技术解决方案、流程、架构或文化变革——这不仅仅是编写代码。我经常参与许多需要构建依赖数据工程工具和服务的团队的技术设计审查过程。除了参与倡导技术项目外，我的一个重点领域是改进文化或流程变革。

我内心深处的一个领域，我相信我在我的组织中发挥了重要作用，是关于事件管理和分析。我一直与公司的弹性团队合作，以改进我们的事件分析流程，但对于我的数据工程组织，我非常参与推动我们的一般随叫随到期望和结构，同时采用公司的事件响应结构。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

赞助对我来说实际上是一个重要的领域，因为我专注于与许多与我一起工作的人建立良好的关系，并且我坚信我们需要互相提升。通过我获得Staff Engineer的旅程以及与我自己的冒名顶替综合症作斗争，我有机会与赞助我的优秀人士合作，并对我的成长产生了巨大的影响。我合作过的一些人，并且一直是我的导师和榜样是[乔希·威尔斯 \(Josh Wills\)](#)，[斯坦·巴布林 \(Stan Babourine\)](#)，[博格丹·加扎 \(Bogdan Gaza\)](#)和[特拉维斯·克劳福德 \(Travis Crawford\)](#)。

指导和培养我周围的人对我来说一直很重要，并且担任Staff+角色，你拥有一种其他人没有的特权和权力，我尽力利用这一点来帮助和提升我周围的人。

你最初在Slack获得了Staff engineer的头衔。你是作为Staff engineer被雇用的吗？如果不是，晋升为Staff的流程是什么？

我加入 Slack 时是一名中级工程师，一年后我获得了高级工程师的晋升。作为一名高级工程师，我有机会参与多个对整个组织/公司产生影响的项目，其中许多项目直接关系到我们公司业务指标的计算方式，这对于公司准备上市至关重要。

在担任高级职位两年后，我的经理告诉我，我的工作水平已经达到了下一个级别，他认为有充分的理由可以晋升我，并计划推荐我晋升。在 Slack，Staff+ 工程师的晋升需要有一个晋升方案，其中包含清晰的细节和可衡量的信息，以说明一个人在特定级别上的工作表现。主要的关注领域是：技术质量、影响力、协作和执行力。我们一起编写并填写了晋升方案所需的所有细节。作为一名 IC（个人贡献者），我强烈建议，如果可能的话，与你的经理合作，共同撰写这份文件：这应该是一项团队努力。方案准备好后，晋升方案将由一个特殊的晋升委员会进行评估，该委员会由来自整个公司的一些领导和 Staff+ 工程师组成。

您达到 Staff 级别最重要的两三个因素是什么？您加入的公司、您的地理位置或您的教育背景对您的职业道路有何影响？

当我回顾并思考我作为一名初级工程师时对此的感受和想法时，晋升为 Staff Engineer 的主要因素实际上是相信**你能做到**并且不要让冒名顶替综合症（impostor syndrome）战胜你。

总的来说，我一直试图对我的职业选择非常有意识，通常我每年都会花一些时间来思考我正在做的事情以及我想关注的成长领域。我发现这非常有价值，因为它让我退后一步，评估我目前正在做的事情，询问我是否还在当前环境中成长，并思考新的机会。

所以在 2015 年底，当我决定离开 Twitter 时，我发现 Slack 开始组建他们的数据工程团队。能够从头开始构建和设计系统、服务和框架对我来说非常令人兴奋。加入 Slack 一个新成立的团队是一个独特的机会，这无疑有助于我晋升为 Staff Engineer。它给了我机会参与对组织或公司范围产生影响的项目。例如，我参与的第一个大型项目将大约 25% 的负载从生产 MySQL 数据库转移到数据仓库，为公司节省了数百万美元。

影响我成为 Staff Engineer 的另一个关键因素是我周围的人，我很幸运在我的团队中拥有出色的榜样和导师。当我加入 Slack 时，我是一个非常资深的团队中的第四个人（其他所有人都是 Senior Staff），这

促使我渴望证明自己并表明我属于这里。在每个项目中建立指导、可见性和技术质量的记录也有助于我走向 Staff 的道路，我没有把我的工作仅仅看作是一份工作，而是把大量的热情投入到我们试图解决的每个项目或问题中。

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

不，我没有被分配到“Staff 项目”，这也不是 Slack 晋升过程的一部分。有一个职业阶梯描述了每个级别的一般期望和影响范围，对于 Staff+ 级别，这种影响范围开始从组织范围的影响扩展到公司范围的影响。

我通常总是尝试挑战自己，并且一直在寻找推动变革和影响我所在组织的方法。我认为我参与并为我晋升为 Staff Engineer 做出贡献的最有影响力的项目是参与思考和实施关于我们公司业务指标（例如：ARR）如何计算的技术设计，以确保该过程可靠、可扩展，最重要的是可重现。这是一个关键的举措，因为 Slack 正在完成一家上市公司的准备过程。

你还记得关于晋升到 Staff 的任何特别有用的建议吗？回想起来，有没有一条更容易的晋升到 Staff 的道路，你可以走吗？

我觉得非常有帮助的一点是，要理解 Staff+ 工程师的工作和责任不仅仅是编写代码。基本上，让你达到高级水平的东西不会让你达到 Staff+。重要的是要了解这个角色在你公司中的期望，以及整个行业的期望，因为公司之间存在一些差异。

与你的经理或更资深的同事合作，找到能够挑战你并扩大你的工作范围的项目。对我来说非常有帮助的一点是，我开始投入更多精力来发展我的领导力和沟通技巧。我也开始以不同的方式来构建和思考某些事情，当我开始感到压力或不确定自己的能力时，这通常表明我正在成长，并且偶然进入了一个提供大量成长机会的领域。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

晋升到Staff Engineer带来了很多责任，你应该始终是你同事的坚定拥护者。作为一名IC（个人贡献者），我认为执行和亲力亲为总是“容易”的事情，而困难的事情实际上是在你的组织中推动变革和产生影响。

我认为在你作为Staff工程师的不同时期，你可能会发现自己专注于不同的事情，这是可以接受的，也是预料之中的。对于Staff Engineer应该做什么，没有一个明确的定义。

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

这实际上是我每隔几年就会问自己的一个问题。每次我进行自我反思并思考这个问题的答案时，目前的答案是否定的——我不想成为一名经理。我太喜欢编码了，而且我坚信，要成为一名成功的经理，你不应该编写代码，而应该完全专注于培养你的团队。我太喜欢参与技术决策和思考技术解决方案了，以至于不想放弃这种亲身实践的经验，即使当你担任更高级的职位时，你花在编码上的时间也会减少。

不成为工程经理并不意味着你不能影响和帮助人们成长。作为一名Staff+工程师，你需要许多核心管理技能，即使你不是一名经理，而且我发现阅读管理书籍非常有帮助。我实际上认为这两个角色，即使它们在独立的平行轨道上，也比人们想象的更接近。

在某个时间点，这个问题的答案可能会改变，这是可以接受的。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

我广泛使用 Twitter，但我主要是一个消费者，关注很多科技界人士。我通常关注那些我在会议上见过发言的人，或者我曾与之共事的人，我发现他们的内容与我相关。以下是一些，没有特定顺序：[卡米尔·富尼埃 \(Camille Fournier\)](#)，[拉拉·霍根 \(Lara Hogan\)](#)，[乔希·威尔斯](#)

[\(Josh Wills\)](#), [维姬·博伊基斯 \(Vicki Boykis\)](#), [大卫·加斯卡 \(David Gasca\)](#), [朱莉娅·格蕾丝 \(Julia Grace\)](#), [霍尔顿·卡劳 \(Holden Karau\)](#), [约翰·奥尔斯帕 \(John Allspaw\)](#), [慈善·梅杰斯 \(Charity Majors\)](#), [西奥·施洛斯纳格尔 \(Theo Schlossnagle\)](#), [杰西卡·乔伊·克尔](#), [莎拉·卡坦扎罗](#), [橙皮书](#).

我也喜欢阅读（我每年阅读大约 50 本书），而且从去年开始，我总是尝试在[我的 Goodreads 帐户](#)上为我读的每本书留下一个迷你评论，但这里有一些我发现有用的书：

- [感谢反馈](#)
- [激进坦诚](#)
- [经理之路：技术领导者驾驭成长和变革的指南](#)
- [领导力和自我欺骗：走出盒子](#)
- [教练习惯：少说多问，永远改变你的领导方式](#)
- [首先，打破所有规则：世界上最伟大的管理者与众不同之处](#)
- [《被讨厌的勇气：如何解放自己，改变生活，实现真正的幸福》](#)
- [《付出与收获：一种革命性的成功方法》](#)
- [《错误已铸成（但不是我犯的）：我们为何为愚蠢的信念、糟糕的决定和伤人的行为辩护》](#)

这并非详尽的清单！

Dan Na - Squarespace 的 Staff Engineer 和团队负责人

故事记录于2020年3月。在他的网站上了解更多关于Dan的信息[博客](#), [Twitter](#), 和[Linkedin](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是Squarespace的Staff Engineer。Squarespace是领先的一体化平台，用于构建精美的在线形象：网站、域名、在线商店、营销工具、安排预约等。我还担任国际化平台团队的团队负责人，该团队负责构建和维护Squarespace产品国际化的基础原语。工程师使用我们拥有的工具和库来创建本地化产品。

Squarespace的Staff-plus工程师做什么？你如何度过你的时间？

我认为在实践中，Staff-plus工程师的日常职责各不相同，具体取决于您的精确角色以及您的职责在组织中的映射方式。

我作为团队负责人的职位意味着我完全负责团队的产出，无论是从业务角度还是技术角度。在业务方面，我花大量时间与公司内不同的团队和职能部门会面。这些利益相关者包括产品、战略、客户运营等。我想确保我尽可能多地获得输入，以验证我团队的路线图反映了我们公司最重要的优先事项。

在技术方面，我经常发现自己审查技术文档或在白板上为我的团队规划工作范围团队在飞行中的工作。我的角色已经演变为减少动手编码工作，更多的是询问有关架构决策和部署策略的深入问题。一个讽刺之处在于，作为一名资深工程师，我实际编写的代码比我作为非资深工程师时要少得多。这绝不是这个角色的普遍现象，但在我的团队的背景下，关闭 vim 并以更具战略性/监督性的角色运作是我利用时间的最高杠杆。我很幸运，我的团队已经由优秀的工程师组成，因此我的具体代码贡献对我们的产出影响较小。

但在Squarespace，许多Staff-plus工程师并不是团队负责人，并且编写大量代码。其他人则专注于工程流程和文化。总的来说，我认为Staff-plus工程师的职责具有高度的背景相关性。

作为一名Staff-plus工程师，你觉得在哪里最有影响力？作为一名Staff-plus工程师，你做了哪些你在早期职位上无法做或不会做的事情？

我可以在高于个人项目和团队的更高层次的工程讨论中占有一席之地。我们有定期的员工工程会议，讨论跨团队的问题，这些问题本质上既有技术性的，也有非技术性的。作为一个假设的例子，在这种类型的会议中，我会很乐意提出我认为工程入职流程中的缺点。将像工程入职这样的主题归因于特定团队可能很困难，但缺乏正式的所有权并不会使其变得不那么重要。我认为Staff-plus的一个关键责任是愿意拥有所有有助于（或阻碍）工程产出的事物，包括技术战略和文化。

关于已经改变的事情，在日常生活中，我的头衔从一开始就赋予了很高的可信度对话。虽然我不提倡一种重视头衔胜过想法的文化，但如果我说它没有帮助我升级或推动我以前可能难以解决的问题，那我就是在撒谎。

你是否花时间倡导技术、实践、流程或架构变更？你能分享一个影响你组织的故事吗？

我并没有真正从类别方面考虑倡导。我主要只是希望我们的工程团队和产品能够做到最好，并解决经验告诉我我可以帮助改变的事情。

一些例子：

- 当我刚加入公司时，我们正处于员工人数大幅增长的时期，我注意到除非你碰巧一起参与一个项目，否则很难了解其他团队的任何人。因此，我创建了一个slack房间——#connect-engineering——它使用一个机器人每两周随机配对工程部门的两个人喝咖啡。这个房间已经配对人们喝咖啡两年多了。

- 根据个人经验，我知道工程领导职位可能会让人感到孤立，与同事交谈时，我可以听到一些孤独感。因此，一些同行和我[创建了一个非官方的工程管理读书俱乐部](#)，面向团队负责人和工程经理开放。现在有两个自发组织的读书俱乐部，每个俱乐部约有10名参与者，为新的和有经验的领导者提供了一个相互支持的安全空间。关于读书俱乐部的反馈非常积极。

公平地说，这两个例子都不需要Staff-plus的头衔。但我确实认为，成为一名有效的Staff-plus工程师的一部分，是像关注技术一样关注和解决文化差距差距。

你最初在Squarespace获得了Staff工程师的头衔。晋升为Staff的过程是怎样的？

我被聘为高级软件工程师II（比Staff低一个级别）。我很幸运地加入了一个从事高影响力项目的团队，我可以立即为之做出贡献。该项目最困难的部分涉及一个我已经熟悉的领域——跨代码库的广泛而彻底的更改——我提出、原型设计并最终交付了一个我认为能更好地帮助公司取得成功的替代架构。这成为了我们的前端翻译系统，我在我们的工程博客上写了关于它的文章：[构建前端翻译系统](#)。

我还负责围绕新翻译系统的沟通和教育工作，在内部会议上介绍该架构，并发送有关项目状态的相关电子邮件。将这种技术贡献与一些有意义的文化倡议（其他内部演示、#connect-engineering等）结合起来，我的经理有一个很好的晋升理由，并得到了工程总监的同意。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

我觉得在职业阶梯上进步是一个累加的练习，它迫使你关心比以前更多的事情。关心更多的事情是很难的。

举一个简单的例子：实习生关心他们可以在三个月内构建的功能的小方面。团队的全职工程师关心该功能的整个生命周期。团队负责人/经

理关心构成产品的整套功能。总监关心其组织拥有的整套产品。等等。

梯子上的每一级都意味着你关心另一层抽象，除了关心你当前层之下的所有层。

我觉得 Staff Engineering 的角色也很相似，因为你离开了特定技术领域的舒适区，进入了一个更通用的问题领域：工程。作为领导者，你离开了潜在的技术舒适区，进入了影响工程产出的系统挑战领域。阻碍工程团队发展的最大问题是什么？这些问题现在是你的问题，此外还有你技术领域的所有问题。

因此，虽然 Staff 是一个令人向往的头衔，但它也包含着重大的额外责任。你现在是一名领导者，无论你是否愿意。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

我认为赞助是任何高级职位的一项关键责任，并且对任何工程组织的成长都至关重要。我想“赞助”的定义各不相同，但对我来说，一种切实可行的方式是提供曝光机会。例如：

- 让资历较浅的队友有机会在更广泛的会议上拥有并展示他们的工作。
- 联系一个刚刚发布了一个很棒的功能的团队，为我们的工程博客写一篇文章。
- 鼓励我在 #connect-engineering 咖啡馆遇到的一位具有独特经验或视角的人进行内部演示。
- 确保会议不会被少数人的观点所主导，并征求房间里每个人的意见。
- 在一个大型的 slack 房间里公开表扬那些做了很棒的事情但大家都没看到的人。

Lara Hogan 有一篇关于赞助实践的精彩文章：[赞助是什么样的？](#)

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

是的，我仍然积极地考虑它。我知道将这两个阶梯视为互斥的阶梯会更方便，但我并不这么认为。

我仍然喜欢发布代码和管理团队，我认为在高水平上同时做到这两者的能力对于长期的工程成功至关重要。Charity Majors在这方面有一篇很棒的博客文章，我建议阅读：[“工程师/经理的钟摆”](#)。

慈善机构认为，“经理职业道路与工程职业道路”是一种错误的二分法，花时间在两种角色之间交替可以让你在两种角色上都做得更好。这与我自己的经验相符。我之所以成为一个更好的管理者，是因为我知道在一个计划不周的项目中担任个人贡献者有多么糟糕，而我之所以成为一个更好的个人贡献者，是因为我知道如何在项目进展不顺利时发出警报。

我认为构建软件最重要的战略技能之一是朝着务实的决策能力靠拢。我反复看到的一种失败模式是，当产品经理提出业务需求，而工程师提出技术上的反对意见时，双方都不愿意让步。能够同情两方面的动机并驾驭这种紧张关系是完成任何事情的唯一途径，而建立这种同情心的最好方法就是同时坐在两个位置上。

为了具体回答这个问题：在加入 Squarespace 之前，我之前的职位是工程经理。我喜欢当一名工程经理，但我希望保持我的技术技能，所以我接受了一个 IC 职位。然后我被提升为 Staff。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

在工程领导力的背景下，有两本书脱颖而出。

我最喜欢的工程领导力书籍是[安德鲁·格鲁夫的《高产出管理》](#)。我每年都会从书架上拿起它，并不由自主地重读一遍。格鲁夫书中的许多想法深刻地影响了我对工作和领导力的看法：“衡量一个经理的标准是

他们下属组织的产出”，“授权不是放弃”，工程/管理杠杆的概念等等。在传达工程领导力的战术方面，我仍然认为格鲁夫的书是最好的。

在领导力的人性方面，我真的很喜欢[拉拉·霍根 \(Lara Hogan\)](#)的书：[弹性管理](#)。我非常幸运地在 2013 年在 Etsy 开始了我在纽约的科技职业生涯，Lara 是我的第一位工程经理。Lara 擅长拆解和解决关于驾驭情绪和个性、培养心理安全感和赞助同事的最困难的部分。并且在她手下直接工作了近四年，她完全是真实的，并且实践了她所宣扬的。

就非书籍而言，我订阅并喜欢阅读“[非理性繁荣](#)”，“威尔·拉森定期在他的博客上以高度务实和战略的视角撰写关于工程管理的文章。我最近还发现并喜欢阅读马蒂·卡根的“[洞察博客](#)”，主要是因为产品领导力是我不太熟悉的领域，并且有兴趣了解更多。

我的榜样是我多年来密切合作的一些了不起的同事。我坐在[丹尼尔·埃斯佩塞特](#)在Etsy工作了四年，学到了很多关于将技术执行与文化影响相结合的知识。我学到了很多，比如看着Lara倡导并实现我们工程团队的薪酬公平。我学到了很多，比如看着现在的同事[Tanya Reilly](#)制定并改进我们的工程流程，以适应我们不断增长的规模。我最受那些我亲眼目睹有勇气为了让公司变得更好而改变公司的人的启发，尽管他们一路上遇到了各种摩擦。

乔伊·埃伯茨 - Split 高级职员软件工程师

故事记录于2020年3月。在她的网站上了解更多关于Joy的信息[博客](#), [Twitter](#), 和[Linkedin](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是Split.io的高级职员软件工程师，负责我们称之为COE团队的后端工作。Split是一个功能标记和实验框架。我们专注于使我们的客户能够在CI/CD中分离部署和发布，并支持A/B测试。我的团队负责我们Web应用程序的大部分主要业务逻辑，包括从数据存储到API的所有内容。有一个单独的团队专注于实验方面，包括所有详细的统计数据，因此我们能够更多地关注主要平台。

在 Split, Staff-plus 工程师是做什么的？你如何安排你的时间？

我仍然有点新，所以我仍在努力定义我的角色，这也是更高级职位的美妙之处。今天，我仍在适应中，所以我可能将大约一半到四分之三的时间花在我的特定 scrum 团队的任务上，就像这里的任何其他工程师一样。在剩下的时间里，我参与对话并与其他工程师合作，以定义我们许多更长期的架构和战略，包括我们未来的 API 和平台战略，我们希望如何开发我们的授权框架，分解和解耦我们的构建等等。最近，我还接管了我们的后端章节的领导权，现在与另一位工程师共同领导，我们正在努力整合后端技术愿景，优先考虑技术项目并领导标准讨论。如果这还不够，我还继续定期在我的博客上写作并在会议上发言。

作为一名Staff-plus工程师，你觉得在哪里最有影响力？作为一名Staff-plus工程师，你做了哪些你在早期职位上无法做或不会做的事情？

当我可以促进为一个领域设定技术愿景并让人们朝着该愿景前进时，我感到最有影响力。我想我们都会同意，我们希望我们的代码在架构

上比现在更好，或者以某种方式得到改进。但是，我发现人们常常对想要变得更好有一种模糊的感觉，但并不清楚他们想要的东西是什么。我喜欢帮助团队就他们试图达到的目标达成共识（即使我们永远无法达到目标也没关系），并提出一个实现目标的大致计划。这样我们都在朝着同一个方向前进。清楚地了解我们想要什么，可以让我们与产品部门合作，使其获得优先考虑。即使我们永远无法将整个事情优先考虑，了解如何实现目标，也能让我们慢慢地做出改变，从而引导我们朝着那个方向前进。例如，如果我无论如何都要接触一个文件，并且可以进行一些调整，使我更接近该愿景，我就会这样做。如果不知道该愿景，这些调整永远不会发生。仅仅有愿景是不够的，我们需要每个人都理解并内化该愿景。我刚才提到的小改变的力量在于，如果每个人都在他们的正常编码中进行这些改变。突然，我们每个人都在朝着一个共同的目标努力。

我认为现在和我刚入行时最大的区别在于我的主人翁意识和责任感。我一直愿意反驳或推动改进。但是，当我刚入行时，我经常认为某件事是别人的问题。现在，这都是我的问题。我可以选择优先考虑某件事因为我认为它不如其他事情重要，或者我可以选择委派或将问题转交给其他人，但我仍然认为这是我的问题。我不再认为其他人会处理某件事。我仍然坚信要选择我的战斗，我不会处理所有事情——那太多了。但是，我也不会认为其他人会处理，所以如果值得完成，那么要么由我来完成，要么由我转交给其他人。

你是否花时间倡导技术、实践、流程或架构变革？你倡导过什么？你能分享一个影响你组织的故事吗？

是的。所有这些。在目前的角色中，我会说这是我工作的重要组成部分。虽然作为一名工程师，我也在一个 scrum 团队中做出贡献，但我会说我的很多工作是留意我以前见过的陷阱或更大的问题模式。我认为我的工作是为所有工程部门更有效率——无论是通过技术、架构还是流程。但是，我不应该为了改变而改变。多年来，我倡导了很多事情，从重写我们的电子邮件通知系统到重新思考测试，再到重做几个授权框架。

对于某些事情，比如电子邮件大修，我没有做任何大的或伟大的事情，我只是每次产品部门想要添加通知时，都会提醒他们我们的系统已经准备好崩溃，在我们修复它之前，我们真的不能再添加任何东西了。当我反驳时，我周围的工程师也意识到他们可以反驳。起初，产品部门大多选择不添加更多通知，但最终他们决定修复系统。在这种情况下，这主要是一个向他们解释系统风险的问题，并坚持我认为在保持系统运行方面正确的行动方案。

对于其他事情，比如授权框架，我当时负责寻找解决方案。在这些情况下，即使人们想要新的/更好的解决方案，你仍然需要说服他们你选择了正确的东西。对于极其复杂的系统，人们常常会认为他们发现了你遗忘的东西（也许他们确实发现了），因此尽早并经常寻求反馈，并仔细记录和沟通你选择的内容和原因，以及你考虑过的其他内容以及为什么没有选择其他内容，这一点非常重要。人们需要感到被倾听，并且需要知道你充分考虑了他们的担忧。他们也想了解你的思考过程，但更重要的是，他们想了解你是否进行了彻底的研究，而不仅仅是选择了第一个出现的東西。事实上，当我在审查别人的设计时，这是我真正关注的事情之一——还考虑了什么？

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

是的。一旦你担任任何更高级的职位，这始终是你职责的一部分，前提是你选择利用它。由于我刚到Split不久，所以在这里还没有太多机会，但我相信这种情况会改变。有时赞助是大事——推荐人们领导项目或管理团队，但很多赞助是小事——鼓励那些对自己有点不确定的人，向他们通常无法接触到的更高级的人展示他们的成就，找到将你的工作委派给那些可以从中获得成长机会的人的方法。我认为有可能成为一名没有赞助的高级工程师，但我不确定是否有可能成为一名没有赞助的伟大的高级工程师。赞助是我们能够培养周围人的最有效方式之一，我会说培养他人是我们职责中最重要的方面之一。

你第一次获得Staff工程师的头衔是在Box。晋升为Staff的过程是怎样的？

在Box，我们提交一份晋升案例，其中概述了根据工程评估标准，我们已经在下一个级别上运作。我们的经理也会提交他们的推荐信，这两份材料会提交给由经理和个人贡献者（至少比我们申请的级别高一个级别）组成的晋升委员会。他们审查案例，召集经理回答任何问题，然后提出建议。我们的副总裁可以更改任何决定（尽管据我所知，这种情况从未发生过）。如果答案是否定的，会给出反馈说明原因，你可以用补充信息撤销决定，或者下次再试。申诉有时确实会通过，所以如果你不同意反馈，值得尝试。我喜欢这个过程，因为它允许对我们的成就最有发言权的人来撰写，并且允许你即使你的经理不同意也可以申请晋升。另一方面，我不喜欢这个过程，因为它微妙地歧视那些自信心稍差和不擅长自我推销的人。这也导致经理们在启动晋升流程时的主动性降低（让工程师来找他们说他们想要晋升，而不是建议他们晋升）。

您达到 Staff 级别最重要的两三个因素是什么？您加入的公司、您的地理位置或您的教育背景对您的职业道路有何影响？

我想说我的地理位置可能没有太大关系。教育在早期帮助我获得面试机会，但此后（至少直接地）帮助不大。我想说对我来说，三个最重要的因素是公司、知名度和机会。

我认为在很多公司都有晋升的机会。但是，我发现身处一家快速发展的初创公司真的对我很有帮助。当我加入Box时，工程团队大约有30人，当我8年后离开时，有几百人，但大部分增长都在前半段。这让我能够进入一个较小的工程环境，在那里可以真正了解环境、人员和代码。然后，因为我们正在成长，所以对于那些有动力并愿意接受挑战的人来说，有很多领导机会和技术挑战。因为我们成长了，机会也随着我成长。与此同时，我周围也有足够的人可以向他们学习（我之前在一家非常小的初创公司——2-4人，在那里真的没有这种机会）。

通过知名度，我的意思只是找到某种方式让别人知道你。我一直在现场工作，我发现这更容易一些，但我认为即使你是远程工作也是可能的（尽管可能更具挑战性）。如果你做了非常出色的工作，但没有人知道，那么在晋升的时候，你会被忽略。此外，随着你变得更资深，

你的一部分工作变成了指导和教导他人，并帮助你的公司创建一个技术品牌——所有这些都是定义上的可见的。知名度可以采取多种形式，但对我来说，我想说有几件事做出了贡献。我非常活跃于我们的 Slack 讨论论坛，尽可能地回答人们的问题。我还做了很多博客，并在内部和外部做了一些演讲。最后，我积极参与我们的女性技术团体，这让我能够与整个工程部门的各种人建立联系。

最后，机会。这些机会也可能看起来大相径庭。对我来说，有一个特别有帮助的机会——我加入了我们的 API 标准委员会。一开始我有点犹豫，因为我不认为自己是 API 方面的专家，但在阅读了几本关于 REST 的（简短）书籍，以及我之前在各种 API 上的工作之后，我对它有了相当扎实的掌握。这个小组的强大之处在于它横跨工程部门的许多团队，这让我有机会与很多人一起工作来自不同工程师（并给了我我刚才提到的那种可见性）。它还让我能够清楚地指出一些东西，以便影响他人，并成为为一个为质量而战的人。我们在那里的项目对整个工程部门产生了广泛的影响，并让我能够从整体上思考一些东西（在本例中是我们的 API）。

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

实际上，我并没有真正的 Staff 项目。在我晋升的时候，我大约在 6 个月前从管理岗位转回，所以我参考了我管理期间的一些领导经验。当时，我（从技术方面）领导着 Box 团队中一个非常小的团队，参与一个跨公司的合作项目，该项目涉及了解另一家公司的开发团队的需求，并弄清楚如何在满足他们需求的同时尽可能少地构建。我是一个工程范围内的 API 工作组的成员，负责建立和维护我们的 API 标准，并且我还有几个正在进行中的副项目。我想说，所有这些都对我的晋升的各个方面做出了贡献，并共同帮助我确立了我可以展示他们期望的所有方面。

你还记得关于晋升到 Staff 的任何特别有用的建议吗？有没有一条更容易晋升到 Staff 的途径？

我得到的建议之一是在某个时候放大我的优势。我们所有人都有优点和缺点，我们花很多时间谈论“需要改进的领域”。很容易让人觉得，进步的最佳方式就是消除所有这些缺点。然而，如果这真的是我们薄弱的领域，那么即使稍微有所进步，也可能需要大量的工作和精力。显然，你仍然要确保你没有任何真正糟糕的领域，但是假设你已经做到了这一点，那就专注于放大你的优势。你如何将你擅长的东西变成你的超能力？另一个需要考虑的问题是，你如何利用你擅长的东西来弥补你薄弱的东西？例如，我是一个巨大的内向者，不太喜欢与我不认识的人交往。我不擅长与陌生人建立联系。然而，我擅长写作并且喜欢写作。我利用我在公共博客上的写作来结识我原本不会认识的人，并更广泛地获得曝光。事实上，我确信我从中获得的远比我去参加许多、许多聚会所获得的要多。

另一个更具策略性的想法与我们在 Box 编写晋升案例的过程直接相关。有人向我提出了一些建议——首先，在我确定我准备好晋升之前很久就写好晋升案例。这可以让你看到可能存在哪些差距，并可以给你非常具体的工作要做。（或者你可能会感到惊讶，并意识到你比你想象的更早准备好晋升）。第二点是要非常清楚这些差距在哪里。当晋升委员会阅读晋升案例时，所有的案例都会非常积极。当他们晋升时，没有人会说任何负面的事情。因此，委员会不会寻找负面的东西，而是会寻找没有说出的东西。空白点在哪里？似乎在回避或含糊其辞的是什麼。从这个角度看看你的案例——可能缺少什么？你正在掩盖什么？一定要努力解决这些问题。最后，讲述你的故事。

我们的晋升案例有带有明确问题的模板，但是，尤其是在更高的级别上，每个人都不一样，我们也不希望他们一样。不要只是回答问题，而是首先思考你的优势是什么。你的超能力是什么？你的故事是什么？然后弄清楚如何将这个故事融入到提示中。如果你在其中包含你最好的优势，你将会有一个更好的整体案例。

如果我没有在管理层中徘徊，我可能会更快地晋升到 Staff。也就是说，我不后悔这样做，我学到了很多关于人们如何思考、组织如何运作以及更大的项目如何被优先考虑的知识。所有这些都继续帮助我在

IC 轨道上完成我的工作，并且可能进一步帮助我晋升到高级 Staff。虽然我确实认为这很可能减慢了我晋升到 Staff 的速度，但我实际上不太确定下一个级别——我认为我很有可能在没有它的情况下在 Staff 待更长时间。所有这些都是想说，即使我没有采取最直接的路线，我仍然学到了很多对我的长期发展有帮助的东西。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

你越资深，你的工作就越少与代码有关。当然，与人事经理不同，你仍然有非常技术性的倾向，甚至通过负责人，你可能至少会做一些编码。然而，你职位越高，你的工作就越是关于指导和培养你周围的人（以及更广泛的范围），通过建立你公司的公共技术品牌来建设你的团队，注意到可以改进或纠正的更大的技术趋势，帮助为你的团队或公司设定技术愿景，并倡导为技术债务项目提供资源。它变得更多的是关于看到更广泛的事情并让其他人加入。突然之间，沟通、领导力和说服力比以前更加重要。

你有没有考虑过工程管理，如果有的话，你是如何决定走上 Staff 工程师这条道路的？

实际上，我在 Box 工作期间管理了大约一年半的时间，发现我讨厌它（你可以在我的[关于这个话题的博客文章中找到更多信息](#)）。也就是说，我发现实际上，在大多数公司中，管理层和员工+角色之间存在很多重叠。这两种角色都需要指导他人、领导和说服他人的能力。它们需要更宏大的思考和对更长远的关注——无论是在技术方面还是在人员方面。虽然我不打算回到管理岗位，但我确实[在我管理期间学到了很多](#)而且这段经历实际上帮助了我晋升到 Staff 及以上职位。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

我不太倾向于追随任何特定的人，而是向我周围的几乎所有人学习并从中获得灵感。我在这里列举一些，但老实说，我会说我从各个级别（包括许多比我资历浅的人）的无数不同的人那里学到了东西。

我有一个经理，每次我带着问题去找他时，他总是反过来问我，问我应该怎么做。到了后来，我甚至能听到他告诉我直接给某人反馈，或者告诉我如何解决问题，而我根本不需要和他谈。他真的教会了我，虽然作为一名经理，他愿意支持我，但如果我自己能做到，我将学到最多，成为最好的自己。他教会了我对一切负责。

作为对比，我还想提一下一位与我共事过的首席工程师，他后来教会我，我不需要尝试自己做所有事情。在我学会承担责任之后，我开始忘记我不是孤身一人。当然，我听人们谈论过授权，但听听或在冲刺任务中思考是一回事，而授权让某事优先处理，或授权为团队制定技术愿景，或授权跟进一项倡议则是另一回事。

我和另一位同事一起工作，她有时会让我非常恼火，因为她解决问题的方法和我的截然不同。当我认为显而易见时，她会要求澄清，当我认为每个人都在同一页面上时，她会要求详细解释。然而，她也是我合作过的最聪明的工程师之一，与她合作让我意识到，不仅不同的风格同样出色，而且有时将两种冲突的风格结合在一起，可以产生比我们任何一个人单独取得的更好的结果。她发现了我认为显而易见的事情中的漏洞，虽然她有时让我很恼火，但我们完成了一些了不起的事情，我因此变得更好。

达米安·申克尔曼 - Auth0 首席工程师

故事记录于2020年8月。了解更多关于Damian的信息，请访问他的[博客](#)，[Twitter](#)，和[Linkedin](#)。

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是Auth0的一名首席工程师，Auth0是一个身份即服务平台。我在系统架构组工作，该组目前有三名首席工程师。我们与不同的团队合作开展战略计划，并塑造Auth0的[技术战略](#)，架构决策和指南。

在撰写本文时，我正在与一个[身份和访问管理](#)（IAM）团队合作，担任大型新产品功能的技术负责人，并与其他团队一起推动与可靠性和扩展相关的计划。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

在工程部门内部，我们按领域组织（目前为身份和访问管理、开发者体验、服务管理和平台）。Auth0 的 Staff Engineer 是可以在领域内技术领导团队的人员。Staff Engineer 通常是领域内单个团队的成员，同时也能积极为整个领域的计划做出贡献。

Principal 是我们职业阶梯中的下一个级别。Principal Engineer 可以在特定团队中（深度），也可以与多个团队合作，他们的范围涵盖整个组织（广度）。今天我以“广度模式”运作。这意味着既要从事具体计划，也要定义技术战略、我们平台的技术选择，并领导设计和架构工作组（又名 DNA）。

DNA 有 6 名成员（3 名常任 Principal，以及 3 名每 6 个月轮换一次的 Staff/Senior II）。该工作组定义决策和指南，以帮助推动 Auth0 的技术朝着特定方向发展（例如，避免语言扩散，以便我们可以构建

一次库，人们可以轻松切换团队），并与团队合作进行大型计划的技术审查。

我的角色最大的不同之处在于，因为我在公司工作了 6 年以上，其中 3 年以上担任工程总监，所以我拥有“最广泛的范围”。我与产品和平台团队合作开展计划，并且经常与公司的其他部门合作：参与与重要潜在客户的对话，与我们的法律团队合作处理合同语言，或与市场营销部门合作。

你每天如何度过你的时间？

这变化很大。典型的一周涉及很多会议，所以我正在尝试一种新方法：将会议集中安排在周一、周三和周五。周四完全屏蔽，周二只处理紧急事务。因为我们是远程办公，所以所有会议都在 Zoom 上进行。

在开会日，我经常有：* 1:1s：与我的经理（工程副总裁）或团队经理或技术主管进行沟通。这些对话对于了解他们的最新情况和了解他们的挑战非常有用。我觉得如果过于脱离这些，会影响我有效完成工作的能力。* 团队会议：工程领导力、设计与架构工作组。

也会举行非定期会议。一些示例主题可能包括：

- 我担任技术负责人的具体项目
- 帮助一组团队弄清楚如何启动某件事
- 进行同步设计审查

在周四（以及尽可能在周二）我花时间思考：

- 当前的计划以及它们的进展情况
- 我们将来可以/应该做什么（下个季度，明年）
- 撰写文档、指南、博客文章
- （不经常）进行 POC 和/或编写小型工具

作为一名高级工程师，你觉得在哪里最有影响力？如果能讲一个具体的故事就更好了。

最大的影响来自于能够帮助实现“人员规模化”，积极影响尽可能多的人的内部工作。这本书[Scaling Up Excellence](#)提供了一个容易理解的类比：规模化是一场地面战，而不是一次性的空袭。这需要大量的时间和耐心，但要达到你的目标，你需要让整个公司在目标以及如何实现目标方面保持一致。

作为一名首席工程师，我试图找到我认为可以为尽可能多的人在长期内设定方向的机会/差距。围绕某个特定主题，让我们产品交付组织中的约 200 人保持一致，比我自己为某个问题编写解决方案更有价值。前者影响更大，扩展性更好。

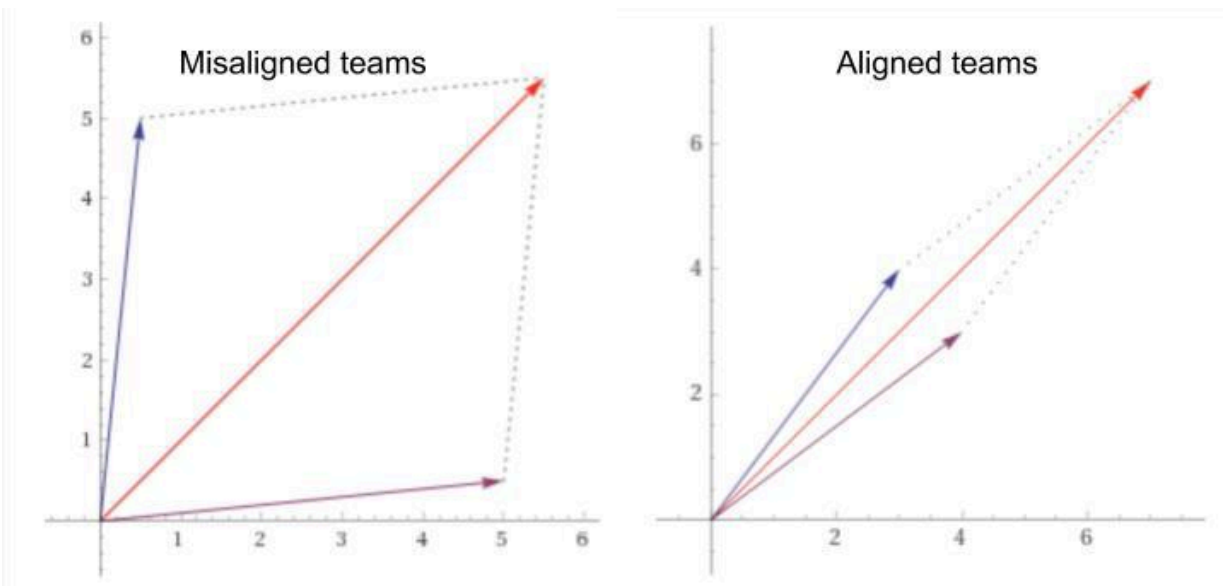
你能想到你作为一名高级工程师所做过的，在你达到这个头衔之前你无法做到或不会做的事情吗？

在成为首席工程师之前，我曾在 Auth0 担任工程总监。最有趣的是，作为一名首席工程师当我在提供反馈时，人们的防御性会降低很多，而且他们在一对一的交流中似乎更加开放。我认为这可能与这样一个事实有关，即作为一名首席工程师，你并不“代表组织的一部分”。

在这方面，作为一名个人贡献者感觉好多了。

你是否花时间倡导技术、实践、流程或架构变革？你倡导过什么？你能分享一个影响你组织的故事吗？

快速发展的公司面临的一个常见问题是，通常存在某种“缺乏清晰度”的情况。在我们的案例中，对于未来会发生什么存在很多困惑，这使得我们在做出技术决策时变得缓慢而低效。团队不确定是否应该使用某种特定的技术，因为他们不知道该技术在未来是否会得到支持；他们不确定是否应该以某种方式构建某个特定的产品，因为他们不知道该方法是否符合我们的长期技术战略等等。自然，这导致了效率低下。



我们认为我们需要一个长期的方向来解释今天如何处理问题的技术实现，以及如何弥合我们最初的情况和未来愿景之间的差距。更准确地说，我们需要一份有据可查的技术战略，详细说明我们应该做什么和不应该做什么才能在长期内取得成功。

在与很多人交谈后，我了解到他们都接触过不一致的信息和谣言，这让他们害怕做出决定，例如：“我听说公司将来会朝着 X 方向发展”或“我听说我们的平台团队将不支持这项特定的 Y 技术。”一个特殊的谣言引起了很多混乱，即我们将支持某个客户的需求及其技术影响。人们一直在听到这件事，但从未宣布具体的计划。我写下了这些问题，连接所有的点，旨在将这些信息转化为知识。我意识到我们需要短期和长期的解决方案。

短期：我们必须填补与一些更紧迫和短期事项相关的不确定性。团队需要做出技术决策，不能等待完全成熟的技术愿景和路线图。我们还意识到，一旦我们有了长期的愿景和决策，自然就需要审查特定例外的决策。我组建了“设计和架构”（DNA）小组，该小组还编写了指南和建议，包括“批准的”技术选择，以指导团队做出不需要审查的独立决策，并建立了 RFC 审查流程。

长期：我想出了一系列我认为公司需要做出决定的主题。我根据两种不同的受众——高管和技术人员——调整了我的演示文稿。对于高管受众，我开发了一个简洁的演示文稿，应用非技术性的类比和解释，并提供可操作的解决方案。技术演示文稿更加详细，包括许多技术术语。我使用了 [nemawashi](#)（一种非正式的过程，通过与相关人员交谈、收集支持和反馈等方式，悄悄地为一些拟议的变更或项目奠定基础），并与我的工程副总裁、其他高管、我的同事和其他高级领导分享，以在正式做出决定之前获得支持。更具体地说，我主动询问人们的想法和意见，以确保获得支持，这样当我们开会讨论我们的决定时，他们就不会是第一次看到这些想法。我们最终见面，讨论了权衡，并达成了一系列决定。所有决定都记录在决策日志中，我们以书面形式指定了具体的负责人来推进这些决定。

当您花在实际开发上的时间越来越少时，您如何与事物的实际运作方式保持联系？

这两个方面，一是跟上总体技术发展，二是跟上 Auth0 的发展以及工程团队当前的“状况”。

以下是我与 Auth0 相关事物保持联系的方式：* 内部：通过 Slack 和与一些技术负责人和工程经理进行 1:1 的交流，密切关注情况。这有助于我第一手了解他们遇到的挑战，并找到模式或得出全局解决方案，而不是局部解决方案。* 外部：与客户/潜在客户交谈，了解他们如何使用产品，阅读提及 Auth0 和身份领域的推文和新闻。

我不觉得我在技术方面“保持联系”的程度达到了我想要的程度，但我确实在努力。我们行业每个月都在发生很多重要的新事物，很难跟上。接受会议和不亲力亲为意味着我可能不如我希望的那样了解情况这一事实很重要。一旦我接受了这一点，我就可以开始优先考虑有价值的东西。

我读书，抽出时间做一些 POC，或者阅读关于特定主题的博客/论文，并要求领导特定的倡议，以便即使我不经常编码也能了解我们的发展情况。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

是的，很多！我是我们工程领导团队的成员。我们每周会面两次，讨论有关组织的话题。这，加上密切关注情况，以及参与关于中期计划的会议，帮助我了解（有时提出）可能存在的机会。

每当发生这种情况时，我通常会提出我认为会从该机会中受益的人的名字，解释为什么我认为他们会擅长，并且，如果有帮助，提供指导，以防出现任何感知到的技能差距。

你最初在Auth0获得了首席工程师的头衔。你是作为首席工程师被雇佣的吗？如果不是，晋升为首席工程师的过程是怎样的？

我的故事比较特殊。我于2014年5月加入Auth0，是第五位工程师，大约是第十位员工。当时没有头衔，没有阶梯，什么都没有。大约在2015年，我开始指导并与几位新员工进行1对1的交流。到2015年底，我一直在做我的倡议，并且领导其他人，帮助招聘等等。2015年末[Matias Woloski](#)，Auth0的首席技术官和联合创始人，正在寻找领导工程团队的人，他问我是否愿意担任工程总监。

我很荣幸能够以一种最大化学习和解决难题机会的方式来对待我的职业生涯。这是帮助我做出决定的主要原则。当他向我，一个住在阿根廷的25岁年轻人，提供领导一家呈指数增长的“硅谷”远程优先公司的工程组织的机会时，我自然而然地说了“是”。我从未想过“我想管理”，这只是因为我想学习和解决难题而发生的。



事情进展得很顺利。我学到了很多关于组建团队、组织、领导人员等方面的知识。因为我是最早的工程师之一，构建了很多系统，并且我喜欢技术方面的对话，所以我也在这个角色中做了很多技术领导工作，包括与产品和平台/基础设施团队。在2019年初，作为平台总监，我开始觉得我学习的速度不如以前快了，并且我想要比仅仅在我们的平台上工作更广泛的范围。在与许多人交谈之后[克里斯蒂安·麦卡里克](#)，当时Auth0的工程副总裁，我意识到我想要挑战的是接下来要做的是成为 Auth0 的技术领导者之一。我于 2019 年 8 月转任首席工程师。

在你晋升为负责人过程中，最重要的两三个因素是什么？你加入的公司、你的所在地或你的教育背景对你的职业道路产生了什么影响？

我喜欢塞内加的一句名言是“当准备遇到机会时，就会发生幸运。”晋升到 Principal 职位需要做对一些事情，但也需要很多运气。我想指出一些让我晋升到 Principal 职位的关键因素，并展示运气在其中所起的作用。

第一份工作：在阿根廷，在大学期间开始工作是很常见的。当我高中毕业时，我在一家名为[南方工厂](#)关于那个地方，有两个关键点：

- 公司从事尖端技术的项目，这给了我很多磨练学习技能的机会
- 公司主要作为微软美国供应商远程工作，这意味着不仅技术技能受到重视，而且我们还可以练习沟通、期望管理和其他人际交往技能。

我高中毕业后就能从事软件工作的原因是，我十一岁时开始告诉妈妈我想“制作电子游戏”，我的父母找到并支付了一所教授编程的高中。

运气所起的作用：我正要到另一家公司工作时，我高中时的一个朋友告诉我，她的哥哥在 Southworks 工作，他们正在招聘初级人员。他很好地向我推销了这家公司，我决定暂停另一个机会，看看我是否可以进入 Southworks。

Auth0：我是 Auth0 的第一批工程师之一，多年来，我参与了其产品和基础设施的许多部分，这使我能够轻松地帮助他人并就各种主题提供有价值的意见。担任工程总监也帮助我了解了我们业务的许多方面，这有助于我成为一个更有效的贡献者。

运气所起的作用：任何初创公司的成功都需要在许多不同的时间点拥有很多运气。如果 Auth0 没有像现在这样发展，我就不会有机会学习我所学到的东西并达到现在的地位。这一点尤其重要，因为我住在阿根廷，那里的软件行业比美国小得多，而且大多数公司都没有双轨制。

团队运动：我小时候和青少年时期都打篮球，我早就意识到，通过获得任何数量的分数来赢得比赛，比通过获得大量分数来输掉比赛感觉好得多。这以两种方式塑造了我的工作方式：* 它促使我经常帮助团队成员，看看我们如何作为一个团队取得成功 * 它促使我去学习和做那些“弥补差距”所需的事情，这帮助我培养了领导能力和人际交往能力，这些能力在我职业生涯的成长过程中非常有用

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

我没有。因为我在 Auth0 的成长方式，我有点“跳过了那部分”。作为一家初创公司的总监，我有机会在技术上领导许多大型、关键的举措，但没有具体/明确的“员工/主要项目”。

我认为，最接近“员工项目”的事情，就是我在 2017 年和 2018 年领导的工作，以提高 Auth0 的可靠性和可扩展性，领导一些项目以提供更高的 SLA 为我们的一些主要客户。

对于刚开始担任 Staff 工程师的人，你有什么建议？

Staff 在不同的地方意味着不同的事情，所以我给出的第一个建议是尽可能多地与人交谈，以定义他们所在地方的期望。

我要告诉人们的下一件事是要有耐心。他们之所以能达到现在的地位，可能是因为他们技术相当精湛，并且取得了成果，但随着你在阶梯上不断攀升，你的工作成果需要时间才能显现。你可能同时在做更多的事情，而且这些事情的影响有更长的时间跨度。你现在也在影响更多不同角色的人，有时他们需要更长的时间才能“看到”你可能清楚地看到的事情。耐心、逐步地影响他人和教导他人，从长远来看是有回报的。

最后：习惯于把事情写下来并重复给别人听。写下想法、计划、推理和标准是你扩展自己的方式。当你记录下一些东西时，你可以让任何人在将来轻松地访问和阅读它，它更容易被引用。这比“只是谈论它”要好得多：它扩展性更好，而且还可以减少被误解的机会。重复也是必要的，因为仅仅发布书面文件是没有用的，所以你必须与人们分享你的想法。举办 AMA、棕色袋子会议和其他会议来解释你的想法非常有价值。

你有没有考虑过工程管理，如果有的话，你是如何决定走上 Staff 工程师这条道路的？

我没有计划这样做，但是当有机会成为总监时，我抓住了它。但是，我的想法是[有一个钟摆](#)在那里你可以在两条路径之间来回切换。这有多容易将取决于公司以及如何即使作为员工/负责人的技能是专门的，但我认为这是可能的。

现在我对继续发展我的技术技能和领导技能非常感兴趣，因为我认为这会带来最有价值的学习和挑战。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

我尝试在 Twitter 上关注那些我认为正在做有趣事情并且我可以向他们学习的人。有很多人在做有趣的事情，而且有很多东西要学习！我想到的名字有：

- [Aphyr](#)与...合作的作品[杰普森](#)以及关于分布式系统的一般内容很棒。
- [Tanya Reilly](#)有一些非常好的内容，比如[Squarespace 的 RFC 流程和作为粘合剂](#)。
- [大卫·福勒](#)分享了很多关于.NET框架和ASP.NET内部结构的内容，我觉得很有趣。还有一个他分享自己如何成为[ASP.NET 架构师](#)。
- 在 Auth0，我与[Jon Allie](#)共事，他是一位出色的工程师和人。他追求简洁，能非常清楚地解释事情，并且考虑到他所知道的知识，他非常谦虚。

我还没有找到很多专门针对高级“个人贡献者”的书籍或类似内容（写一本可能会很有趣）。我最近读了[软件架构基础](#)，它在描述这个角色方面做得相当不错，同时也理解了它的细微差别和灰色地带。

一些关于管理的书籍对于获得组织意识以及帮助指导、一对一会议、招聘很有用，这些都是作为一名高级工程师通常会帮助的事情。在[高产出管理](#)中，安德鲁·格鲁夫将“技术型经理”定义为“可能不直接监督任何人，但即使没有严格的组织权力，也会影响和影响他人工作的人”，这听起来很像高级工程师。我强烈推荐[管理人性](#)因为它分享的故事

事既简单又有趣，而且还有助于与管理者产生共鸣，这对于一名高级工程师来说非常重要。[高效能人士的七个习惯](#)也是一本对高级工程师有很多有益教训的书。

[Accelerate](#)是另一本很棒的书，它以一种有用的方式将公司成功与工程实践和成果联系起来，从而影响利益相关者，尤其是在高管层面。

德米特里·彼得拉什科 - Stripe 基础设施主管技术顾问

这个故事录制于2020年5月。在以下网址了解更多关于德米特里的信息[Twitter](#)或者[Linkedin](#)，并阅读他的[演示文稿](#)。

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是一名高级工程师，也是Stripe基础设施负责人的技术顾问。

我目前的团队是Stripe基础设施的全部，负责Stripe的基础设施服务——计算、网络、存储、数据库、数据工程、性能与效率、可观测性服务和开发者工具。我们的工作使Stripe工程师能够专注于产品。

我“成长”的团队是开发者生产力团队，该团队构建了Stripe产品开发过程中使用的流程、工具和核心库，包括测试框架、代码检查器、类型检查器、构建工具、用于逐步推广的库等等。我最初是该团队的工程师（当时它还是一个单独的团队），最终成为该团队的支柱技术负责人。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

Stripe 的 Staff Engineer 不是一个职位，而是一个级别，对应于对影响、沟通、人员和项目领导技能的期望。Staff Engineer 担任不同的角色，我目前担任的是技术顾问 (TA)。在这个角色中，我与 Foundation 的负责人 Rahul Patil 密切合作，负责提前研究关键主题，深入探讨关键问题（设计、代码、分析），集思广益技术行动项，协助紧急技术跟进，为数据收集配置代码等是我们的目标。这个角色的目的是扩大 Rahul 的带宽和战略思维，但并不直接做出技术决策。

作为该角色的垫脚石，我也曾担任 Pillar Tech Lead（支柱技术负责人）。随着我们拥有更多的 PTL，期望也更加明确：

- PTL 帮助他们的团队做出技术决策，这些决策能够与其他团队在 Stripe 所做的技术决策良好地协同工作。Stripe 的团队自己做出大多数技术决策，但经验丰富的 PTL 可以帮助微调这些决策，以实现更好的结果。当团队在技术主题上无法达成一致时，PTL 也会作为仲裁者。
- PTL 指导 Stripe 的技术方向，为要解决的最重要的问题提供意见，并为解决这些问题设定高层次的技术方法。
- PTL 通过向其他 Pillar Tech Lead 代表他们的组织，并将其他地方做出的技术决策带回他们工作的团队以创建一致性，从而帮助他们的组织。
- PTL 为其他工程师创造机会来承担有影响力的项目并帮助他们成功。

在 PTL 角色中，我过去常常与开发者生产力主管和团队内部的经理密切合作。我们交流背景信息，并朝着共同的目标努力。

这两个角色（PTL 和 TA）的相似之处在于，他们与工程经理合作，并分享对我们用户的需求以及我们可用于解决这些需求的工具的见解，而 EM 更了解 Stripe 范围内的非技术约束（例如，资源约束）。

你每天如何度过你的时间？

在完美的一周里，我会在周一、周三和周五参加会议或工作组：无论是 1:1 的会议还是团队会议，合作制定计划和战略，包括短期和长期。我完美一周的周二和周四会独自编码。实际上，根据当时团队的需求，我最终可能会有更多的会议或更多的编码时间。如果我正在努力启动一个新项目，我通常会先用一周的时间减少会议：专注于项目简报，思考设计、可交付成果/里程碑以及安全/可靠性影响；然后用一周的时间在公司内部推广该提案并解决反馈。

虽然有时可能觉得很难找到时间编写代码，但我认为这很重要，因为它使我能够与工程保持紧密的联系，并成为业务需求/优先级排序与 PTL 需要的工程约束之间的桥梁。

作为一名Staff-plus工程师，你觉得在哪里最有影响力？

首席工程师，尤其是 Pillar Tech Leads，经常帮助确定新项目的方向。当我可以帮助改进一个善意且解决了实际需求的提案时，我感到特别有影响力，但起草该提案的团队缺乏经验或背景来编写一个好的计划来抓住机会。在这种情况下，拥有一个结构良好的计划可以帮助大幅缩小范围，同时获得大部分价值，从而更快地展示影响力。或者，或者，看到手头的提案比团队最初预期的解决了更多的用例，并将项目重新聚焦于团队不知道的用例，这将导致更大的商业影响：在这两种情况下，我都感到通过授权其他工程师而产生影响。

你能想到你作为一名高级工程师所做过的，在你达到这个头衔之前你无法做到或不会做的事情吗？

不，Stripe 并不打算将 Staff 徽章作为进入新机会的门槛，我相信我们在这方面做得很好。PTL 角色也是如此。我们选择擅长代表他人意见的工程师担任 PTL 职位。甚至在我成为 PTL 之前，我就觉得之前的 PTL，[保罗·塔扬](#)，总是确保我的观点被呈现出来。

你是否花时间倡导技术、实践、流程或架构变革？你倡导过什么？你能分享一个影响你组织的故事吗？

我被专门聘用来在 Stripe 中引入 Ruby 的类型检查。这包括，与[纳尔逊](#)和[保罗](#)，一起，架构和实现类型检查器，[冰糕](#)，并发展围绕使用它的文化。

在 Sorbet 的早期，我们根据 Stripe 最需要的使用案例，仔细选择要添加的功能。我相信我们已经成功地用一个类型系统覆盖了 Stripe 的大部分使用案例，同时保持了简单性：很容易进入一个类型系统和文化，为了它而提倡复杂性和精英主义，我很高兴我们的努力避免了从无类型性走向另一个极端。

今天，作为技术顾问，我倡导进行能够产生巨大影响的变革，最常见的是在 Stripe 的可靠性、可扩展性、安全性和生产力方面。这可能包

括改变数据的分片/存储方式，或者改变我们处理变更管理的方式。但最大的区别在于与我在 Sorbet 持续数年的项目不同，我希望找到/培养一个人来尽快接手该项目——在组织认可，并且有一个具有明确里程碑和受控风险的计划之后。我将与推动这些重要项目的人员保持频繁的沟通，目标是帮助减轻这些风险并发现更快交付项目的机会，因此我的参与仅在项目的早期阶段可见。

当您花在实际开发上的时间越来越少时，您如何与事物的实际运作方式保持联系？

当我是 PTL（团队负责人）时，我每周至少有几天可以编写代码。我与团队中的其他工程师密切合作，我们不断地互相学习。

作为一名技术顾问，我编写代码的机会不如我担任 PTL 时那么多。我主要在以下情况下编写代码[代码黄色警报情况](#)但这个角色的成功取决于拥有良好的洞察力和深刻的工程理解。为了做到这一点，我会与我们的内部客户沟通，并随时掌握我所支持的团队的设计，特别是故障阈值和故障模式。

在我的角色中，了解客户的需求至关重要。一个有用的资源是开发者生产力团队组织的 Stripe 全公司工程师调查，我们希望从中找出哪些是阻碍我们的工程师提高生产力的最大因素：也许自上次调查以来，某些工具变得缓慢，或者某些用例的用户群增长但没有得到很好的支持。虽然这项调查很少发现我们不知道的事情，但它是一个很好的相对优先级排序工具：我们可以比较有多少人抱怨这些事情，并相应地确定它们的优先级。

此外，在 Covid-19 引起的封锁之前，我过去常常在 Stripe 随机加入餐桌共进晚餐。我会问 3 个问题：

- 你在做什么？
- 是什么让它变得困难？
- 基础设施团队如何提供帮助？

这在两个方面成为一个很棒的工具：1) 将我与我的用户联系起来，帮助发现他们的需求；2) 通过进行类似于“是的，我同意我们可以通过做 X 来帮助你，现在，让我们一起来看看我们应该停止做什么来为此创造空间”的讨论，来帮助缓解尚未获得支持的团队的不满，在这种情况下，一个人经常会发现，虽然他们希望我们解决他们的痛点，但他们不希望以我们降低当前项目的优先级为代价来解决。

当我要从 PTL 的角色过渡时，我创建了一个名为 DevProd Assembly 的小组，该小组聚集了开发者生产力团队的领导者。该小组的每个成员都应该与 2-3 个产品团队建立高度信任的关系，每月采访他们，并将反馈与其他 Assembly 成员汇总。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

虽然赞助其他工程师不是 Staff Engineer 的必需条件，但我认为它有助于你成功，因为它通过为他人创造机会并帮助他们成功来帮助你产生更大的影响。

是的，我帮助确定范围、启动和降低风险的多个项目，同时也帮助培养一个人在我完成下一件事时接替我。

指导和赞助之间也存在区别，我两者都做。指导是帮助人们成长并产生影响。赞助是帮助一个人进入能够展示他们交付能力的职位更大的影响。在与我的团队合作时，我尝试帮助人们从事一些超出他们舒适区的项目，在这一点上我赞助他们，然后，我可以指导他们以帮助项目成功。

你首先在你现在的公司获得了 Staff engineer 的头衔。你是作为 Staff engineer 被雇佣的吗？如果不是，晋升为 Staff 的过程是怎样的？

我不是作为 Staff Engineer 被雇佣的。我不得不升级两次才能达到 Stripe 的 Staff 级别。这两次升级都很相似：Stripe 在员工已经在下一

个级别上运行了相当一段时间后进行升级，这调整了他们有望继续在该级别上运行的期望。

你认为达到 *Staff* 级别最重要的两三个因素是什么？

按重要性递减的顺序排列：

1. 专注于对业务和公司的影响。
2. 保持协作：通过参加会议/工作组，你应该帮助实现更好的结果。
3. 技术知识。

对我个人而言，在成为 Staff Engineer 之前需要擅长的领域是第二项。我已经产生了影响，并且被认为是寻求技术建议的人。我需要提高我的沟通技巧和协作技巧，以便建设性地帮助团队之外的人，他们可能是第一次见到我，并且尽管他们的项目背后有良好的意图，但可能没有最好的计划来交付它。

帮助我的一种技巧是在会议结束后立即在私聊中寻求反馈，尤其是在那些进行得不太顺利的会议之后。这使我能够了解我做了什么可能导致其他方在这些会议中感到不舒服，并且在少数情况下，真正询问如何才能做得更好，有助于纠正已经进行得很糟糕的会议的结果。

你的工作经历和教育背景如何影响你的职业道路？

公司：我很欣赏 Stripe 有这么多产生影响的机会，这确实帮助了我。

教育：我碰巧获得了非常实用的博士学位（关于如何构建快速且可维护的编译器），几乎完全转化为适用于我工作的知识：帮助公司扩展工程。而且，虽然它对我很有帮助，但我认为这里面有很多运气成分：我碰巧在正确的时间加入了正确的实验室（当 Scala3 诞生的条件变得重要时，我在实验室的时间足够长，不会太“生疏”，但仍然足够早，还没有完全确定我的研究方向）。我不确定我是否会建议其他人攻读博士学位：从我的角度来看，实际上，我的许多朋友可以通过在 Stripe/Google/Facebook 构建系统来学到同样多的东西，而这需要 4 年以上的时间才能完成博士学位。如果你想学习数据库是如何工作的

——你可能不仅会在研究数据库的实验室里学到，而且还会在对数据库有最高需求并有团队致力于改进数据库的公司里学到。也就是说，博士学位对我来说是一个改变我的位置的好工具。

地点：我从瑞士来到美国加入 Stripe。我从俄罗斯来到瑞士，加入了计算机科学领域最好的博士项目之一。我从乌克兰来到俄罗斯，加入了前苏联最好的大学之一。在每一次搬迁中，我都觉得自己是在进行地理套利：我试图逃离我属于最好的地方，去到我属于平均水平的地方。在其中一些搬迁中，我认为我不是最佳人选。通过加入那里的人们并向他们学习，我成长了很多。我很难说美国与欧盟哪个在职业发展方面更好，但我可以肯定地从我的经验中看出，改变地点帮助我成长了很多。

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

回顾起来，这对我来说是一个很难回答的问题。这是因为：据我所知，Sorbet 足够大，可以成为我的 Staff 工程师项目，但是由于 Nelson 和 Paul 在那里，我们彼此密切而快速地合作，因此很难将项目的成功归功于特定个人，而不是整个团队。

在对项目进行第一次绩效评估时，我们三个人都收到了反馈，认为我们应该有更好的方法来确定哪些影响是哪些人造成的。虽然我很想说，我们在下一次绩效评估中没有遇到类似的问题，是因为我们采取了有意的行动，但我认为事实并非如此：我认为项目只是自然而然地进入了一个更大的阶段，因此我们不需要“在相同的 10 个文件中快速迭代”，自然而然地导致我们拥有更清晰、更大的所有权领域。

我成了“内部架构/子类型”负责人，以及“与用户交流”负责人，而保罗成了“修改代码以使类型检查器喜欢它”负责人。尼尔森显然更了解 Stripe 的其他系统是如何工作的，因此帮助将该工具与它们集成。所有这些都发挥了我们的优势：我之前有类型检查器的经验（这是我的博士学位的内容），保罗在程序化代码修改方面有很强的技能，尼尔森既在系统方面非常知识渊博，并且在 Stripe 工作的时间足够长，也

足够早，几乎了解Stripe的每个系统。在项目的这个阶段（稳定、推出），所有这些都变得非常重要因此，让一个人成为某个领域的直接责任人（DRI）变得容易得多，其他人偶尔提供帮助。

在冰沙项目之后，我在短时间内（6个月）交付了其他几个有影响力的项目，我相信这最终促成了我获得Staff Engineer级别，但是，如果我必须选择一个，我仍然会选择冰沙项目，因为它具有广泛的项目范围：包括技术和文化。

你还记得关于达到 Staff 级别的任何对你特别有用的建议吗？

1. 在学术界与 Martin Odersky 和 Ondrej Lhotak 共事，帮助我理解了复杂系统如何协同工作，以及如何清晰地解释这一点。
2. Brian Goetz 帮助我理解了一个简单而强大的系统背后需要付出多少努力，才能经受住广泛的应用和设计。
3. 保罗·塔扬向我展示了调整我的沟通方式以促成所有相关方的建设性成果的重要性。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

至少在 Stripe，Staff Engineer 在非常不同的领域工作。确保你与你的汇报链达成一致，即你应该实现的影响是什么，以及为了达到该影响，你允许在哪些方面做出妥协。清楚地沟通你正在做的妥协以及原因。

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

过去每次我考虑这个问题时，都会问自己和周围的人“这是否是一种带来更大影响的方式。”到目前为止，每次的答案都是“似乎没有”。

也就是说，我发现即使在个人贡献者的角色中，从优秀的管理者（就我而言，是 James Iry、Scott MacVicar、Will Larson、Christian Anderson 和 Shane O’Sullivan）那里学习一些管理技能也能带来巨大的好处。

斯蒂芬·万 - Samsara 职员工程师

这个故事录制于2020年9月。了解更多关于斯蒂芬的信息，请访问他的[Github](#), [Twitter](#), 和[Linkedin](#).

请简单介绍一下您目前的角色：您的头衔、您所在的公司，以及您的团队通常做什么样的工作？

我是一名在职工程师，在[轮回](#)。

我四年前加入这家公司，当时公司成立一年左右，我们有大约50名员工。现在，我们公司有超过一千名员工，在湾区、亚特兰大和伦敦都有工程团队。

当我刚开始工作时，我们还没有组建真正的团队，我们十几个工程师负责处理任何出现的问题。九个月后，我们的人数增加了一倍，并围绕当时的一些核心产品组建了产品团队。我曾短暂地领导过一个产品团队，然后转到我们新兴的基础设施团队，开始组建我们的前端基础设施团队。多年来，我逐渐向下转移，也曾在我们的后端和可观测性系统上工作过。

今天，我在我们的基础设施与平台（I&P）小组工作，大部分时间都花在开发者体验团队，该团队构建工具，以保持我们的全栈开发工作流程高效。

在贵公司，“正常”的Staff-plus工程师是做什么的？你的角色是那样吗，还是有所不同？

我们的大部分 Staff+ 工程师都在专门的角色中，要么在 Web 基础设施中，要么在设备固件中。我想这让我占据了多数，但我们的 Staff 工程师所做的工作各不相同，所以很难声称在这个角色中是“正常的”。

在考察 IC（个人贡献者）和管理路线之间的相似之处时，Staff 被认为是相当于总监的角色。Staff 工程师可以选择参与许多通常为管理者保留的流程。我们被邀请参加跨工程总监会议，至少在 I&P 中，我们参与路线图规划和管理同步。最近，我们引入了 Staff 工程师来参与一些晋升校准会议。

这种访问权限在某种程度上让人感觉一只脚踩在两边。这个角色与高级级别有很大的不同，它不太像一个“个人”的软件贡献角色，但它也与我们更注重人的管理路线有很大的不同。

在[Will 的 Staff 原型](#)中，我认为我的角色介于解决者和技术负责人之间。对我来说，乐趣的一部分在于每 6-12 个月跳入一个不同的角色，与不同的人群一起深入研究系统的不同部分。

你每天如何度过你的时间？

每天的情况都大不相同。现在，我尽量把周二和周四定为会议日，这样我就可以在一周的剩余时间里有专门的专注时间。

我的会议日通常包括与我密切合作的人员的 1:1 会议以及员工会议。我还会花一些时间与个人一起进行代码和设计审查，或者进行更开放的设计讨论。

在其他日子里，我的专注时间花在调查模式上，试图找出当前的問題，并为未来的项目铺平基础。哪些系统需要投资？我们的团队执行情况如何？我们的小组应该为哪些即将到来的变化做好准备？这段时间花在了[塑造](#)从更广义的层面来说。

回顾过去，这种关注点与我担任Staff之前的角色截然不同。我不再是在个人层面直接参与项目或团队的执行，而是将时间集中在更广阔的视角和更长远的目标上。

值得注意的是，我很难保证一天以上的时间都用来编写代码。在考虑工程路线图带宽时，我不会被计算在内，尽管我确实尽量每周留出至少一天的时间来编写一些代码。

在微观层面，我维护着一份名为“斯蒂芬在做什么？”的文档，它驱动着我以小时为单位的工作。其中有一个主要部分是关于当前周的，还有一些关于未来周的提醒。每个星期一，我都会重新开始——上周末开始的项目会被删除，很少有项目能保留到下周。

🍌 what is stephen doing?

quarter commitments

- investigation: lightstep/tracing poc
- design partner: 99.9% availability measurement
- design partner: good error handling in frontend
- design partner: org isolation rollout plan with security
- design partner: deploy state migration plan

10/14

- ☐ +Infra and Platform Roadmap Brainstorming
- ☐ product team rotation plan for adam
- ☐ catch up with federation design with will
- ☒ terraform splitting chat with kelly
- ☐ review dev exp okr draft
- ☒ review: <https://github.com> [REDACTED]

10/20

- ☐ revise +💡 RFC: [Buildkite workload improvements](#) with adam/aashish
 - edit rollout schedule
- ☐ re-evaluate latency monitor false positive/negatives set from 10/9

斯蒂芬在做什么？

这种默认删除的意图最终帮助我保持专注，不会感到过于分散。很长一段时间，我试图整理一个待办事项清单，但它主要起到了让我感到压力的作用。大多数待办事项最终都会被删除，而且一个月后仍然不完整。

你是否花时间倡导技术、实践、流程或架构的改变？你倡导过什么？你能分享一个影响你组织的故事吗？

是的。具体的技术或方法论每个季度都在变化，但倡导最终占据了我大部分时间。在较小的方面，我帮助编写了关于我们如何尝试处理设计文档、代码审查或代码所有权规则的文化文档。

举一个更大的例子，我花了几个季度的时间帮助我们的产品团队采用服务级别目标（SLO）。

当时，我们已经有了了一套相当完善的功能和客户群，但我们对正常运行时间的衡量仍然很原始。在中断期间，很难了解对客户的影响，因为我们缺乏沟通的区别和定义（“有多少百分比的客户受到影响？是读取和写入都有问题吗？这是中断还是现有错误？”），即使我们有很多指标和仪表板可以查看。

虽然引入 SLO 肯定需要新的工程工作，但我估计我在这项目上花费的大部分时间都用于编写文档、与人交谈以及与团队进行咨询式工作。我们希望大家能够端到端地理解可靠性目标：如何定义目标，如何在中断期间讨论它，如何在系统中衡量它，如何随着时间的推移跟踪它，以及当它变得不健康时如何做出反应。这种深度最终需要大量的消息传递和润色才能深入人心。

与我们大多数大型“迁移”一样，让许多团队采用 SLO 是迭代进行的。我们首先在一个提供高接触支持的团队中试用了我们的新工具，然后才弄清楚了针对组织其余部分的广泛消息传递。我认为我的一个关键作用是能够以具体的、我该如何使用的层面与工程师讨论新工具，同时说服主管级别的人员相信，用 SLO 进行沟通是值得他们努力的。

作为一名资深工程师，这种模式在我的大多数项目中都适用。我的角色最终是在各团队之间进行交易，以在整个组织中推广变革。

当您花在实际开发上的时间越来越少时，您如何与事物的实际运作方式保持联系？

我每周都会花一些时间编程和进行代码审查，即使只是为了修复一个小错误。我尽量花时间参与其他 IC 经历的日常活动——代码审查、浏览文档、中断情况等。

当然，这不足以在我的脑海中保持一个高保真模型——跨越太多团队发生的事情太多，无法跟踪。剩下的就是有意识地寻求反馈并听取他人的第一手经验。

我还试图帮助将反馈循环融入到我们的组织中。我帮助我们开始进行半年一次的开发团队调查，其中包含关于我们的技术系统和工程文化的问题。这些调查的回复对于了解组织从基层开始的感受非常有帮助。

你如何赞助其他工程师？赞助其他工程师是你角色中的一个重要方面吗？

是的。我一直试图有意识地放弃我的状态，退后一步，让其他人积累专业知识。

在组织层面，我认为有一些方法可以在结构上赞助他人，推动其他工程师担任主题专家。例如，去年年底我致力于引入一种新的分布式跟踪系统。我们的核心 Web 应用程序由许多不同的后端系统提供支持，多年来，这些系统之间的数据流变得越来越难以理解和页面性能遭受了损失。我们需要一个工具来帮助我们摆脱困境。

我之前曾参与过我们性能工具的早期迭代，对这些系统的了解主要停留在我的脑海中。在新项目中，我的一个具体目标是让更多人尽快上手。仅仅在系统设计中奠定技术基础是不够的：我在项目中的队友必须在未来几年内掌握该领域的专业知识。

实际上，这意味着我将花费更多的时间与即将成为追踪系统负责人的人讨论和配对，而减少直接贡献代码或设计的时间。当我们与产品团队进行beta测试时，我会推动另一位工程师负责销售宣传，或者弄清楚演示，或者让团队加入。

如今，追踪系统已被广泛使用，并由我们的SRE和可观测性团队全面管理。当时与我合作的人现在是解决性能问题的首选团队。

在更个人的层面上，总有一些小地方我可以帮助推动其他人成为焦点。赞助机会可以从小处开始。特别是如果我与职业生涯早期的人合作，我可能会建议他们承担更多新系统设计中未知的模块，或者为新文档撰写草稿，或者在我们的全体会议上演示我们的成果。

那一点小小的推动可能就是某人开始行动所需要的全部，但有时我认为它也能很好地转化为指导和配对的机会。有些事情（比如第一次构建幻灯片）在做几次之前可能会让人觉得难以处理。在那里，赞助和指导方面最终都感觉很有影响力。

我也觉得需要找到合适的平衡点。我们希望培养人们担任可以拥有更多自主权并做出系统决策的职位，但我们也希望这些决策的制定方式以及我们努力的方向保持一致。这是一项艰苦的工作——最终需要投入大量的精力，既要避免把关，又要取得令人满意的结果。

你首先在你现在的公司获得了 *Staff engineer* 的头衔。你是作为 *Staff engineer* 被雇佣的吗？如果不是，晋升为 *Staff* 的过程是怎样的？

当我加入公司时，我们没有IC职称。在2019年初引入职级制度时，我被评定为Staff级别。

我认为作为团队的早期工程师，我有一个很大的优势。这段历史对我理解我们过去的决策非常有帮助——了解我们已经遇到的陷阱，并帮助将新项目顺利落地。

在每个发展阶段，我们都会增加更多的人员和管理层级，并且组织的工作方式会有一个“重新学习”的时期。随着时间的推移，团队的范围会缩小，只能看到拼图的一小部分。将大部分工程历史记在我的脑海中，不仅帮助我将各个部门联系起来，而且还让我能够先人一步地与

我不再直接合作的组织部门保持个人联系。这种广度自然而然地让我能够弄清楚什么对组织最有影响力。

您达到 Staff 级别最重要的两三个因素是什么？您加入的公司、您的地理位置或您的教育背景对您的职业道路有何影响？

我的背景有点不太传统——我学的是电气工程，而不是计算机科学，而且在完成学位之前就辍学了。这段经历迫使我在经验上更加自学成才，但也让我产生了许多冒名顶替综合症。早期，我因为没有合适的资历而在很多软件面试中失败。在我的职业生涯早期，那种冒名顶替的感觉让我真的很想尽可能多地学习可以弥补我害怕自己不知道的东西。

在我辍学前的那个夏天，我在Stripe实习。我记得，也许是通过玫瑰色的眼镜，我被那里的工程文化所激励：对客户体验的高度关注，以及对构建技术来实现这一目标的热情。这段经历最终对我希望工作场所的感觉产生了相当大的影响。

后来我离开学校后，开始在一家较小的创业公司全职工作，我真的不知道自己在做什么。在那段时间里，业务有点漫无目的，但我很幸运能与有思想、资深的工程师密切合作，他们有指导的爱好。在那里工作最终给了我很大的灵活性，让我可以学习我想学的东西，这对我有好处，但可能对公司不利。

作为最后一点背景，我在高中时曾在一家电脑营地工作了几个夏天，教适龄儿童基本的计算机知识。这种教学心态无疑让我对人们最终如何理解和与计算机系统互动有了更多的同情心。

到我加入Samsara时，这些经历让我清楚地了解了我希望工作的感觉——成为一名早期员工让我有影响力来塑造那里的方式。

最后一块拼图是我在Samsara的头三年。我很幸运能在那个时候与这么多有思想的合作者一起工作。我可以很容易地追溯到我今天拥有的

许多工作习惯、思维模式和举止到那些人。我无法想象如果没有他们的影响，我的职业生涯会走到今天这一步。

有一种流行的观点认为，成为资深工程师需要完成一个“资深项目”。你是否有过资深项目？如果有，那是什么？

不，我没有指定的Staff项目。回顾过去，这些年来有一些项目可能积累成了一个大型Staff项目，但这并不是我们在级别划分中明确谈论的事情。

作为一个概念，我怀疑那种单一关注的项目，并担心它们会让人陷入一种[英雄心态](#)当我们真正想要重视那些能够建设组织，而不是拖累组织的工程师时。我更乐于看到随着时间的推移，迭代改进和持续执行：一种深思熟虑的工程记录。

我很高兴轮回似乎同意这个评估。我们的职业道路文件最终更多地谈论了持续的执行，而不是一个大型项目。

对于刚开始担任 Staff 工程师的人，你有什么建议吗？

我想到了几件事。

习惯多说话。我认为高级职位和Staff职位之间的一个重大转变最终会集中在人身上：调和相互竞争的优先事项，消除误解，使人们在同一页面上保持一致。即使他们通常没有直接下属，Staff工程师也在技术和人员的系统中工作：最大的影响将来自对两者施加影响。

尽力不要让自己筋疲力尽。当我过渡到Staff职位时，很容易陷入一种心态，即我对正在发生的一切负责，并且必须将我的注意力分散到太多的事情上。我花了一段时间才意识到，这个角色并不意味着我必须更加努力地参与到所有事情中，而是我需要通过组织中的其他人来指导变革。信任他人，指出问题，并期望他们解决问题。

你有没有考虑过工程管理，如果有的话，你是如何决定走上Staff工程师这条道路的？

早在2016年，我记得我和我的经理进行了一次关于追求IC与管理道路的初步对话。当时，我仍然觉得我的职业生涯还处于早期阶段，并希望继续投资于我的核心技术经验。

我大约每年重新评估一次这个决定，最终得出了同样的结论——我还没有完成在技术方面的亲力亲为。在那段时间里，我所做的很多工作都集中在为公司的人们构建开发经验。这些努力最终促使我做了更多类似Staff的工作，我自然而然地从那里进步了。

你从哪些资源（书籍、博客、人等）中学习过？你在该领域的榜样是谁？

我倾向于珍视那些用简单的英语谈论复杂话题的文学作品，包括小说和非小说。

我记得读过小说家[村上春树](#)先用英语写他的第一部小说，然后再翻译回日语，以此来塑造他的表达风格。他指出，“我只能用简单、简短的[英语]句子写作。这意味着，无论我脑海中涌现的想法多么复杂和繁多，我都无法尝试将它们原封不动地记录下来。语言必须简单，我的想法必须以一种容易理解的方式表达。”

编写软件是一个完全不同的领域，但感觉这种观点符合我真正重视的关于沟通的某些原则：头脑中有一个理解是一半的战斗——能够表达这种理解同样困难和有价值。

我喜欢阅读那些真正深入技术领域的博客和论文。这些年来，我经常回顾以下几个：

- [Bob Nystrom的博客文章](#)关于编程语言
- [Vyacheslav Egorov的博客](#)关于编译器和V8内部结构（Chrome的JS引擎）
- [Brandur的博客](#)关于各种系统主题
- 尼尔森·埃尔哈格的[意外的二次方](#)
- [Vicki Pfau的博客](#)关于开发GameBoy Advance模拟器

- [fail0overflow的博客](#)并讨论控制台架构和漏洞利用
- [Bungie的工程出版物](#)关于构建和制作最初的Halo游戏

作为一个轶事，在我职业生涯的早期，我对编程语言的内部结构产生了浓厚的兴趣，并拿起了一本编译器教科书（“龙书”）来学习。这是一本很难读完的书。也许和教授以及几个同学一起读完是合理的，但对我来说，从阅读中获得一个可行的心理模型确实很困难。后来，我发现了Bob Nystrom的[Crafting Interpreters](#)它采用了一种更加实用的方法，感觉就像一股清新的空气。

我也是阅读代码库的忠实粉丝。在我的职业生涯早期，我记得调试一个棘手的 React 问题，其中一些回调没有按照我预期的顺序发生。阅读文档没有帮助。放入打印语句是不够的。我当时的导师让我阅读一些源代码，以更好地理解发生了什么，这真的让我大吃一惊。我得到了一个错误修复，但也更深入地了解了 React 的工作原理。

那真是一个转折点。能够快速深入并跳过不熟悉的代码真的感觉像是一种超能力，并为我提供了更大的模式匹配库，用于不同的软件设计方法。最近最喜欢的是阅读[esbuild](#)的设计和代码，这是一个超快的 javascript 打包器。

最后，我最近几年最喜欢的一些非小说类读物是关于[BART 的历史](#)，[施乐 PARC 的历史](#)，和[现代日本文化概览](#)。在每个领域，我都发现历史和背景非常引人入胜，因为看似微小、独立的事件和决定最终演变成了当今世界运作的方式。

结束语



通常情况下，当一群 Staff-plus 工程师坐下来聊天时，他们会花时间哀叹行业中糟糕的技术领导力发展道路。这是真的，它确实很糟糕。但并非不可避免。行业对技术领导力的看法是每个在 Staff-plus 工程师岗位上工作、渴望晋升或管理 Staff-plus 工程师的人每天采取的集体行动的总和。通过改变我们的习惯和方法，我们将改变这个行业。

通过提升自己，我们让行业变得更好。制定一个计划，赞助你组织中的其他人。综合一份战略文件，花时间让别人感到被倾听。建立一个同行网络，并邀请更多人加入该网络。当你做这些工作时，你的工程组织将从你的榜样中学习。

你的经理和管理链也将从你的榜样中学习。许多经理从未与一位高效的 Staff-plus 工程师合作过。相反，他们通过之前的灾难来看待这个角色。与他们合作，了解他们的优先事项和顾虑，以建立一致性。让自己承担与你的经理相同的压力。克服他们对可能出错的事情的恐惧，并为可能做对的事情创造兴奋感。

谈论害怕 Staff-plus 角色是否感觉奇怪？确实如此，但是这是不可避免的。在大多数公司，阻止采用有影响力的 Staff-plus 角色的主要因素是害怕创造一个专制的架构师。这种恐惧源于他们之前与无能和管理不善的技术领导者的经验。只有当我们共同补充优秀的 Staff-plus 榜样的地下水时，这种恐惧才会消退。

我希望阅读资深工程师已经为你提供了一张地图，可以跟随它走向有回报的 Staff-plus 职业生涯，或者激励你继续走你已经找到的道路。软件技术行业仍处于起步阶段，它将变成什么完全掌握在我们手中。

资源



关于 Staff-plus 工程师的补充资源

我采访过的 Staff Engineer 没有一个是独自成功的。他们中的大多数要么通过贪婪的阅读，要么通过建立强大的同事网络来实现。本节是推荐资源的集合。

你的网络

几乎一致地，Staff-plus 工程师最有价值的学习资源不是书籍、博客、演讲或论文，而是他们的同行和导师网络。如果你只有一个小时来发展自己作为一名工程师，你最好的选择是[建立一个从事类似角色的人的网络](#)。

如果你正在寻找一个 Slack 社区，`#staff-principal-engineering`在[Rands Leadership Slack](#)是一个相当活跃的房间。

Staff-plus 工程师是做什么的？

- [在 Skyscanner 担任首席工程师](#)作者：Nicky Wrightson
- [定义杰出工程师](#)作者：Jessie Frazelle
- [我在 Heroku 如何担任 Staff 工程师](#)作者：Amy Unger
- [并非所有的工程领导者都是工程经理](#)作者：Tanya Reilly
- [与 Tanya Reilly 探讨细节](#)
- [关于成为首席工程师](#)作者：Silvia Botros
- [关于成为高级工程师](#)作者：John Allspaw
- [Staff Engineering](#)作者：Sam Kleinman
- [在技术领导力道路上茁壮成长](#)作者：Keavy McMinn
- [高级工程师的工作是什么？](#)作者：Julia Evans
- [高级职员软件工程师的实际工作，第一部分：角色和我的任务和第二部分：角色的心态和重点](#)作者：Joy Ebertz
- [在 GitLab，职员级别意味着什么？](#)作者：Charlie Ablett

成为一名 Staff-plus 工程师

- [成为一名职员工程师——采访 Intuit 的职员 iOS 工程师 Kristina Fox](#)作者：Kaya Thomas
- [关于成为一名高级技术领导者](#)作者：Jesse Pollak
- [关于中年职业和管理者](#)作者：Ryn Daniels
- [如何在谷歌成为一名 Staff 软件工程师？](#)在 Quora 上
- [工程师/经理的钟摆](#)作者：Charity Majors
- [关于工程级别需要了解的事情](#)作者：Charity Majors

作为 Staff-plus 工程师运营

- [作为粘合剂](#)作者：Tanya Reilly
- [计算机是可以理解的](#)作者：Nelson Elhage
- [代码和系统的有效思维模型](#)作者：Cindy Sridharan
- [“我不会从这里开始。”如何做出重大的技术变革](#)作者：Tanya Reilly
- [迁移：解决技术债务的唯一可扩展方法](#)作者：Will Larson
- [关于职业中期和团队动态](#)作者：Ryn Daniels
- [在组织支线任务中生存](#)作者：Tanya Reilly
- [难以详细理解的系统](#)作者：Nelson Elhage
- [团队目标](#)作者：Marty Cagan
- [技术决策](#)作者：Cindy Sridharan
- [技术与研究准备](#)作者：Keavy McMinn
- [技术领导的幕后工作](#)作者：Jean Hsu
- [理解项目管理将改善你的开发者工作](#)作者：Daniel Na
- [赞助是什么样的？](#)作者：Lara Hogan
- [从哪里开始](#)作者：Keavy McMinn
- [设计文档、Markdown 和 Git](#)作者：Caitie McCaffrey

技术规格

- [编写技术规范的实用指南](#)

- [Google 的设计文档](#)
- [设计文档、Markdown 和 Git](#)
- [记录架构决策](#)
- [如何编写更好的技术设计文档](#)
- [远程文化中的技术决策和协调](#)
- [编写技术设计文档](#)

工程策略

- [负责任的创新框架](#)
- [Slack 如何进行大型技术变更 - 几个人在编码](#)
- [关于起草工程策略](#)
- [定义技术战略](#)
- [实现架构战略](#)
- [垫脚石而非里程碑](#)
- [通过技术战略实现一致性和效率](#)
- [困难的青少年时期：发布后制定技术战略](#)作者：安娜·希普曼
- [学习拥有工程愿景](#)

工程策略示例：

- [减少运行软件](#)作者：里奇·阿奇博尔德

关于策略的其他方面也有很多很棒的资源，例如，Marty Cagan 关于“的产品战略”系列[产品战略](#)。

书籍

虽然我发现很多人不太读书，但当我向 Staff 工程师询问他们最有价值的资源时，他们总是提到一位私人导师或一本书。他们可能会提到与更具体的问题相关的博客文章和技术讲座，但他们受这种更大、更纸质的形式的影响最大。

一些被推荐的书籍：

- [软件设计哲学](#)作者：约翰·奥斯特豪特
- [加速：构建和扩展高性能技术组织](#)作者：Forsgren、Humble 和 Kim
- [成为技术领导者：一种有机的解决问题的方法](#)作者：Gerald Weinberg
- [构建进化型架构](#)作者：Ford、Parsons 和 Kua
- [逃离构建陷阱：有效的产品管理如何创造真正的价值](#)作者：Melissa Perri
- [好战略，坏战略：区别以及为什么重要](#)作者：Richard Rumelt
- [高产出管理](#)作者：Andy Grove
- [经理之路：技术领导者驾驭成长和变革的指南](#)卡米尔·富尼埃
- [人月神话](#)作者：弗雷德·布鲁克斯
- [凤凰项目](#)作者：金、贝尔和斯帕福德。
- [《充满激情的程序员》](#)作者：查德·福勒
- [《程序员修炼之道》](#)作者：亨特和托马斯
- [弹性管理](#)作者：Lara Hogan
- [软件设计X射线：用行为代码分析修复技术债务](#)作者：亚当·托恩希尔
- [系统思考：入门](#)作者：多内拉·梅多斯

如果你正在寻找更多，推荐书单比比皆是，包括我自己在[非理性繁荣的最佳书籍](#)。

演讲

我聊过的 Staff-plus 工程师通常提到，对他们来说，做演讲比听演讲更有价值，但当然也有一些很棒的演讲。[辛迪·斯里达兰 \(Twitter\)](#) 是精彩演讲的最佳来源，特别是她的文章[2019 年科技演讲精选](#)，[2018 年科技演讲精选](#)，和[2017 年科技演讲精选](#)。

论文

相对而言，很少有 Staff-plus 工程师是计算机科学论文的狂热读者。然而，大多数人都熟悉一些基础论文，而那些花时间阅读论文的一小部分人往往能从中获得很多。

如果你渴望加入经常阅读论文的行列，那么没有比 Adrian Colyer 的更好的地方了[the morning paper](#)，它会在每个工作日向你发送一篇计算机科学论文的摘要。如果你更有兴趣了解一些著名论文的基础知识，首先阅读 Peter Klein 的[如何阅读学术文章](#)或者 S. Keshav 的[如何阅读论文](#)，然后进入这份推荐论文列表：

- [Dynamo：亚马逊的高可用键值存储](#)
- [关于设计和部署互联网规模服务](#)
- [没有银弹 - 软件工程中的本质与偶然](#)
- [走出焦油坑](#)
- [Chubby 锁服务，用于松散耦合的分布式系统](#)
- [Bigtable：用于结构化数据的分布式存储系统](#)
- [Raft：寻找一种易于理解的共识算法](#)
- [Paxos 变得简单](#)
- [SWIM：可扩展的弱一致性感染式进程组群成员协议](#)
- [计算机系统设计提示](#)
- [大泥球](#)
- [Google 文件系统](#)
- [CAP 十二年后：规则如何改变](#)
- [收获、产量和可扩展的容错系统](#)
- [MapReduce：大型集群上的简化数据处理](#)
- [Dapper：大规模分布式系统跟踪基础设施](#)
- [Kafka：用于日志处理的分布式消息传递系统](#)
- [Google 使用 Borg 进行的大规模集群管理](#)
- [Mesos：数据中心中细粒度资源共享的平台](#)

可能找到高质量论文的最佳地点是[我们热爱的论文](#)，他们也举办聚会来讨论论文。其他一些资源是[ACM SIGOPS 名人堂奖项列表](#)和[《非理性繁荣》论文集](#)。

其他好东西

在为这些资源做研究时，我发现了一些其他文章，它们不太适合上面的任何地方，但我认为它们很好，值得一看：

- [在生产环境中进行测试，安全的方法和在生产环境中进行测试：困难的部分](#)作者：Cindy Sridharan
- [科技领域十年回顾](#)作者：Cindy Sridharan
- [虚构的难题](#)作者：Dan Na

如果您发现更多，请发给我！

Staff-plus 工程师在组织中扮演什么角色？

当我在进行工程组织的组织设计时，我会思考很多关于“[组织数学](#),” 每支团队应该有一名经理和六到八名工程师的指导方针，以及每位经理的经理应该支持四到六名经理的指导方针。从这些数字中，您可以快速确定一个适合您组织的结构，并且效果会相当好。它可能不是完美，但它会工作。

当我在设计多个组织时应用这种方法时，反复出现的极端情况之一是决定最高级的工程师应该向谁汇报。他们应该像组织结构图所规定的那样，向组织叶节点中的经理汇报吗？或者，作为您组织中的关键领导者，他们应该向更高级的领导者汇报，以便更好地获得他们所需的信息和授权，从而在他们的角色中脱颖而出？

在回答之前，有必要描述一下你今天在公司里能找到的最常见的配置，特别是配置在不同公司之间的差异[员工+原型](#)：

- **技术主管**通常向负责一个团队的经理汇报。较少情况下，他们会向负责两到四个团队的经理汇报。在这两种情况下，他们的运营范围与该经理相同。例子：[丹·娜](#)向国际化平台经理汇报。
- **建筑师**通常向更高级别的经理汇报，通常是经理的经理。通常，他们将负责该经理职责范围内的横向切片，例如，数据建模。例子：[基维·麦克明](#)向首席技术官汇报。
- **解决者**通常在具有“[弱团队概念](#)”的公司中运作，并且在这些公司中，报告层级结构通常不太明确或经过深思熟虑。最常见的是看到他们向团队经理汇报，但你会发现各种各样的情况。另一种常见的模式是将这些人聚集到“[首席技术官办公室](#)”或者“[首席执行官办公室](#)”他们向一位指导他们工作的主管汇报。例子：[里图·文森特](#)是向首席执行官汇报的孵化器的一部分。
- **左右手**向高级领导汇报，通常是负责一百人或更多人的经理，以该领导的权力运作。例子：[里克·布恩](#)向基础设施副总裁汇报；米歇尔·布向[首席产品官](#)。

了解这些不同的原型通常如何在组织中以不同的方式汇报，有助于解码一些报告结构的表面上的随意性。

首席技术官办公室

关于“**首席技术官办公室**”概念的题外话，因为很多人在工作中没有遇到过。通常，首席技术官（有时首席执行官也会做类似的事情）会有两到八名以上的员工工程师直接向他们汇报。这些人被视为高级领导，他们会得到一个机会去追求，很少的管理支持，并且在必要时可以依靠高管的支持。

在这些办公室里，你会发现各种各样的人*建筑师, 解决者和左右手*。

通常情况下，**首席技术官办公室**通常在公司发展的后期才会出现，并且通常是作为解决现有组织问题的权宜之计而引入的，例如，Staff-plus工程师和管理团队之间缺乏信任，或者CTO无法授权。如果你发现自己过早地*过早地*在公司发展过程中使用它，问问自己是否在逃避一个应该解决的问题，而不是将这个概念引入你的结构中。

但在实践中.....

根据原型，通常每个工程师都有一个理论上正确的位置可以融入组织，但你会发现，在实践中，很少有组织能够完全将其实际报告结构与该理论结构对齐。

有时这是由于你的管理团队对组织结构缺乏关注。在其他情况下，这是由于管理带宽不足以支持处于正确位置的人员。例如，“正确”的经理已经在管理一个由12人组成的团队，无法有效地支持另一位工程师。另一个常见的情况是，结构变化得足够频繁，以至于经理们不愿再次更换工程师的经理*再次*，尤其是经理变动通常会导致抽象、平庸的绩效评估。

如果你发现自己向一个你认为不合适的经理汇报工作，那么与你的经理进行一次合理的对话是可行的，但值得承认的是，许多经理会对暗示他们不适合你的说法做出防御反应。如果你的经理很成熟，并且你与他们关系良好，那就继续进行对话。如果不是，那么与他们的上级领导进行一次更抽象的讨论，了解Staff-plus工程师在他们的组织中向谁汇报，可能风险更小。

在你急于倡导变革之前，先问问自己，如果你的汇报结构发生变化，你认为会有什么不同。汇报结构是一种权威形式，一般来说，人们高估了权威对他们的帮助。当然，经典的陷阱是，那些从额外权威中获益最多的人——少数族裔和女性——是那些经理最有可能对变革的建议做出防御反应的人。

应该如何运作？

如果你想为你的管理团队设计一个关于他们应该如何随着时间的推移进行这类组织调整的提案，有几件事需要考虑。在可能的情况下，**汇报变更应该立即发生**。延迟会迫使人们经历两次过渡，包括在前任和新任角色之间一个特别具有挑战性的中间环境。经历一次角色过渡的风险总是较低的，即使这意味着如果你的新经理因工作量过大而无法提供支持，你也同意减少经理的支持。

如果你不能立即做到，**始终设定一个时间表**以便在角色变更后向正确的经理汇报。如果你没有明确重新开放结构的时间框架，那么这种情况发生的可能性相对较小。

大多数公司都在努力建立这种组织架构，以充分支持 Staff-plus 工程师作为真正的领导者，因此你应该将此视为一个你希望取得进展的问题多年，而不是一夜之间就能解决的问题。如果你期望一个立即、永久的解决方案，你可能会遇到一些麻烦。

管理 Staff-plus 工程师

在获得关于 [StaffEng](#) 的反馈时，有人要求提供更多关于管理 Staff-plus 工程师的内容。这不太符合主题——该工作重点关注的是 Staff 工程师本身，而不是公司或经理——但这是一个有趣的话题，也是一个值得添加的附录。

当然，管理 Staff-plus 人员并非所有方面都是该级别独有的：有一些基本原则适用于管理任何角色中的任何人，例如[进行有效的 1 对 1 会议](#)或者[给予反馈](#)。对于这类事情，请阅读 Lara Hogan 的[弹性管理](#)或者 Camille Fournier 的[管理者之路](#)我想在这里探讨的是，管理 Staff-plus 级别的人员与管理高级工程师有何不同。

这些角色在不同的公司之间差异很大，以至于管理你的 Staff-plus 工程师的某些方面将取决于[Staff 原型](#) 贵公司强调和 [你的 Staff-plus 工程师应该在工程组织中处于什么位置](#)，但有一些方法在大多数配置中对您都有帮助。

- **赞助和支持比你指导的更多。** 如果你每天都在指导你的技术骨干工程师，那么你就是在使用错误的角色中使用他们。如果你没有每周给他们反馈，那么你就是延缓他们的成长。如果你没有为他们的倡议提供支持，那么你就会把他们的积极性扼杀掉。
- **帮助他们重新定义成功的定义。** 在高绩效的产品工程团队中工作是一个积极反馈的飞轮。你的产品经理欣赏你的工作。你的工程经理与团队互动。你的同事们喜欢一起工作。你的用户喜欢你的产品。你的企业喜欢用户的采纳。相反，Staff工程师的反馈飞轮就没有那么直接了。你花更多的时间来处理冲突。你的工作时间更长。你代表着重要的优先事项，这需要降低某些业务或产品目标的优先级。很多人没有意识到这种转变，一年后醒来讨厌他们的新角色，作为他们的经理，你可以帮助他们认识到这种转变，并找到补偿策略[保持精力充沛](#)。
- **给予反馈。** 一个特别重要的策略是重写他们对成功的定义——并保持他们的成长——是给予频繁的反馈。如果他们选择了错误的

战斗，告诉他们，并告诉他们为什么如果他们优先考虑你不会做的工作，告诉他们，并告诉他们为什么没有什么比不知道自己做得怎么样更让一个高绩效者感到压力了！如果你不给予反馈，特别是关于他们最好的工作，他们会不断改变他们的方法，直到你给予反馈（通常让你后悔）。

- **让他们随时了解情况**作为一名经理，很容易忘记你比与你合作的工程师拥有更多的信息。现实情况是，大多数组织围绕经理向其他经理传达关键信息来构建其信息流。如果你没有找到一种有意的、可重复的过程来与他们分享你的背景信息，你的 Staff-plus 工程师将会受到阻碍。有些人会在他们的 1:1 会议开始时这样做，这还可以，但我更喜欢在事情发生时将它们放入团队的聊天频道中，并将它们汇总到我的[每周电子邮件更新](#)。
- **让他们参与计划和优先级排序。**许多工程师感到沮丧，认为“正确的工作永远无法优先考虑”，解决这个问题最好的方法之一是主动让更多工程师参与到规划过程中。这在两个方面起作用。首先，他们更了解竞争性工作以及为什么这项工作很重要，其次，他们将能够更有效地倡导他们认为缺失的技术工作。
- **就如何在独立行动的同时保持一致达成一致。**当你推动 Staff-plus 工程师时支持他们走向领导岗位，他们会开始更多地领导，这有时会让你感到惊讶。如果你合作的领导者从不让你感到惊讶，那么你授权不够，但如果他们经常让你感到惊讶，那么明确[明确建立你的控制](#)。
- **为他们创造思考的空间，同时不要让他们脱离组织的日常现实。**这些角色中的许多人受到影响和“为企业做正确的事情”的激励，以至于他们会在没有外部干预的情况下把自己累垮。如果你是他们的经理，那么“外部干预”意味着你。如果你看到他们花费太多时间救火和帮助解决紧急工作，与他们合作，为深度思考工作争取更多时间。相反，如果你看到他们只做深度思考工作，他们可能会失去背景，如果他们不调整这种组合，可能会失去同行的尊重和业务。
- **提醒他们，他们是榜样。**就像他们对经理所做的那样，组织中的工程师会观察 Staff-plus 工程师，以了解哪些行为和行动会得到奖励（和容忍）。这是一个巨大的责任，但也是一个巨大的影响机

会：通过践行积极的价值观，他们有机会在他们周围创建一个积极的组织。

- **尽量减少经理的负担。**在追求[效率高干效果](#)，许多公司让他们的经理陷入了大量的协调和官僚主义之中。当你快要淹死时，你会尽一切可能寻求帮助，在很多情况下，这会导致经理将管理工作转移给他们的Staff工程师。这种情况绝对会发生——你与你管理的Staff-plus工程师的关系是一种伙伴关系——但要尽量减少这种工作量，并确保这只是一种暂时的负担，而不是永久性的。
- **给他们未经提炼的问题。**这是一个高级职位你应该给他们一个问题空间，让他们将其缩小为一个更具体的问题和解决方案。他们比你拥有更好的技术背景，如果你过于精确地指出你认为的问题所在，你就无法从他们的判断中获益。选择正确的问题与找到正确的解决方案至少具有相同的影响，并且只有在你创造空间时才有可能。
- **将空间让给他们的领导。**当你管理Staff-plus工程师时，找到方法将你所有权的一部分明确地转移到他们的责任范围内。例如，你如何让他们对团队的技术质量负责，而不是你来做？这为你和Staff-plus工程师创造了杠杆作用和所有权感。
- **欣赏他们。**优秀的 Staff-plus 工程师可以相当独立地工作，因此当组织陷入困境时，很容易降低他们的优先级。忽视你最重要的人[是“小吃”的经理版本](#)——感觉很重要，但通常不是正确的优先事项。所以保留你的 1:1 会议，并且通常记得参加，特别是如果他们不是那种会要求的人。
- **建立并坚持与业务保持一致。**有些工程师成功了，尽管他们怀有一种技术工作比需要这项工作的业务更重要的心态。这种心态通常是有害的，但当由 Staff-plus 工程师持有时，它尤其有害。这是一个更广泛组织的榜样，将他们扩展到这种观点之外对于他们继续担任领导角色至关重要。公司会破坏并最终驱逐与业务不一致的领导者。
- **让他们对整个角色负责。**虽然很少有人存在技术方面有重大缺陷的情况下达到 Staff-plus 角色，但我的亲身经历是，许多人在领导力或行为挑战方面受到严重阻碍而达到这些角色。这些人获得了头衔，但往往徘徊在 Staff 炼狱中，在那里他们被期望领导，

但却被排除在大多数领导机会之外。他们被认为太不可靠或“参与成本太高”。你必须就他们的差距向这些人提供反馈，并让他们对整个角色的期望负责。不要让他们无限期地作为准领导者徘徊。也许他们最初是通过[头衔膨胀](#)获得了这个角色，所以你决定只是掩盖他们的差距而不是解决它——不要这样做，相反，制定一个计划来支持他们[同时将该责任转移给他们](#)。

- **让他们可以进入房间，但不要把它当作一种地位的象征。**人们常常执着于地位的象征，而工程师们特别关注的一个是“[在房间里](#)。”有时会议是工作发生的地方，但大多数例行报告会议都有太多的人参加，你可以通过在不同的会议中分摊出席人数，而不是在所有会议中都增加一倍的人数，为你和你所支持的Staff-plus工程师创造大量的时间和空间。

从高级工程师到Staff-plus工程师的转变是一个重大的转变，它改变了正在完成的工作的类型，而之前的转变通常只改变了工作的范围。很多人都在努力适应这种转变，而且很多经理也不确定如何帮助支持与他们合作的Staff-plus工程师。当然，这并不是是一份完整的你可以做的事情来支持他们的清单，但希望这是一个有用的起点。

设计 Staff-plus 面试流程

当我们谈论设计一个Staff-plus工程师面试流程时，首先要谈论的是，绝对没有人对他们的Staff-plus面试流程运作良好有信心。许多流程最终都在寻找一个高级工程师，他*真的*擅长解决问题，但这并不能反映实际的角色。其他人则专注于沟通技巧，这是是这个角色的一个关键部分，但肯定不是它的全部。甚至有一些公司构建他们的流程来评估候选人是否*感觉*像他们现有高级工程团队的成员一样，将卓越与熟悉混为一谈。

即使没有人对他们的面试流程感到满意，仍然有很多集体经验可以融入到你设计的流程中，这就是我们在这里要讨论的内容。我们将首先检查Staff+级别面试流程中常见的失败模式，并讨论你所需要的信号*做*想在这些循环中进行测试，最后讨论一些面试形式，这些形式对于评估这些信号可能很有用。

挑战

虽然技术面试总体上有点混乱，但对非常资深的候选人的面试本身就存在足够多的问题，足以专门安排一辆挣扎巴士。在考虑什么可能更有效之前，首先了解一些常见的失败模式是有帮助的：

- **一位高级工程师，但更好。**许多面试过程将Staff-plus工程师视为在所有方面都做得更好一点的高级工程师。他们速度更快一点。他们的沟通更清晰一点。他们在架构讨论中更细致一点。这源于大多数人对Staff角色的不熟悉，并且通常会导致Staff-plus工程师在这些循环中表现不佳。特别是，大多数Staff-plus工程师的编程量少于高级工程师，因此在死记硬背的编程任务中，他们是“更慢”而不是“更快”。你会发现一些仍然非常快速的程序员Staff-plus工程师，但你不会发现这种速度与他们的影响力之间存在太多关联。循环。特别是，大多数Staff-plus工程师的编程量少于高级工程师，因此*更慢*而不是*更快*在死记硬背的编程任务中。你会发现

一些仍然非常快速的程序员Staff-plus工程师，但你不会发现这种速度与他们的影响力之间存在太多关联。

- **一个高级工程师，但更糟糕。**相反，其他的面试流程认识到，Staff-plus工程师花费在编程上的时间更少，并且预计在一些更平凡的编程练习中表现会更慢。这使得Staff-plus工程师更有可能在面试中成功，但无法获得关于是什么让这类工程师特别有影响力的信号。如果你不增加额外的面试来捕捉这些优势，那么在死记硬背工作上的速度降低可能会在一定程度上与资历相关，但它与其他许多因素的相关性更强。
- **一个高级工程师，但他们会接受这份工作。**另一种失败模式是，一些公司正在努力招聘高级工程师，并决定在不改变角色期望的情况下抬高他们的职称。在这些情况下，面试与实际工作相符，但职称不符。由于公司和候选人都不完全愿意承认这种膨胀，该公司未来所有Staff-plus的招聘都带上了一层不确定性。
- **和我们一样的人。**许多面试环节侧重于Staff-plus候选人是否表现出与公司现有Staff-plus工程师相同的智慧和信心，面试总结可能包括诸如“他们感觉会成为这个团队的自然组成部分”之类的陈述。这种方法更有可能锚定在半任意的特征上，比如他们如何展现自己的信心，而不是候选人的能力。
- **不比我强。**尤其是在招聘你的第一批Staff-plus工程师时，你经常会发现一些早期职业的面试官低估了候选人的优势，而是锚定在候选人执行的能力上面面试官目前的职位。你可能会遇到一位令人印象深刻的 Staff-plus 候选人，但面试小组会对他们作为中级工程师的能力持怀疑态度。这种情况似乎最常发生在女性和少数族裔候选人身上。
- **反向过滤器。**某些类型的面试表明，作为一个组织，你不知道如何使用 Staff-plus 工程师：白板算法面试，面试官主要是在职业生涯早期等等。许多 Staff-plus 流程导致最优秀的候选人在流程早期退出，这通常对招聘指标来说是有些隐形的。
- **过于注重头衔？**在达到一定成就水平后，人们不太关心内部等级，通常是因为他们已经经济稳定。这给新达到 Staff-plus 级别的人带来了一种特殊的压力，即避免显得过于追求职业或头衔，这使得人们更难首次获得该头衔。

其中一些问题很难解决，另一些问题则很容易解决，只要你记住它们，但当你开始设计或重新构想你的 Staff-plus 面试流程时，所有这些问题都值得考虑。

信号

这个[最佳面试循环](#)从你想要捕获的信号向前推导出面试主题和形式，这意味着要回答的第一个重要问题是，“哪些信号对于招聘成功的 Staff-plus 工程师至关重要？”

我建议关注的信号是：

- **自我意识**他们是否对错误负责？他们是否在以前较弱的领域表现出成长？
- **判断力**。他们是否能够看到问题的潜在风险？他们是否能够应对广泛、模糊的问题？他们是否能够有效地调解在权衡或设计方面存在争议的人员？他们是否能够降低执行难题的风险？
- **协作**。他们是否与他人合作良好？对于那些经验不如他们的人呢？比他们更有经验的人呢？他们的经理呢？跨职能部门呢？高管呢？
- **沟通**。他们是否是优秀的倾听者，能够理解他人提出的观点？他们是否能够清晰地表达自己的想法？他们是否能够以贵公司所依赖的格式（书面、口头等）进行沟通？
- **发展**。他们是否能促进周围其他人的成长？在他们领导的领域，“组织人才储备”是增长还是萎缩？破损的系统和流程是否得到清理？

有趣的是，许多人不会认为这些是明确的技术技能。领域专业知识是他们成功的关键因素，但正是将这种专业知识与其他关键技能和行为结合起来，才能将一位资深的资深工程师转变成为一名 Staff-plus 工程师。

格式和结构

在设计面试流程时，需要问自己的两个关键问题始终是：

1. 这个人需要哪些任务和行为才能在日常工作中取得成功？
2. 我们如何让他们展示实际操作这些任务和行为？

大多数资深候选人变得越来越有外交手腕，而且询问他们关于工作的事情，永远不如观察他们做这项工作更有帮助。如果指导是最重要的活动，不要依赖他们谈论指导，而是想办法看到他们指导别人。如果是架构方面，展示你当前的系统，让他们带着问题来，看看他们对不同意的决定有何反应——摆脱模棱两可的抽象。

除了典型的一对一讨论和编程面试之外，我发现一些面试形式和结构对于评估Staff-plus信号特别有效：

- **结构化演示。**让候选人准备并向一组同行进行二十分钟到三十分钟的关于一个狭窄主题的演示。这种形式可以很好地反映结构化思维、沟通、倾听和回答问题的能力。根据你选择的主题，你可以获得关于你选择的一个或两个领域的强烈信号。这种形式在了解人们如何谈论他们的同行和同事方面尤其有效。
- **代码审查。**准备一个拉取请求，并要求候选人提供反馈，重点关注同理心、清晰度和实用性。
- **数据建模、接口和架构。**让候选人逐步完成一个系统的设计，通常侧重于使其不断发展以满足不断变化的需求。这些面试通常试图做太多事情：缩小你的关注范围，并在问题中添加层次，以便你可以继续深入挖掘那些取得重大进展的候选人。
- **主题专业知识。**测试他们领域专业知识的面试。例如，前端工程师可能会与设计师和产品经理合作，讨论技术约束将如何影响拟议的设计和发布时间表。对于后端工程师，你可以为候选人提供一个损坏的软件或环境，让他们调试问题并修复。
- **导师小组**如果你的面试小组完全由职业生涯早期的员工组成，那么招聘 Staff-plus 候选人会很有挑战性，但如果 Staff-plus 候选人没有展示出指导职业生涯早期员工的成功经验，那么招聘他们同样具有风险。让三到四位他们可能会指导的员工组成面试小

组，并提出问题。观察他们如何将粗略的问题转化为有用的讨论，可以特别深入地了解他们在新角色中指导他人的能力。

如果这些形式还不够，那就开始四处打听！大多数公司都为他们的 Staff-plus 评估流程设计了定制的方法，你可以从讨论中学习到很多。

如何整合

你可能会理所当然地认为，这篇文章会以一个预先编译好的面试流程结束，供你的公司用来评估 Staff-plus 工程师，我很遗憾让你失望了，但我认为大部分价值来自于思考对你来说重要的信号，并设计出能够以一种与你和你的公司产生共鸣的方式来获取这些信号的形式。

无论你最终使用什么面试方式，都要测试它们，收集候选人的反馈，并不断改进它们，让它们变得更好！

员工晋升阶梯

有所以现在公开分享了许多不同的职业阶梯，没有理由不是在尝试设计自己的职业阶梯之前，先阅读六个不同的职业阶梯。

一些特别值得阅读的职业阶梯：

- [Rent the Runway \(服装租赁平台\)](#)
- [Kickstarter \(众筹平台\)](#)
- [Patreon \(赞助平台\)](#)

你可以在以下网址找到更多收集的信息：[progression.fyi](#) Charity Majors 也写了一篇有用的指南，关于[关于工程级别的须知](#)。

我认为重要的是要认识到职业阶梯只对人群有效。它们很少，甚至从未，干净利落地适用于任何个人。这种效应在 Staff-plus 角色中尤其明显，这些角色通常只有几个人。阶梯是必不可少的，但不要陷入相信它们是事物实际运作方式的地图，而不是事物应该如何运作的神话。